

RaschPy

Version 1.2.2

Mark Elliott

August 2026

Abstract

RaschPy (Elliott, 2026) is a Python package for Rasch analysis which can estimate parameters for a variety of Rasch models, generate a range of model fit statistics and output tables and graphical plots. *RaschPy* also contains simulation functionality (used for the simulations in this work). *RaschPy* is open source and free to download. Specifications are subject to change as the software is developed.

RaschPy is capable of estimating parameters and generating tables of estimates and fit statistics plus assorted plotting and simulation functionality, for the following Rasch models:

- Simple Logistic Model (SLM), aka dichotomous Rasch model (Rasch, 1960, 1968)
- Partial Credit Model (PCM) (Masters, 1982)
- Rating Scale Model (RSM) (Andrich, 1978)
- Many-Facet Rasch Model (MFRM) (Linacre, 1994), RSM formulation
- Extended Many-Facet Rasch Models (Extended MFRM) (Elliott & Buttery, 2022b), RSM formulation

Parameter estimation uses non-iterative conditional pairwise methods: PAIR (Choppin, 1968, 1985) and the Eigenvector Method (EVM) (Garner & Engelhard, 2002, 2009) for SLM and PCM, and the Conditional Pairwise Adjacent Thresholds (CPAT) method (Elliott & Buttery, 2022a) for the RSM and MFRM models.

For each model, methods are grouped under the following sections:

- Preliminaries: Loading data and editing labels.
- Core functions: Functions for category probabilities, expected scores, variance and kurtosis.
- Parameter estimation: Estimating item, person and (where applicable) rater parameters and generating bootstrapped standard errors of estimation.
- Statistical output: Generating tables of estimates and fit statistics for items, persons and (where applicable) raters, plus overall test statistics, residual correlation analysis and category counts.
- Plotting functionality: Plots for a variety of item-level and test-level functions, showing theoretical curves with the option to overlay relevant features such as observed class category responses and threshold lines.

Contents

1	About <i>RaschPy</i>	10
2	Loading data	12
2.1	SLM	12
2.1.1	loadup_slm	12
2.2	PCM	14
2.2.1	loadup_pcm	14
2.3	RSM	16
2.3.1	loadup_rsm	16
2.4	MFRM	18
2.4.1	loadup_mfrm_single	18
2.4.2	loadup_mfrm_xlsx_tabs	20
2.4.3	loadup_mfrm_multiple	22
2.5	Exogenous covariates (for DIF analysis)	25
2.5.1	loadup_exogenous	25
3	class SLM	26
3.1	Preliminaries	26
3.1.1	SLM	26
3.1.2	check_data_connectivity	28
3.1.3	rename_item	29
3.1.4	rename_items_all	29
3.1.5	rename_person	30
3.1.6	rename_persons_all	30
3.2	Core functions	31
3.2.1	cat_prob	31
3.2.2	exp_score	32
3.2.3	variance	33
3.2.4	kurtosis	34
3.3	Parameter estimation	35
3.3.1	calibrate	35
3.3.2	std_errors	36
3.3.3	calibrate_anchor	38
3.3.4	person	42
3.3.5	person_estimates	44
3.3.6	score_lookup	45
3.3.7	score_lookup_table	46
3.3.8	csem	47
3.4	Statistical output	48
3.4.1	item_stats_df	48
3.4.2	person_stats_df	51
3.4.3	test_stats_df	54

3.4.4	andersen_lr_test	57
3.4.5	dif_test	59
3.4.6	res_corr_analysis	64
3.4.7	save_stats	66
3.4.8	save_residuals	67
3.4.9	category_counts_df	68
3.5	Plotting functionality	69
3.5.1	Shared plotting arguments	69
3.5.2	icc	70
3.5.3	crcs	72
3.5.4	iic	74
3.5.5	tcc	76
3.5.6	test_info	78
3.5.7	test_csem	80
3.5.8	std_residuals_plot	81
3.5.9	plot_anchor_selection	82
4	class PCM	84
4.1	Preliminaries	84
4.1.1	PCM	84
4.1.2	check_data_connectivity	86
4.1.3	rename_item	87
4.1.4	rename_items_all	87
4.1.5	rename_person	88
4.1.6	rename_persons_all	88
4.2	Core functions	89
4.2.1	cat_prob_centred	89
4.2.2	cat_prob_uncentred	90
4.2.3	exp_score_centred	91
4.2.4	exp_score_uncentred	92
4.2.5	variance_centred	93
4.2.6	variance_uncentred	94
4.2.7	kurtosis_centred	95
4.2.8	kurtosis_uncentred	96
4.3	Parameter estimation	97
4.3.1	calibrate	97
4.3.2	std_errors	99
4.3.3	calibrate_anchor	101
4.3.4	person	106
4.3.5	person_estimates	108
4.3.6	score_lookup	110
4.3.7	score_lookup_table	111
4.3.8	csem	112
4.4	Statistical output	113

4.4.1	item_stats_df	113
4.4.2	threshold_stats_df	116
4.4.3	category_stats_df	119
4.4.4	person_stats_df	121
4.4.5	test_stats_df	124
4.4.6	model_selection	127
4.4.7	andersen_lr_test	129
4.4.8	dif_test	131
4.4.9	res_corr_analysis	137
4.4.10	save_stats	139
4.4.11	save_residuals	140
4.4.12	category_counts_df	141
4.5	Plotting functionality	142
4.5.1	Shared plotting arguments	142
4.5.2	icc	143
4.5.3	crcs	145
4.5.4	threshold_ccs	147
4.5.5	iic	149
4.5.6	tcc	151
4.5.7	test_info	153
4.5.8	test_csem	154
4.5.9	std_residuals_plot	155
4.5.10	plot_anchor_selection	156
5	class RSM	158
5.1	Preliminaries	158
5.1.1	RSM	158
5.1.2	check_data_connectivity	160
5.1.3	rename_item	161
5.1.4	rename_items_all	161
5.1.5	rename_person	162
5.1.6	rename_persons_all	162
5.2	Core functions	163
5.2.1	cat_prob	163
5.2.2	exp_score	164
5.2.3	variance	165
5.2.4	kurtosis	166
5.3	Parameter estimation	167
5.3.1	calibrate	167
5.3.2	std_errors	169
5.3.3	calibrate_anchor	171
5.3.4	person	176
5.3.5	person_estimates	177
5.3.6	score_lookup	178

5.3.7	score_lookup_table	179
5.3.8	csem	180
5.4	Statistical output	181
5.4.1	item_stats_df	181
5.4.2	threshold_stats_df	184
5.4.3	category_stats_df	187
5.4.4	person_stats_df	189
5.4.5	test_stats_df	192
5.4.6	model_selection	195
5.4.7	andersen_lr_test	197
5.4.8	dif_test	199
5.4.9	res_corr_analysis	205
5.4.10	save_stats	207
5.4.11	save_residuals	208
5.4.12	category_counts_df	209
5.5	Plotting functionality	210
5.5.1	Shared plotting arguments	210
5.5.2	icc	211
5.5.3	crcs	213
5.5.4	threshold_ccs	215
5.5.5	iic	217
5.5.6	tcc	219
5.5.7	test_info	221
5.5.8	test_csem	222
5.5.9	std_residuals_plot	223
5.5.10	plot_anchor_selection	224
6	class MFRM	226
6.1	Preliminaries	226
6.1.1	MFRM	226
6.1.2	check_data_connectivity	229
6.1.3	rename_item	230
6.1.4	rename_items_all	230
6.1.5	rename_person	231
6.1.6	rename_persons_all	231
6.1.7	rename_facet_element	232
6.1.8	rename_facet_elements_all	232
6.2	Core functions	233
6.2.1	cat_prob	233
6.2.2	exp_score	239
6.2.3	variance	243
6.2.4	kurtosis	247
6.3	Parameter estimation	251
6.3.1	calibrate	251

6.3.2	calibrate_anchor	260
6.3.3	std_errors	265
6.3.4	anchor_std_errors	272
6.3.5	person	274
6.3.6	person_estimates	276
6.3.7	score_lookup	279
6.3.8	score_lookup_table	281
6.3.9	csem	283
6.3.10	model_selection	285
6.3.11	per_rater_model_selection	288
6.4	Statistical output	292
6.4.1	item_stats_df	292
6.4.2	threshold_stats_df	296
6.4.3	category_stats_df	301
6.4.4	rater_stats_df	303
6.4.5	check_anchor_homogeneity	309
6.4.6	person_stats_df	312
6.4.7	test_stats_df	315
6.4.8	item_res_corr_analysis	318
6.4.9	rater_res_corr_analysis	321
6.4.10	save_stats	324
6.4.11	save_residuals_items / save_residuals_raters	326
6.4.12	category_counts_df	327
6.5	Plotting functionality	329
6.5.1	Shared plotting arguments	329
6.5.2	icc	330
6.5.3	crCs	332
6.5.4	threshold_ccs	334
6.5.5	iic	336
6.5.6	tcc	338
6.5.7	test_info	340
6.5.8	test_csem	342
6.5.9	std_residuals_plot	344
7	class SLM_Sim	345
7.1	Generating an SLM_Sim simulation	345
7.1.1	SLM_Sim	345
7.2	Customising an SLM_Sim simulation	348
7.2.1	rename_item	348
7.2.2	rename_items_all	348
7.2.3	rename_person	349
7.2.4	rename_persons_all	349
8	class PCM_Sim	350

8.1	Generating a PCM_Sim simulation	350
8.1.1	PCM_Sim	350
8.2	Customising an PCM_Sim simulation	354
8.2.1	rename_item	354
8.2.2	rename_items_all	354
8.2.3	rename_person	355
8.2.4	rename_persons_all	355
9	class RSM_Sim	356
9.1	Generating an RSM_Sim simulation	356
9.1.1	RSM_Sim	356
9.2	Customising an RSM_Sim simulation	360
9.2.1	rename_item	360
9.2.2	rename_items_all	360
9.2.3	rename_person	361
9.2.4	rename_persons_all	361
10	class MFRM_Sim_Global	362
10.1	Generating an MFRM_Sim_Global simulation	362
10.1.1	MFRM_Sim_Global	362
10.2	Customising an MFRM_Sim_Global simulation	367
10.2.1	rename_item	367
10.2.2	rename_items_all	367
10.2.3	rename_person	368
10.2.4	rename_persons_all	368
10.2.5	rename_rater	369
10.2.6	rename_raters_all	369
11	class MFRM_Sim_Items	370
11.1	Generating an MFRM_Sim_Items simulation	370
11.1.1	MFRM_Sim_Items	370
11.2	Customising an MFRM_Sim_Items simulation	375
11.2.1	rename_item	375
11.2.2	rename_items_all	375
11.2.3	rename_person	376
11.2.4	rename_persons_all	376
11.2.5	rename_rater	377
11.2.6	rename_raters_all	377
12	class MFRM_Sim_Thresholds	378
12.1	Generating an MFRM_Sim_Thresholds simulation	378
12.1.1	MFRM_Sim_Thresholds	378
12.2	Customising an MFRM_Sim_Thresholds simulation	383
12.2.1	rename_item	383
12.2.2	rename_items_all	383

12.2.3	rename_person	384
12.2.4	rename_persons_all	384
12.2.5	rename_rater	385
12.2.6	rename_raters_all	385
13	class MFRM_Sim_Matrix	386
13.1	Generating an MFRM_Sim_Matrix simulation	386
13.1.1	MFRM_Sim_Matrix	386
13.2	Customising an MFRM_Sim_Matrix simulation	391
13.2.1	rename_item	391
13.2.2	rename_items_all	391
13.2.3	rename_person	392
13.2.4	rename_persons_all	392
13.2.5	rename_rater	393
13.2.6	rename_raters_all	393
14	class MFRM_Sim_Bivector	394
14.1	Generating an MFRM_Sim_Bivector simulation	394
14.1.1	MFRM_Sim_Bivector	394
14.2	Customising an MFRM_Sim_Bivector simulation	399
14.2.1	rename_item	399
14.2.2	rename_items_all	399
14.2.3	rename_person	400
14.2.4	rename_persons_all	400
14.2.5	rename_rater	401
14.2.6	rename_raters_all	401
15	class MFRM_Sim_Centrality	402
15.1	Generating an MFRM_Sim_Centrality simulation	402
15.1.1	MFRM_Sim_Centrality	402
15.2	Customising an MFRM_Sim_Centrality simulation	406
15.2.1	rename_item	406
15.2.2	rename_items_all	406
15.2.3	rename_person	407
15.2.4	rename_persons_all	407
15.2.5	rename_rater	408
15.2.6	rename_raters_all	408
16	class MFRM_Sim_PseudoHalo	409
16.1	Generating an MFRM_Sim_PseudoHalo simulation	409
16.1.1	MFRM_Sim_PseudoHalo	409
16.2	Customising an MFRM_Sim_PseudoHalo simulation	413
16.2.1	rename_item	413
16.2.2	rename_items_all	413
16.2.3	rename_person	414

16.2.4	<code>rename_persons_all</code>	414
16.2.5	<code>rename_rater</code>	415
16.2.6	<code>rename_raters_all</code>	415
17	class <code>MFRM_Sim_Bistretch</code>	416
17.1	Generating an <code>MFRM_Sim_Bistretch</code> simulation	416
17.1.1	<code>MFRM_Sim_Bistretch</code>	416
17.2	Customising an <code>MFRM_Sim_Bistretch</code> simulation	421
17.2.1	<code>rename_item</code>	421
17.2.2	<code>rename_items_all</code>	421
17.2.3	<code>rename_person</code>	422
17.2.4	<code>rename_persons_all</code>	422
17.2.5	<code>rename_rater</code>	423
17.2.6	<code>rename_raters_all</code>	423
	References	424

1 About *RaschPy*

Built-in dependencies: `math`, `string`, `warnings`. Non-built-in dependencies: `numpy`, `pandas`, `matplotlib`, `scipy`, `sklearn`, `seaborn`, `openpyxl`. Non-built-in dependencies, not part of the core anaconda installation: `seaborn` and `openpyxl`.

Maintainer: Dr Mark Elliott <markelliott@cantab.net>

URL: <https://github.com/MarkElliott999/RaschPy>

Issues: <https://github.com/MarkElliott999/RaschPy/issues>

RaschPy is offered free under an Apache 2.0 licence, but please cite when used, using the following format:

Elliott, M. (2026). *RaschPy Rasch analysis in Python*. URL: <https://github.com/MarkElliott999/RaschPy>

Model and simulation class hierarchy

Model class	Description	Simulation class
SLM	Simple Logistic Model (dichotomous)	SLM_Sim
PCM	Partial Credit Model (polytomous, item-varying thresholds)	PCM_Sim
RSM	Rating Scale Model (polytomous, shared thresholds)	RSM_Sim
MFRM	MFRM — global rater representation	MFRM_Sim_Global
	MFRM — by-item rater representation	MFRM_Sim_Items
	MFRM — by-threshold rater representation	MFRM_Sim_Thresholds
	MFRM — full facet-effect matrix representation	MFRM_Sim_Matrix
	MFRM — bivector representation	MFRM_Sim_Bivector

Table 1: Model and simulation classes in *RaschPy*. All four analysis classes inherit from a common Rasch base class; the simulation classes from *Rasch_Sim*.

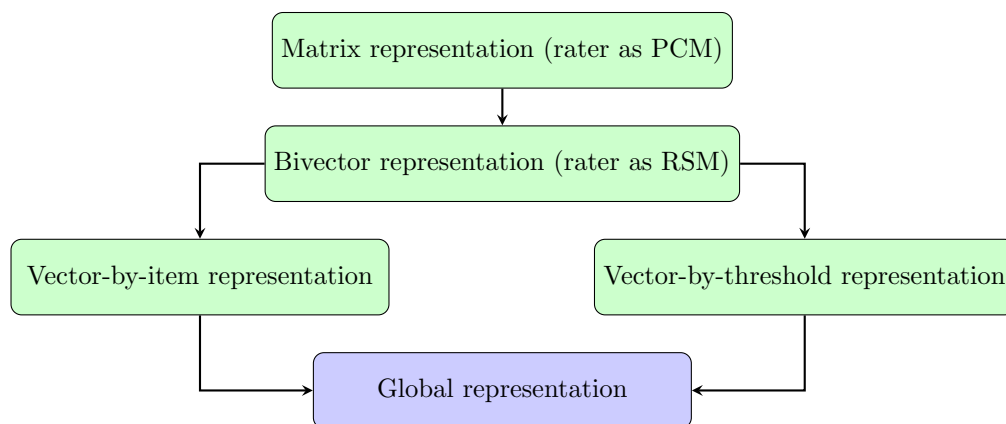


Figure 1: Hierarchy of extended many-facet Rasch models: an arrow flows from a more general model to a restricted form that is a special case of the more general form. The green boxes indicate extended rater representation models. From Elliott (2025).

Reproducibility

Every stochastic operation in *RaschPy* – simulation (all `*_Sim` classes), bootstrap standard errors (`std_errors`, `anchor_std_errors`), and subsampling for large-sample log-likelihood evaluation (`model_selection`, `per_rater_model_selection`) – accepts a `seed` argument. Each stochastic method draws its own independent generator via `numpy.random.default_rng(seed)`, rather than relying on `numpy`’s shared global random state. This means:

- `seed=None` (the default everywhere): fresh entropy is drawn each call, exactly as in earlier versions of *RaschPy* – no change to existing default behaviour.
- `seed=an integer`: the call becomes fully deterministic – repeating it with the same seed and the same data reproduces identical results, useful for regression testing, reviewer replication, and reporting a fixed simulation seed alongside published results.

Any method that auto-triggers one of the above internally – `fit_statistics`, `item_stats_df`, `test_stats_df`, `threshold_stats_df` and `rater_stats_df` (MFRM), `calibrate_anchor`, `dif_test`, and `check_anchor_homogeneity` (MFRM) – also accepts `seed`, which it forwards to whichever internal bootstrap/subsampling step it triggers. The seed is only consulted if that internal step runs, so if for example, standard errors have already been computed, `item_stats_df(seed=42)` has nothing left to seed). `calibrate` itself is deterministic (PAIR/CPAT involve no randomness) and takes no `seed` argument.

Example – a fully reproducible SLM simulation and bootstrap:

```
import raschpy as rp
sim = rp.SLM_Sim(no_of_items=10, no_of_persons=500, seed=42)
slm = rp.SLM(sim)
slm.calibrate()
slm.std_errors(no_of_samples=200, seed=42)
```

These five lines of code will always produce identical `slm.items` and `slm.item_se`.

2 Loading data

2.1 SLM

2.1.1 `loadup_slm`

Description

Function to load dichotomously scored data for use with class SLM.

Usage

```
loadup_slm(filename, item_names=True, person_names=True, long=False)
```

Arguments

<code>filename</code>	The full filename, including suffix <code>.csv</code> , <code>.xlsx</code> or <code>.json</code> .
<code>item_names</code>	Boolean. If <code>True</code> , the first row of data will be read as the item names. If <code>False</code> , item names will be allocated following the format <code>Item_1</code> etc. Default value is <code>True</code> .
<code>person_names</code>	Boolean. If <code>True</code> , the first column of data will be read as the person names. If <code>False</code> , person names will be allocated following the format <code>Person_1</code> etc. Default value is <code>True</code> .
<code>long</code>	Boolean. If value is <code>True</code> , data will be expected in long format; if value is <code>False</code> , data will be expected in wide format, as per toy example below. Default value is <code>False</code> .

Input file format

	Item_1	Item_2
Person_1	1	0
Person_2	0	1
Person_3	1	

Table 2: Wide format SLM data

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	0
Person_2	Item_1	0
Person_2	Item_2	1
Person_3	Item_1	1
Person_3	Item_2	

Table 3: Long format SLM data

Input files can be in wide format, with `long=False` (the default) or long format, with `long=True`. It is not necessary to specify whether the file is a csv, xlsx or json file; this will be inferred from the suffix `.csv`, `.xlsx` or `.json`. Examples of the required formats can be found in Table 2, which shows a toy example with three persons and two items in wide format, and Table 3, which shows the same data in long format. Missing data should be left blank, as shown for Person_3 and Item_2. Person and item names are optional, but if they are omitted, arguments `person_names=False` and/or `item_names=False` must be passed.

Returns

- pandas dataframe of responses in required format for class SLM
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load a csv file in wide format with item and person labels:

```
my_slm_df, my_invalid_responses = loadup_slm('my_slm_data.csv')
```

To load a csv file in wide format with no item or person labels:

```
my_slm_df, my_invalid_responses = loadup_slm('my_slm_data.csv', item_names=False,  
                                             person_names=False)
```

To load an xlsx file in long format:

```
my_slm_df, my_invalid_responses = loadup_slm('my_slm_data.xlsx', long=True)
```

2.2 PCM

2.2.1 loadup_pcm

Description

Function to load polytomously scored data for use with class PCM.

Usage

```
loadup_pcm(filename, max_score_vector=None, item_names=True, person_names=True,  
            long=False)
```

Arguments

<code>filename</code>	The full filename, including suffix <code>.csv</code> , <code>.xlsx</code> or <code>.json</code> .
<code>max_score_vector</code>	A vector of the maximum score for each item, as a list, numpy array or pandas Series. If omitted or <code>max_score_vector=None</code> , the maximum scores will be inferred from the data (passing a vector is recommended, however).
<code>item_names</code>	Boolean. If <code>True</code> , the first row of data will be read as the item names. If <code>False</code> , item names will be allocated following the format <code>Item_1</code> etc. Default value is <code>True</code> .
<code>person_names</code>	Boolean. If <code>True</code> , the first column of data will be read as the person names. If <code>False</code> , person names will be allocated following the format <code>Person_1</code> etc. Default value is <code>True</code> .
<code>long</code>	Boolean. If value is <code>True</code> , data will be expected in long format; if value is <code>False</code> , data will be expected in wide format, as per toy example below. Default value is <code>False</code> .

Input file format

	Item_1	Item_2
Person_1	1	3
Person_2	2	2
Person_3	2	

Table 4: Wide format PCM data

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	3
Person_2	Item_1	2
Person_2	Item_2	2
Person_3	Item_1	2
Person_3	Item_2	

Table 5: Long format PCM data

Input files can be in wide format, with `long=False` (the default) or long format, with `long=True`. It is not necessary to specify whether the file is a csv, xlsx or json file; this will be inferred from the suffix `.csv`, `.xlsx` or `.json`. Examples of the required formats can be found in Table 4, which shows a toy example with three persons and two items in wide format, and Table 5, which shows the same data in long format. Missing data should be left blank, as shown for Person_3 and Item_2. Person and item names are optional, but if they are omitted, arguments `person_names=False` and/or `item_names=False` must be passed.

Returns

- pandas dataframe of responses in required format for class PCM
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load a csv file in wide format for six items with item and person labels, passing a list of maximum scores:

```
my_pcm_df, my_invalid_responses = loadup_pcm('my_pcm_data.csv',
                                             max_score_vector=[2, 2, 3, 3, 5, 5])
```

To load a csv file in wide format with no item or person labels, passing a the variable name for a pre-existing vectors of maximum scores:

```
my_pcm_df, my_invalid_responses = loadup_pcm('my_pcm_data.csv',
                                             max_score_vector=my_max_score_vector,
                                             item_names=False, person_names=False)
```

To load an .xlsx file in long format, inferring the maximum scores from the data:

```
my_pcm_df, my_invalid_responses = loadup_pcm('my_pcm_data.xlsx', long=True)
```

2.3 RSM

2.3.1 `loadup_rsm`

Description

Function to load polytomously scored data for use with class RSM.

Usage

```
loadup_rsm(filename, max_score=None, item_names=True, person_names=True, long=False)
```

Arguments

<code>filename</code>	The full filename, including suffix <code>.csv</code> , <code>.xlsx</code> or <code>.json</code> .
<code>max_score</code>	The maximum score available. If omitted or <code>max_score=None</code> , the maximum scores will be inferred from the data (passing a maximum score is recommended, however).
<code>item_names</code>	Boolean. If <code>True</code> , the first row of data will be read as the item names. If <code>False</code> , item names will be allocated following the format <code>Item_1</code> etc. Default value is <code>True</code> .
<code>person_names</code>	Boolean. If <code>True</code> , the first column of data will be read as the person names. If <code>False</code> , person names will be allocated following the format <code>Person_1</code> etc. Default value is <code>True</code> .
<code>long</code>	Boolean. If value is <code>True</code> , data will be expected in long format; if value is <code>False</code> , data will be expected in wide format. Default value is <code>False</code> .

Input file format

	Item_1	Item_2
Person_1	1	3
Person_2	3	2
Person_3	2	

Table 6: Wide format RSM data

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	3
Person_2	Item_1	3
Person_2	Item_2	2
Person_3	Item_1	2
Person_3	Item_2	

Table 7: Long format RSM data

Input files can be in wide format, with `long=False` – the default – or long format, with `long=True`. Data loaded in long format will be converted to wide format for analysis. It is not necessary to specify whether the file is a csv, xlsx or json file; this will be inferred from the suffix `.csv`, `.xlsx` or `.json`. Examples of the required formats can be found in Table 6, which shows a toy example with three persons and two items in wide format, and Table 7, which shows the same data in long format. Missing data should be left blank, as shown for Person_3 and Item_2. Person and item names are optional, but if they are omitted, arguments `person_names=False` and/or `item_names=False` must be passed.

Returns

- pandas dataframe of responses in required format for class RSM
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load a csv file in wide format with item and person labels, with a maximum score of 5:

```
my_rsm_df, my_invalid_responses = loadup_rsm('my_rsm_data.csv', max_score=5)
```

To load a csv file in wide format with no item or person labels, passing a maximum scores stored as a variable `my_max_score`:

```
my_rsm_df, my_invalid_responses = loadup_rsm('my_rsm_data.csv',
                                             max_score=my_max_score,
                                             item_names=False, person_names=False)
```

To load an .xlsx file in long format, inferring the maximum score from the data:

```
my_rsm_df, my_invalid_responses = loadup_rsm('my_rsm_data.xlsx', long=True)
```

2.4 MFRM

2.4.1 `loadup_mfrm_single`

Description

Function to load polytomously scored data for use with class `MFRM` from a single csv file, xlsx or json tab.

Usage

```
loadup_mfrm_single(filename, max_score=None, item_names=True, long=False)
```

Arguments

<code>filename</code>	The full filename, including suffix <code>.csv</code> , <code>.xlsx</code> or <code>.json</code> .
<code>max_score</code>	The maximum score available. If omitted or <code>max_score=None</code> , the maximum scores will be inferred from the data (passing a maximum score is recommended, however).
<code>item_names</code>	Boolean. If <code>True</code> , the first row of data will be read as the item names. If <code>False</code> , item names will be allocated following the format <code>Item_1</code> etc. Default value is <code>True</code> .
<code>long</code>	Boolean. If value is <code>True</code> , data will be expected in long format; if value is <code>False</code> , data will be expected in wide format. Default value is <code>False</code> .

Input file format

		Item_1	Item_2
Rater_1	Person_1	1	3
Rater_1	Person_2	3	2
Rater_2	Person_1	2	2
Rater_2	Person_2	4	

Table 8: Wide format MFRM data, single sheet

Rater	Person	Item	Score
Rater_1	Person_1	Item_1	1
Rater_1	Person_1	Item_2	3
Rater_1	Person_2	Item_1	3
Rater_1	Person_2	Item_2	2
Rater_2	Person_1	Item_1	2
Rater_2	Person_1	Item_2	2
Rater_2	Person_2	Item_1	4
Rater_2	Person_2	Item_2	

Table 9: Long format MFRM data, single sheet

Input files can be in wide format, with `long=False` – the default – or long format, with `long=True`. Data loaded in long format will be converted to wide format for analysis. It is not necessary to specify whether the file is a csv, xlsx or json file; this will be inferred from the suffix `.csv`, `.xlsx` or `.json`. Examples of the required formats can be found in Table 8, which shows a toy example with two persons and two items

rated by two raters in wide format, and Table 9, which shows the same data in long format. Missing data should either be left blank, as shown for Person_2 and Item_2, rated by Rater_2, or the row may be omitted entirely if there are no observations (for any individual observation in the case of long form). Item names are optional, but if they are omitted, the argument `item_names=False` must be passed. Unlike for SLM, PCM and RSM, person names are mandatory for MFRM data.

Returns

- pandas dataframe of responses in required format for class MFRM
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load a csv file in wide format with item labels, with a maximum score of 5:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_single('my_mfrm_data.csv', max_score=5)
```

To load a json file in wide format with no item labels, passing a maximum scores stored as a variable `my_max_score` :

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_single('my_mfrm_data.json',
                                                    max_score=my_max_score,
                                                    item_names=False)
```

To load an xlsx file in long format, inferring the maximum score from the data:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_single('my_mfrm_data.xlsx', long=True)
```

2.4.2 loadup_mfrm_xlsx_tabs

Description

Function to load polytomously scored data for use with class `MFRM` from an `xlsx` file with multiple tabs: one tab for each rater.

Usage

```
loadup_mfrm_xlsx_tabs(filename, max_score, item_names=True, missing=None, long=False)
```

Arguments

<code>filename</code>	The full filename, including suffix <code>.xlsx</code> .
<code>max_score</code>	The maximum score available. If omitted or <code>max_score=None</code> , the maximum scores will be inferred from the data (passing a maximum score is recommended, however).
<code>item_names</code>	Boolean. If <code>True</code> , the first row of data will be read as the item names. If <code>False</code> , item names will be allocated following the format <code>Item_1</code> etc. Default value is <code>True</code> .
<code>missing</code>	String, int, float, list or <code>None</code> : value(s) in the raw data to treat as missing (<code>NaN</code>) before coercion to scores, e.g. a code such as <code>-99</code> or <code>'M'</code> . Default is <code>missing=None</code> .
<code>long</code>	Boolean. If value is <code>True</code> , data will be expected in long format; if value is <code>False</code> , data will be expected in wide format. Default value is <code>False</code> .

Input file format

	Item_1	Item_2
Person_1	1	3
Person_3	5	4

Table 10: Wide format `MFRM` data, multiple `xlsx` tabs: Rater_1

	Item_1	Item_2
Person_2	3	4
Person_3	5	

Table 11: Wide format `MFRM` data, multiple `xlsx` tabs: Rater_2

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	3
Person_3	Item_1	5
Person_3	Item_2	4

Table 12: Long format MFRM data,
multiple xlsx tabs: Rater_1

Person	Item	Score
Person_2	Item_1	3
Person_2	Item_2	4
Person_3	Item_1	5

Table 13: Long format MFRM data,
multiple xlsx tabs: Rater_2

Input files can be in wide format, with `long=False` – the default – or long format, with `long=True`. Data loaded in long format will be converted to wide format for analysis. Examples of the required formats can be found in Tables 10 and 11, which show a toy example with three persons and two items rated by two raters in wide format, and Tables 12 and 13, which show the same data in long format. The names of the tabs should match the raters, and will automatically be processed as the rater names (in particular, they should be unique). Missing data should either be left blank, as shown for Person_3 and Item_2, rated by Rater_2, or the row may be omitted entirely if there are no observations (or for any individual observation in the case of long form). Item names are optional, but if they are omitted, the argument `item_names=False` must be passed. Unlike for SLM, PCM and RSM, person names are mandatory for MFRM data.

Returns

- pandas dataframe of responses in required format for class MFRM
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load an xlsx file in wide format with item labels, with a maximum score of 5:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_xlsx_tabs('my_mfrm_data.xlsx', max_score=5)
```

To load an xlsx file in wide format with no item labels, passing a maximum scores stored as a variable `my_max_score`:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_xlsx_tabs('my_mfrm_data.xlsx',
                                                         max_score=my_max_score,
                                                         item_names=False)
```

To load an xlsx file in long format, inferring the maximum score from the data:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_xlsx_tabs('my_mfrm_data.xlsx', long=True)
```

2.4.3 loadup_mfrm_multiple

Description

Function to load polytomously scored data for use with class `MFRM` from multiple csv, xlsx or json files, with one file for each rater.

Usage

```
loadup_mfrm_multiple(filename_dict, max_score, item_names=True, missing=None, long=False)
```

Arguments

<code>filename_dict</code>	A dictionary in the format <code>{'Rater_1': filename_1, 'Rater_2': filename_2, ... }</code> with the full filenames, including suffixes <code>.csv</code> or <code>.xlsx</code> . An equivalent pandas Series may also be passed, as <code>my_series.to_dict()</code> .
<code>max_score</code>	The maximum score available. If omitted or <code>max_score=None</code> , the maximum scores will be inferred from the data (passing a maximum score is recommended, however).
<code>item_names</code>	Boolean. If <code>True</code> , the first row of data will be read as the item names. If <code>False</code> , item names will be allocated following the format <code>Item_1</code> etc. Default value is <code>True</code> .
<code>missing</code>	String, int, float, list or <code>None</code> : value(s) in the raw data to treat as missing (NaN) before coercion to scores, e.g. a code such as <code>-99</code> or <code>'M'</code> . Default is <code>missing=None</code> .
<code>long</code>	Boolean. If value is <code>True</code> , data will be expected in long format; if value is <code>False</code> , data will be expected in wide format. Default value is <code>False</code> .

Input file format

	Item_1	Item_2
Person_1	1	3
Person_3	5	4

Table 14: Wide format MFRM data, multiple files: Rater_1

	Item_1	Item_2
Person_2	3	4
Person_3	5	

Table 15: Wide format MFRM data, multiple files: Rater_2

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	3
Person_3	Item_1	5
Person_3	Item_2	4

Table 16: Long format MFRM data,
multiple files: Rater_1

Person	Item	Score
Person_2	Item_1	3
Person_2	Item_2	4
Person_3	Item_1	5

Table 17: Long format MFRM data,
multiple files: Rater_2

Input files can be in wide format, with `long=False` – the default – or long format, with `long=True`. Data loaded in long format will be converted to wide format for analysis. Examples of the required formats can be found in Tables 14 and 15, which show a toy example with three persons and two items rated by two raters in wide format, and Tables 16 and 17, which show the same data in long format. Missing data should either be left blank, as shown for Person_3 and Item_2, rated by Rater_2, or the row may be omitted entirely if there are no observations (or for any individual observation in the case of long form). Item names are optional, but if they are omitted, the argument `item_names=False` must be passed. Unlike for SLM, PCM and RSM, person names are mandatory for MFRM data.

Returns

- pandas dataframe of responses in required format for class MFRM
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load data from two csv files in wide format with item labels, with a maximum score of 5:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_multiple({'Rater_1': 'my_mfrm_data_1.csv',
                                                         'Rater_2': 'my_mfrm_data_2.csv'},
                                                         max_score=5)
```

To load data from two csv files in wide format with no item labels, passing a maximum score stored as a variable `my_max_score`:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_multiple({'Rater_1': 'my_mfrm_data_1.csv',
                                                         'Rater_2': 'my_mfrm_data_2.csv'},
                                                         max_score=my_max_score,
                                                         item_names=False)
```

To load data from a csv file and an xlsx file in long format, inferring the maximum score from the data:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_multiple({'Rater_1': 'my_mfrm_data_1.csv',  
                                                       'Rater_2': 'my_mfrm_data_2.csv'},  
                                                       long=True)
```


2.5 Exogenous covariates (for DIF analysis)

2.5.1 `loadup_exogenous`

Description

Function to load person-level exogenous covariates (e.g. gender, age or first language) for use in Differential Item Functioning (DIF) analysis with classes `SLM`, `PCM` and `RSM` (see Sections 3.4.4 and 3.4.5 for `SLM`, Sections 4.4.7 and 4.4.8 for `PCM` and Sections 5.4.7 and 5.4.8 for `RSM`). Unlike the response loaders above, values are read as raw category labels rather than validated numeric scores, since covariates are typically categorical (e.g. 'Female', '21 ' or 'Mandarin') rather than item responses. Loaded independently of the response file, rather than as extra columns within it, so that a covariate can never accidentally be parsed as an item score.

Usage

```
loadup_exogenous(filename, person_names=True)
```

Arguments

<code>filename</code>	The full filename, including suffix <code>.csv</code> , <code>.xlsx</code> or <code>.json</code> . Format is inferred from the suffix; anything other than <code>.xlsx</code> or <code>.json</code> is read as <code>csv</code> .
<code>person_names</code>	Boolean. If <code>True</code> , the first column of data is read as the person names and used as the index. If <code>False</code> , persons are labelled <code>Person_1</code> , <code>Person_2</code> etc., in file row order. Default value is <code>True</code> .

Input file format

A person-indexed table of covariate columns, e.g. a first column of person names followed by columns such as `Gender`, `L1`. Persons need not appear in the same order as (or even be fully identical to) the response data — see the constructor documentation for `SLM` (Section 3.1.1) for how mismatches between the response data and the exogenous data are handled.

Returns

pandas dataframe: person-indexed covariate table. Pass directly as the `exogenous` argument to the `SLM`, `PCM` or `RSM` constructor.

Examples

To load a csv file of exogenous covariates with person labels:

```
my_exogenous_df = loadup_exogenous('my_exogenous_data.csv')
```

To load an xlsx file of exogenous covariates with no person labels:

```
my_exogenous_df = loadup_exogenous('my_exogenous_data.xlsx', person_names=False)
```

3 class SLM

3.1 Preliminaries

3.1.1 SLM

Description

Creates an object of the class `SLM` from a pandas dataframe of dichotomously scored data for analysis. No analysis can be run until an object is created.

Usage

```
SLM(dataframe, extreme_persons=True, no_of_classes=5, validate=True, exogenous=None)
```

Arguments

<code>dataframe</code>	pandas dataframe with items as columns (item names as column names) and persons as index (person names as row names).
<code>extreme_persons</code>	Boolean: if <code>False</code> , all persons with extreme scores (all responses correct or all responses incorrect) are removed from the response dataframe. Default is <code>extreme_persons=True</code> .
<code>no_of_classes</code>	Integer: the number of classes of persons grouped by location for overplotting observed responses on theoretical curves. Default is <code>no_of_classes=5</code>
<code>validate</code>	Boolean: if <code>True</code> , automatically calls <code>check_data_connectivity</code> (Section 3.1.2) on creation and issues a <code>UserWarning</code> if the item response network is disconnected, or if any item has a structurally unresolvable zero in the pairwise calibration matrix. Default is <code>validate=True</code> .
<code>exogenous</code>	pandas dataframe or <code>None</code> : optional person-level covariates (e.g. gender, age or first language) for Differential Item Functioning (DIF) analysis (Section 3.4.5), indexed by person identifier – typically produced by <code>loadup_exogenous</code> (Section 2.5.1). Values are kept as raw category labels, not pre-dummy-coded, so multi-level covariates work without rework. Persons present in <code>dataframe</code> without a matching exogenous record, and vice versa, are permitted rather than raising an error – exogenous data (e.g. demographic covariates) is often optional to supply, for instance for GDPR reasons – and any such gaps are reported via a <code>UserWarning</code> and the <code>self.no_exogenous_persons/self.exogenous_only_persons</code> attributes below. Default is <code>exogenous=None</code> .

Returns

Object of class `SLM`. Analyses are run using methods defined on the `SLM` object, with results stored as attributes of the `SLM` object.

Several attributes of object `SLM` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: Dataframe of valid responses.
<code>self.invalid_responses</code>	pandas dataframe: Dataframe of invalid responses (persons with no responses to any items, i.e. all missing data).
<code>self.extreme_persons</code>	pandas dataframe: Persons with extreme scores (all-correct or all-incorrect). When <code>True</code> (the default), extreme scorers are retained in <code>self.responses</code> and this attribute is empty; when <code>False</code> , they are removed and stored here.
<code>self.no_of_items</code>	Integer: Number of items.
<code>self.item_names</code> <code>self.item_ids</code>	List: List of item names. <code>self.item_ids</code> is an alias for users who find <code>*_ids</code> more natural than <code>*_names</code> .
<code>self.no_of_persons</code>	Integer: Number of persons.
<code>self.person_names</code> <code>self.person_ids</code>	List: List of person names. <code>self.person_ids</code> is an alias for users who find <code>*_ids</code> more natural than <code>*_names</code> .
<code>self.generating</code>	Namespace object (<code>_SimParams</code>): Only present when the model is instantiated by passing a simulation object (an <code>SLM.Sim</code> object for class <code>SLM</code>) as the <code>dataframe</code> argument. Contains all attributes of the simulation object (e.g. <code>self.generating.persons</code> , <code>self.generating.items</code>), enabling direct comparison of generating and estimated parameters.
<code>self.connectivity_status</code>	Dictionary: result of <code>check_data_connectivity</code> (Section 3.1.2), present only when <code>validate=True</code> . Always contains at least 'connected' (Boolean) and 'components.count' (Integer).
<code>self.exogenous</code>	pandas dataframe or <code>None</code> : person-level covariates passed via the <code>exogenous</code> argument, re-indexed onto <code>self.person_names</code> . <code>None</code> if not supplied.
<code>self.no_exogenous_persons</code>	pandas index: Persons present in the response data with no matching exogenous record. Empty if <code>exogenous</code> was not supplied or every person has a record.
<code>self.exogenous_only_persons</code>	pandas index: Persons present in the exogenous data but not in the response data (and so ignored). Empty if <code>exogenous</code> was not supplied or every exogenous person is also in the response data.

Example

To create an object from a dataframe `my_slm_dataframe`, with 10 observed classes:

```
my_slm = SLM(my_slm_dataframe, no_of_classes=10)
```

To create an object from a dataframe `my_slm_dataframe`, additionally carrying exogenous covariates for DIF analysis loaded into `my_exogenous_df`:

```
my_slm = SLM(my_slm_dataframe, exogenous=my_exogenous_df)
```

3.1.2 `check_data_connectivity`

Description

Validates whether the item response network is fully connected, in the specific sense required for PAIR/CPAT pairwise calibration: two items are connected if at least one person's response differentiates between them (e.g. scores higher on one than the other) – not merely that some person answered both, since a person who scores identically on two items contributes nothing to their relative calibration. Builds the same directed pairwise comparison matrix that `calibrate` (Section 3.3.1) itself uses, then runs a breadth-first search over item-level adjacency to find connected components. A disconnected network means item locations are estimated independently per component and cannot be placed on a common scale.

Also flags a subtler 'directionally isolated' case: an item with a structurally zero row or column in the directed matrix (no 1s or no 0s for SLM) can still appear connected under the undirected view – it has an edge in at least one direction – but breaks `calibrate`'s directed matrix with a zero that matrix powering cannot resolve, which can silently produce NaN/overflow rather than a clear error.

Called automatically on object creation when `validate=True` (the default; see Section 3.1.1), storing its result as `self.connectivity_status` and issuing a `UserWarning` for either condition above, but can also be re-run manually, e.g. after removing items or persons from `self.responses` by hand.

Usage

```
self.check_data_connectivity()
```

Returns

Dictionary. Always contains:

'connected'	Boolean: <code>True</code> if the item response network is fully connected.
'components_count'	Integer: number of connected components (1 if fully connected).
'directionally_isolated_items'	List: items with a structurally zero row or column in the directed pairwise matrix. These pass the undirected connectivity check but will break <code>calibrate</code> .

If disconnected, additionally contains:

'isolated_items'	list: items forming singleton components.
'all_sub_groups'	List of lists: all components.

Example

To manually re-check connectivity on object `my_slm` after editing `self.responses` directly, and inspect the result:

```
my_slm.connectivity_status = my_slm.check_data_connectivity()
my_slm.connectivity_status['connected']
```

3.1.3 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_slm` from `Item_1` to `my_new_item_name`:

```
my_slm.rename_item('Item_1', 'my_new_item_name')
```

3.1.4 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_slm` with item names in a list stored as a variable `my_new_item_names`:

```
my_slm.rename_items_all(my_new_item_names)
```

3.1.5 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the index of `self.responses` with new name. Also replaces person name in `self.exogenous` or `self.no_exogenous_persons` as appropriate if `self.exogenous` exists.

Example

To rename a person in object `my_slm` from `Person_1` to `my_new_person_name`:

```
my_slm.rename_person('Person_1', 'my_new_person_name')
```

3.1.6 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the index of `self.responses` with new names. Also replaces all person names in `self.exogenous` and `self.no_exogenous_persons` if `self.exogenous` exists.

Example

To rename all persons in object `my_slm` with person names in a list stored as a variable `my_new_person_names`:

```
my_slm.rename_persons_all(my_new_person_names)
```

3.2 Core functions

3.2.1 cat_prob

Description

Category probability function which calculates the probability $P(X_{ni} = k)$ of scoring k , with $k \in \{0, 1\}$ from person location and item location. For a person n with location β_n attempting an item i with location δ_i , the probability of obtaining a score of k is given by:

$$P(X_{ni} = k) = \frac{e^{k(\beta_n - \delta_i)}}{1 + e^{\beta_n - \delta_i}} \quad (1)$$

Usage

```
self.cat_prob(person_location, item_location, category)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Item location
<code>category</code>	Integer: Response category k , with $k \in \{0, 1\}$.

Returns

Float: probability of obtaining score k .

Example

To obtain the probability of a person with location 0.5 scoring 0 on an item with location 0 and store the result as a variable `my_cat_prob`:

```
my_cat_prob = self.cat_prob(0.5, 0, 0)
```

3.2.2 exp_score

Description

Expected score function which calculates the expected score $E(X_{ni})$ from person location and item location. The expected score is given by:

$$E(X_{ni}) = \sum_{k=0}^1 kP(X_{ni} = k) \quad (2)$$

where $P(X_{ni} = k)$ is as described in Section 3.2.1.

In the dichotomous case, this is also the probability of obtaining a correct response; for a person n with location β_n attempting an item i with location δ_i , the equation reduces to the most familiar formulation of the SLM:

$$E(X_{ni}) = \frac{e^{\beta_n - \delta_i}}{1 + e^{\beta_n - \delta_i}} \quad (3)$$

Usage

```
self.exp_score(person_location, item_location)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Item location

Returns

Float: expected score (also the probability of obtaining a correct response).

Example

To obtain the expected score for a person of location 0.5 attempting an item of location 0 and store the result as a variable `my_exp_score`:

```
my_exp_score = self.exp_score(0.5, 0)
```


3.2.3 variance

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location and item location. The variance is given by:

$$V(X_{ni}) = \sum_{k=0}^1 P(X_{ni} = k)(k - E(X_{ni}))^2 \quad (4)$$

where $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 3.2.1 and 3.2.2 respectively.

In the dichotomous case, since it is a Bernoulli variable, the variance reduces to $p(1 - p)$, where p is the probability of obtaining a correct response; for a person n with location β_n attempting an item i with location δ_i , the variance is therefore given by:

$$V(X_{ni}) = \frac{e^{\beta_n - \delta_i}}{(1 + e^{\beta_n - \delta_i})^2} \quad (5)$$

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

```
self.variance(person_location, item_location)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Item location

Returns

Float: variance (also the Fisher information provided by the response).

Example

To obtain the variance for a person of location 0.5 attempting an item of location 0 and store the result as a variable `my_variance`:

```
my_variance = self.variance(0.5, 0)
```

3.2.4 kurtosis

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location and item location. The variance is given by:

$$\kappa(X_{ni}) = \sum_{k=0}^1 P(X_{ni} = k)(k - E(X_{ni}))^4 \quad (6)$$

where $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 3.2.1 and 3.2.2 respectively.

Usage

```
self.kurtosis(person_location, item_location)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Item location

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 attempting an item of location 0 and store the result as a variable `my_kurtosis`:

```
my_kurtosis = self.kurtosis(0.5, 0)
```

3.3 Parameter estimation

3.3.1 `calibrate`

Description

Produces item location estimates using PAIR (Choppin, 1968, 1985), eigenvector method (Garner & Engelhard, 2002) or related conditional pairwise methods.

Usage

```
self.calibrate(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001)
```

Arguments

<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> . This is a finite-sample safeguard against empty pairwise comparison cells (for which the log-ratio underlying PAIR/CPAT estimation is undefined), not a regularisation or smoothing choice in the usual statistical sense. <code>constant</code> only prevents zero counts from breaking the estimator on small or sparse datasets, and its value is not expected to materially change well-supported estimates.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b) is raised. Higher powers increase the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method. Ignored for other methods.

Returns

Attribute `self.items`: pandas series of item estimates with item names as keys and estimates as values.

Examples

To generate a set of estimates using the cosine similarity method, with additive smoothing constant of 0.1:

```
self.calibrate()
```

To generate a set of estimates using the log-likelihood method, with matrix raised to power 7 and a convergence stopping criterion of 0.00000001:

```
self.calibrate(method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```

3.3.2 std_errors

Description

Produces bootstrapped estimates for the standard errors of item location estimates.

Usage

```
self.std_errors(interval=None, no_of_samples=500, constant=0.1, method='cos',  
                matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval. Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate. More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> .
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method. Ignored for other methods.
<code>seed</code>	Integer or <code>None</code> : seed for the bootstrap resampling random number generator. Pass an integer for fully reproducible standard errors; <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

<code>self.item_se</code>	pandas series: tem names as keys and item standard errors as values.
<code>self.item_bootstrap</code>	pandas dataframe: Contains the full bootstrap results, with a row for each bootstrap sample and a column for each item estimate.

If `interval` is specified, also returns:

<code>self.item_low</code>	Float: The lower bound of the specified interval.
<code>self.item_high</code>	Float: The upper bound of the specified interval.

Example

To generate item standard errors with a 95% interval from 200 samples:

```
self.std_errors(interval=0.95, no_of_samples=200)
```

Modifications to the estimation method are discussed in Section 3.3.1.

3.3.3 `calibrate_anchor`

Description

Anchors item location estimates onto externally-supplied reference values, supporting item banking: after running a standard calibration (Section 3.3.1), the whole scale is shifted by a translation constant so that a subset of common (“anchor”) items line up with externally-supplied reference locations, e.g. from a bank of previously-calibrated items. Since all items have parallel item response functions under the SLM, this is a simple mean-shift.

By default (`selection_method='robust_z'`), the translation constant is computed via a robust, iterative outlier-trimming procedure based on the modified z-score (Iglewicz & Hoaglin, 1993): anchor items whose estimated values have drifted relative to the other anchor items when compared to the externally-supplied reference values are excluded before the shift is computed, so that a handful of stale or otherwise misbehaving anchor items cannot distort the shift applied to the rest of the scale. `selection_method='wald'` is an alternative that uses a formal significance test per anchor item instead of a descriptive-statistics trim, and requires bootstrap item standard errors (auto-triggers `std_errors`, Section 3.3.2, if not already computed). `selection_method='none'` takes a plain mean shift over all supplied anchors, with no trimming. Diagnostics from either selection procedure can be visualised with `plot_anchor_selection` (Section 3.5.9).

Usage

```
self.calibrate_anchor(anchors, calibrate=False, selection_method='robust_z',
                      corr_tol=0.95, sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05,
                      no_of_samples=500, adj=None, overwrite_anchors='none',
                      plot=True, plot_kwargs=None, constant=0.1, method='cos',
                      matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>anchors</code>	Dictionary or pandas series: reference locations for a subset of items, keyed/indexed by item name.
<code>calibrate</code>	Boolean: if <code>True</code> , (re-)runs <code>calibrate</code> before anchoring. If <code>False</code> , <code>calibrate</code> is still auto-triggered if it has not already been run. Default is <code>calibrate=False</code> .
<code>selection_method</code>	String, one of <code>'robust_z'</code> , <code>'wald'</code> or <code>'none'</code> . <code>'robust_z'</code> (default): excludes outlying anchor items via the modified z-score procedure (Iglewicz & Hoaglin, 1993); uses <code>corr_tol</code> , <code>sd_ratio_tol</code> , <code>min_anchors</code> . <code>'wald'</code> : a formal per-item sequential significance test based on z-scores computed from item standard errors – needs bootstrap standard errors, so auto-triggers <code>std_errors</code> if not already computed; uses <code>wald_alpha</code> and <code>min_anchors</code> , not <code>corr_tol</code> / <code>sd_ratio_tol</code> . <code>'none'</code> : plain mean shift over every supplied anchor, no trimming.

Arguments continue on the next page.

Arguments (continued)

<code>corr_tol</code>	Float: minimum acceptable Pearson correlation between anchor and observed values on the surviving (non-outlier) item set (<code>selection_method='robust_z'</code> only). Default is <code>corr_tol=0.95</code> .
<code>sd_ratio_tol</code>	Float: the ratio of observed to anchor standard deviations on the surviving item set must fall within $[1/\text{sd_ratio_tol}, \text{sd_ratio_tol}]$ (<code>selection_method='robust_z'</code> only). Default is <code>sd_ratio_tol=1.1</code> , i.e. the ratio must lie in $[0.909, 1.1]$.
<code>min_anchors</code>	Integer: floor on the number of surviving anchor items – selection stops (with a warning) once this floor is reached, even if the active method’s tolerance/significance criterion is not yet satisfied, and is skipped altogether if the initial common item set is at or below this floor (<code>selection_method</code> in <code>'robust_z'/'wald'</code> only). Default is <code>min_anchors=6</code> .
<code>wald_alpha</code>	Float: significance level for each anchor item’s Wald test (<code>selection_method='wald'</code> only). Default is <code>wald_alpha=0.05</code> .
<code>no_of_samples</code>	Integer: bootstrap samples for the auto-triggered <code>std_errors</code> call, only used if item standard errors are not already computed (<code>selection_method='wald'</code> only). Default is <code>no_of_samples=500</code> .
<code>adj</code>	Float or <code>None</code> : if provided, applied as translation constant and selection step skipped. Intended for reuse across bootstrap resamples (Section 3.3.2), where recomputing anchor selection every time would inflate standard errors with anchor-item sampling variance. Default <code>adj=None</code> .
<code>overwrite_anchors</code>	String, one of <code>'none'</code> , <code>'rejected'</code> or <code>'all'</code> : controls what <code>self.anchor_items</code> holds for the anchor items themselves, as opposed to non-anchor items, which always receive the shifted (calibrated + translation constant) value. <code>'none'</code> : every anchor item keeps exactly its externally-supplied value, unchanged – the usual convention, since an anchor’s value is fixed by definition, not re-estimated. <code>'rejected'</code> : retained anchors keep their supplied values, but rejected anchors have their values overwritten with new, freshly-calibrated values; requires <code>selection_method</code> in <code>'robust_z'/'wald'</code> and no <code>adj</code> override (otherwise falls back to <code>'none'</code> with a warning). <code>'all'</code> : every anchor item has its value overwritten with a new, freshly-calibrated value like any other item. Default is <code>overwrite_anchors='none'</code> .
<code>plot</code>	Boolean: if <code>True</code> and <code>selection_method</code> in <code>'robust_z'/'wald'</code> (so a selection table exists), calls <code>plot_anchor_selection</code> (Section 3.5.9). Has no effect when <code>selection_method='none'</code> or <code>adj</code> is supplied. Default <code>plot=True</code> .

Arguments continue on the next page.

Arguments (continued)

<code>plot_kwargs</code>	Dictionary or None: extra keyword arguments forwarded to <code>plot_anchor_selection</code> when <code>plot=True</code> (e.g. <code>filename</code> , <code>x_min/x_max</code> , <code>graph_title</code>). Default is <code>plot_kwargs=None</code> .
<code>constant</code> , <code>method</code> , <code>matrix_power</code> , <code>log_lik_tol</code>	Calibration/bootstrap arguments, used only if <code>calibrate</code> or <code>std_errors</code> (<code>selection_method='wald'</code>) triggered (Section 3.3.1).
<code>seed</code>	Integer or None: seed passed through to the internal <code>std_errors</code> call (only used if item standard errors are not already computed, <code>selection_method='wald'</code> only). None (default) draws fresh entropy each call. See Section 1.

Returns

<code>self.anchor_items</code>	pandas series: item locations shifted onto the anchor scale.
<code>self.anchor_item_names</code>	pandas index: names of the items supplied as anchors.
<code>self.anchor_adj</code>	Float: the translation constant actually applied.
<code>self.anchor_selection</code>	pandas dataframe or None: per-item diagnostics from whichever selection method was used – Anchor, Observed, Deviation, Robust z, Selected (<code>'robust_z'</code>), or Anchor, Observed, SE, Deviation, z, p, Selected (<code>'wald'</code>). None if <code>selection_method='none'</code> or <code>adj</code> supplied.
<code>self.anchor_selected_items</code> , <code>self.anchor_dropped_items</code>	pandas index or None: items retained / excluded as outliers. None if <code>selection_method='none'</code> or <code>adj</code> supplied.
<code>self.anchor_original_corr</code> , <code>self.anchor_original_sd_ratio</code>	Float or None: correlation / SD ratio before trimming.
<code>self.anchor_corr</code> <code>self.anchor_sd_ratio</code>	Float or None: correlation / SD ratio after trimming.
<code>self.anchor_summary</code>	pandas series: one-line summary – anchors supplied/common/selected/dropped, correlation and SD ratio before/after trimming, translation constant applied.
<code>self.anchor_plot</code>	matplotlib figure or None: figure from the auto-triggered <code>plot_anchor_selection</code> call. None if <code>plot=False</code> , <code>selection_method='none'</code> , or <code>adj</code> was supplied directly.
<code>self.calibrate_anchor_runs</code>	Dictionary: every call's results above, keyed by the sorted anchor items supplied, so results from an earlier call survive a later call with a different anchor set instead of being overwritten in place.

Examples

To anchor item locations onto a bank of reference values stored in a dictionary `my_bank`, with automatic outlier trimming and diagnostic plot:

```
self.calibrate_anchor(my_bank)
```

To anchor without outlier trimming, taking a plain mean shift over all common items, with no plot:

```
self.calibrate_anchor(my_bank, selection_method='none', plot=False)
```

To anchor using the Wald significance test for trimming, with a fully reproducible bootstrap:

```
self.calibrate_anchor(my_bank, selection_method='wald', wald_alpha=0.05, seed=42)
```

To anchor with a stricter correlation tolerance and smaller floor on the number of retained anchors, overwriting rejected anchors' own values with freshly-calibrated estimates:

```
self.calibrate_anchor(my_bank, corr_tol=0.98, min_anchors=4, overwrite_anchors='rejected')
```

3.3.4 person

Description

Generates a location estimate for a person using the Newton-Raphson method to produce a maximum likelihood estimate, with optional Warm bias correction (Warm, 1989).

Usage

```
self.person(persons, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,  
            ext_score_adjustment=0.5, missing_as_incorrect=False)
```

Arguments

persons	String or list: person name(s) for the location(s) being estimated. Pass 'all' to estimate for all persons in <code>self.responses</code> .
items	List: List of names of a subset of items, from which to generate estimate. Default <code>items=None</code> , which generates a location based on full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default <code>tolerance=0.00001</code> .
max_iters	Integer: maximum Newton-Raphson iterations. Default is <code>max_iters=100</code> .
ext_score_adjustment	Float: Value in range (0,1) to ensure a estimate is returned if the person has an extreme score (all items correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
missing_as_incorrect	Boolean: if <code>True</code> , treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is <code>missing_as_incorrect=False</code> .

Returns

pandas series: person location estimate(s), in logits, indexed by person identifier. Returns `NaN` for any person for whom estimation fails.

Examples

To generate a person location estimate for `Person_1` using the default settings and store the result as a variable, `my_person_location`:

```
my_person_location = self.person('Person_1')
```

To generate an MLE person location estimate without Warm bias correction for `Person_1` based on the first three items and store the result as a variable, `my_person_location`:

```
my_person_location = self.person('Person_1', ['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

3.3.5 person_estimates

Description

Generates location estimates for all persons using the Newton-Raphson method to produce maximum likelihood estimates, with optional Warm bias correction (Warm, 1989).

Usage

```
self.person_estimates(items=None, warm_corr=True, tolerance=0.00001, max_iters=100,  
                      ext_score_adjustment=0.5, missing_as_incorrect=False)
```

Arguments

<code>items</code>	List: List of names of a subset of items, from which to generate estimate. Default <code>items=None</code> , which generates a location based on full set of items. Only use when an estimate based on a subset of items is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations. Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>missing_as_incorrect</code>	Boolean: if <code>True</code> , treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is <code>missing_as_incorrect=False</code> .

Returns

Attribute `self.persons`: pandas series with person names as keys and location estimates as values.

Example

To generate a set of person location estimates with Warm bias correction:

```
self.person_estimates()
```

To generate a set of person location estimates without Warm bias correction, on a subset of the first three items only:

```
self.person_estimates(items=['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

3.3.6 score_lookup

Description

Generates location estimate for a given raw score on a given set of items using the Newton-Raphson method to produce a maximum likelihood estimate, optional Warm bias correction (Warm, 1989).

Usage

```
self.score_lookup(score, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
                  ext_score_adjustment=0.5)
```

Arguments

<code>score</code>	Integer: The raw score for which location is being estimated.
<code>items</code>	List: List of names of a subset of items, from which to generate estimate. Default <code>items=None</code> , which generates a location based on full set of items. Only use when an estimate based on a subset of items is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations. Default <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Returns

pandas series with raw scores as keys and person location estimates as values.

Examples

To generate a location estimate for a score of 10 on all items, with Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = self.score_lookup(10)
```

To generate a location estimate for a score of 10 on a subset of items saved as a variable `my_items`, without Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = self.score_lookup(10, items=my_items, warm_corr=False)
```

3.3.7 score_lookup_table

Description

Generates a score-to-location estimate correspondence for all raw scores on a set of items, using the Newton-Raphson method to produce maximum likelihood estimates, with optional Warm bias correction (Warm, 1989).

Usage

```
self.score_lookup_table(items=None, ext_scores=True, warm_corr=True, tolerance=0.00001,
                        max_iters=100, ext_score_adjustment=0.5)
```

Arguments

<code>items</code>	List: List of names of a subset of items, from which to generate estimate. Default <code>items=None</code> , which generates a location based on full set of items. Only use when an estimate based on a subset of items is required.
<code>ext_scores</code>	Boolean: If <code>True</code> , location estimates for extreme scores (all correct/all incorrect) will be generated using the <code>ext_score_adjustment</code> argument. Default is <code>ext_scores=True</code> .
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations. Default <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Returns

Attribute `self.score_table`: pandas series with raw scores as keys and location estimates as values.

Examples

To generate a location lookup table for all items, including extreme scores, with Warm bias correction:

```
self.score_lookup_table()
```

To generate a location lookup table for a subset of items saved as a variable `my_items`), without extreme scores and without Warm bias correction:

```
self.score_lookup_table(items=my_items, ext_scores=False)
```

3.3.8 csem

Description

Calculates conditional standard error of measurement for one or more persons.

Usage

```
self.csem(persons=None, person_locations=None, items=None)
```

Arguments

<code>persons</code>	List or string: person name(s). Overrides <code>person_locations</code> if provided. Default is <code>persons=None</code> , which uses all persons.
<code>person_locations</code>	pandas series, float, list, numpy array, or dictionary, with person names as keys (where applicable) and locations as values. If <code>None</code> , uses <code>self.persons</code> , automatically generating via <code>self.person_estimates()</code> if necessary. A raw location value (or values not indexed by a real person, e.g. a raw score lookup) is treated as hypothetical: since there is no observed response data for it, all items in <code>items</code> are treated as answered. Default is <code>person_locations=None</code> .
<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only for when an estimate based on a subset of items is required.

Returns

pandas series: conditional standard error of measurement in logits for each person (or location), indexed by person name (or by a location label, for hypothetical locations).

Examples

To generate the CSEM for all persons on all items and save the result as a variable, `my_csem`:

```
my_csem = self.csem()
```

To generate the CSEM for `Person_1` on all items and save the result as a variable, `my_csem`:

```
my_csem = self.csem(persons='Person_1')
```

To generate the CSEM for a raw score of 3 on a subset of items saved to a variable `my_items` and save the result as a variable, `my_csem`:

```
my_csem = self.csem(persons=3, person_locations=self.score_table, items=my_items)
```

where `self.score_table` is the output from running `self.score_lookup_table(items=my_items)`, as described in Section 3.3.7.

3.4 Statistical output

3.4.1 `item_stats_df`

Description

Produces a table of item statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

```
self.item_stats_df(full=False, zstd=False, disc=False, point_measure_corr=False, dp=3,
                  se=True, warm_corr=True, tolerance=1e-05, max_iters=100,
                  ext_score_adjustment=0.5, method='cos', constant=0.1, matrix_power=3,
                  log_lik_tol=0.000001, no_of_samples=500, interval=None, seed=None)
```

Arguments

<code>full</code>	Boolean: If <code>True</code> , a full table with all statistics is produced. Default is <code>full=False</code> for a table with the most commonly reported statistics.
<code>zstd</code>	Boolean: If <code>True</code> , infit/outfit standardised z-scores are reported with mean square statistics. Default is <code>zstd=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>disc</code>	Boolean: If <code>True</code> , item discrimination is reported. The discrimination of the empirical item slope relative to the ideal logistic ogive, with 1 perfect, > 1 showing overfit and < 1 showing underfit; discrimination is similar to the 2PL IRT discrimination parameter (Linacre, 2023), but is a descriptive statistic in the SLM rather than an item parameter.
<code>point_measure_corr</code>	Boolean: If <code>True</code> , point-biserial correlation between person location and item score is reported, with its expected value for an ideal item. Default is <code>point_measure_corr=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>se</code>	Boolean: If <code>True</code> , computes and reports the SE column (and CI if <code>interval=True</code>). If <code>False</code> , skips the bootstrap – faster when only Infit MS/Outfit MS needed (e.g. repeated simulations), since those do not depend on the bootstrap. Forces <code>interval=None</code> when <code>False</code> . Default <code>se=True</code> .
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's (1989) bias correction is applied during generation of person estimates (see Section 3.3.4). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is <code>max_iters=100</code> .

Arguments continue on the next page.

Arguments (continued)

<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 3.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 3.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 3.3.1). Default value is <code>constant=0.1</code> .
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates (see Section 3.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 3.3.1). Ignored for other methods.
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 3.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Unused if <code>se=False</code> . Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 3.3.2). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Ignored if <code>se=False</code> . Default is <code>interval=None</code> .
<code>seed</code>	Integer or None: seed passed through to the internal <code>std_errors/fit_statistics</code> calls (only used if not already computed). None (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.item_stats`, a pandas dataframe with one row for each item and the following columns:

Estimate	Item location estimate.
SE	Bootstrapped standard error of item estimate. Only produced if <code>se=True</code> .
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 2.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 5%. Only produced if <code>se=True</code> and <code>interval</code> is set.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 97.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 95%. Only produced if <code>se=True</code> and <code>interval</code> is set.
Count	Count of responses.
Facility	Item facility: proportion of correct responses.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if <code>zstd=True</code> .
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if <code>zstd=True</code> .
Discrim	Item discrimination. Only produced if <code>disc=True</code> .
PM corr	Point-biserial correlation. Only produced if <code>point_measure_corr=True</code> .
Exp PM corr	Expected value of point-measure biserial correlation. Only produced if <code>point_measure_corr=True</code> .

Examples

To produce a summary `self.item_stats` table with the most commonly reported statistics:

```
self.item_stats_df()
```

To produce a full `self.item_stats` table with all statistics:

```
self.item_stats_df(full=True)
```

To produce a `self.item_stats` table with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
self.item_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

3.4.2 `person_stats_df`

Description

Produces a table of person statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

```
self.person_stats_df(full=False, rsem=False, dp=3, se=True, warm_corr=True, tolerance=0.00001,
                    max_iters=100, ext_score_adjustment=0.5, method='cos', constant=0.1)
```

Arguments

<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>rsem</code>	Boolean: If <code>True</code> , realistic standard error of measurement (RSEM), which takes into account item fit (Wright, 1996), is reported alongside the conditional standard error of measurement (CSEM). Default is <code>rsem=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>se</code>	Boolean: passed through to the internal <code>fit_statistics</code> call (only used if not already computed). If <code>False</code> , skips the bootstrap entirely – this table’s own columns (CSEM, RSEM, Infit/Outfit) do not depend on it, so <code>se=False</code> is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is <code>se=True</code> .
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm’s bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 3.3.4). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 3.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person’s score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Arguments continue on the next page.

Arguments (continued)

<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 3.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 3.3.1). Default value is <code>constant=0.1</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 3.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 3.3.2). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .

Returns

Attribute `self.person_stats`, a pandas dataframe with one row for each person and the following columns:

<code>Estimate</code>	Item location estimate.
<code>CSEM</code>	Conditional standard error of measurement for person location estimate.
<code>RSEM</code>	Realistic standard error of measurement for person location estimate. Only produced if <code>rsem=True</code>
<code>Score</code>	Number of correct responses.
<code>Max score</code>	Maximum available score (number of items attempted).
<code>p</code>	Proportion of correct responses.
<code>Infit MS</code>	Infit mean square.
<code>Infit Z</code>	Standardised infit z-score.
<code>Outfit MS</code>	Outfit mean square.
<code>Outfit Z</code>	Standardised outfit z-score.

Examples

To produce a summary `self.person_stats` table with the most commonly reported statistics:

```
self.person_stats_df()
```

To produce a full `self.person_stats` table with all statistics:

```
self.person_stats_df(full=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

3.4.3 test_stats_df

Description

Produces a table of test-level statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or $\text{\LaTeX}$.

Usage

```
self.test_stats_df(dp=3, warm_corr=True, tolerance=0.00001, max_iters=100,  
                   ext_score_adjustment=0.5, method='cos', constant=0.1, alpha=False,  
                   seed=None)
```

Arguments

dp	Integer: Number of decimal places reported in output.
warm_corr	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 3.3.4). Default is <code>warm_corr=True</code> .
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is <code>tolerance=0.00001</code> .
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is <code>max_iters=100</code> .
ext_score_adjustment	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 3.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 3.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 3.3.1). Default value is <code>constant=0.1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>alpha</code>	Boolean: if <code>True</code> , adds a <code>Cronbach alpha</code> row to the <code>Persons</code> column of the returned table (the <code>Items</code> column is left blank, since Cronbach's (1951) alpha is a person-side reliability statistic). Computed on complete cases; with missing responses, a warning is raised and alpha computed after listwise deletion (may underestimate true value). Default <code>alpha=False</code> .
<code>seed</code>	Integer or <code>None</code> : seed passed through to the internal <code>fit_statistics</code> call (only used if not already computed). <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.test_stats`, a pandas dataframe with two columns, `Items` and `Persons` and rows for a range of descriptive statistics describing the distributions of the estimates and different statistics related to reliability – these statistics describe the suitability of the data for estimating and differentiating the parameters, rather than properties of the parameters themselves. The statistics are:

<code>Mean</code>	The mean of the estimates.
<code>SD</code>	The standard deviation of the estimates.
<code>Separation ratio</code>	The separation ratio (Wright, 1996; Wright & Masters, 1982), which is the standard deviation of person locations reported as a ratio of standard error units. For persons, $G_p = \sigma_p / \sqrt{\sum_n SE_n^2}$, where σ_p is the variance of the person estimates and SE_n is the RSEM (see Section 3.4.2) for person n . The formula is symmetrical for items, substituting the standard error of estimation for RSEM.
<code>Strata</code>	The number of statistically distinct levels of either person location or item location as strata with centers three measurement errors apart (Wright & Masters, 1982:106). For persons, $H_p = (4G_p + 1)/3$, with symmetrical results for items.
<code>Reliability</code>	A Rasch-specific reliability statistic (Wright, 1996), derived from PSI and which is a Rasch-specific reliability statistic similar to Cronbach's Alpha (Cronbach, 1951), and which may be interpreted the same way – as the proportion of variance of the estimates which stems from variation in location or location rather than estimation error. For persons, $R_p = G_p^2 / (1 + G_p^2)$, with symmetrical results for items.
<code>Cronbach alpha</code>	Optional (<code>alpha=True</code>), <code>Persons</code> column only – classical Cronbach's alpha (Cronbach, 1951), computed directly from the raw response data rather than derived from Rasch estimates. Complete cases only; a warning is raised if missing data are present, since alpha may then be an underestimate. Not computed by default (<code>alpha=False</code>).

Example

To produce a `self.test_stats` table:

```
self.test_stats_df()
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

3.4.4 andersen_lr_test

Description

Andersen's (1973) likelihood ratio test of parameter invariance (Andersen, 1973a): splits persons into two groups, fits the model separately in each group, and tests whether item location estimates are invariant across the groups. Groups can be formed either by a median split on person location or raw score (a general model-fit check, not tied to any particular grouping variable), or by an exogenous person covariate such as gender (Differential Item Functioning, or DIF – see Holland and Wainer, 1993), in which case this serves as an omnibus test of whether *any* item shows DIF with respect to that covariate, complementing the per-item test in `dif_test` (Section 3.4.5).

Usage

```
self.andersen_lr_test(split_by='person_location', covariate=None, warm_corr=True,
                      tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                      constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001)
```

Arguments

<code>split_by</code>	String, one of 'person_location', 'score' or 'exogenous': split criterion. 'person_location' splits on the median of person location estimates (Section 3.3.4). 'score' splits on the median of raw scores (a warning is issued if data are missing, since raw scores are then not comparable across persons). 'exogenous' splits by an external person covariate, requiring <code>covariate</code> and <code>self.exogenous</code> to be set (Section 3.1.1), and requires the covariate to have exactly two distinct non-null values – covariates with more than two levels are not supported here; use <code>dif_test</code> (Section 3.4.5) instead, which supports them directly. Default is <code>split_by='person_location'</code> .
<code>covariate</code>	String or None: column name in <code>self.exogenous</code> to split by. Required (and only used) when <code>split_by='exogenous'</code> . Persons with a missing value for this covariate are excluded from both groups and from the full-sample comparison fit, so the likelihood-ratio decomposition stays valid. Default is <code>covariate=None</code> .
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_adjustment</code>	Person-estimation arguments passed to the group models – see Section 3.3.4.
<code>constant</code> , <code>method</code> , <code>matrix_power</code> , <code>log_lik_tol</code>	Calibration arguments passed to the group models – see Section 3.3.1.

Returns

<code>self.andersen_lr</code>	Float: the likelihood ratio statistic.
<code>self.andersen_df</code>	Integer: degrees of freedom (<code>self.no_of_items - 1</code>).
<code>self.andersen_p</code>	Float: p-value from the chi-squared distribution. <code>self.andersen_lr</code> is floored at 0 (with a warning) if the raw statistic is negative, which can occur since unlike CMLE (Andersen, 1973b), PAIR does not maximise likelihood directly; a floored statistic indicates no evidence of misfit.
<code>self.andersen_groups</code>	Dictionary: <code>{group_name: SLM}</code> , the fitted group models, for inspection. Group names are 'low'/'high' for <code>split_by='person_location'/'score'</code> , or the two observed covariate values for <code>split_by='exogenous'</code> .
<code>self.andersen_summary</code>	pandas series: the LR statistic, degrees of freedom and p-value together.

Examples

To test parameter invariance by a median split on person location:

```
self.andersen_lr_test()
```

To test parameter invariance by a median split on raw score:

```
self.andersen_lr_test(split_by='score')
```

To test for DIF by an exogenous covariate 'Gender' (which must have exactly two levels), having supplied exogenous to the constructor:

```
self.andersen_lr_test(split_by='exogenous', covariate='Gender')
```

3.4.5 dif_test

Description

Differential Item Functioning (DIF) test to test whether any items function differently for different subgroups of persons as defined by an exogenous person covariate (e.g. gender, first language) (Holland & Wainer, 1993). Splits non-extreme persons into groups by the named column in `self.exogenous`, designates one group as the reference (by default the largest), and compares each of the other (“focal”) groups against it individually – a reference-group design, not all-pairwise, so covariates with more than two levels are supported directly (unlike `andersen_lr_test`, Section 3.4.4).

For each focal group, both groups are calibrated independently (each self-centred), then purified onto a common scale – by default via a formal per-item Wald significance test (`selection_method='wald'`), with optional Welch correction and small-reference-group size adjustment (pass `selection_welch='True'`), or optionally via the robust modified z-score outlier-trimming procedure used by `calibrate_anchor` (Section 3.3.3; Iglewicz & Hoaglin, 1993) – the reference group’s item location estimates supply the “anchor” values, the focal group’s supply the “observed” values – so that genuine DIF items cannot contaminate the common scale used to test for DIF in the first place. A per-item test is then run on the purified scale for every item (not only the ones used to define the scale), using bootstrap standard errors from `std_errors` (Section 3.3.2). An item is flagged only if it clears both a (corrected) p-value threshold and an absolute logit-difference threshold so that a statistically significant but practically negligible difference is not flagged.

`dif_test` also supports several further options: – a per-item likelihood-ratio test alongside or instead of the Wald test (`test='lr'/'both'`), an omnibus LR test of whether *any* item shows DIF (`omnibus`), which runs by default, and ETS-style A/B/C severity-category classification (Zwick et al., 1997) (`category`, `category_thresholds`, `category_alpha`), as described in the Table 18:

Category	Criteria	Interpretation
A	$ \text{Difference} < 0.43$, or fails the significance test below	Negligible DIF
B+ / B-	Both $ \text{Difference} \geq 0.43$ and $p(\text{DIF} = 0) < \text{category_alpha}$ (two-sided Wald/Welch test), but not C	Slight to moderate DIF
C+ / C-	Both $ \text{Difference} \geq 0.64$ and $p(\text{Difference} \leq 0.43) < \text{category_alpha}$ (one-sided test that the difference significantly exceeds the B boundary)	Moderate to large DIF

Table 18: ETS-style DIF category classification. Sign: + indicates that an item is harder for the focal group; - indicates that an item is easier for the focal group. The 0.43/0.64 logit thresholds are the Rasch-logit equivalents of the classic ETS Mantel-Haenszel delta-scale B/C boundaries (Tristan, 2006); the significance level (default `category_alpha=0.05`) is independent of `alpha`, which governs the separate `Flagged/Flagged_LR` columns.

Usage

```
self.dif_test(covariate, reference=None, selection_method='wald', corr_tol=0.95,
              sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05, test='wald',
              omnibus=True, welch=False, size_adjust=False, reference_n=100,
              correction='bh', alpha=0.05, logit_threshold=0.43, category=False,
              category_thresholds=(0.43, 0.64), category_alpha=0.05, no_of_samples=500,
              plot=False, plot_kwargs=None, warm_corr=True, tolerance=0.00001,
              max_iters=100, ext_score_adjustment=0.5, constant=0.1, method='cos',
              matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>covariate</code>	String: column name in <code>self.exogenous</code> to group persons by.
<code>reference</code>	String or <code>None</code> : value of <code>covariate</code> to use as the reference group. If <code>None</code> , the largest group is used. Default is <code>reference=None</code> .
<code>selection_method</code>	String, one of 'wald', 'robust_z' or 'none': how the common scale is purified before testing – see <code>calibrate_anchor</code> (Section 3.3.3) for what each option does. Default is <code>selection_method='wald'</code> .
<code>corr_tol</code> , <code>sd_ratio_tol</code> , <code>min_anchors</code> , <code>wald_alpha</code>	As for <code>calibrate_anchor</code> (Section 3.3.3), used only by the corresponding <code>selection_method</code> . Defaults are <code>corr_tol=0.95</code> , <code>sd_ratio_tol=1.1</code> , <code>min_anchors=6</code> , <code>wald_alpha=0.05</code> .
<code>test</code>	String, one of 'wald', 'lr' or 'both': per-item DIF test(s) to compute. 'wald' (default) matches the original behaviour exactly, at no added cost (the bootstrap standard errors it needs are computed anyway). 'lr'/'both' additionally compute a per-item likelihood-ratio test (extra cost: two person location re-estimations per item per focal group) – H1 is each group's own natively-calibrated fit (computed once); H0 for item i pools that item's location across groups (precision-weighted), then re-estimates that group's person locations under the modified item set. The existing <code>Flagged</code> column always reflects the Wald test; a separate <code>Flagged_LR</code> column is added when <code>test</code> is 'lr' or 'both'.
<code>omnibus</code>	Boolean: if <code>True</code> (default), also runs an Andersen-style omnibus LR test (reference vs. focal, every common item jointly) per focal group, stored in <code>self.dif_omnibus_table</code> – answering “is there DIF at all” before the per-item table answers “in which items”. The H1 side plugs in person location estimates from the pooled reference+focal model rather than each group's own separately-fit locations, since re-optimising them independently on both sides of the comparison inflates the LR statistic beyond what $df = k - 1$ accounts for.

Arguments continue on the next page.

Arguments (continued)

<code>welch</code>	Boolean: if <code>True</code> , the per-item Wald test (<code>test='wald'</code> or <code>'both'</code>) becomes a Welch's t-test: anchor selection and statistic are unchanged (<code>Difference / SE</code>), but p-value computed against a t-distribution with Welch-Satterthwaite degrees of freedom. Adds a <code>df</code> column to <code>self.dif_table</code> . Only used for the Wald test. Default is <code>welch=False</code> .
<code>size_adjust</code>	Boolean: if <code>True</code> , rescales each group's own item SE to what it would be at a standard reference sample size (Tristan, 2006) – the opposite problem from <code>welch</code> 's: at very large N , SE shrinks enough that trivial differences become “significant”, swamping the logit-magnitude thresholds' intent. Each group's own SE is rescaled independently by $\sqrt{n_{\text{actual}}/\text{reference_n}}$ using that group's own person count, then combined as usual – so unequal reference/focal group sizes are put on equal footing too, not just corrected for extremity. Applied to the Wald/Welch test and (if <code>welch=True</code>) the Welch-Satterthwaite df, which then also uses <code>reference_n</code> rather than each group's actual n , for internal consistency. Does not affect the LR test (a nuisance-parameter estimation detail, not the significance-test artefact this targets) or the omnibus test (an exact chi-squared test, not SE-based). Default is <code>size_adjust=False</code> .
<code>reference_n</code>	Integer: reference sample size for <code>size_adjust=True</code> . 100 (default) is Tristán's (2006) recommendation; 60 is suggested there for closer alignment with ETS Category B boundary. Unused unless <code>size_adjust=True</code> .
<code>correction</code>	String or <code>None</code> , one of <code>'bh'</code> , <code>'bonferroni'</code> or <code>None</code> : multiple-comparison correction applied across items within each focal-group comparison – Benjamini-Hochberg (1995) false discovery rate control or Bonferroni – applied separately to the Wald and per-item LR p-values if both computed. Not applied to omnibus test (one test per focal group, not a family of item-level comparisons). Default <code>correction='bh'</code> .
<code>alpha</code>	Float: p-value threshold for flagging, compared against the corrected p-value if <code>correction</code> is not <code>None</code> . Default is <code>alpha=0.05</code> .
<code>logit_threshold</code>	Float: absolute logit-difference threshold for flagging (ETS-style convention). An item is flagged only if both this and the p-value threshold are met. Default is <code>logit_threshold=0.43</code> .
<code>category</code>	Boolean: if <code>True</code> , adds an ETS-style <code>Category</code> column to <code>self.dif_table</code> , as described in Table 18 above, following Zwick et al. (1997). Independent of <code>test/logit_threshold/Flagged</code> – this reproduces the ETS scheme exactly, not a repackaging of the existing Wald flag. Default is <code>category=False</code> .

Arguments continue on the next page.

Arguments (continued)

<code>category_thresholds</code>	Tuple of two floats: (B boundary, C boundary) in logits, per Zwick et al. (1997). Override with your own values if working from a different source or convention. Default is <code>category_thresholds=(0.43, 0.64)</code> .
<code>category_alpha</code>	Float: significance level for both category tests – ETS standard 0.05, independent of <code>alpha</code> (which governs <code>Flagged/Flagged_LR</code>) so overriding one does not change the other. Default <code>category_alpha=0.05</code> .
<code>no_of_samples</code>	Integer: bootstrap resamples for each group's <code>std_errors</code> call (Section 3.3.2). Default is <code>no_of_samples=500</code> .
<code>plot</code>	Boolean: if <code>True</code> , calls <code>plot_anchor_selection</code> (Section 3.5.9) for each focal group that has a selection table (<code>selection_method</code> in <code>'wald'/'robust_z'</code> only) and stores the resulting figures in <code>self.dif_plots</code> . Defaults <code>plot=False</code> – unlike <code>calibrate_anchor</code> 's <code>plot=True</code> default – since <code>dif_test</code> can compare multiple focal groups against the reference in one call.
<code>plot_kwargs</code>	Dictionary or <code>None</code> : extra keyword arguments for <code>plot_anchor_selection</code> for every focal group when <code>plot=True</code> . Default <code>plot_kwargs=None</code> .
<code>warm_corr,</code> <code>tolerance,</code> <code>max_iters,</code> <code>ext_score_adjustment</code>	Person-estimation arguments, passed through to the group models for signature consistency with <code>andersen_lr_test</code> (Section 3.4.4); person estimates are only used if <code>test</code> is <code>'lr'/'both'</code> or <code>omnibus=True</code> .
<code>constant,</code> <code>method,</code> <code>matrix_power,</code> <code>log_lik_tol</code>	Calibration arguments passed to the group models – see Section 3.3.1.
<code>seed</code>	Integer or <code>None</code> : seed passed through to each group's internal <code>std_errors</code> call. <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

<code>self.dif_table</code>	pandas dataframe: one row per (item, focal group) pair, indexed by item. Columns: <code>Group</code> (focal group value), <code>Reference/Focal/Focal (purified)</code> (item location estimates), <code>Difference (purified focal minus reference)</code> , <code>SE</code> , <code>z</code> , <code>p</code> , <code>p (corrected)</code> , <code>Selected</code> (used to define the purified scale) and <code>Flagged</code> (both flagging thresholds met). If <code>welch=True</code> , also <code>df</code> (Welch-Satterthwaite), and <code>p/p (corrected)</code> are t-based rather than normal-based (<code>z</code> is unchanged either way). If <code>test</code> is <code>'lr'</code> or <code>'both'</code> , also <code>LR</code> , <code>p_LR</code> , <code>p_LR (corrected)</code> , <code>Flagged_LR</code> . If <code>category=True</code> , also <code>Category ('A', 'B+', 'B-', 'C+', or 'C-')</code> .
-----------------------------	---

Returns continue on the next page.

Returns (continued)

<code>self.dif_omnibus_table</code>	pandas dataframe or <code>None</code> : one row per focal group – LR, df, p, Flagged – the Andersen-style joint test of every common item at once. <code>None</code> if <code>omnibus=False</code> .
<code>self.dif_reference</code>	The reference group value.
<code>self.dif_covariate</code>	The covariate column name used.
<code>self.dif_reference_model</code>	SLM object: fitted model for the reference group.
<code>self.dif_focal_models</code>	Dictionary: <code>{focal_value: SLM}</code> , fitted models for each focal group.
<code>self.dif_tc</code>	Dictionary: <code>{focal_value: float}</code> , the translation constant applied to each focal group.
<code>self.dif_anchor_selection</code>	Dictionary: <code>{focal_value: dataframe or None}</code> , per-item selection diagnostics for each focal group (<code>None</code> if <code>selection_method='none'</code>).
<code>self.dif_plots</code>	Dictionary: <code>{focal_value: figure or None}</code> , the <code>plot_anchor_selection</code> figure for each focal group, if <code>plot=True</code> and a selection table exists for that group.
<code>self.dif_group_sizes</code>	Dictionary: <code>{group_value: int}</code> , non-extreme, non-missing-covariate N used for each group.

Examples

To test for DIF by an exogenous covariate 'L1' (which may have any number of levels), having supplied exogenous to the constructor, using the largest level as the reference group by default:

```
self.dif_test('L1')
```

To test for DIF against a specific reference level 'English', with 1000 bootstrap samples and diagnostic plots for every focal group:

```
self.dif_test('L1', reference='English', no_of_samples=1000, plot=True)
```

To additionally compute the per-item likelihood-ratio test and ETS-style DIF categories:

```
self.dif_test('L1', test='both', category=True)
```

3.4.6 res_corr_analysis

Description

Analysis of correlations of residuals to tests for violations of local item interdependence and unidimensionality requirements.

Usage

```
self.res_corr_analysis(warm_corr=True, tolerance=0.00001, max_iters=100,  
                       ext_score_adjustment=0.5, constant=0.1, method='cos',  
                       matrix_power=3, log_lik_tol=0.000001, se=True)
```

Arguments

<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's (1989) bias correction is applied during generation of person location estimates (see Section 3.3.4). Default <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 3.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates (see Section 3.3.1). Default value is <code>constant=0.1</code> .
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of item location estimates (see Section 3.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates (see Section 3.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).

Arguments continue on the next page.

Arguments (continued)

<code>log_lik_tol</code>	Float: convergence criterion for log-likelihood method for generation of item location estimates (see Section 3.3.1). Ignored for other methods.
<code>se</code>	Boolean: passed through to the internal <code>fit_statistics</code> call (used if not already computed). If <code>False</code> , skips the bootstrap. This analysis (residual correlations, PCA) does not depend on it, so <code>se=False</code> faster e.g. for repeated simulation runs with no effect on output. Default <code>se=True</code> .

Returns

For tests of violation of the requirement for local item independence (Andrich & Kreiner, 2010; Marais, 2012):

<code>self.residual_correlations</code>	A pandas dataframe of pairwise correlations between item residuals.
---	---

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

<code>self.eigenvectors</code>	The eigenvectors of the standardised residual correlations matrix.
<code>self.eigenvalues</code>	The eigenvalues corresponding to the eigenvectors.
<code>self.variance_explained</code>	The variance explained by each principal component.
<code>self.loadings</code>	The loading of each item onto each of the principal components, for the the first of which large loadings ('large' typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce a residual correlation analysis:

```
self.res_corr_analysis()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

3.4.7 save_stats

Description

Exports `item_stats`, `person_stats` and `test_stats` to file in a single call. Auto-triggers `item_stats_df`, `person_stats_df` and `test_stats_df` (Sections 3.4.1, 3.4.2, 3.4.3) if any have not already been run – with their default arguments plus whichever of `dp/warm_corr/etc.` below are passed, so call those methods directly first (e.g. with `full=True`) if non-default table columns are wanted in the saved output.

Usage

```
self.save_stats(filename, format='csv', dp=3, warm_corr=True, tolerance=0.00001,
                max_iters=100, ext_score_adjustment=0.5, method='cos', constant=0.1,
                no_of_samples=500, interval=None)
```

Arguments

<code>filename</code>	String: output filename or path. For <code>format='csv'</code> , used as a prefix (any trailing <code>.csv</code> is stripped first) for three files, <code>_item_stats.csv</code> , <code>_person_stats.csv</code> and <code>_test_stats.csv</code> . For <code>format='xlsx'</code> , used as a single workbook filename (<code>.xlsx</code> appended automatically if not already present), with the three tables on separate sheets.
<code>format</code>	String: <code>'csv'</code> or <code>'xlsx'</code> . Default is <code>format='csv'</code> .
<code>dp</code> , <code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_adjustment</code> , <code>method</code> , <code>constant</code> , <code>no_of_samples</code> , <code>interval</code>	Passed through to <code>item_stats_df/person_stats_df/test_stats_df</code> (Sections 3.4.1, 3.4.2, 3.4.3), used only for whichever of those tables has not already been computed. See those sections for details of each.

Returns

None. Writes the three tables to file as described above.

Example

To save item, person and test statistics as three CSV files prefixed `my_slm_results`:

```
self.save_stats('my_slm_results')
```

To save all three tables to separate sheets of a single Excel workbook:

```
self.save_stats('my_slm_results', format='xlsx')
```

3.4.8 save_residuals

Description

Exports the residual correlation analysis (`self.eigenvectors`, `self.eigenvalues`, `self.variance_explained` and `self.loadings`) to file. Auto-triggers `res_corr_analysis` (Section 3.4.6) if not already run.

Usage

```
self.save_residuals(filename, format='csv', single=True, dp=3, warm_corr=True,
                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                    method='cos', constant=0.1)
```

Arguments

filename	String: output filename or path (extension appended automatically if not already present).
format	String: 'csv' or 'xlsx'. Default is <code>format='csv'</code> .
single	Boolean: if <code>True</code> , writes all four tables (eigenvectors, eigenvalues, variance explained, loadings) into a single file – sequentially, separated by blank lines, for <code>format='csv'</code> ; at successive row offsets on one sheet ('Item residual analysis'), for <code>format='xlsx'</code> . If <code>False</code> , writes each table to its own file (csv) or sheet (xlsx). Default is <code>single=True</code> .
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant	dp controls rounding of the saved values; the rest are passed through to <code>res_corr_analysis</code> (Section 3.4.6), used only if it has not already been run. See that section for details of each.

Returns

None. Writes the residual correlation analysis tables to file as described above.

Example

To save the residual correlation analysis as a single CSV file:

```
self.save_residuals('my_slm_residuals')
```

To save each table to its own sheet of a single Excel workbook:

```
self.save_residuals('my_slm_residuals', format='xlsx', single=False)
```

3.4.9 category_counts_df

Description

Produces a table of counts of scores in each category, plus total responses and missing responses.

Usage

```
self.category_counts_df(persons=None, items=None, counts_name=None)
```

Arguments

<code>persons</code>	List or String: Person or subset of persons over which to count responses. Default <code>persons=None</code> , which returns counts for all persons.
<code>items</code>	List or String: Item or subset of items over which to count responses. Default <code>items=None</code> , which returns counts for all items.
<code>counts_name</code>	String or Int: Name under which to store counts on a subset of persons or items so that multiple sets of counts can be made available. Useful if e.g. the user wants to compare counts between different subgroups of persons according to a criterion such as an exogenous variable. If passed, stores counts within a namespace, <code>self.counts</code> . Default <code>counts_name=None</code> , when counts are returned as a simple attribute, <code>self.category_counts_table</code> .

Returns

If `counts_name=None`:

Attribute `self.category_counts_table`, a pandas dataframe of category counts with one row per item and one column per response category, plus total and missing responses per item, and grand total row.

If `counts_name=my_counts_name`:

Attribute `self.counts.my_counts_name` stored in the namespace `self.counts`, a pandas dataframe of category counts with one row per item and one column per response category, plus total and missing responses per item and grand total row.

Example

To produce a dataframe of category counts for all persons and items:

```
self.category_counts_df()
```

To produce a dataframe of category counts for a subset of persons saved as a list `my_persons`, with a matching attribute name within the namespace `self.counts`:

```
self.category_counts_df(persons=my_persons, counts_name=my_persons)
```

3.5 Plotting functionality

3.5.1 Shared plotting arguments

All the plotting methods described in this section share a set of arguments which may be used to customise the appearance of the plot or save the plot to file automatically. These arguments are:

<code>title</code>	String: Title for the plot, to appear in the image. Default is <code>title=None</code> .
<code>xmin</code>	Float: Minimum displayed point on x-axis, in logits. Default is <code>xmin=-5</code> .
<code>xmax</code>	Float: Maximum displayed point on x-axis, in logits. Default is <code>xmax=5</code> .
<code>plot_style</code>	String: Plot style to use. Available styles are 'white', producing a plot on a white background, and 'dark', producing a plot on a grey background. The styles correspond to Seaborn (Waskom, 2021) styles <code>whitegrid</code> and <code>darkgrid</code> . Default is <code>plot_style='white'</code> .
<code>palette</code>	String: Controls colours of lines and overplotted class intervals dots. Options are 'dark blue', 'light blue', 'dark red', 'light red', 'dark green', 'light green', 'dark grey', 'light grey', 'dark multi' and 'light multi'. Default is <code>palette='dark blue'</code> . Not used for <code>std_residual_plot</code> .
<code>black</code>	Boolean: If <code>True</code> , the plot will be rendered in black and white. Default is <code>black=False</code> .
<code>font</code>	String: The font to use in the plot. Default is <code>font='Times New Roman'</code> .
<code>title_font_size</code>	Float: The size of the title font in points. Default is <code>title_font_size=15</code> .
<code>axis_font_size</code>	Float: The size of the axis label font in points. Default is <code>axis_font_size=12</code> .
<code>labelsize</code>	Float: The size of the axis tick label font in points. Default is <code>labelsize=12</code> .
<code>filename</code>	String: The filename for the saved plot, with no suffix for format. If <code>None</code> , no file will be saved. Default is <code>filename=None</code> .
<code>file_format</code>	The format of the file: <code>png</code> , <code>jpg</code> or <code>svg</code> . Default is <code>file_format=png</code> .
<code>dpi</code>	The resolution of the plot in dpi (dots per inch) – higher resolution plots are better quality but have larger file sizes. Default is <code>dpi=300</code> .

3.5.2 `icc`

Description

Plots the item characteristic curves (or item response function) for an item: person location on the x-axis against expected score on the y-axis. Options to plot observed responses, item threshold line and lines showing locations corresponding to specified expected scores, and to highlight a specified response category.

Usage

```
self.icc(item, obs=False, no_of_classes=5, thresh_line=False, score_lines=None,  
         score_labels=False, cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot.
<code>obs</code>	Boolean: If <code>True</code> , mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is <code>obs=False</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_line</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the item location (the threshold between the location regions for which 0 or 1 are the most probable score) will be plotted. Default is <code>thresh_line=False</code> .
<code>score_lines</code>	List of floats between 0 and 1: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is <code>score_lines=None</code> .
<code>score_labels</code>	Boolean: If <code>True</code> , scores and locations corresponding to arguments passed to <code>score_lines</code> will be labelled on the plot. Default is <code>score_labels=False</code> .
<code>cat_highlight</code>	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item characteristic curve for `Item_1` and store the output as a variable `my_icc_plot` and save it to file as `my_icc_plot.png`:

```
my_icc_plot = self.icc('Item_1', filename=my_icc_plot)
```

To plot an item characteristic curve for `Item_1` with observed responses for 8 response classes and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc('Item_1', obs=True, no_of_classes=8)
```

To plot an item characteristic curve for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc('Item_1', thresh_line=True, cat_highlight=0)
```

To plot an item characteristic curve for `Item_1` with lines showing the locations corresponding to expected scores of 0.3 and 0.7, with the expected score and corresponding location labelled, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc('Item_1', score_lines=[0.3, 0.7], score_labels=True)
```

3.5.3 crcs

Description

Plots category response curves for an item: person location on the x-axis against the probability of obtaining a score in each category (0 or 1) on the y-axis. Options to plot observed proportions and item threshold line, and to highlight a specified response category.

Usage

```
self.crcs(item, obs=None, no_of_classes=5, thresh_line=False, cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot.
<code>obs</code>	List: List of integers (0 or 1). For each value, mean observed proportions in each ordered response category scoring in that category are plotted against the mean location of the corresponding response class. Default is <code>obs=None</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_line</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the item location (the threshold between the location regions for which 0 or 1 are the most probable score) will be plotted. Default is <code>thresh_line=False</code> .
<code>cat_highlight</code>	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot basic category response curves for `Item_1` and store the output as a variable `my_crcs_plot` and save it to file as `my_crcs_plot.png`:

```
my_crcs_plot = self.crcs('Item_1', filename=my_crcs_plot)
```

To plot category response curves for `Item_1` with observed response proportions for category 0 for 8 response classes and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = self.crcs('Item_1', obs=[0], no_of_classes=8)
```


To plot category response curves for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = self.crcs('Item_1', thresh_line=True, cat_highlight=0)
```

3.5.4 iic

Description

Plots the item information curve for an item: person location on the x-axis against Fisher information on the y-axis. Options to plot item threshold line and lines showing Fisher information corresponding to specified locations, and to highlight a specified response category.

Usage

```
self.iic(item, ymax=None, thresh_line=False, point_info_lines=None, point_info_labels=False,
         cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot.
<code>ymax</code>	Float: The maximum value to show on the y-axis. If <code>None</code> , will infer, plotting a maximum of 1.1 times the maximum item information. Default is <code>ymax=None</code>
<code>thresh_line</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the item location, which the location for which scores of 0 and 1 are equally probable, will be plotted. Default is <code>thresh_line=False</code> .
<code>point_info_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the Fisher information corresponding to the location. Default is <code>point_info_lines=None</code> .
<code>point_info_labels</code>	Boolean: If <code>True</code> , locations and Fisher information corresponding to arguments passed to <code>point_info_lines</code> will be labelled on the plot. Default is <code>point_info_labels=False</code> .
<code>cat_highlight</code>	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item information curve for `Item_1` and store the output as a variable `my_iic_plot` and save it to file as `my_iic_plot.png`:

```
my_iic_plot = self.iic('Item_1', filename='my_iic_plot')
```

To plot an item information curve for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = self.iic('Item_1', thresh_line=True, cat_highlight=0)
```

To plot an item information curve for `Item_1` with lines showing the Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding Fisher information labelled, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = self.icc('Item_1', point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

3.5.5 tcc

Description

Plots the test characteristic curve (or test response function) for a set of items: person location on the x-axis against expected score on the y-axis. Options to plot observed responses and lines showing locations corresponding to specified expected scores.

Usage

```
self.tcc(items=None, obs=False, no_of_classes=5, score_lines=None, score_labels=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>obs</code>	Boolean: If <code>True</code> , mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is <code>obs=False</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>score_lines</code>	List of floats between 0 and 1: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is <code>score_lines=None</code> .
<code>score_labels</code>	Boolean: If <code>True</code> , scores and locations corresponding to arguments passed to <code>score_lines</code> will be labelled on the plot. Default is <code>score_labels=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic test characteristic curve for all items and store the output as a variable `my_tcc_plot` and save it to file as `my_tcc_plot.png`:

```
my_tcc_plot = self.tcc(filename=my_tcc_plot)
```

To plot a test characteristic curve for `Item_1` for a subset of items stored as a list `my_item_list`, with observed responses for 8 response classes and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = self.tcc(obs=True, no_of_classes=8)
```

To plot a test characteristic curve for `Item_1` for all items with lines showing the locations corresponding to expected scores of 13 and 20, with the expected score and corresponding location labelled, and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = self.tcc(score_lines=[13, 20], score_labels=True)
```

3.5.6 test_info

Description

Plots the test information curve: person location on the x-axis against total Fisher information on the y-axis. Option to plot lines showing Fisher information corresponding to specified locations.

Usage

```
self.test_info(items=None, ymax=None, point_info_lines=None, point_info_labels=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>ymax</code>	Float: The maximum value to show on the y-axis. If <code>None</code> , will infer, plotting a maximum of 1.1 times the maximum test information. Default is <code>ymax=None</code>
<code>point_info_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the total Fisher information corresponding to the location. Default is <code>point_info_lines=None</code> .
<code>point_info_labels</code>	Boolean: If <code>True</code> , locations and total Fisher information corresponding to arguments passed to <code>point_info_lines</code> will be labelled on the plot. Default is <code>point_info_labels=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic test information curve and store the output as a variable `my_test_info_plot` and save it to file as `my_test_info_plot.png`:

```
my_test_info_plot = self.test_info(filename='my_test_info_plot')
```

To plot a test information curve for a subset of items stored as a list `my_item_list` and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = self.test_info(items=my_item_list)
```

To plot a test information curve with lines showing the total Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding total Fisher information labelled, and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = self.test_info(point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

3.5.7 test_csem

Description

Plots the test conditional standard error of measurement (CSEM) curve: person location on the x-axis against CSEM (in logits) on the y-axis. Option to plot lines showing CSEM corresponding to specified locations.

Usage

```
self.test_csem(items=None, point_csem_lines=None, point_csem_labels=False, ymax=5)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>ymax</code>	Float: The maximum value to show on the y-axis, in logits. Default is <code>ymax=5</code>
<code>point_csem_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the CSEM curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the CSEM corresponding to the location. Default is <code>point_csem_lines=None</code> .
<code>point_csem_labels</code>	Boolean: If <code>True</code> , locations and CSEM corresponding to <code>point_csem_lines</code> labelled on the plot. Default <code>point_csem_labels=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic CSEM curve and store the output as a variable `my_test_csem_plot` and save it to file as `my_test_csem_plot.png`:

```
my_test_csem_plot = self.test_csem(filename='my_test_csem_plot')
```

To plot a CSEM curve for a subset of items stored as a list `my_item_list` and store the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = self.test_csem(items=my_item_list)
```

To plot a CSEM curve with lines showing the CSEM corresponding to locations of -0.3 and 0.7, with the location and corresponding CSEM labelled, and store the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = self.test_csem(point_csem_lines=[-0.3, 0.7], point_csem_labels=True)
```


3.5.8 std_residuals_plot

Description

Plots histogram of standardised residuals, with optional overplotting of standard Normal distribution.

Usage

```
self.std_residuals_plot(items=None, bin_width=0.5, normal=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>bin_width</code>	Float: The width of the histogram bins along the x-axis. Default is <code>bin_width=0.5</code> .
<code>normal</code>	Boolean: If <code>True</code> , plots a standard normal distribution over the standardised residual histogram for comparison. Default is <code>normal=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot and display a basic standardised residuals histogram and save it to file as `my_std_residuals_plot.png`:

```
self.std_residuals_plot(filename='my_std_residuals_plot')
```

To plot and display a standardised residuals histogram with bin width 1, with standard normal curve:

```
self.std_residuals_plot(bin_width=1, normal=True)
```

To plot and display a standardised residuals histogram on a subset of items stored as a list in a variable `my_item_list`:

```
self.std_residuals_plot(items=my_item_list)
```

3.5.9 plot_anchor_selection

Description

Scatter plot of anchor item locations (x-axis) against observed values for the same items (y-axis) to visualise the outlier-trimming diagnostics produced by `calibrate_anchor` (Section 3.3.3) or `dif_test` (Section 3.4.5). Selected and dropped items are plotted in different colours, with a solid $y = x$ identity line for reference and a dotted OLS best-fit line through the selected points only. `plot_anchor_selection` does not use the shared plotting arguments described in Section 3.5.1; its own, smaller set of arguments is described in full below.

Usage

```
self.plot_anchor_selection(result, xmin=-4, xmax=4, ymin=-3, ymax=3, figsize=(8, 6),
                           font='Times New Roman', title_font_size=15, axis_font_size=12,
                           labels_size=12, title='', dpi=300, filename=None,
                           file_format='png')
```

Arguments

<code>result</code>	dict or pandas dataframe: Either the full dictionary returned internally by the robust anchor-selection routine, or just its 'table', i.e. one of <code>self.anchor_selection</code> (from <code>calibrate_anchor</code>) or <code>self.dif_anchor_selection</code> (from <code>dif_test</code>), each a dataframe with 'Anchor', 'Observed' and boolean 'Selected' columns.
<code>xmin, xmax, ymin, ymax</code>	Floats: axis limits. Defaults are <code>xmin=-4, xmax=4, ymin=-3, ymax=3</code> .
<code>figsize</code>	Tuple of two floats: figure size in inches (width, height). Default is <code>figsize=(8, 6)</code> .
<code>font</code>	String: the font to use in the plot. Default is <code>font='Times New Roman'</code> .
<code>title_font_size,</code> <code>axis_font_size,</code> <code>labels_size</code>	Integers: font sizes in points for the title, axis labels and axis tick labels respectively. Defaults are <code>title_font_size=15, axis_font_size=12, labels_size=12</code> .
<code>title</code>	String: plot title. Default is <code>title=''</code> (no title).
<code>filename</code>	String or None: if provided, saves the plot to this path (with the suffix determined by <code>file_format</code>). Default is <code>filename=None</code> (not saved).
<code>file_format</code>	String: file format for saved plots, e.g. 'png', 'pdf', 'svg'. Default is <code>file_format='png'</code> .
<code>dpi</code>	Integer: output resolution in dpi (dots per inch) when saving to file. Default is <code>dpi=300</code> .

Returns

matplotlib figure object. If a `filename` argument is passed, also saves the image to file. `calibrate_anchor` and `dif_test` both call this method automatically when their own `plot` argument is `True` (see Sections 3.3.3 and 3.4.5), so it need not usually be called directly; it is documented separately here since the resulting figure objects (`self.anchor_plot`, `self.dif_plots`) can also be regenerated with different display arguments by calling it directly on the stored selection table.

Example

To regenerate the anchor-selection plot for an existing calibration's diagnostics, stored in `self.anchor_selection`, with wider axis limits and a custom title:

```
self.plot_anchor_selection(self.anchor_selection, xmin=-6, xmax=6, ymin=-6, ymax=6,  
                           title='Anchor calibration')
```

4 class PCM

4.1 Preliminaries

4.1.1 PCM

Description

Creates an object of the class `PCM` from a pandas dataframe of polytomously scored data of items with individual Rasch-Andrich threshold structures, and potentially different maximum scores, for analysis. No analysis can be run until an object is created.

Usage

```
PCM(responses, max_score_vector=None, extreme_persons=True, no_of_classes=5,  
    validate=True, exogenous=None)
```

Arguments

<code>responses</code>	pandas dataframe with items as columns (item names as column names) and persons as index (person names as row names).
<code>max_score_vector</code>	List or 1-dimensional numpy array of integers: List with the maximum possible score for each item. If no vector is passed, <code>max_score_vector</code> will be inferred from the data, although passing an argument is recommended. Default is <code>max_score_vector=None</code> .
<code>extreme_persons</code>	Boolean: if <code>False</code> , all persons with extreme scores (all responses correct or all responses incorrect) are removed from the response dataframe. Default is <code>extreme_persons=True</code> .
<code>no_of_classes</code>	Integer: the number of classes of persons grouped by location for overplotting observed responses on theoretical curves. Default is <code>no_of_classes=5</code>
<code>validate</code>	Boolean: if <code>True</code> , automatically calls <code>check_data_connectivity</code> (Section 4.1.2) on creation and issues a <code>UserWarning</code> if the item response network is disconnected, or if any item has a structurally unresolvable zero in the pairwise calibration matrix. Default is <code>validate=True</code> .
<code>exogenous</code>	pandas dataframe or <code>None</code> : optional person-level covariates for Differential Item Functioning (DIF) analysis (Section 4.4.8). See the equivalent argument on the <code>SLM</code> constructor (Section 3.1.1) for full details – behaviour is identical. Default is <code>exogenous=None</code> .

Returns

Object of class `PCM`. Analyses are run using methods defined on the `PCM` object, with results stored as attributes of the `PCM` object.

Several attributes of object `PCM` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: Dataframe of valid responses.
<code>self.invalid_responses</code>	pandas dataframe: Dataframe of invalid responses (persons with no responses to any items, i.e. all missing data).
<code>self.extreme_persons</code>	pandas dataframe: Dataframe of persons with extreme scores (all-correct or all-incorrect). When <code>extreme_persons=True</code> (the default), extreme scorers are retained in <code>self.responses</code> and this attribute is empty; when <code>extreme_persons=False</code> , they are removed and stored here.
<code>self.max_score_vector</code>	List or 1-dimensional numpy array of integers: List with the maximum possible score for each item.
<code>self.no_of_items</code>	Integer: Number of items.
<code>self.item_names</code> <code>self.item_ids</code>	List: List of item names. <code>self.item_ids</code> is an alias for users who find <code>*_ids</code> more natural than <code>*_names</code> .
<code>self.no_of_persons</code>	Integer: Number of persons.
<code>self.person_names</code> <code>self.person_ids</code>	List: List of person names. <code>self.person_ids</code> is an alias for users who find <code>*_ids</code> more natural than <code>*_names</code> .
<code>self.generating</code>	Namespace object (<code>_SimParams</code>): Only present when the model is instantiated by passing a simulation object (e.g. <code>PCM.Sim</code>) as the <code>responses</code> argument. Contains all attributes of the simulation object (<code>self.generating.persons</code> , <code>self.generating.items</code> , etc.), enabling direct comparison of generating and estimated parameters.
<code>self.connectivity_status</code>	Dictionary: result of <code>check_data_connectivity</code> (Section 4.1.2), present only when <code>validate=True</code> . Always contains at least 'connected' (Boolean) and 'components_count' (Integer).
<code>self.exogenous</code>	pandas dataframe or None: person-level covariates passed via the <code>exogenous</code> argument, re-indexed onto <code>self.person_names</code> . None if not supplied.
<code>self.no_exogenous_persons</code>	pandas index: Persons present in the response data with no matching exogenous record.
<code>self.exogenous_only_persons</code>	pandas index: Persons present in the exogenous data but not in the response data.

Example

To create an object from a dataframe `my_pcm_dataframe`, with four items with maximum scores of 3, 3, 5 and 5, and with 10 observed classes:

```
my_pcm = PCM(my_pcm_dataframe, max_score_vector=[3, 3, 5, 5], no_of_classes=10)
```

To create an object from a dataframe `my_pcm_dataframe`, additionally carrying exogenous covariates for DIF analysis loaded into `my_exogenous_df`:

```
my_pcm = PCM(my_pcm_dataframe, max_score_vector=[3, 3, 5, 5], exogenous=my_exogenous_df)
```

4.1.2 `check_data_connectivity`

Description

Validates whether the item response network is fully connected: two items are connected if at least one person's response differentiates between them. For PCM, the underlying directed matrix is built at item-*category* granularity (one row per item-category, matching `calibrate`'s own matrix – Section 4.3.1) rather than one row per item, since each item can have its own threshold structure. Category-rows are collapsed back to their owning item for the connectivity check, so two items are adjacent if any pair of their categories has a nonzero directed comparison in either direction.

The 'directionally isolated' check is correspondingly at category granularity: an item is flagged if *any* of its categories has a structurally zero row or column – most commonly a category that is never observed at all (e.g. no person ever scores the top category of an item). An item can therefore be flagged here even when it otherwise looks well-observed overall.

Called automatically on object creation when `validate=True` (the default; see Section 4.1.1), storing its result as `self.connectivity_status`.

Usage

```
self.check_data_connectivity()
```

Returns

Dictionary: 'connected', 'components_count', 'directionally_isolated_items' always present; if disconnected, 'isolated_items'/'all_sub_groups' also present.

Example

To manually re-check connectivity on object `my_pcm` and inspect which items, if any, have an unobserved category:

```
my_pcm.connectivity_status = my_pcm.check_data_connectivity()
my_pcm.connectivity_status['directionally_isolated_items']
```

4.1.3 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_pcm` from `Item_1` to `my_new_item_name`:

```
my_pcm.rename_item('Item_1', 'my_new_item_name')
```

4.1.4 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_pcm` with item names in a list stored as a variable `my_new_item_names`:

```
my_pcm.rename_items_all(my_new_item_names)
```

4.1.5 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the index of `self.responses` with new name. Also replaces person name in `self.exogenous` or `self.no_exogenous_persons` as appropriate if `self.exogenous` exists.

Example

To rename a person in object `my_pcm` from `Person_1` to `my_new_person_name`:

```
my_pcm.rename_person('Person_1', 'my_new_person_name')
```

4.1.6 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the index of `self.responses` with new names. Also replaces all person names in `self.exogenous` and `self.no_exogenous_persons` if `self.exogenous` exists.

Example

To rename all persons in object `my_pcm` with person names in a list stored as a variable `my_new_person_names`:

```
my_pcm.rename_persons_all(my_new_person_names)
```


4.2 Core functions

4.2.1 cat_prob_centred

Description

Category probability function which calculates the probability $P(X_{ni} = k)$ of scoring k , with $k \in \{0, m\}$, where m is the maximum score, from person location, central item location and Rasch-Andrich thresholds (see Section 4.2.2 below for the uncentred thresholds formulation). For a person n with location β_n attempting an item i with central item location δ_i and Rasch-Andrich thresholds $\{\tau_{i0}, \dots, \tau_{im}\}$, the probability of obtaining a score of k is given by:

$$P(X_{ni} = k) = \frac{e^{k(\beta_n - \delta_i) - \sum_{t=0}^k \tau_{it}}}{\sum_{k=0}^m e^{k(\beta_n - \delta_i) - \sum_{t=0}^k \tau_{it}}} \quad (7)$$

In this formulation, an item is defined by a central item location, δ_i and a set of centred Rasch-Andrich thresholds $\{\tau_{ik}\}$, $k \in \{0, \dots, m\}$ which sum to zero: an alternative formulation is to define the item solely by a set of uncentred thresholds, $\{\tau'_{ik}\}$, $k \in \{0, \dots, m\}$, where $\tau'_{ik} = \delta_i + \tau_{ik}$ for $i = 1, \dots, m$ and $\tau'_{ik} = 0$. Separate methods for all core functions are defined using uncentred thresholds.

Usage

```
self.cat_prob_centred(person_location, item_location, category, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Central item location
<code>category</code>	Integer: Response category k , with $k \in \{0, 1\}$.
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Returns

Float: probability of obtaining score k .

Example

To obtain the probability of a person of location 0.5 scoring 0 on an item of central location 0 with a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_cat_prob`:

```
my_cat_prob = self.cat_prob_centred(0.5, 0, 0, my_thresholds)
```

4.2.2 cat_prob_uncentred

Description

Category probability function which calculates the probability $P(X_{ni} = k)$ of scoring k , with $k \in \{0, m\}$, where m is the maximum score, from person location and uncentred thresholds (see Section 4.2.1 above for the centred thresholds formulation). For a person n with location β_n attempting an item i with thresholds $\{\tau_{i1}, \dots, \tau_{im}\}$, the probability of obtaining a score of k is given by:

$$P(X_{ni} = k) = \frac{e^{k\beta_n - \sum_{t=0}^k \tau_{it}}}{\sum_{k=0}^m e^{k\beta_n - \sum_{t=0}^k \tau_{it}}} \quad (8)$$

In this formulation, an item is defined by a set of thresholds $\{\tau_{ik}\}$, $k \in \{1, \dots, m\}$: an alternative formulation is to define the item a central item location, δ_i and a set of centred Rasch-Andrich thresholds $\{\tau'_k\}$, $k \in \{0, \dots, m\}$ which sum to zero, where $\delta_i = \sum_i = 1^m \tau_{ik} / m$ and $\tau'_{ik} = \tau_{ik} - \delta_i$. Separate methods for all core functions are defined using centred thresholds.

Usage

```
self.cat_prob_uncentred(person_location, category, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>category</code>	Integer: Response category k , with $k \in \{0, 1\}$.
<code>thresholds</code>	List or numpy array: Set of m uncentred thresholds for scores 1 to m , where m is the maximum score.

Returns

Float: probability of obtaining score k .

Example

To obtain the probability of a person of location 0.5 scoring 0 on an item with a set of uncentred thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_cat_prob`:

```
my_cat_prob = self.cat_prob_uncentred(0.5, 0, my_thresholds)
```

4.2.3 exp_score_centred

Description

Expected score function which calculates the expected score $E(X_{ni})$ from person location, central item location and set of centred Rasch-Andrich thresholds. The expected score is given by:

$$E(X_{ni}) = \sum_{k=0}^m kP(X_{ni} = k) \quad (9)$$

where m is the maximum score for item i and $P(X_{ni} = k)$ is as described in Section 4.2.1.

Usage

```
self.exp_score_centred(person_location, item_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Central item location
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Returns

Float: expected score.

Example

To obtain the expected score for a person of location 0.5 attempting an item of location 0 with a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_exp_score`:

```
my_exp_score = self.exp_score_centred(0.5, 0, my_thresholds)
```

4.2.4 exp_score_uncentred

Description

Expected score function which calculates the expected score $E(X_{ni})$ from person location and a set of uncentred thresholds. The expected score is given by:

$$E(X_{ni}) = \sum_{k=0}^m kP(X_{ni} = k) \quad (10)$$

where m is the maximum score for item i and $P(X_{ni} = k)$ is as described in Section 4.2.2.

Usage

```
self.exp_score_uncentred(person_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>thresholds</code>	List or numpy array: Set of m thresholds, where m is the maximum score.

Returns

Float: expected score.

Example

To obtain the expected score for a person of location 0.5 attempting an item of location 0 with a set of thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_exp_score`:

```
my_exp_score = self.exp_score_uncentred(0.5, my_thresholds)
```

4.2.5 variance_centred

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location, central item location and set of Rasch-Andrich thresholds. The variance is given by:

$$V(X_{ni}) = \sum_{k=0}^m P(X_{ni} = k)(k - E(X_{ni}))^2 \quad (11)$$

where m is the maximum score for item i and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 4.2.1 and 4.2.3 respectively.

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

```
self.variance_centred(person_location, item_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Central item location
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Returns

Float: variance (also the Fisher information provided by the response).

Example

To obtain the variance for a person of location 0.5 attempting an item with central location 0 and a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_variance`:

```
my_variance = self.variance_centred(0.5, 0, my_thresholds)
```

4.2.6 variance_uncentred

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location and set of uncentred thresholds. The variance is given by:

$$V(X_{ni}) = \sum_{k=0}^m P(X_{ni} = k)(k - E(X_{ni}))^2 \quad (12)$$

where m is the maximum score for item i and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 4.2.2 and 4.2.4 respectively.

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

```
self.variance_uncentred(person_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>thresholds</code>	List or numpy array: Set of m thresholds, where m is the maximum score.

Returns

Float: variance (also the Fisher information provided by the response).

Example

To obtain the variance for a person of location 0.5 attempting an item with a set of uncentred thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_variance`:

```
my_variance = self.variance_uncentred(0.5, my_thresholds)
```

4.2.7 kurtosis_centred

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location, central item location and Rasch-Andrich thresholds. The variance is given by:

$$\kappa(X_{ni}) = \sum_{k=0}^m P(X_{ni} = k)(k - E(X_{ni}))^4 \quad (13)$$

where m is the maximum score for item i and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 4.2.1 and 4.2.3 respectively.

Usage

```
self.kurtosis_centred(person_location, item_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Central item location
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 attempting an item with central location 0 and a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_kurtosis`:

```
my_kurtosis = self.kurtosis_centred(0.5, 0, my_thresholds)
```

4.2.8 kurtosis_uncentred

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location, central item location and Rasch-Andrich thresholds. The variance is given by:

$$\kappa(X_{ni}) = \sum_{k=0}^m P(X_{ni} = k)(k - E(X_{ni}))^4 \quad (14)$$

where m is the maximum score for item i and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 4.2.2 and 4.2.4 respectively.

Usage

```
self.kurtosis_uncentred(person_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>thresholds</code>	List or numpy array: Set of m uncentred thresholds for scores 1 to m , where m is the maximum score.

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 attempting an item with a set of uncentred thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_kurtosis`:

```
my_kurtosis = self.kurtosis_uncentred(0.5, my_thresholds)
```


4.3 Parameter estimation

4.3.1 `calibrate`

Description

Produces item estimates in two formats: central item location and Rasch-Andrich threshold estimates (See sections 4.2.1 and 4.2.2 for details), plus uncentred thresholds using PAIR (Choppin, 1968, 1985), eigenvector method (Garner & Engelhard, 2002) or related conditional pairwise methods.

Usage

```
self.calibrate(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001)
```

Arguments

<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> .
<code>method</code>	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.

Returns

Returns three attributes:

<code>self.items</code>	pandas series: Central item location estimates (mean of uncentred thresholds) with item names as keys and estimates as values.
<code>self.thresholds</code>	pandas DataFrame: Centred threshold estimates, one row per item, one column per threshold position. Values sum to zero within each item. NaN-padded for items with fewer thresholds than the maximum.
<code>self.thresholds_uncentred</code>	pandas DataFrame: Uncentred threshold estimates, one row per item, one column per threshold position. NaN-padded for items with fewer thresholds than the maximum.

Examples

To generate a set of estimates using the cosine similarity method for central item locations, with additive smoothing constant of 0.1:

```
self.calibrate()
```

To generate a set of estimates using the log-likelihood method for central item locations, with matrix raised to power 7 and a convergence stopping criterion of 0.00000001:

```
self.calibrate(method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```

4.3.2 std_errors

Description

Produces bootstrapped estimates for the standard errors of central item location estimates and threshold estimates. Threshold standard errors are shared between centred and uncentred thresholds, while central item location estimates are calculated separately.

Usage

```
self.std_errors(interval=None, no_of_samples=500, constant=0.1, method='cos',  
                matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval. Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate. More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> .
<code>method</code>	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix for central item location estimates (Elliott & Buttery, 2022b:991) is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
<code>seed</code>	Integer or <code>None</code> : seed for the bootstrap resampling random number generator. Pass an integer for fully reproducible standard errors; <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attributes:

<code>self.central_bootstrap</code>	pandas dataframe: Full central item location bootstrap results, with a row for each bootstrap sample and a column for each central item location estimate.
<code>self.central_se</code>	pandas series: Item names as keys and item standard errors as values.
<code>self.threshold_bootstrap</code>	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each threshold estimate. <code>self.threshold_bootstrap</code> contains uncentred threshold estimates (standard errors would be identical with centred thresholds).
<code>self.threshold_se</code>	pandas series: Threshold numbers as keys and threshold standard errors as values.
<code>self.cat_width_bootstrap</code>	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each category width estimate.
<code>self.cat_width_se</code>	pandas series: Category numbers as keys and category width standard errors as values.

If an argument is passed to `interval`, also returns:

<code>self.item_low</code>	Lower bound of the specified interval for item estimates.
<code>self.item_high</code>	Upper bound of the specified interval for item estimates.
<code>self.threshold_low</code>	Lower bound of the specified interval for threshold estimates.
<code>self.threshold_high</code>	Upper bound of the specified interval for threshold estimates.
<code>self.cat_width_low</code>	Lower bound of the specified interval for category estimates.
<code>self.cat_width_high</code>	Upper bound of the specified interval for category estimates.

Example

To generate item standard errors with a 95% interval from 200 samples: `self.std_errors(interval=0.95, no_of_samples=200)`

Modifications to the estimation method are discussed in Section 4.3.1.

4.3.3 `calibrate_anchor`

Description

Anchors item location estimates onto externally-supplied reference values, supporting item banking – item locations are calibrated as usual, then the item-location scale is shifted by a translation constant (by default via robust modified z-score outlier trimming (Iglewicz & Hoaglin, 1993)) so that a subset of common anchor items line up with externally-supplied reference locations. This is a mean-shift translation only; each item’s own centred threshold offsets (`self.thresholds`) are left untouched by the anchoring process.

Optionally, if reference threshold structures `anchor_thresholds` are also supplied for some anchor items (thresholds are not needed for every anchor item), likelihood-ratio tests check whether the reference thresholds are compatible with the estimated thresholds. This can be run per item (`threshold_check='per_item'`, one independent test per supplied item, BH/Bonferroni-corrected for multiple comparisons (Benjamini & Hochberg, 1995)), or as a single combined test across every supplied item at once (`threshold_check='aggregate'`), or both (default).

Usage

```
self.calibrate_anchor(anchors, calibrate=False, selection_method='robust_z',
                      corr_tol=0.95, sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05,
                      no_of_samples=500, adj=None, overwrite_anchors='none',
                      plot=True, plot_kwargs=None, anchor_thresholds=None,
                      threshold_check='both', alpha=0.05, correction='bh',
                      warm_corr=True, tolerance=0.00001, max_iters=100,
                      ext_score_adjustment=0.5, constant=0.1, method='cos',
                      matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>anchors</code>	Dictionary or pandas series: reference locations for a subset of items, keyed/indexed by item name.
<code>calibrate</code>	Boolean: if True, (re-)runs <code>calibrate</code> before anchoring. If False, <code>calibrate</code> is still auto-triggered if it has not already been run. Default is <code>calibrate=False</code> .
<code>selection_method</code>	String: 'robust_z', 'wald' or 'none'. 'robust_z' (default): excludes outlying anchor items via modified z-score procedure (Iglewicz & Hoaglin, 1993); uses <code>corr_tol</code> , <code>sd_ratio_tol</code> , <code>min_anchors</code> . 'wald': formal per-item sequential significance test based on z-scores from item standard errors – auto-triggers <code>std_errors</code> if not already computed; uses <code>wald_alpha</code> and <code>min_anchors</code> , not <code>corr_tol</code> / <code>sd_ratio_tol</code> . 'none': plain mean shift over every supplied anchor, no trimming.

Arguments continue on the next page.

Arguments (continued)

<code>corr_tol</code>	Float: minimum acceptable Pearson correlation between anchor and observed values on the surviving (non-outlier) item set (<code>selection_method='robust_z'</code> only). Default is <code>corr_tol=0.95</code> .
<code>sd_ratio_tol</code>	Float: the ratio of observed to anchor standard deviations on the surviving item set must fall within $[1/\text{sd_ratio_tol}, \text{sd_ratio_tol}]$ (<code>selection_method='robust_z'</code> only). Default is <code>sd_ratio_tol=1.1</code> , i.e. the ratio must lie in $[0.909, 1.1]$.
<code>min_anchors</code>	Integer: floor on the number of surviving anchor items – selection stops (with a warning) once this floor is reached, even if the active method's tolerance/significance criterion is not yet satisfied, and is skipped altogether if the initial common item set is at or below this floor (<code>selection_method</code> in <code>'robust_z'/'wald'</code> only). Default is <code>min_anchors=6</code> .
<code>wald_alpha</code>	Float: significance level for each anchor item's Wald test (<code>selection_method='wald'</code> only). Default is <code>wald_alpha=0.05</code> .
<code>no_of_samples</code>	Integer: bootstrap samples for the auto-triggered <code>std_errors</code> call, only used if item standard errors are not already computed (<code>selection_method='wald'</code> only). Default is <code>no_of_samples=500</code> .
<code>adj</code>	Float or <code>None</code> : if provided, applied as translation constant and selection step skipped. Intended for reuse across bootstrap resamples (Section 4.3.2), where recomputing anchor selection every time would inflate standard errors with anchor-item sampling variance. Default <code>adj=None</code> .
<code>overwrite_anchors</code>	String, one of <code>'none'</code> , <code>'rejected'</code> or <code>'all'</code> : controls what <code>self.anchor_items</code> holds for the anchor items themselves, as opposed to non-anchor items, which always receive the shifted (calibrated + translation constant) value. <code>'none'</code> : every anchor item keeps exactly its externally-supplied value, unchanged – the usual convention, since an anchor's value is fixed by definition, not re-estimated. <code>'rejected'</code> : retained anchors keep their supplied values, but rejected anchors have their values overwritten with new, freshly-calibrated values; requires <code>selection_method</code> in <code>'robust_z'/'wald'</code> and no <code>adj</code> override (otherwise falls back to <code>'none'</code> with a warning). <code>'all'</code> : every anchor item has its value overwritten with a new, freshly-calibrated value like any other item. Default is <code>overwrite_anchors='none'</code> .
<code>plot</code>	Boolean: if <code>True</code> and <code>selection_method</code> in <code>'robust_z'/'wald'</code> (so a selection table exists), calls <code>plot_anchor_selection</code> (Section 3.5.9). Has no effect when <code>selection_method='none'</code> or <code>adj</code> is supplied. Default <code>plot=True</code> .

Arguments continue on the next page.

Arguments (continued)

<code>plot_kwargs</code>	Dictionary or <code>None</code> : extra keyword arguments forwarded to <code>plot_anchor_selection</code> when <code>plot=True</code> (e.g. <code>filename</code> , <code>xmin/xmax</code> , <code>title</code>). Default is <code>plot_kwargs=None</code> .
<code>anchor_thresholds</code>	Dictionary or <code>None</code> : <code>{item_name: array-like}</code> of externally-supplied reference threshold offsets, matching the shape of <code>self.thresholds</code> for that item (that item's own centred category-structure shape, not an absolute/uncentred value). Only items present in this dictionary are checked; need not cover every anchor item. If <code>None</code> , the threshold-structure check is skipped entirely regardless of <code>threshold_check</code> . Default is <code>anchor_thresholds=None</code> .
<code>threshold_check</code>	String: one of <code>'per_item'</code> , <code>'aggregate'</code> , <code>'both'</code> or <code>'none'</code> . <code>'per_item'</code> runs one independent LR test per item in <code>anchor_thresholds</code> (df = that item's max score -1 each), correction-adjusted. <code>'aggregate'</code> runs a single combined LR test swapping every supplied item's thresholds in simultaneously (df = sum of each item's max score -1), needing no multiple-comparison correction since it is a single test. <code>'both'</code> runs both tests. <code>'none'</code> skips the checks. Default is <code>threshold_check='both'</code>
<code>alpha</code>	Float: significance level for all LR tests (compared against the corrected p-value for per-item tests, raw p for the aggregate test). Default is <code>alpha=0.05</code> .
<code>correction</code>	String or <code>None</code> , one of <code>'bh'</code> , <code>'bonferroni'</code> or <code>None</code> : multiple-comparison correction across the per-item tests only – Benjamini-Hochberg false discovery rate control (Benjamini & Hochberg, 1995) or Bonferroni. <code>None</code> uses raw, uncorrected p-values. Ignored for the aggregate test. Default is <code>correction='bh'</code> .
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_adjustment</code>	Person-estimation arguments, used only by the <code>threshold_check</code> comparison (locations must be re-estimated under each candidate threshold table before the log-likelihoods are comparable), or if <code>calibrate/std_errors</code> (<code>selection_method='wald'</code>) triggered.
<code>constant</code> , <code>method</code> , <code>matrix_power</code> , <code>log_lik_tol</code>	Calibration/bootstrap arguments, used only if <code>calibrate</code> or <code>std_errors</code> (<code>selection_method='wald'</code>) triggered – see Section 4.3.1.
<code>seed</code>	Integer or <code>None</code> : seed passed through to the internal <code>std_errors</code> call (only used if item standard errors are not already computed, <code>selection_method='wald'</code> only). <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

<code>self.anchor_items</code>	pandas series: item locations shifted onto the anchor scale.
<code>self.anchor_item_names</code>	pandas index: names of the items supplied as anchors.
<code>self.anchor_adj</code>	Float: the translation constant actually applied.
<code>self.anchor_selection</code>	pandas dataframe or <code>None</code> : per-item diagnostics from whichever selection method was used – Anchor, Observed, Deviation, Robust z, Selected ('robust_z'), or Anchor, Observed, SE, Deviation, z, p, Selected ('wald'). <code>None</code> if <code>selection_method='none'</code> or <code>adj</code> supplied.
<code>self.anchor_selected_items,</code> <code>self.anchor_dropped_items</code>	pandas index or <code>None</code> : items retained / excluded as outliers. <code>None</code> if <code>selection_method='none'</code> or <code>adj</code> supplied.
<code>self.anchor_original_corr,</code> <code>self.anchor_original_sd_ratio</code>	Float or <code>None</code> : correlation / SD ratio before trimming.
<code>self.anchor_corr</code> <code>self.anchor_sd_ratio</code>	Float or <code>None</code> : correlation / SD ratio after trimming.
<code>self.anchor_summary</code>	pandas series: one-line summary – anchors supplied/common/selected/dropped, correlation and SD ratio before/after trimming, translation constant applied.
<code>self.anchor_plot</code>	matplotlib figure or <code>None</code> : figure from the auto-triggered <code>plot_anchor_selection</code> call. <code>None</code> if <code>plot=False</code> , <code>selection_method='none'</code> , or <code>adj</code> was supplied directly.
<code>self.calibrate_anchor_runs</code>	Dictionary: every call's results above, keyed by the sorted anchor items supplied, so results from an earlier call survive a later call with a different anchor set instead of being overwritten in place.

The threshold-structure check additionally sets:

<code>self.anchor_threshold_test</code>	pandas dataframe or <code>None</code> : per-item LR test table (LL_estimated, LL_given, LR, df, p, p_corrected, Flagged), indexed by item. <code>None</code> if <code>anchor_thresholds</code> not supplied, <code>threshold_check='none'</code> , or <code>threshold_check='aggregate'</code> only.
<code>self.anchor_threshold_test_aggregate</code>	pandas series or <code>None</code> : combined LR test across every item in <code>anchor_thresholds</code> at once (LL_estimated, LL_given, LR, df, p, Flagged). <code>None</code> if <code>anchor_thresholds</code> not supplied, <code>threshold_check='none'</code> , or <code>threshold_check='per_item'</code> only.

Examples

To anchor item locations onto a bank of reference values stored in a dictionary `my_bank`, with automatic outlier trimming and diagnostic plot:

```
self.calibrate_anchor(my_bank)
```

To additionally check compatibility with reference threshold structures for a subset of items, stored as `my_bank_thresholds`, using only the per-item test with Bonferroni correction:

```
self.calibrate_anchor(my_bank, anchor_thresholds=my_bank_thresholds,  
                      threshold_check='per_item', correction='bonferroni')
```

4.3.4 person

Description

Generates a location estimate for a person using the Newton-Raphson method to produce a maximum likelihood estimate, with optional Warm bias correction (Warm, 1989).

Usage

```
self.person(persons, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,  
            ext_score_adjustment=0.5, missing_as_incorrect=False)
```

Arguments

<code>persons</code>	String or list: person name(s) for the location(s) being estimated. Pass 'all' to estimate for all persons in <code>self.responses</code> .
<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations. Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>missing_as_incorrect</code>	Boolean: if <code>True</code> , treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is <code>missing_as_incorrect=False</code> .

Returns

pandas series: person location estimate(s), in logits, indexed by person identifier. Returns `NaN` for any person for whom estimation fails.

Examples

To generate a person location estimate for `Person_1` using the default settings and store the result as a variable, `my_person_location`:

```
my_person_location = my_pcm.person('Person_1')
```

To generate an MLE person location estimate without Warm bias correction for `Person_1` based on the first three items and store the result as a variable, `my_person_location`:

```
my_person_location = my_pcm.person('Person_1', ['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

4.3.5 person_estimates

Description

Generates location estimates for all persons using the Newton-Raphson method to produce maximum likelihood estimates, with optional Warm bias correction (Warm, 1989).

Usage

```
self.person_estimates(items=None, warm_corr=True, tolerance=0.00001, max_iters=100,  
                      ext_score_adjustment=0.5, missing_as_incorrect=False)
```

Arguments

<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations. Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>missing_as_incorrect</code>	Boolean: if <code>True</code> , treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is <code>missing_as_incorrect=False</code> .

Returns

Attribute `self.persons`: pandas series with person names as keys and location estimates as values.

Examples

To generate a set of person location estimates with Warm bias correction:

```
self.person_estimates()
```

To generate a set of person location estimates without Warm bias correction, on a subset of the first three items only:

```
my_pcm.person_estimates(items=['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

4.3.6 score_lookup

Description

Generates a location estimate for a given raw score on responses to a given set of items using the Newton-Raphson method to produce a maximum likelihood estimate, with optional Warm bias correction (Warm, 1989).

Usage

```
self.score_lookup(score, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
                  ext_score_adjustment=0.5)
```

Arguments

<code>score</code>	Integer: The raw score for which location is being estimated.
<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations. Default <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Returns

pandas series with raw scores as keys and person location estimates as values.

Examples

To generate a location estimate for a score of 10 on all items, with Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = my_pcm.score_lookup(10)
```

To generate a location estimate for a score of 10 on a subset of items saved as a variable `my_items`, without Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = my_pcm.score_lookup(10, items=my_items, warm_corr=False)
```

4.3.7 score_lookup_table

Description

Generates a lookup table of location estimates corresponding to all available raw scores on a set of items with no missing responses, using the Newton-Raphson method to produce maximum likelihood estimates and with optional Warm bias correction (Warm, 1989).

Usage

```
self.score_lookup_table(items=None, ext_scores=True, warm_corr=True, tolerance=0.00001,
                        max_iters=100, ext_score_adjustment=0.5)
```

Arguments

<code>items</code>	List: List of items names on which to generate estimates. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
<code>ext_scores</code>	Boolean: If <code>True</code> , location estimates for extreme scores (all correct/all incorrect) generated using <code>ext_score_adjustment</code> . Default <code>ext_scores=True</code> .
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations. Default <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Returns

Attribute `self.score_table`: pandas series with raw scores as keys and corresponding location estimates as values.

Examples

To generate a location lookup table for all items, including extreme scores, with Warm bias correction:

```
my_pcm.score_lookup_table()
```

To generate a location lookup table for a subset of items saved as a variable `my_items`, without extreme scores and without Warm bias correction:

```
my_pcm.score_lookup_table(items=my_items, ext_scores=False)
```

4.3.8 csem

Description

Calculates conditional standard error of measurement for one or more persons.

Usage

```
self.csem(persons=None, person_locations=None, items=None)
```

Arguments

<code>persons</code>	List or string: person name(s). Overrides <code>person_locations</code> if provided. Default is <code>persons=None</code> , which uses all persons.
<code>person_locations</code>	pandas series, float, list, numpy array, or dictionary, with person names as keys (where applicable) and locations as values. If <code>None</code> , uses <code>self.persons</code> , automatically generating via <code>self.person_estimates()</code> if necessary. A raw location value (or values not indexed by a real person, e.g. a raw score lookup) is treated as hypothetical: since there is no observed response data for it, all items in <code>items</code> are treated as answered. Default is <code>person_locations=None</code> .
<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.

Returns

pandas series: conditional standard error of measurement in logits for each person (or location), indexed by person name (or by a location label, for hypothetical locations).

Examples

To generate the CSEM for `Person_1` on all items and save the result as a variable, `my_csem`:

```
my_csem = my_pcm.csem('Person_1')
```

To generate the CSEM for a raw score of 3 on a subset of items saved to a variable `my_items` and save the result as a variable, `my_csem`:

```
my_csem = my_pcm.csem(3, person_locations=self.score_table, items=my_items)
```

where `self.score_table` is the output from `self.score_lookup_table(items=my_items)` (see Section 4.3.7).

4.4 Statistical output

4.4.1 `item_stats_df`

Description

Produces a table of item statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

```
self.item_stats_df(full=False, zstd=False, point_measure_corr=False, dp=3, se=True,
                   warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                   method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                   no_of_samples=500, interval=None, seed=None)
```

Arguments

<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>zstd</code>	Boolean: If <code>True</code> , infit and outfit standardised z-scores are reported. Default is <code>zstd=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>point_measure_corr</code>	Boolean: If <code>True</code> , point-polyserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure polyserial correlation for an ideal item. Default is <code>point_measure_corr=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>se</code>	Boolean: If <code>True</code> , computes and reports the SE column (and CI bound columns, if <code>interval</code> is set). If <code>False</code> , skips the bootstrap entirely – useful when only Infit MS/Outfit MS are needed (e.g. repeated simulation runs), since those do not depend on the bootstrap and are unaffected by this flag. Forces <code>interval=None</code> when <code>False</code> . Default is <code>se=True</code> .
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 4.3.4). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is <code>max_iters=100</code> .

Arguments continue on the next page.

Arguments (continued)

<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure an item location estimate is returned if a person has an extreme score (all responses correct or incorrect). Since there is no finite ML location estimate for extreme scores, adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimation.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 4.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 4.3.1). Default value is <code>constant=0.1</code> .
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates (see Section 4.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 4.3.1). Ignored for other methods.
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 4.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Unused if <code>se=False</code> . Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 4.3.2). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Ignored if <code>se=False</code> . Default is <code>interval=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed passed through to the internal <code>std_errors/fit_statistics</code> calls (only used if not already computed). <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.item_stats`, a pandas dataframe with one row for each item and the following columns:

Estimate	Central item location estimate.
SE	Bootstrapped standard error of central item location estimate. Only produced if <code>se=True</code> .
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 2.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 5%. Only produced if <code>se=True</code> and <code>interval</code> is set.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 97.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 95%. Only produced if <code>se=True</code> and <code>interval</code> is set.
Count	Count of responses.
Facility	Item facility: proportion of correct responses.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if <code>zstd=True</code> .
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if <code>zstd=True</code> .
PM corr	Point-polyserial correlation. Only produced if <code>point_measure_corr=True</code> .
Exp PM corr	Expected value of point-measure polyserial correlation. Only produced if <code>point_measure_corr=True</code> .
Disordered	If <code>True</code> , threshold estimates indicate the presence of disordered thresholds within the item (Andrich, 2010).

Examples

To produce a summary `item_stats` table with the most commonly reported statistics:

```
my_pcm.item_stats_df()
```

To produce a full `item_stats` table with all statistics:

```
my_pcm.item_stats_df(full=True)
```

To produce an `item_stats` table with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
my_pcm.item_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of central item location and/or person location estimation.

4.4.2 threshold_stats_df

Description

Produces a table of threshold statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

```
self.threshold_stats_df(full=False, zstd=False, disc=False, point_measure_corr=False, dp=3,
                        warm_corr=True, tolerance=0.00001, max_iters=100,
                        ext_score_adjustment=0.5, method='cos', constant=0.1,
                        no_of_samples=500, interval=None)
```

Arguments

<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a table with the most commonly reported statistics.
<code>zstd</code>	Boolean: If <code>True</code> , infit and outfit standardised z-scores are reported. Default is <code>zstd=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>disc</code>	Boolean: If <code>True</code> , item discrimination is reported. The discrimination of the empirical threshold slope, which functions as an SLM item conditioned on responses being in the categories either side, relative to the ideal logistic ogive, with 1 perfect, > 1 showing overfit and < 1 showing underfit; discrimination is similar to the 2PL IRT discrimination parameter (Linacre, 2023), but is a descriptive statistic here rather than an item parameter.
<code>point_measure_corr</code>	Boolean: If <code>True</code> , point-biserial correlation between person location and score $k - 1$ versus k is reported, together with its expected value for an ideal item. Default is <code>point_measure_corr=False</code> . Always <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's (1989) bias correction is applied during generation of person location estimates (see Section 4.3.4). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations during generation of person estimates (Section 4.3.4). Default <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a person location estimate is returned if a person has an extreme score (all responses correct or incorrect). Adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimation.

Arguments continue on the next page.

Arguments (continued)

<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 4.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 4.3.1). Default value is <code>constant=0.1</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 4.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .

Returns

Two attributes: `self.threshold_stats_centred` and `self.threshold_stats_uncentred`, pandas dataframes which differ only in their first column, `Estimate`. The dataframes have one row for each individual threshold and the following columns:

<code>Estimate</code>	Threshold location estimate. For the <code>threshold_stats_centred</code> table, this contains the estimates of the centred Rasch-Andrich thresholds, while for the <code>threshold_stats_uncentred</code> table, it contains the estimates of the uncentred thresholds.
<code>SE</code>	Bootstrapped standard error of item location estimate.
<code>2.5%</code>	Lower bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 2.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 5%.
<code>97.5%</code>	Upper bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 97.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 95%.
<code>Infit MS</code>	Infit mean square.
<code>Infit Z</code>	Standardised infit z-score. Only produced if <code>zstd=True</code> .

Returns continue on the next page.

Returns (continued)

Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if <code>zstd=True</code> .
Discrim	Item discrimination. Only produced if <code>disc=True</code> .
PM corr	Point-biserial correlation. Only produced if <code>point_measure_corr=True</code> .
Exp PM corr	Expected value of point-measure biserial correlation. Only produced if <code>point_measure_corr=True</code> .

Examples

To produce summary `threshold_stats` tables with the most commonly reported statistics:

```
self.threshold_stats_df()
```

To produce a full `my_pcm.threshold_stats` table with all statistics:

```
my_pcm.threshold_stats_df(full=True)
```

To produce `threshold_stats` tables with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
my_pcm.threshold_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of central item location, threshold and person location estimation.

4.4.3 category_stats_df

Description

Produces a table of category width statistics – `threshold_stats_df` (Section 4.4.2) reports threshold locations; this reports the difference between consecutive thresholds for each item, $\tau_{k+1} - \tau_k$. Each item’s zero-summed threshold vector has only `max_score_vector[item]-1` true degrees of freedom, so the per-threshold standard errors in `threshold_stats_df` are correlated rather than independent, and understate the true amount of error. Widths are identical whether computed from centred or uncentred thresholds, so there is only one `category_stats` table. No infit/outfit or other fit statistics are reported, since these apply to thresholds rather than categories. Auto-triggers `calibrate` (Section 4.3.1) and `std_errors` (Section 4.3.2) directly if not already run.

A category width can be negative – a negative width at category k means thresholds τ_k and τ_{k+1} are disordered for that item (category k is never the most likely response at any person location). Prop `disordered` makes this a continuous diagnostic rather than a straightforward yes/no: the proportion of bootstrap resamples in which that item’s category width was negative, reasonably read as the probability that the true category is disordered.

Usage

```
self.category_stats_df(dp=3, constant=0.1, method='cos', matrix_power=3,  
                       log_lik_tol=0.000001, no_of_samples=500, interval=None, seed=None)
```

Arguments

<code>dp</code>	Integer: Number of decimal places reported in output. Default is <code>dp=3</code> .
<code>constant</code> , <code>matrix_power</code> , <code>log_lik_tol</code>	<code>method</code> , Calibration keyword arguments, used only if <code>calibrate</code> (Section 4.3.1) has not already been run. See Section 4.3.1 for details of each.
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates, used only if <code>std_errors</code> (Section 4.3.2) has not already been run. Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed passed through to <code>std_errors</code> , used only if not already run. <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.category_stats`, a pandas dataframe with a (item, category) multiindex – one fewer row per item than `threshold_stats`, since a width is defined between each pair of consecutive thresholds – and the following columns:

Estimate	Category width (can be negative).
SE	Bootstrapped standard error of the width, differenced <i>within</i> each bootstrap resample before taking the standard deviation, so it correctly reflects the covariance between adjacent thresholds rather than naively combining their individual standard errors.
2.5% / 97.5%	Bounds of the empirical bootstrap estimate interval, if <code>interval</code> is available (see <code>threshold_stats_df</code> , Section 4.4.2, for the column-naming convention).
Disordered	Boolean: True if Estimate is negative.
Prop disordered	Proportion of bootstrap resamples in which this category's width was negative – reasonably read as the probability that the true category is disordered. Useful for interpreting <code>Disordered</code> in both directions: a True that is only weakly supported (<code>Prop disordered</code> close to 0.5) versus robust, or a False that is nonetheless uncertain versus clear-cut.

Examples

To produce the `my_pcm.category_stats` table, alongside the threshold-level table:

```
my_pcm.threshold_stats_df()
my_pcm.category_stats_df()
```

To check which categories, if any, are disordered:

```
my_pcm.category_stats_df()
my_pcm.category_stats[my_pcm.category_stats['Disordered']]
```


4.4.4 `person_stats_df`

Description

Produces a table of person statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

```
self.person_stats_df(full=False, rsem=False, dp=3, se=True, warm_corr=True, tolerance=0.00001,
                    max_iters=100, ext_score_adjustment=0.5, method='cos', constant=0.1)
```

Arguments

<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>rsem</code>	Boolean: If <code>True</code> , realistic standard error of measurement (RSEM), which takes into account item fit (Wright, 1996), is reported alongside the conditional standard error of measurement (CSEM). Default is <code>rsem=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>se</code>	Boolean: passed through to the internal <code>fit_statistics</code> call (only used if not already computed). If <code>False</code> , skips the bootstrap entirely – this table’s own columns (CSEM, RSEM, Infit/Outfit) do not depend on it, so <code>se=False</code> is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is <code>se=True</code> .
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's (1989) bias correction is applied during generation of person estimates (see Section 4.3.4). Default <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 4.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Arguments continue on the next page.

Arguments (continued)

<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 4.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 4.3.1). Default value is <code>constant=0.1</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 4.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 4.3.2). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .

Returns

Attribute `self.person_stats`, a pandas dataframe with one row for each person and the following columns:

<code>Estimate</code>	Item location estimate.
<code>CSEM</code>	Conditional standard error of measurement for person location estimate.
<code>RSEM</code>	Realistic standard error of measurement for person location estimate. Only produced if <code>rsem=True</code>
<code>Score</code>	Number of correct responses.
<code>Max score</code>	Maximum available score (number of items attempted).
<code>p</code>	Proportion of correct responses.
<code>Infit MS</code>	Infit mean square.
<code>Infit Z</code>	Standardised infit z-score.
<code>Outfit MS</code>	Outfit mean square.
<code>Outfit Z</code>	Standardised outfit z-score.

Examples

To produce a summary `person_stats` table with the most commonly reported statistics:

```
my_pcm.person_stats_df()
```

To produce a full `self.person_stats` table with all statistics:

```
my_pcm.person_stats_df(full=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

4.4.5 test_stats_df

Description

Produces a table of test-level statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or $\text{\LaTeX}$.

Usage

```
self.test_stats_df(dp=3, warm_corr=True, tolerance=0.00001, max_iters=100,  
                   ext_score_adjustment=0.5, method='cos', constant=0.1, alpha=False,  
                   seed=None)
```

Arguments

dp	Integer: Number of decimal places reported in output.
warm_corr	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 4.3.4). Default is <code>warm_corr=True</code> .
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is <code>tolerance=0.00001</code> .
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is <code>max_iters=100</code> .
ext_score_adjustment	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 4.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 4.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 4.3.1). Default value is <code>constant=0.1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>alpha</code>	Boolean: if <code>True</code> , adds a <code>Cronbach alpha</code> row to the <code>Persons</code> column of the returned table (the <code>Items</code> and <code>Thresholds</code> columns are left blank, since Cronbach's alpha (Cronbach, 1951) is a person-side reliability statistic). Computed on complete cases; if the data contain missing responses, a warning is raised and alpha is computed after listwise deletion, which may underestimate the true value. Default is <code>alpha=False</code> .
<code>seed</code>	Integer or <code>None</code> : seed passed through to the internal <code>fit_statistics</code> call (only used if not already computed). <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.test_stats`, a pandas dataframe with three columns, `Items`, `Thresholds` and `Persons` and rows for a range of descriptive statistics describing the distributions of the estimates and different statistics related to reliability – these statistics describe the suitability of the data for estimating and differentiating the parameters, rather than properties of the parameters themselves. The statistics are:

<code>Mean</code>	The mean of the estimates.
<code>SD</code>	The standard deviation of the estimates.
<code>Separation ratio</code>	The separation ratio (Wright, 1996; Wright & Masters, 1982), which is the standard deviation of person locations reported as a ratio of standard error units. For persons: $G_p = \sigma_p / \sqrt{\sum_n SE_n^2}$ where σ_p is the variance of the person estimates and SE_n is the RSEM (see Section 4.4.4) for person n. The formula is symmetrical for items, substituting the standard error of estimation for RSEM.
<code>Strata</code>	The number of statistically distinct levels of either person location or item location as strata with centers three measurement errors apart (Wright & Masters, 1982:106). For persons: $H_p = (4G_p + 1)/3$ with symmetrical results for items.
<code>Reliability</code>	A Rasch-specific reliability statistic (Wright, 1996), derived from PSI and which is a Rasch-specific reliability statistic similar to Cronbach's Alpha (Cronbach, 1951), and which may be interpreted the same way – as the proportion of variance of the estimates which stems from variation in location or location rather than estimation error. For persons: $R_p = G_p^2 / (1 + G_p^2)$ with symmetrical results for items.

Returns continue on the next page.

Returns (continued)

Cronbach alpha	Optional (<code>alpha=True</code>), <code>Persons</code> column only – classical Cronbach’s alpha (Cronbach, 1951), computed directly from the raw response data rather than derived from Rasch estimates. Complete cases only; a warning is raised if missing data are present, since alpha may then be an underestimate. Not computed by default (<code>alpha=False</code>).
----------------	--

Example

To produce a `my_pcm.test_stats` table:

```
my_pcm.test_stats_df()
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

4.4.6 model_selection

Description

Compares PCM against RSM – RSM is the constrained (nested) model, sharing one threshold structure across all items; PCM is the unconstrained model, giving each item its own threshold structure – via a likelihood ratio test, AIC, or BIC, to help decide which model is the more appropriate, parsimonious fit for a given dataset. Requires uniform maximum scores across items (a precondition for RSM itself to apply at all). Internally constructs and calibrates an RSM object from the same response data for comparison; auto-triggers `calibrate` (Section 4.3.1) and `person` (Section 4.3.4) on `self` directly if not already run.

Usage

```
self.model_selection(test='AIC', aic_sig_test=True, alpha=0.05, sampling='dynamic',
                    warm_corr=True, tolerance=0.00001, max_iters=100,
                    ext_score_adjustment=0.5, constant=0.1, method='cos',
                    log_lik_tol=0.000001, seed=None)
```

Arguments

<code>test</code>	String: 'LR', 'AIC', or 'BIC'. Default is <code>test='AIC'</code> .
<code>aic_sig_test</code>	Boolean: When <code>test='AIC'</code> , if <code>True</code> , applies a significance test with RSM as the null hypothesis. The p-value is the relative likelihood of RSM vs PCM, $p = e^{-\Delta/2}$ where $\Delta = \text{AIC}_{\text{RSM}} - \text{AIC}_{\text{PCM}}$. PCM is preferred only if $p < \alpha$; otherwise RSM is retained as the more parsimonious default. Default is <code>aic_sig_test=True</code> .
<code>alpha</code>	Float: significance level for <code>test='LR'</code> or <code>test='AIC'</code> with <code>aic_sig_test=True</code> . Default is <code>alpha=0.05</code> .
<code>sampling</code>	None, 'dynamic', or Integer: Subsamples T non-extreme persons before computing log-likelihoods: at very large N , the LR/AIC/BIC comparison becomes biased toward the more complex (PCM) model, since log-likelihood differences scale with N ; capping the effective N mitigates this. An integer defines T . 'dynamic' uses $T = \min(20(I - 1)(m - 1), 1500)$, where I is the number of items and m the shared maximum score; subsampling is skipped whenever the actual non-extreme sample size is $\leq T$. None applies no subsampling. Default is <code>sampling='dynamic'</code> .
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_adjustment</code>	Person location estimation keyword arguments, used only if <code>person</code> (Section 4.3.4) has not already been run on <code>self</code> . Always used for the internally-constructed RSM comparison object. See Section 4.3.4 for details of each.

Arguments continue on the next page.

Arguments (continued)

<code>constant, method, log_lik_tol</code>	Calibration keyword arguments, used only if <code>calibrate</code> (Section 4.3.1) has not already been run on <code>self</code> . Always used for the internally-constructed RSM comparison object. See Section 4.3.1 for details of each.
<code>seed</code>	Integer or <code>None</code> : seed for the person-subsampling random number generator, only used when <code>sampling</code> triggers. Pass an integer for a reproducible comparison; <code>None</code> (default) draws fresh entropy. See Section 1.

Returns

Sets whichever of the following attributes correspond to `test` (all named from the RSM-vs-PCM comparison regardless of which class this method is called from, since the comparison itself is symmetric):

<code>self.model_comparison_rsm_pcm_lr, df, _p, _lr_preferred, _lr_summary</code>	Likelihood ratio statistic, degrees of freedom, p-value, the preferred model as a string (RSM is the null; PCM is preferred if $p < \alpha$), and a one-line pandas Series summary (including a Preferred row). Only set if <code>test='LR'</code> .
<code>self.model_comparison_rsm_pcm_aic, _aic_preferred, _aic_summary</code>	AIC values for both models, the preferred model as a string, and a summary Series. Only set if <code>test='AIC'</code> .
<code>self.model_comparison_rsm_pcm_aic_p</code>	The AIC relative-likelihood p-value described above. Only set if <code>test='AIC'</code> and <code>aic_sig_test=True</code> .
<code>self.model_comparison_rsm_pcm_bic, _bic_preferred, _bic_summary</code>	BIC values for both models, the preferred model as a string, and a summary Series. Only set if <code>test='BIC'</code> .

Examples

To compare `my_pcm` against the equivalent RSM using the default AIC significance test:

```
my_pcm.model_selection()
my_pcm.model_comparison_rsm_pcm_aic_summary
```

To instead run a likelihood ratio test, with a fixed subsampling size of 800 persons to guard against large- N bias:

```
my_pcm.model_selection(test='LR', sampling=800)
```


4.4.7 andersen_lr_test

Description

Andersen's (1973) likelihood ratio test of parameter invariance (Andersen, 1973a): splits persons into two groups, fits the model separately in each group, and tests whether item threshold estimates are invariant across the groups. Groups can be formed either by a median split on person location or raw score (a general model-fit check, not tied to any particular grouping variable), or by an exogenous person covariate such as gender (Differential Item Functioning, or DIF – see Holland and Wainer, 1993), in which case this serves as an omnibus test of whether *any* item shows DIF with respect to that covariate, complementing the per-item and per-threshold tests in `dif_test` (Section 4.4.8).

Usage

```
self.andersen_lr_test(split_by='person_location', covariate=None, warm_corr=True,
                      tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                      constant=0.1, method='cos', log_lik_tol=0.000001)
```

Arguments

<code>split_by</code>	String, one of 'person_location', 'score' or 'exogenous': split criterion. 'person_location' splits on the median of person location estimates (Section 4.3.4). 'score' splits on the median of raw scores (a warning is issued if data are missing, since raw scores are then not comparable across persons). 'exogenous' splits by an external person covariate, requiring <code>covariate</code> and <code>self.exogenous</code> to be set (Section 4.1.1), and requires the covariate to have exactly two distinct non-null values – covariates with more than two levels are not supported here; use <code>dif_test</code> (Section 4.4.8) instead, which supports them directly. Default is <code>split_by='person_location'</code> .
<code>covariate</code>	String or <code>None</code> : column name in <code>self.exogenous</code> to split by. Required (and only used) when <code>split_by='exogenous'</code> . Persons with a missing value for this covariate are excluded from both groups and from the full-sample comparison fit, so the likelihood-ratio decomposition stays valid. Default is <code>covariate=None</code> .
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_</code> <code>adjustment</code>	Person-estimation arguments passed to the group models – see Section 4.3.4.
<code>constant</code> , <code>method</code> , <code>log_</code> <code>lik_tol</code>	Calibration arguments passed to the group models – see Section 4.3.1.

Returns

<code>self.andersen_lr</code>	Float: the likelihood ratio statistic.
<code>self.andersen_df</code>	Integer: degrees of freedom (total number of item thresholds across all items, minus 1).
<code>self.andersen_p</code>	Float: p-value from the chi-squared distribution. <code>self.andersen_lr</code> is floored at 0 (with a warning) if the raw statistic is negative, which can occur since PAIR/CPAT are approximate, not exact, estimation methods; a floored statistic indicates no evidence of misfit.
<code>self.andersen_groups</code>	Dictionary: <code>{group_name: PCM}</code> , the fitted group models, for inspection. Group names are 'low'/'high' for <code>split_by='person_location'/'score'</code> , or the two observed covariate values for <code>split_by='exogenous'</code> .
<code>self.andersen_summary</code>	pandas series: the LR statistic, degrees of freedom and p-value together.

Examples

To test parameter invariance by a median split on person location:

```
self.andersen_lr_test()
```

To test parameter invariance by a median split on raw score:

```
self.andersen_lr_test(split_by='score')
```

To test for DIF by an exogenous covariate 'Gender' (which must have exactly two levels), having supplied exogenous to the constructor:

```
self.andersen_lr_test(split_by='exogenous', covariate='Gender')
```

4.4.8 dif_test

Description

Differential Item Functioning (DIF) test by an exogenous person covariate (e.g. gender, first language) (Holland & Wainer, 1993). Splits non-extreme persons into groups by the named column in `self.exogenous`, designates one group as the reference (by default the largest), and compares each other (“focal”) group against it individually – a reference-group design, not all-pairwise, so covariates with more than two levels are supported directly (unlike `andersen_lr_test`, Section 4.4.7). Two independent components are tested:

1. **Item-location DIF.** Both groups are calibrated independently (each self-centred), then purified onto a common item scale via either a per-item Wald significance test (`selection_method='wald'`, default) or the robust modified z-score outlier-trimming procedure used by `calibrate_anchor` (Section 4.3.3; Iglewicz & Hoaglin, 1993) – the reference group’s central locations supply the “anchor” values, the focal group’s supply the “observed” values – so that genuine DIF items cannot contaminate the common scale used to test for DIF in the first place. Item-level DIF is then tested via a per-item Wald test (`test='wald'`, default), a per-item likelihood-ratio test (`test='lr'`), or both (`test='both'`), using bootstrap standard errors from `std.errors` (Section 4.3.2) for the Wald test. An omnibus LR test (Andersen-style: reference vs. focal, every common item jointly) runs by default (`omnibus=True`). Results in `self.dif_table` and `self.dif_omnibus_table`.
2. **Threshold-structure DIF** (differential category functioning, or DCF). PCM’s per-item centred threshold offsets (`self.thresholds`) describe each item’s own category structure relative to its own central location, not its location, so they are not translated by the item-scale purification above. Tested via each item’s category widths (consecutive threshold differences) rather than threshold locations – a change to one category width imputes changes in all threshold locations, potentially causing false positives elsewhere; widths isolate the effect to the category that actually changed. Each item is Wald-tested category by category, reference versus focal, using a bootstrap standard error of the width itself (differenced *within* each bootstrap resample before taking the standard deviation, so it already reflects the covariance between adjacent thresholds). Each item’s own category-width tests form their own multiple-comparison correction pool, independent of other items and of the item-location correction pool above. Not affected by `test/omnibus` – those only govern component (1). Results in `self.threshold_dif_table`.

An item or threshold cell is flagged only if it clears both a (corrected) p-value threshold and an absolute logit-difference threshold, the latter an ETS-style convention, so that a statistically significant but practically negligible difference is not flagged. `dif_test` also supports ETS-style A/B/C severity-category classification (Zwick et al., 1997) (`category`, `category_thresholds`, `category_alpha`) for the item-location component only – the 0.43/0.64 logit defaults are calibrated for item-location DIF specifically, so this does not extend to `self.threshold_dif_table` – as described in Table 19:

Category	Criteria	Interpretation
A	$ \text{Difference} < 0.43$, or fails the significance test below	Negligible DIF
B+ / B-	Both $ \text{Difference} \geq 0.43$ and $p(\text{DIF} = 0) < \text{category_alpha}$ (two-sided Wald/Welch test), but not C	Slight to moderate DIF
C+ / C-	Both $ \text{Difference} \geq 0.64$ and $p(\text{Difference} \leq 0.43) < \text{category_alpha}$ (one-sided test that the difference significantly exceeds the B boundary)	Moderate to large DIF

Table 19: ETS-style DIF category classification (item-location component only). Sign: + indicates that an item is harder for the focal group; - indicates that an item is easier for the focal group. The 0.43/0.64 logit thresholds are the Rasch-logit equivalents of the classic ETS Mantel-Haenszel delta-scale B/C boundaries (Tristan, 2006); the significance level (default `category_alpha=0.05`) is independent of `alpha`, which governs the separate `Flagged/Flagged_LR` columns.

Usage

```
self.dif_test(covariate, reference=None, selection_method='wald', corr_tol=0.95,
              sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05, test='wald',
              omnibus=True, omnibus_scope='item', welch=False, size_adjust=False,
              reference_n=100, correction='bh', alpha=0.05, logit_threshold=0.43,
              category=False, category_thresholds=(0.43, 0.64), category_alpha=0.05,
              no_of_samples=500, plot=False, plot_kwargs=None, warm_corr=True,
              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5, constant=0.1,
              method='cos', matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>covariate</code>	String: column name in <code>self.exogenous</code> to group persons by.
<code>reference</code>	String or <code>None</code> : value of <code>covariate</code> to use as the reference group. If <code>None</code> , the largest group is used. Default is <code>reference=None</code> .
<code>selection_method</code>	String, one of 'wald', 'robust_z' or 'none': how the common item-location scale is purified before testing (never the threshold-structure component) – see <code>calibrate_anchor</code> (Section 4.3.3) for what each option does. Default is <code>selection_method='wald'</code> .
<code>corr_tol</code> , <code>sd_ratio_tol</code> , <code>min_anchors</code> , <code>wald_alpha</code>	As for <code>calibrate_anchor</code> (Section 4.3.3), used only by the corresponding <code>selection_method</code> . Defaults are <code>corr_tol=0.95</code> , <code>sd_ratio_tol=1.1</code> , <code>min_anchors=6</code> , <code>wald_alpha=0.05</code> .

Arguments continue on the next page.

Arguments (continued)

<code>test</code>	String, one of 'wald', 'lr' or 'both': per-item item-location DIF test(s) to compute – affects only the item-location component, never <code>self.threshold_dif_table</code> . 'wald' (default) matches the original behaviour exactly, at no added cost. 'lr'/'both' additionally compute a per-item likelihood-ratio test (extra cost: two person location re-estimations per item per focal group). The existing <code>Flagged</code> column always reflects the Wald test; a separate <code>Flagged_LR</code> column is added when <code>test</code> is 'lr' or 'both'. Default is <code>test='wald'</code> .
<code>omnibus</code>	Boolean: if <code>True</code> (default), also runs an Andersen-style omnibus LR test (reference vs. focal, every common item jointly) per focal group for the item-location component, stored in <code>self.dif_omnibus_table</code> . Cheap relative to the per-item LR test – one extra combined-group model fit per focal group, not per item. See <code>omnibus_scope</code> below for how PCM's per-item threshold freedom is accounted for.
<code>omnibus_scope</code>	String, one of 'item' or 'full': unlike SLM (no thresholds) or RSM (one threshold vector shared by the whole test), PCM has a separate threshold/category structure per item, which otherwise adds unaccounted-for free parameters to the omnibus comparison. 'item' (default): an omnibus test of uniform DIF only, matching SLM/RSM's $df = k - 1$ scope – holds each item's category step-shape fixed at the pooled model's values and re-optimises only a single location shift per item, isolating exactly the item-location difference the df accounts for. 'full': a joint omnibus test of uniform DIF <i>and</i> differential category functioning together – widens df to $(k - 1) + \sum(\text{max_score} - 1)$ to properly charge for the full threshold freedom. A single joint p-value, not a decomposition: for a localised, independently-tested breakdown of the two components, use <code>self.dif_table</code> and <code>self.threshold_dif_table</code> directly. Default is <code>omnibus_scope='item'</code> .
<code>welch</code>	Boolean: if <code>True</code> , every Wald test here (item-location <i>and</i> threshold-structure) becomes a Welch's t-test: statistic unchanged (<code>Difference / SE</code>), but p-value computed against a t-distribution with Welch-Satterthwaite degrees of freedom instead of a normal distribution. Adds a <code>df</code> column to both <code>self.dif_table</code> and <code>self.threshold_dif_table</code> . Does not affect the LR test or omnibus test (already exact chi-squared tests). Default is <code>welch=False</code> .

Arguments continue on the next page.

Arguments (continued)

<code>size_adjust</code>	Boolean: if <code>True</code> , rescales each group’s own SE (item location <i>and</i> threshold-cell – same scope as <code>welch</code> , unlike <code>category</code> below) to what it would be at a standard reference sample size (Tristan, 2006) – the opposite problem from <code>welch</code> ’s: at very large N , SE shrinks enough that trivial differences become “significant”. Each group’s own SE is rescaled independently by $\sqrt{n_{\text{actual}}/\text{reference_n}}$ using that group’s own person count, then combined as usual. Applied to the Wald/Welch test, the category tests, and (if <code>welch=True</code>) the Welch-Satterthwaite df, which then also uses <code>reference_n</code> rather than each group’s actual n , for internal consistency. Default is <code>size_adjust=False</code> .
<code>reference_n</code>	Integer: reference sample size for <code>size_adjust=True</code> . 100 (the default) is Tristán’s own recommendation; around 60 is suggested there for closer alignment with the ETS Category B boundary specifically. Unused unless <code>size_adjust=True</code> .
<code>correction</code>	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction applied across items (within each focal-group comparison) and, separately, across every item $\times$ category cell (within each focal-group comparison) – Benjamini-Hochberg false discovery rate control (Benjamini & Hochberg, 1995) or Bonferroni. Default is <code>correction='bh'</code> .
<code>alpha</code>	Float: p-value threshold for flagging in both components (Wald, per-item LR, and omnibus alike), compared against the corrected p-value if <code>correction</code> is not None. Default is <code>alpha=0.05</code> .
<code>logit_threshold</code>	Float: absolute logit-difference threshold for flagging in both components (ETS-style convention). Default is <code>logit_threshold=0.43</code> .
<code>category</code>	Boolean: if <code>True</code> , adds an ETS-style <code>Category</code> column to <code>self.dif_table</code> (item-location component only – see Table 19 above). Default is <code>category=False</code> .
<code>category_thresholds</code>	Tuple of two floats: (B boundary, C boundary) in logits, per Zwick et al. (1997). Default is <code>category_thresholds=(0.43, 0.64)</code> .
<code>category_alpha</code>	Float: significance level for both category tests, independent of <code>alpha</code> . Default is <code>category_alpha=0.05</code> .
<code>no_of_samples</code>	Integer: bootstrap resamples for each group’s <code>std_errors</code> call (Section 4.3.2). Default is <code>no_of_samples=500</code> .

Arguments continue on the next page.

Arguments (continued)

<code>plot</code>	Boolean: if <code>True</code> , calls <code>plot_anchor_selection</code> (Section 3.5.9) for each focal group that has a selection table (<code>selection_method</code> in <code>'wald'/'robust_z'</code> only) and stores the resulting figures in <code>self.dif_plots</code> . Default is <code>plot=False</code> .
<code>plot_kwargs</code>	Dictionary or <code>None</code> : extra keyword arguments forwarded to <code>plot_anchor_selection</code> for every focal group when <code>plot=True</code> . Default is <code>plot_kwargs=None</code> .
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_</code> <code>adjustment</code>	Person-estimation arguments, passed through to the group models – see Section 4.3.4. Only used if <code>test</code> is <code>'lr'/'both'</code> or <code>omnibus=True</code> .
<code>constant</code> , <code>method</code> , <code>matrix_power</code> , <code>log_lik_</code> <code>tol</code>	Calibration arguments passed to the group models – see Section 4.3.1.
<code>seed</code>	Integer or <code>None</code> : seed passed through to each group’s internal <code>std_errors</code> call. <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

<code>self.dif_reference</code>	The reference group value.
<code>self.dif_covariate</code>	The covariate column name used.
<code>self.dif_reference_</code> <code>model</code>	PCM object: fitted model for the reference group.
<code>self.dif_focal_models</code>	Dictionary: <code>{focal_value: PCM}</code> , fitted models for each focal group.
<code>self.dif_tc</code>	Dictionary: <code>{focal_value: float}</code> , the item-scale translation constant applied to each focal group.
<code>self.dif_anchor_</code> <code>selection</code>	Dictionary: <code>{focal_value: dataframe or None}</code> , per-item selection diagnostics for each focal group (<code>None</code> if <code>selection_method='none'</code>).
<code>self.dif_plots</code>	Dictionary: <code>{focal_value: figure or None}</code> , the <code>plot_anchor_selection</code> figure for each focal group, if <code>plot=True</code> and a selection table exists for that group.
<code>self.dif_group_sizes</code>	Dictionary: <code>{group_value: int}</code> , non-extreme, non-missing-covariate N used for each group.

The two tested components additionally set:

<code>self.dif_table</code>	pandas dataframe: one row per (item, focal group) pair, indexed by item. Columns: Group, Reference/Focal/Focal (purified) (central location estimates), Difference, SE, z, p, p (corrected), Selected and Flagged. If <code>welch=True</code> , also <code>df</code> . If test is 'lr' or 'both', also LR, p_LR, p_LR (corrected), Flagged_LR. If <code>category=True</code> , also Category.
<code>self.dif_omnibus_table</code>	pandas dataframe or None: one row per focal group – LR, df, p, Flagged. Item-location component only if <code>omnibus_scope='item'</code> (default); item location and threshold/category-structure jointly if <code>omnibus_scope='full'</code> . None if <code>omnibus=False</code> .
<code>self.threshold_dif_table</code>	pandas dataframe: differential category functioning, tested via category widths rather than threshold locations. One row per (item, category, focal group), indexed by (Item, Category) where category k is the width between threshold k and $k+1$. Columns: Group, Reference/Focal (category widths), Difference, SE (bootstrap SE of the width itself), z, p, p (corrected), Flagged – and <code>df</code> if <code>welch=True</code> – corrected independently of <code>self.dif_table</code> , per item. Items whose maximum score differs between the reference and a focal group, or whose maximum score is 1 (no category width to test), are excluded from this table for that group, the former with a warning.

Examples

To test for DIF by an exogenous covariate 'L1' (which may have any number of levels), having supplied `exogenous` to the constructor, using the largest level as the reference group by default:

```
self.dif_test('L1')
```

To inspect the threshold-structure DIF results for a specific item 'Item_3' after running the test:

```
self.threshold_dif_table.loc['Item_3']
```

To run the joint omnibus test of uniform DIF and differential category functioning together, rather than uniform DIF alone:

```
self.dif_test('L1', omnibus_scope='full')
```


4.4.9 res_corr_analysis

Description

Analysis of correlations of standardised residuals to test for violations of local item interdependence and unidimensionality requirements.

Usage

```
self.res_corr_analysis(warm_corr=True, tolerance=0.00001, max_iters=100,  
                       ext_score_adjustment=0.5, constant=0.1, method='cos',  
                       matrix_power=3, log_lik_tol=0.000001, se=True)
```

Arguments

warm_corr	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 4.3.4). Default is <code>warm_corr=True</code> .
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is <code>tolerance=0.00001</code> .
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is <code>max_iters=100</code> .
ext_score_adjustment	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 4.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates (see Section 4.3.1). Default value is <code>constant=0.1</code> .
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of central item location estimates (see Section 4.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
matrix_power	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of central item location estimates (see Section 4.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).

Arguments continue on the next page.

Arguments (continued)

<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 4.3.1). Ignored for other methods.
<code>se</code>	Boolean: passed through to the internal <code>fit_statistics</code> call (only used if not already computed). If <code>False</code> , skips the bootstrap entirely – this analysis’s own output (residual correlations, PCA) does not depend on it, so <code>se=False</code> is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is <code>se=True</code> .

Returns

For tests of violation of the requirement for local item independence (Andrich & Kreiner, 2010; Marais, 2012):

<code>self.residual_correlations</code>	A pandas dataframe of pairwise correlations between item standardised residuals.
---	--

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

<code>self.eigenvectors</code>	The eigenvectors of the standardised residual correlations matrix.
<code>self.eigenvalues</code>	The eigenvalues corresponding to the eigenvectors.
<code>self.variance_explained</code>	The variance explained by each principal component.
<code>self.loadings</code>	The loading of each item onto each of the principal components, for the the first of which large loadings ('large' typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce a residual correlation analysis:

```
my_pcm.res_corr_analysis()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

4.4.10 save_stats

Description

Exports `item_stats`, `threshold_stats` (both centred and uncentred), `person_stats` and `test_stats` to file in a single call. Auto-triggers `item_stats_df`, `threshold_stats_df`, `person_stats_df` and `test_stats_df` (Sections 4.4.1, 4.4.2, 4.4.4, 4.4.5) if any have not already been run – with their default arguments plus whichever of `dp/warm_corr/etc.` below are passed, so call those methods directly first (e.g. with `full=True`) if non-default table columns are wanted in the saved output. Does not include `category_stats` (Section 4.4.3) – save that table separately with `self.category_stats.to_csv(...)` if needed.

Usage

```
self.save_stats(filename, format='csv', dp=3, warm_corr=True, tolerance=0.00001,
                max_iters=100, ext_score_adjustment=0.5, method='cos', constant=0.1,
                no_of_samples=500, interval=None)
```

Arguments

<code>filename</code>	String: output filename or path. For <code>format='csv'</code> , used as a prefix (any trailing <code>.csv</code> is stripped first) for five files, <code>_item_stats.csv</code> , <code>_threshold_stats_uncentred.csv</code> , <code>_threshold_stats_centred.csv</code> , <code>_person_stats.csv</code> and <code>_test_stats.csv</code> . For <code>format='xlsx'</code> , used as a single workbook filename (<code>.xlsx</code> appended automatically if not already present), with the five tables on separate sheets.
<code>format</code>	String: <code>'csv'</code> or <code>'xlsx'</code> . Default is <code>format='csv'</code> .
<code>dp</code> , <code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_adjustment</code> , <code>method</code> , <code>constant</code> , <code>no_of_samples</code> , <code>interval</code>	Passed through to <code>item_stats_df/threshold_stats_df/person_stats_df/test_stats_df</code> (Sections 4.4.1, 4.4.2, 4.4.4, 4.4.5), used only for whichever of those tables has not already been computed. See those sections for details of each.

Returns

None. Writes the five tables to file as described above.

Example

To save item, threshold, person and test statistics as five CSV files prefixed `my_pcm_results`:

```
my_pcm.save_stats('my_pcm_results')
```

To save all tables to separate sheets of a single Excel workbook:

```
my_pcm.save_stats('my_pcm_results', format='xlsx')
```

4.4.11 save_residuals

Description

Exports the residual correlation analysis (`self.eigenvectors`, `self.eigenvalues`, `self.variance_explained` and `self.loadings`) to file. Auto-triggers `res_corr_analysis` (Section 4.4.9) if not already run.

Usage

```
self.save_residuals(filename, format='csv', single=True, dp=3, warm_corr=True,
                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                    method='cos', constant=0.1)
```

Arguments

filename	String: output filename or path (extension appended automatically if not already present).
format	String: 'csv' or 'xlsx'. Default is format='csv'.
single	Boolean: if True, writes all four tables (eigenvectors, eigenvalues, variance explained, loadings) into a single file – sequentially, separated by blank lines, for format='csv'; at successive row offsets on one sheet ('Item residual analysis'), for format='xlsx'. If False, writes each table to its own file (csv) or sheet (xlsx). Default is single=True.
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant	dp controls rounding of the saved values; the rest are passed through to <code>res_corr_analysis</code> (Section 4.4.9), used only if it has not already been run. See that section for details of each.

Returns

None. Writes the residual correlation analysis tables to file as described above.

Example

To save the residual correlation analysis as a single CSV file:

```
my_pcm.save_residuals('my_pcm_residuals')
```

To save each table to its own sheet of a single Excel workbook:

```
my_pcm.save_residuals('my_pcm_residuals', format='xlsx', single=False)
```

4.4.12 `category_counts_df`

Description

Produces a table of counts of scores in each category, plus total and missing responses. Response categories run from 0 to the highest maximum score among the included items; items whose own maximum score is lower show blank cells (not 0) for the categories above it, to avoid implying those categories exist for that item.

Usage

```
self.category_counts_df(persons=None, items=None, counts_name=None)
```

Arguments

<code>persons</code>	List or String: Person or subset of persons over which to count responses. Default <code>persons=None</code> , which returns counts for all persons.
<code>items</code>	List or String: Item or subset of items over which to count responses. Default <code>items=None</code> , which returns counts for all items.
<code>counts_name</code>	String or Int: Name under which to store counts on a subset of persons or items so multiple sets of counts can be made available. Useful if e.g. user wants to compare counts between different subgroups of persons according to a criterion such as an exogenous variable. If passed, stores counts within a namespace, <code>self.counts</code> . Default <code>counts_name=None</code> , when counts are returned as a simple attribute, <code>self.category_counts_table</code> .

Returns

If `counts_name=None`:

Attribute `self.category_counts_table`, a pandas dataframe of category counts with one row per item and one column per response category (0 to the highest included max score, blank above an item's own max score), plus total and missing responses per item, and grand total row.

If `counts_name=my_counts_name`:

Attribute `self.counts.my_counts_name` stored in the namespace `self.counts`, a pandas dataframe of category counts with one row per item and one column per response category (0 to the highest included max score, blank above an item's own max score), plus total and missing responses per item and grand total row.

Example

To produce a dataframe of category counts for all persons and items:

```
my_pcm.category_counts_df()
```

To produce a dataframe of category counts for a subset of persons saved as a list `my_persons`, with a matching attribute name within the namespace `self.counts`:

```
my_pcm.category_counts_df(persons=my_persons, counts_name=my_persons)
```

4.5 Plotting functionality

4.5.1 Shared plotting arguments

All the plotting methods described in this section share a set of arguments which may be used to customise the appearance of the plot or save the plot to file automatically. These arguments are:

<code>title</code>	String: Title for the plot, to appear in the image. Default is <code>title=None</code> .
<code>xmin</code>	Float: Minimum displayed point on x-axis, in logits. Default is <code>xmin=-5</code> .
<code>xmax</code>	Float: Maximum displayed point on x-axis, in logits. Default is <code>xmax=5</code> .
<code>plot_style</code>	String: Plot style to use. Available styles are 'white', producing a plot on a white background, and 'dark', producing a plot on a grey background. The styles correspond to Seaborn (Waskom, 2021) styles <code>whitegrid</code> and <code>darkgrid</code> . Default is <code>plot_style='white'</code> .
<code>palette</code>	String: Controls colours of lines and overplotted class intervals dots. Options are 'dark blue', 'light blue', 'dark red', 'light red', 'dark green', 'light green', 'dark grey', 'light grey', 'dark multi' and 'light multi'. Default is <code>palette='dark blue'</code> . Not used for <code>std_residual_plot</code> .
<code>black</code>	Boolean: If <code>True</code> , the plot will be rendered in black and white. Default is <code>black=False</code> .
<code>font</code>	String: The font to use in the plot. Default is <code>font='Times New Roman'</code> .
<code>title_font_size</code>	Float: The size of the title font in points. Default is <code>title_font_size=15</code> .
<code>axis_font_size</code>	Float: The size of the axis label font in points. Default is <code>axis_font_size=12</code> .
<code>labelsize</code>	Float: The size of the axis tick label font in points. Default is <code>labelsize=12</code> .
<code>filename</code>	String: The filename for the saved plot, with no suffix for format. If <code>None</code> , no file will be saved. Default is <code>filename=None</code> .
<code>file_format</code>	The format of the file: <code>png</code> , <code>jpg</code> or <code>svg</code> . Default is <code>file_format=png</code> .
<code>dpi</code>	The resolution of the plot in dpi (dots per inch) – higher resolution plots are better quality but have larger file sizes. Default is <code>dpi=300</code> .

4.5.2 `icc`

Description

Plots the item characteristic curves (or item response function) for an item: person location on the x-axis against expected score on the y-axis. Options to plot observed responses, item threshold line and lines showing locations corresponding to specified expected scores, and to highlight a specified response category.

Usage

```
self.icc(item, obs=False, no_of_classes=5, thresh_lines=False, central_location=False,  
         score_lines=None, score_labels=False, cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot.
<code>obs</code>	Boolean: If <code>True</code> , mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is <code>obs=False</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is <code>central_location=False</code> .
<code>score_lines</code>	List of floats between 0 and the item's maximum score: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score.
<code>score_labels</code>	Boolean: If <code>True</code> , scores and locations corresponding to arguments passed to <code>score_lines</code> will be labelled on the plot. Default is <code>score_labels=False</code> .
<code>cat_highlight</code>	Integer: Passing a score between 0 and the item's maximum score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item characteristic curve for `Item_1` and store the output as a variable `my_icc_plot` and save it to file as `my_icc_plot.png`:

```
my_icc_plot = my_pcm.icc('Item_1', filename=my_icc_plot)
```

To plot an item characteristic curve for `Item_1` with observed responses for 8 response classes and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc('Item_1', obs=True, no_of_classes=8)
```

To plot an item characteristic curve for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_pcm.icc('Item_1', thresh_lines=True, cat_highlight=0)
```

To plot an item characteristic curve for `Item_1` with lines showing the locations corresponding to expected scores of 0.7 and 1.6, with the expected score and corresponding location labelled, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_pcm.icc('Item_1', score_lines=[0.7, 1.6], score_labels=True)
```


4.5.3 crcs

Description

Plots category response curves for an item: person location on the x-axis against the probability of obtaining a score in each category, from 0 to the item's maximum score, on the y-axis. Options to plot observed proportions and item threshold line, and to highlight a specified response category.

Usage

```
self.crcs(item, obs=None, no_of_classes=5, thresh_lines=False, central_location=False,
          cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot.
<code>obs</code>	List: List of integers between 0 and <code>self.max_score</code> . For each value, mean observed proportions in each ordered response category scoring in that category are plotted against the mean location of the corresponding response class.
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is <code>central_location=False</code> .
<code>cat_highlight</code>	Integer: Passing a score between 0 and <code>self.max_score</code> will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot basic category response curves for `Item_1` and store the output as a variable `my_crcs_plot` and save it to file as `my_crcs_plot.png`:

```
my_crcs_plot = my_pcm.crcs('Item_1', filename=my_crcs_plot)
```

To plot category response curves for `Item_1` with observed response proportions for category 0 for 8 response classes and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = my_pcm.crcs('Item_1', obs=[0], no_of_classes=8)
```

To plot category response curves for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = my_pcm.crcs('Item_1', thresh_lines=True, cat_highlight=0)
```

4.5.4 threshold_ccs

Description

Plots the threshold characteristic curves (or threshold response functions) for an item. For threshold τ_k , $k \in \{1, \dots, \text{self.max_score}\}$, the threshold characteristic curve is the probability of obtaining a score of k rather than $k - 1$, conditional on the score being either $k - 1$ rather than k , for a given person location. Each threshold characteristic curve functions as a dichotomous item characteristic curve under the SLM (see Sections 3.2.2 and 3.5.2).

For threshold τ_k , `threshold_ccs` plots person location on the x-axis against the probability of obtaining a score of k on the y-axis. Options to plot threshold lines and central item locations, and to highlight a specified response category.

Usage

```
self.threshold_ccs(item, obs=None, no_of_classes=5, thresh_lines=False, central_location=False,
                  cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot.
<code>obs</code>	List: List of integers corresponding to thresholds τ_1 to τ_m , where $m = \text{self.max_score}$, or 'all' or 'none'. If <code>obs=[k]</code> , mean proportions of persons obtaining a score of k rather than $k - 1$, conditional on the score being either $k - 1$ or k , for each of the ordered response categories, will be plotted against the mean location of the corresponding response class. Multiple thresholds may be passed. Default is <code>obs=None</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is <code>central_location=False</code> .
<code>cat_highlight</code>	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot basic threshold characteristics curves for `Item_1`, store the output as variable `my_threshold_ccs_plot`, and save it to file as `my_threshold_ccs_plot.png`:

```
my_threshold_ccs_plot = my_pcm.threshold_ccs(item='Item_1', filename='my_threshold_ccs_plot')
```

To plot threshold characteristics curves for `Item_1` with threshold lines and category 1 highlighted, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = my_pcm.threshold_ccs(item='Item_1', thresh_lines=True, cat_highlight=1)
```

To plot threshold characteristics curves for `Item_1` with observed responses plotted for thresholds 2 and 4 and central item location line, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = my_pcm.threshold_ccs(item='Item_1', obs=[2, 4], central_location=True)
```

4.5.5 iic

Description

Plots the item information curve for an item: person location on the x-axis against Fisher information on the y-axis. Options to plot item threshold line and lines showing Fisher information corresponding to specified locations, and to highlight a specified response category.

Usage

```
self.iic(item, ymax=None, thresh_lines=False, central_location=False, point_info_lines=None,
         point_info_labels=False, cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot.
<code>ymax</code>	Float: The maximum value to show on the y-axis. If <code>None</code> , will infer, plotting a maximum of 1.1 times the maximum item information. Default is <code>ymax=None</code>
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is <code>central_location=False</code> .
<code>point_info_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the Fisher information corresponding to the location. Default is <code>point_info_lines=None</code> .
<code>point_info_labels</code>	Boolean: If <code>True</code> , locations and Fisher information corresponding to arguments passed to <code>point_info_lines</code> will be labelled on the plot. Default is <code>point_info_labels=False</code> .
<code>cat_highlight</code>	Integer: Passing a score between 0 and <code>self.max_score</code> will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).
<code>ymax</code>	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item information curve for `Item_1` and store the output as a variable `my_iic_plot` and save it to file as `my_iic_plot.png`:

```
my_iic_plot = my_pcm.iic('Item_1', filename='my_iic_plot')
```

To plot an item information curve for `Item_1` with threshold lines, central item location line and category 1 highlighted, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = my_pcm.iic('Item_1', thresh_lines=True, central_location=True, cat_highlight=1)
```

To plot an item information curve for `Item_1` with lines showing the Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding Fisher information labelled, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = my_pcm.iic('Item_1', point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

4.5.6 tcc

Description

Plots the test characteristic curve (or test response function) for a set of items: person location on the x-axis against expected score on the y-axis. Options to plot observed responses and lines showing locations corresponding to specified expected scores.

Usage

```
self.tcc(items=None, obs=False, no_of_classes=5, score_lines=None, score_labels=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>obs</code>	Boolean: If <code>True</code> , mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is <code>obs=False</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>score_lines</code>	List of floats between 0 and <code>self.max_score</code> : Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is <code>score_lines=None</code> .
<code>score_labels</code>	Boolean: If <code>True</code> , scores and locations corresponding to arguments passed to <code>score_lines</code> will be labelled on the plot. Default is <code>score_labels=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic test characteristic curve for all items and store the output as a variable `my_tcc_plot` and save it to file as `my_tcc_plot.png`:

```
my_tcc_plot = my_pcm.tcc(filename=my_tcc_plot)
```

To plot a test characteristic curve for `Item_1` for a subset of items stored as a list `my_item_list`, with observed responses for 8 response classes and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = my_pcm.tcc(obs=True, no_of_classes=8)
```

To plot a test characteristic curve for `Item_1` for all items with lines showing the locations corresponding to expected scores of 13 and 20, with the expected score and corresponding location labelled, and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = my_pcm.tcc(score_lines=[13, 20], score_labels=True)
```


4.5.7 test_info

Description

Plots the test information curve: person location on the x-axis against total Fisher information on the y-axis. Option to plot lines showing Fisher information corresponding to specified locations.

Usage

```
self.test_info(items=None, ymax=None, point_info_lines=None, point_info_labels=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>point_info_lines</code>	List of floats: Each float is a location for which a vertical line is plotted from the x-axis to its intersection with the item information curve; from there a horizontal line is plotted to the y-axis to show the total Fisher information corresponding to the location. Default <code>point_info_lines=None</code> .
<code>point_info_labels</code>	Boolean: If <code>True</code> , locations and total Fisher information corresponding to arguments passed to <code>point_info_lines</code> will be labelled on the plot. Default is <code>point_info_labels=False</code> .
<code>ymax</code>	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic test information curve and store the output as a variable `my_test_info_plot` and save it to file as `my_test_info_plot.png`:

```
my_test_info_plot = my_pcm.test_info(filename='my_test_info_plot')
```

To plot a test information curve for a subset of items stored as a list `my_item_list` and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = my_pcm.test_info(items=my_item_list)
```

To plot a test information curve with lines showing the total Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding total Fisher information labelled, and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = my_pcm.test_info(point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

4.5.8 test_csem

Description

Plots the test conditional standard error of measurement (CSEM) curve: person location on the x-axis against CSEM (in logits) on the y-axis. Option to plot lines showing CSEM corresponding to specified locations.

Usage

```
self.test_csem(items=None, point_csem_lines=None, point_csem_labels=False, ymax=5)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>point_csem_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the CSEM curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the CSEM corresponding to the location. Default is <code>point_csem_lines=None</code> .
<code>point_csem_labels</code>	Boolean: If <code>True</code> , locations and CSEM corresponding to arguments passed to <code>point_csem_lines</code> will be labelled on the plot. Default is <code>point_csem_labels=False</code> .
<code>ymax</code>	The maximum point displayed on the y-axis, in logits.

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic CSEM curve and store the output as a variable `my_test_csem_plot` and save it to file as `my_test_csem_plot.png`:

```
my_test_csem_plot = my_pcm.test_csem(filename='my_test_csem_plot')
```

To plot a CSEM curve for a subset of items stored as a list `my_item_list` and store the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = my_pcm.test_csem(items=my_item_list)
```

To plot a CSEM curve with lines showing the CSEM corresponding to locations of -0.3 and 0.7, with the location and corresponding CSEM labelled, and store the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = my_pcm.test_csem(point_csem_lines=[-0.3, 0.7], point_csem_labels=True)
```

4.5.9 std_residuals_plot

Description

Plots histogram of standardised residuals, with optional overplotting of standard Normal distribution.

Usage

```
self.std_residuals_plot(items=None, bin_width=0.5, normal=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>bin_width</code>	Float: The width of the histogram bins along the x-axis. Default is <code>bin_width=0.5</code> .
<code>normal</code>	Boolean: If <code>True</code> , plots a standard normal distribution over the standardised residual histogram for comparison. Default is <code>normal=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot and display a basic standardised residuals histogram and save it to file as `my_std_residuals_plot.png`:

```
my_pcm.std_residuals_plot(filename='my_std_residuals_plot')
```

To plot and display a standardised residuals histogram with bin width 1, with standard normal curve:

```
self.std_residuals_plot(bin_width=1, normal=True)
```

To plot and display a standardised residuals histogram on a subset of items stored as a list in a variable `my_item_list`:

```
my_pcm.std_residuals_plot(items=my_item_list)
```

4.5.10 `plot_anchor_selection`

Description

Scatter plot of anchor item locations (x-axis) against observed values for the same items (y-axis) to visualise the outlier-trimming diagnostics produced by `calibrate_anchor` (Section 4.3.3) or `dif_test` (Section 4.4.8). Selected and dropped items are plotted in different colours, with a solid $y = x$ identity line for reference and a dotted OLS best-fit line through the selected points only. `plot_anchor_selection` does not use the shared plotting arguments described in Section 4.5.1; its own, smaller set of arguments is described in full below.

Usage

```
self.plot_anchor_selection(result, xmin=-4, xmax=4, ymin=-3, ymax=3, figsize=(8, 6),
                           font='Times New Roman', title_font_size=15, axis_font_size=12,
                           labels_size=12, title='', dpi=300, filename=None,
                           file_format='png')
```

Arguments

<code>result</code>	dict or pandas dataframe: Either the full dictionary returned internally by the robust anchor-selection routine, or just its 'table', i.e. one of <code>self.anchor_selection</code> (from <code>calibrate_anchor</code>) or <code>self.dif_anchor_selection</code> (from <code>dif_test</code>), each a dataframe with 'Anchor', 'Observed' and boolean 'Selected' columns.
<code>xmin, xmax, ymin, ymax</code>	Floats: axis limits. Defaults are <code>xmin=-4, xmax=4, ymin=-3, ymax=3</code> .
<code>figsize</code>	Tuple of two floats: figure size in inches (width, height). Default is <code>figsize=(8, 6)</code> .
<code>font</code>	String: the font to use in the plot. Default is <code>font='Times New Roman'</code> .
<code>title_font_size,</code> <code>axis_font_size,</code> <code>labels_size</code>	Integers: font sizes in points for the title, axis labels and axis tick labels respectively. Defaults are <code>title_font_size=15, axis_font_size=12, labels_size=12</code> .
<code>title</code>	String: plot title. Default is <code>title=''</code> (no title).
<code>filename</code>	String or None: if provided, saves the plot to this path (with the suffix determined by <code>file_format</code>). Default is <code>filename=None</code> (not saved).
<code>file_format</code>	String: file format for saved plots, e.g. 'png', 'pdf', 'svg'. Default is <code>file_format='png'</code> .
<code>dpi</code>	Integer: output resolution in dpi (dots per inch) when saving to file. Default is <code>dpi=300</code> .

Returns

matplotlib figure object. If a `filename` argument is passed, also saves the image to file. `calibrate_anchor` and `dif_test` both call this method automatically when their own `plot` argument is `True` (see Sections 4.3.3 and 4.4.8), so it need not usually be called directly; it is documented separately here since the resulting figure objects (`self.anchor_plot`, `self.dif_plots`) can also be regenerated with different display arguments by calling it directly on the stored selection table.

Example

To regenerate the anchor-selection plot for an existing calibration's diagnostics, stored in `self.anchor_selection`, with wider axis limits and a custom title:

```
self.plot_anchor_selection(self.anchor_selection, xmin=-6, xmax=6, ymin=-6, ymax=6,  
                           title='Anchor calibration')
```

5 class RSM

5.1 Preliminaries

5.1.1 RSM

Description

Creates an object of the class `RSM` from a pandas dataframe of polytomously scored data of items which share the same maximum score and Rasch-Andrich threshold structure for analysis. No analysis can be run until an object is created.

Usage

```
RSM(responses, max_score=None, extreme_persons=True, no_of_classes=5,  
    validate=True, exogenous=None)
```

Arguments

<code>responses</code>	pandas dataframe with items as columns (item names as column names) and persons as index (person names as row names).
<code>max_score</code>	Integer: The maximum possible score, shared across all items. If no score is passed, <code>max_score</code> will be inferred from the data, although passing an argument is recommended. Default is <code>max_score=None</code> .
<code>extreme_persons</code>	Boolean: if <code>False</code> , all persons with extreme scores (all responses correct or all responses incorrect) are removed from the response dataframe. Default is <code>extreme_persons=True</code> .
<code>no_of_classes</code>	Integer: the number of classes of persons grouped by location for overplotting observed responses on theoretical curves. Default is <code>no_of_classes=5</code>
<code>validate</code>	Boolean: if <code>True</code> , automatically calls <code>check_data_connectivity</code> (Section 5.1.2) on creation and issues a <code>UserWarning</code> if the item response network is disconnected, or if any item has a structurally unresolvable zero in the pairwise calibration matrix. Default is <code>validate=True</code> .
<code>exogenous</code>	pandas dataframe or <code>None</code> : optional person-level covariates for Differential Item Functioning (DIF) analysis (Section 5.4.8). See the equivalent argument on the <code>SLM</code> constructor (Section 3.1.1) for full details – behaviour is identical. Default is <code>exogenous=None</code> .

Returns

Object of class `RSM`. Analyses are run using methods defined on the `RSM` object, with results stored as attributes of the `RSM` object.

Several attributes of object `RSM` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: Dataframe of valid responses.
<code>self.invalid_responses</code>	pandas dataframe: Dataframe of invalid responses (persons with no responses to any items, i.e. all missing data).
<code>self.extreme_persons</code>	pandas dataframe: Dataframe of persons with extreme scores (all-correct or all-incorrect). When <code>extreme_persons=True</code> (the default), extreme scorers are retained in <code>self.responses</code> and this attribute is empty; when <code>extreme_persons=False</code> , they are removed and stored here.
<code>self.max_score</code>	Integer: The maximum possible score, shared across all items.
<code>self.no_of_items</code>	Integer: Number of items.
<code>self.item_names</code> <code>self.item_ids</code>	List: List of item names. <code>self.item_ids</code> is an alias for users who find <code>*_ids</code> more natural than <code>*_names</code> .
<code>self.no_of_persons</code>	Integer: Number of persons.
<code>self.person_names</code> <code>self.person_ids</code>	List: List of person names. <code>self.person_ids</code> is an alias for users who find <code>*_ids</code> more natural than <code>*_names</code> .
<code>self.generating</code>	Namespace object (<code>_SimParams</code>): Only present when the model is instantiated by passing a simulation object (e.g. <code>RSM_Sim</code>) as the <code>responses</code> argument. Contains all attributes of the simulation object (e.g. <code>self.generating.persons</code> , <code>self.generating.items</code> , <code>self.generating.thresholds</code>), enabling direct comparison of generating and estimated parameters.
<code>self.connectivity_status</code>	Dictionary: result of <code>check_data_connectivity</code> (Section 5.1.2), present only when <code>validate=True</code> . Always contains at least 'connected' (Boolean) and 'components_count' (Integer).
<code>self.exogenous</code>	pandas dataframe or None: person-level covariates passed via the <code>exogenous</code> argument, re-indexed onto <code>self.person_names</code> . None if not supplied.
<code>self.no_exogenous_persons</code>	pandas index: Persons present in the response data with no matching exogenous record.
<code>self.exogenous_only_persons</code>	pandas index: Persons present in the exogenous data but not in the response data.

Example

To create an object from a dataframe `my_rsm_dataframe`, with a maximum score of 5 and 10 observed classes:

```
my_rsm = RSM(my_rsm_dataframe, max_score=5, no_of_classes=10)
```

To create an object from a dataframe `my_rsm_dataframe`, additionally carrying exogenous covariates for DIF analysis loaded into `my_exogenous_df`:

```
my_rsm = RSM(my_rsm_dataframe, max_score=5, exogenous=my_exogenous_df)
```

5.1.2 `check_data_connectivity`

Description

Validates whether the item response network is fully connected, in the same sense as SLM's `check_data_connectivity` (Section 3.1.2): two items are connected if at least one person's response differentiates between them. For RSM, the underlying directed matrix is one row per item, with entry (i, j) counting persons who scored exactly one point higher on item i than item j – matching the item-location stage of `calibrate` (Section 5.3.1) exactly, since items share a single threshold structure and so need no category-level breakdown.

Called automatically on object creation when `validate=True` (the default; see Section 5.1.1), storing its result as `self.connectivity_status`.

Usage

```
self.check_data_connectivity()
```

Returns

Dictionary: `'connected'`, `'components_count'`, `'directionally_isolated_items'` always present; if disconnected, `'isolated_items'/'all_sub_groups'` also present.

Example

To manually re-check connectivity on object `my_rsm` and inspect the number of connected components:

```
my_rsm.connectivity_status = my_rsm.check_data_connectivity()
my_rsm.connectivity_status['components_count']
```


5.1.3 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_rsm` from `Item_1` to `my_new_item_name`:

```
my_rsm.rename_item('Item_1', 'my_new_item_name')
```

5.1.4 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_rsm` with item names in a list stored as a variable `my_new_item_names`:

```
my_rsm.rename_items_all(my_new_item_names)
```

5.1.5 `rename_person`

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

<code>old</code>	String: the old name for the person
<code>new</code>	String: the new name for the person

Returns

Replaces specified person name in the index of `self.responses` with new name. Also replaces person name in `self.exogenous` or `self.no_exogenous_persons` as appropriate if `self.exogenous` exists.

Example

To rename a person in object `my_rsm` from `Person_1` to `my_new_person_name`:

```
my_rsm.rename_person('Person_1', 'my_new_person_name')
```

5.1.6 `rename_persons_all`

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

<code>new_names</code>	List of new person names as strings
------------------------	-------------------------------------

Returns

Replaces all person names in the index of `self.responses` with new names. Also replaces all person names in `self.exogenous` and `self.no_exogenous_persons` if `self.exogenous` exists.

Example

To rename all persons in object `my_rsm` with person names in a list stored as a variable `my_new_person_names`:

```
my_rsm.rename_persons_all(my_new_person_names)
```

5.2 Core functions

5.2.1 cat_prob

Description

Category probability function which calculates the probability $P(X_{ni} = k)$ of scoring k , with $k \in \{0, m\}$, where m is the maximum score, from person location, central item location and Rasch-Andrich thresholds. For a person n with location β_n attempting an item i with central item location δ_i and Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$, the probability of obtaining a score of k is given by:

$$P(X_{ni} = k) = \frac{e^{k(\beta_n - \delta_i) - \sum_{t=0}^k \tau_t}}{\sum_{k=0}^m e^{k(\beta_n - \delta_i) - \sum_{t=0}^k \tau_t}} \quad (15)$$

In this formulation, an item is defined by a central item location, δ_i and a set of centred Rasch-Andrich thresholds $\{\tau_k\}$, $k \in \{1, \dots, m\}$ which sum to zero, plus a dummy/sentinel threshold $\tau_t = 0$ for the convenience of Equation 15. An alternative formulation would be to define the item solely by m uncentred thresholds, $\{\tau'_{ik}\}$, $k \in \{1, \dots, m\}$, where $\tau'_{ik} = \delta_i + \tau_k$, in analogy with the partial credit model formulation described in Section 4.2.2.

Usage

```
self.cat_prob(person_location, item_location, category, thresholds)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location
category	Integer: Response category k , with $k \in \{0, 1\}$.
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Returns

Float: probability of obtaining score k .

Example

To obtain the probability of a person of location 0.5 scoring 0 on an item with central location 0 and a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_cat_prob`:

```
my_cat_prob = self.cat_prob(0.5, 0, 0, my_thresholds)
```

5.2.2 exp_score

Description

Expected score function which calculates the expected score $E(X_{ni})$ from person location, central item location and set of Rasch-Andrich thresholds. The expected score is given by:

$$E(X_{ni}) = \sum_{k=0}^m kP(X_{ni} = k) \quad (16)$$

where m is the maximum score and $P(X_{ni} = k)$ is as described in Section 5.2.1.

Usage

```
self.exp_score(person_location, item_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Item location
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Returns

Float: expected score.

Example

To obtain the expected score for a person of location 0.5 attempting an item of location 0 with a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_exp_score`:

```
my_exp_score = self.exp_score(0.5, 0, my_thresholds)
```

5.2.3 variance

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location, central item location and set of Rasch-Andrich thresholds. The variance is given by:

$$V(X_{ni}) = \sum_{k=0}^m P(X_{ni} = k)(k - E(X_{ni}))^2 \quad (17)$$

where m is the maximum score and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 5.2.1 and 5.2.2 respectively.

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

```
self.variance(person_location, item_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Item location
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Returns

Float: variance (also the Fisher information provided by the response).

Example

To obtain the variance for a person of location 0.5 attempting an item with central location 0 and a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_variance`:

```
my_variance = self.variance(0.5, 0, my_thresholds)
```

5.2.4 kurtosis

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location, central item location and Rasch-Andrich thresholds. The variance is given by:

$$\kappa(X_{ni}) = \sum_{k=0}^m P(X_{ni} = k)(k - E(X_{ni}))^4 \quad (18)$$

where m is the maximum score and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 5.2.1 and 5.2.2 respectively.

Usage

```
self.kurtosis(person_location, item_location, thresholds)
```

Arguments

<code>person_location</code>	Float: Person location
<code>item_location</code>	Float: Item location
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 attempting an item of location 0 with a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_kurtosis`:

```
my_kurtosis = self.kurtosis(0.5, 0, my_thresholds)
```

5.3 Parameter estimation

5.3.1 `calibrate`

Description

Produces central item location and Rasch-Andrich threshold estimates using the conditional pairwise estimation (CPAT) algorithm (Elliott & Buttery, 2022b).

Usage

```
self.calibrate(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001)
```

Arguments

<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> .
<code>method</code>	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.

Returns

Returns three attributes:

<code>self.items</code>	pandas series: Item location estimates with the item names as keys and estimates as values.
<code>self.thresholds</code>	pandas series: Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, an array of m estimates, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed; the series now has length m , not $m + 1$.
<code>self.cat_widths</code>	numpy array: Array of $m - 1$ category width estimates, where m is the maximum score.

Examples

To generate a set of estimates using the cosine similarity method for central item locations, with additive smoothing constant of 0.1:

```
self.calibrate()
```

To generate a set of estimates using the log-likelihood method for central item locations, with matrix raised to power 7 and a convergence stopping criterion of 0.00000001:

```
self.calibrate(method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```


5.3.2 std_errors

Description

Produces bootstrapped estimates for the standard errors of central item location estimates, threshold estimates and category width estimates for bounded (non-extreme) categories.

Usage

```
self.std_errors(interval=None, no_of_samples=500, constant=0.1, method='cos',  
                matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval. Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate. More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> .
<code>method</code>	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix for central item location estimates (Elliott & Buttery, 2022b:991) is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
<code>seed</code>	Integer or <code>None</code> : seed for the bootstrap resampling random number generator. Pass an integer for fully reproducible standard errors; <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attributes:

<code>self.item_bootstrap</code>	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each item estimate.
<code>self.item_se</code>	pandas series: Item names as keys and item standard errors as values.
<code>self.threshold_bootstrap</code>	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each threshold estimate.
<code>self.threshold_se</code>	pandas series: Threshold numbers as keys and threshold standard errors as values.
<code>self.cat_width_bootstrap</code>	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each category width estimate.
<code>self.cat_width_se</code>	pandas series: Category numbers as keys and category width standard errors as values.

If an argument is passed to `interval`, also returns:

<code>self.item_low</code>	Lower bound of the specified interval for item estimates.
<code>self.item_high</code>	Upper bound of the specified interval for item estimates.
<code>self.threshold_low</code>	Lower bound of the specified interval for threshold estimates.
<code>self.threshold_high</code>	Upper bound of the specified interval for threshold estimates.
<code>self.cat_width_low</code>	Lower bound of the specified interval for category estimates.
<code>self.cat_width_high</code>	Upper bound of the specified interval for category estimates.

Example

To generate item standard errors with a 95% interval from 200 samples:

```
self.std_errors(interval=0.95, no_of_samples=200)
```

Modifications to the estimation method are discussed in Section 5.3.1.

5.3.3 `calibrate_anchor`

Description

Anchors item location estimates onto externally-supplied reference values, supporting item banking: after running a standard calibration (Section 5.3.1), the whole item-location scale is shifted by a translation constant so that a subset of common (“anchor”) items line up with externally-supplied reference locations, e.g. from a bank of previously-calibrated items. This is a simple mean-shift.

By default (`selection_method='robust.z'`), the translation constant is computed via a robust, iterative outlier-trimming procedure based on the modified z-score (Iglewicz & Hoaglin, 1993): anchor items whose estimated values have drifted relative to the other anchor items when compared to the externally-supplied reference values are excluded before the shift is computed, so that a handful of stale or otherwise misbehaving anchor items cannot distort the shift applied to the rest of the scale. `selection_method='wald'` is an alternative that uses a formal significance test per anchor item instead of a descriptive-statistics trim, and requires bootstrap item standard errors (auto-triggers `std_errors`, Section 5.3.2, if not already computed). `selection_method='none'` takes a plain mean shift over all supplied anchors, with no trimming. Diagnostics from either selection procedure can be visualised with `plot_anchor_selection` (Section 3.5.9).

The shared threshold vector `self.thresholds` is left untouched by anchoring: thresholds describe the relative category structure common to all items, not an item’s location on the location scale, so an item-bank shift does not apply to them. Accordingly, there is no `anchor_thresholds` attribute with a modified version of `self.thresholds`.

Optionally, if an external threshold vector is available to go with the anchor items, passing it via `anchor_thresholds` runs two checks of whether it is compatible with the response data:

1. An omnibus likelihood-ratio test: compares the model fit (person locations re-estimated in each case) using the response data’s estimated thresholds against the given `anchor_thresholds`, holding item locations fixed at `self.anchor_items`. A significantly worse fit under `anchor_thresholds` warns that the rating-scale structures may not actually be shared, undermining the validity of a translation-only equating assumption.
2. A per-category test, based on the response data’s category widths (consecutive threshold differences), with bootstrap standard errors from `cat_width_se` (auto-triggers `std_errors`, Section 5.3.2, if not already computed). Estimated category widths are compared against `anchor_thresholds`’ own widths via a Wald z-test, one per category. This localised test identifies which categories have different widths from the anchor values, in the same way `dif_test`’s category-width (DCF) test does across groups (Section 5.4.8). The test operates on category widths rather than threshold locations since threshold locations, and their standard errors, are mutually interdependent due to the sum-zero constraint.

Usage

```
self.calibrate_anchor(anchors, calibrate=False, selection_method='robust_z',
                      corr_tol=0.95, sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05,
                      no_of_samples=500, adj=None, overwrite_anchors='none',
                      plot=True, plot_kwargs=None, anchor_thresholds=None,
                      alpha=0.05, correction='bh', warm_corr=True,
                      tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                      constant=0.1, method='cos', matrix_power=3,
                      log_lik_tol=0.000001, seed=None)
```

Arguments

<code>anchors</code>	Dictionary or pandas series: reference locations for a subset of items, keyed/indexed by item name.
<code>calibrate</code>	Boolean: if True, (re-)runs <code>calibrate</code> before anchoring. If False, <code>calibrate</code> is still auto-triggered if it has not already been run. Default is <code>calibrate=False</code> .
<code>selection_method</code>	String, one of 'robust_z', 'wald' or 'none'. 'robust_z' (default): excludes outlying anchor items via the modified z-score procedure (Iglewicz & Hoaglin, 1993); uses <code>corr_tol</code> , <code>sd_ratio_tol</code> , <code>min_anchors</code> . 'wald': a formal per-item sequential significance test based on z-scores computed from item standard errors – needs bootstrap standard errors, so auto-triggers <code>std_errors</code> if not already computed; uses <code>wald_alpha</code> and <code>min_anchors</code> , not <code>corr_tol</code> / <code>sd_ratio_tol</code> . 'none': plain mean shift over every supplied anchor, no trimming.
<code>corr_tol</code>	Float: minimum acceptable Pearson correlation between anchor and observed values on the trimmed item set (<code>selection_method='robust_z'</code> only). Default is <code>corr_tol=0.95</code> .
<code>sd_ratio_tol</code>	Float: the ratio of observed to anchor standard deviations on the trimmed item set must fall within $[1/\text{sd_ratio_tol}, \text{sd_ratio_tol}]$ (<code>robust_z</code> only). Default is <code>sd_ratio_tol=1.1</code> , i.e. the ratio must lie in $[0.909, 1.1]$.
<code>min_anchors</code>	Integer: minimum number of anchor items after trimming – selection stops (with a warning) once this floor is reached, even if tolerance/significance criterion is not yet satisfied, and is skipped altogether if the initial common item set is at or below this floor (<code>selection_method</code> in 'robust_z'/'wald' only). Default is <code>min_anchors=6</code> .
<code>wald_alpha</code>	Float: significance level for each anchor item's Wald test (<code>selection_method='wald'</code> only). Default is <code>wald_alpha=0.05</code> .

Arguments continue on the next page.

Arguments (continued)

<code>no_of_samples</code>	Integer: bootstrap samples for the auto-triggered <code>std_errors</code> call, only used if item standard errors are not already computed (<code>selection_method='wald'</code> only). Default is <code>no_of_samples=500</code> .
<code>adj</code>	Float or <code>None</code> : if provided, applied as translation constant and selection step skipped. Intended for reuse across bootstrap resamples (Section 5.3.2), where recomputing anchor selection every time would inflate standard errors with anchor-item sampling variance. Default <code>adj=None</code> .
<code>overwrite_anchors</code>	String, one of <code>'none'</code> , <code>'rejected'</code> or <code>'all'</code> : controls what <code>self.anchor_items</code> holds for the anchor items. <code>'none'</code> : every anchor item keeps exactly its externally-supplied value, unchanged – the usual convention, since an anchor’s value is fixed by definition, not re-estimated. <code>'rejected'</code> : retained anchors keep supplied values, rejected anchors have their values overwritten with new estimates; requires <code>selection_method</code> in <code>'robust_z'/'wald'</code> and no <code>adj</code> override (otherwise falls back to <code>'none'</code> with a warning). <code>'all'</code> : every anchor item has its value overwritten with a new, freshly-calibrated value like any other item. Default is <code>overwrite_anchors='none'</code> .
<code>plot</code>	Boolean: if <code>True</code> and <code>selection_method</code> in <code>'robust_z'/'wald'</code> , calls <code>plot_anchor_selection</code> (Section 3.5.9). Has no effect when <code>selection_method='none'</code> or <code>adj</code> is supplied. Default <code>plot=True</code> .
<code>plot_kwargs</code>	Dictionary or <code>None</code> : extra keyword arguments forwarded to <code>plot_anchor_selection</code> when <code>plot=True</code> (e.g. <code>filename</code> , <code>x_min/x_max</code> , <code>graph_title</code>). Default is <code>plot_kwargs=None</code> .
<code>anchor_thresholds</code>	Array-like or <code>None</code> : externally-supplied reference threshold vector, length <code>max_score</code> . If supplied, runs both checks described above and warns if either flags a difference. If <code>None</code> , both checks are skipped entirely (<code>self.anchor_threshold_test</code> and <code>self.anchor_category_width_test</code> are set to <code>None</code>). Default is <code>anchor_thresholds=None</code> .
<code>alpha</code>	Float: significance level for both the omnibus likelihood-ratio test (estimated thresholds, $df = \text{max_score} - 1$, versus <code>anchor_thresholds</code> , fixed, $df = 0$, chi-squared with $df = \text{max_score} - 1$) and the per-category Wald z-tests (compared against the corrected p -value if <code>correction</code> is set, else raw p -value). Default is <code>alpha=0.05</code> .
<code>correction</code>	String, one of <code>'bh'</code> , <code>'bonferroni'</code> or <code>None</code> : multiple-comparison correction across the per-category width tests only. Not applied to the omnibus test, which is a single test. Default is <code>correction='bh'</code> .

Arguments continue on the next page.

Arguments (continued)

<code>warm_corr,</code> <code>tolerance,</code> <code>max_iters,</code> <code>ext_score_</code> <code>adjustment</code>	Person-estimation arguments, used only by the omnibus <code>anchor_thresholds</code> comparison (locations must be re-estimated under each candidate threshold vector before the log-likelihoods are comparable).
<code>constant,</code> <code>matrix_power,</code> <code>log_lik_tol</code>	Calibration arguments, used only if <code>calibrate</code> is triggered – see Section 5.3.1.
<code>seed</code>	Integer or <code>None</code> : seed passed through to the internal <code>std.errors</code> call (only used if item standard errors are not already computed, <code>selection_method='wald'</code> only). <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

<code>self.anchor_items</code>	pandas series: item locations shifted onto the anchor scale.
<code>self.anchor_item_names</code>	pandas index: names of the items supplied as anchors.
<code>self.anchor_adj</code>	Float: the translation constant actually applied.
<code>self.anchor_selection</code>	pandas dataframe or <code>None</code> : per-item diagnostics from whichever selection method was used – Anchor, Observed, Deviation, Robust z, Selected (<code>'robust_z'</code>), or Anchor, Observed, SE, Deviation, z, p, Selected (<code>'wald'</code>). <code>None</code> if <code>selection_method='none'</code> or <code>adj</code> supplied.
<code>self.anchor_selected_items,</code> <code>self.anchor_dropped_items</code>	pandas index or <code>None</code> : items retained / excluded as outliers. <code>None</code> if <code>selection_method='none'</code> or <code>adj</code> supplied.
<code>self.anchor_original_corr,</code> <code>self.anchor_original_sd_ratio</code>	Float or <code>None</code> : correlation / SD ratio before trimming.
<code>self.anchor_corr</code> <code>self.anchor_sd_ratio</code>	Float or <code>None</code> : correlation / SD ratio after trimming.
<code>self.anchor_summary</code>	pandas series: one-line summary – anchors supplied/common/selected/dropped, correlation and SD ratio before/after trimming, translation constant applied.
<code>self.anchor_plot</code>	matplotlib figure or <code>None</code> : figure from the auto-triggered <code>plot_anchor_selection</code> call. <code>None</code> if <code>plot=False</code> , <code>selection_method='none'</code> , or <code>adj</code> was supplied directly.
<code>self.calibrate_anchor_runs</code>	Dictionary: every call's results above, keyed by the sorted anchor items supplied, so results from an earlier call survive a later call with a different anchor set instead of being overwritten in place.

Optional threshold-structure checks additionally return:

<code>self.anchor_threshold_test</code>	pandas series or None: omnibus test – <code>LL_estimated</code> , <code>LL_given</code> , <code>LR</code> , <code>df</code> , <code>p</code> and a boolean <code>Flagged</code> . None if <code>anchor_thresholds</code> not supplied.
<code>self.anchor_category_width_test</code>	pandas dataframe or None: per-category test, indexed by category – <code>Estimate</code> (response data's estimated category width), <code>Given</code> (<code>anchor_thresholds</code> ' corresponding category width), <code>Difference</code> , <code>SE</code> (from <code>cat_width_se</code>), <code>z</code> , <code>p</code> , <code>p_corrected</code> (if <code>correction</code> is set), boolean <code>Flagged</code> . None if <code>anchor_thresholds</code> not supplied.

Examples

To anchor item locations onto a bank of reference values stored in a dictionary `my_bank`, with automatic outlier trimming and diagnostic plot:

```
self.calibrate_anchor(my_bank)
```

To additionally check compatibility with a reference threshold vector stored as `my_bank_thresholds`:

```
self.calibrate_anchor(my_bank, anchor_thresholds=my_bank_thresholds)
```

5.3.4 person

Description

Generates a location estimate for a person using the Newton-Raphson method to produce a maximum likelihood estimate, with optional Warm bias correction (Warm, 1989).

Usage

```
self.person(person, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,  
            ext_score_adjustment=0.5)
```

Arguments

person	String: The person name for the location being estimated.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
max_iters	Integer: maximum Newton-Raphson iterations. Default <code>max_iters=100</code> .
ext_score_adjustment	Float: Value in range (0,1) to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimation.

Returns

Float: person location estimate.

Examples

To generate a person location estimate for `Person_1` using the default settings and store the result as a variable, `my_person_location`:

```
my_person_location = my_person_location = self.person('Person_1')
```

To generate an MLE person location estimate without Warm bias correction for `Person_1` based on the first three items and store the result as a variable, `my_person_location`:

```
my_person_location = self.person('Person_1', ['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```


5.3.5 `person_estimates`

Description

Generates location estimates for all persons using the Newton-Raphson method to produce maximum likelihood estimates, with optional Warm bias correction (Warm, 1989).

Usage

```
self.person_estimates(items=None, warm_corr=True, tolerance=0.00001, max_iters=100,  
                      ext_score_adjustment=0.5)
```

Arguments

<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations. Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimation.

Returns

Attribute `self.persons`: pandas series with person names as keys and location estimates as values.

Examples

To generate a set of person location estimates with Warm bias correction:

```
self.person_estimates()
```

To generate a set of person location estimates without Warm bias correction, on a subset of the first three items only:

```
self.person_estimates(items=['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

5.3.6 score_lookup

Description

Generates a location estimate for a given raw score on responses to a given set of items using the Newton-Raphson method to produce a maximum likelihood estimate, with optional Warm bias correction (Warm, 1989).

Usage

```
self.score_lookup(score, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
                  ext_score_adjustment=0.5)
```

Arguments

<code>score</code>	Integer: The raw score for which location is being estimated.
<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations. Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Returns

pandas series with raw scores as keys and person location estimates as values.

Examples

To generate a location estimate for a score of 10 on all items, with Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = self.score_lookup(10)
```

To generate a location estimate for a score of 10 on a subset of items saved as a variable `my_items`, without Warm bias correction, and store the result as a variable, `my_score_location`: `my_score_location = self.score_lookup(10, items=my_items, warm_corr=False)`

5.3.7 score_lookup_table

Description

Generates a lookup table of location estimates corresponding to all available raw scores on a set of items with no missing responses, using the Newton-Raphson method to produce maximum likelihood estimates and with optional Warm bias correction (Warm, 1989).

Usage

```
self.score_lookup_table(items=None, ext_scores=True, warm_corr=True, tolerance=0.00001,
                        max_iters=100, ext_score_adjustment=0.5)
```

Arguments

<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
<code>ext_scores</code>	Boolean: If <code>True</code> , estimates for extreme scores (all correct/all incorrect) generated using the <code>ext_score_adjustment</code> argument. Default <code>ext_scores=True</code> .
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence criterion for Newton-Raphson iterations. Default <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations. Default <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the score is extreme (all responses correct or incorrect). Adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimation.

Returns

Attribute `self.score_table`: pandas series with raw scores as keys and location estimates as values.

Examples

To generate a location lookup table for all items, including extreme scores, with Warm bias correction:

```
self.score_lookup_table()
```

To generate a location lookup table for a subset of items saved as a variable `my_items`), without extreme scores and without Warm bias correction:

```
self.score_lookup_table(items=my_items, ext_scores=False)
```

5.3.8 csem

Description

Calculates conditional standard error of measurement for a person.

Usage

```
self.csem(person, person_locations=None, items=None)
```

Arguments

<code>person</code>	Person name.
<code>person_locations</code>	pandas series (or dictionary) with person names as keys and locations as values. If <code>None</code> , uses <code>self.persons</code> , automatically generating if necessary. Default is <code>self.persons=None</code> .
<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.

Returns

Float: conditional standard error of measurement for location estimate.

Examples

To generate the CSEM for `Person_1` on all items and save the result as a variable, `my_csem`:

```
my_csem = self.csem('Person_1')
```

To generate the CSEM for a raw score of 3 on a subset of items saved to a variable `my_items` and save the result as a variable, `my_csem`:

```
my_csem = self.csem(3, person_locations=self.score_table, items=my_items)
```

where `self.score_table` is the output from `self.score_lookup_table(items=my_items)` (see Section 5.3.7).

5.4 Statistical output

5.4.1 `item_stats_df`

Description

Produces a table of item statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

```
self.item_stats_df(full=False, zstd=False, point_measure_corr=False, dp=3, se=True,
                   warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                   method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                   no_of_samples=500, interval=None, seed=None)
```

Arguments

<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>zstd</code>	Boolean: If <code>True</code> , infit and outfit standardised z-scores are reported alongside mean square statistics. Default is <code>zstd=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>point_measure_corr</code>	Boolean: If <code>True</code> , point-polyserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure polyserial correlation for an ideal item. Default is <code>point_measure_corr=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>se</code>	Boolean: If <code>True</code> , computes and reports the SE column (and CI bound columns, if <code>interval</code> is set). If <code>False</code> , skips the bootstrap entirely – useful when only Infit MS/Outfit MS are needed (e.g. repeated simulation runs), since those do not depend on the bootstrap and are unaffected by this flag. Forces <code>interval=None</code> when <code>False</code> . Default is <code>se=True</code> .
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's (1989) bias correction applied during generation of person estimates (see Section 5.3.4). Default <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is <code>max_iters=100</code> .

Arguments continue on the next page.

Arguments (continued)

<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person estimates (see Section 5.3.4) if a person has an extreme score (all responses correct or incorrect). Adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimation.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 5.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 5.3.1). Default value is <code>constant=0.1</code> .
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates (see Section 5.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 5.3.1). Ignored for other methods.
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 5.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Unused if <code>se=False</code> . Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 5.3.2). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Ignored if <code>se=False</code> . Default is <code>interval=None</code> .
<code>seed</code>	Integer or None: seed passed through to the internal <code>std_errors/fit_statistics</code> calls (only used if not already computed). None (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.item_stats`, a pandas dataframe with one row for each item and the following columns:

Estimate	Central item location estimate.
SE	Bootstrapped standard error of central item location estimate. Only produced if <code>se=True</code> .
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 2.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 5%. Only produced if <code>se=True</code> and <code>interval</code> is set.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 97.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 95%. Only produced if <code>se=True</code> and <code>interval</code> is set.
Count	Count of responses.
Facility	Item facility: proportion of correct responses.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if <code>zstd=True</code> .
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if <code>zstd=True</code> .
PM corr	Point-polyserial correlation. Only produced if <code>point_measure_corr=True</code> .
Exp PM corr	Expected value of point-measure polyserial correlation. Only produced if <code>point_measure_corr=True</code> .
Disordered	If <code>True</code> , threshold estimates indicate the presence of disordered thresholds within the item (Andrich, 2010).

Examples

To produce a summary `item_stats` table with the most commonly reported statistics:

```
my_rsm.item_stats_df()
```

To produce a full `item_stats` table with all statistics:

```
my_rsm.item_stats_df(full=True)
```

To produce an `item_stats` table with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
my_rsm.item_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

5.4.2 threshold_stats_df

Description

Produces a table of threshold statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

```
self.threshold_stats_df(full=False, zstd=False, disc=False, point_measure_corr=False, dp=3,
                        warm_corr=True, tolerance=0.00001, max_iters=100,
                        ext_score_adjustment=0.5, method='cos', constant=0.1,
                        no_of_samples=500, interval=None)
```

Arguments

<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>zstd</code>	Boolean: If <code>True</code> , infit and outfit standardised z-scores are reported alongside mean square statistics. Default is <code>zstd=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>disc</code>	Boolean: If <code>True</code> , item discrimination is reported. The discrimination of the empirical threshold slope, which functions as an SLM item conditioned on responses being in the categories either side, relative to the ideal logistic ogive, with 1 perfect, > 1 showing overfit and < 1 showing underfit; discrimination is similar to the 2PL IRT discrimination parameter (Linacre, 2023), but is a descriptive statistic here rather than an item parameter.
<code>point_measure_corr</code>	Boolean: If <code>True</code> , point-biserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure biserial correlation for an ideal item. Default is <code>point_measure_corr=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 5.3.4). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default <code>max_iters=100</code> .

Arguments continue on the next page.

Arguments (continued)

<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 5.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 5.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 5.3.1). Default value is <code>constant=0.1</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 5.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 5.3.2). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .

Returns

Attribute `self.threshold_stats`, a pandas dataframe with one row for each threshold and the following columns:

Estimate	Rasch-Andrich threshold estimate.
SE	Bootstrapped standard error of threshold estimate.
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 2.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 5%.

Returns continue on the next page.

Returns (continued)

97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 97.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 95%.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if <code>zstd=True</code> .
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if <code>zstd=True</code> .
Discrim	Item discrimination. Only produced if <code>disc=True</code> .
PM corr	Point-biserial correlation. Only produced if <code>point_measure_corr=True</code> .
Exp PM corr	Expected value of point-measure biserial correlation. Only produced if <code>point_measure_corr=True</code> .

Examples

To produce a summary `threshold_stats` table with the most commonly reported statistics:

```
my_rsm.threshold_stats_df()
```

To produce a full `threshold_stats` table with all statistics:

```
my_rsm.threshold_stats_df(full=True)
```

To produce an `threshold_stats` table with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
my_rsm.threshold_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of central item location, threshold and person location estimation.

5.4.3 category_stats_df

Description

Produces a table of category *width* statistics – `threshold_stats_df` (Section 5.4.2) reports raw threshold locations; this reports the difference between consecutive thresholds for each item, $\tau_{k+1} - \tau_k$. The shared, zero-summed threshold vector has only `max_score-1` true degrees of freedom, so the per-threshold standard errors in `threshold_stats_df` are correlated rather than independent, and understate the true amount of error. No infit/outfit or other fit statistics are reported, since these apply to thresholds rather than categories. Auto-triggers `calibrate` (Section 5.3.1) and `std_errors` (Section 5.3.2) directly if not already run.

A category width can be negative – a negative width at category k means thresholds τ_k and τ_{k+1} are disordered for that item (category k is never the most likely response at any person location). `Prop disordered` makes this a continuous diagnostic rather than a straightforward yes/no: the proportion of bootstrap resamples in which that item’s category width was negative, reasonably read as the probability that the true category is disordered.

Usage

```
self.category_stats_df(dp=3, constant=0.1, method='cos', matrix_power=3,  
                       log_lik_tol=0.000001, no_of_samples=500, interval=None, seed=None)
```

Arguments

<code>dp</code>	Integer: Number of decimal places reported in output. Default is <code>dp=3</code> .
<code>constant,</code> <code>method,</code> <code>matrix_power,</code> <code>log_lik_</code> <code>tol</code>	Calibration keyword arguments, used only if <code>calibrate</code> (Section 5.3.1) has not already been run. See Section 5.3.1 for details of each.
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates, used only if <code>std_errors</code> (Section 5.3.2) has not already been run. Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float or <code>None</code> : Empirical interval to define quantiles of estimates from bootstrap samples (see Section 5.3.2). Only included in the output if <code>std_errors</code> is triggered by this call, or was already run with an interval – not retroactively added to an existing standard error run made without one. Default is <code>interval=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed passed through to <code>std_errors</code> , used only if not already run. <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.category_stats`, a pandas dataframe with one row per category, indexed `Category 1` to `Category max_score-1` (one fewer row than `threshold_stats`, since a width is defined between each pair of consecutive thresholds) – with the following columns:

Estimate	Category width (can be negative).
SE	Bootstrapped standard error of the width, differenced <i>within</i> each bootstrap resample before taking the standard deviation, so it correctly reflects the covariance between adjacent thresholds rather than naively combining their individual standard errors.
2.5% / 97.5%	Bounds of the empirical bootstrap estimate interval, if <code>interval</code> is available (see <code>threshold_stats_df</code> , Section 5.4.2, for the column-naming convention).
Disordered	Boolean: True if Estimate is negative.
Prop disordered	Proportion of bootstrap resamples in which this category's width was negative – reasonably read as the probability that the true category is disordered. Useful for interpreting <code>Disordered</code> in both directions: a True that is only weakly supported (<code>Prop disordered</code> close to 0.5) versus robust, or a False that is nonetheless uncertain versus clear-cut.

Example

To produce the `my_rsm.category_stats` table, alongside the threshold-level table:

```
my_rsm.threshold_stats_df()
my_rsm.category_stats_df()
```

To check which categories, if any, are disordered:

```
my_rsm.category_stats_df()
my_rsm.category_stats[my_rsm.category_stats['Disordered']]
```

5.4.4 `person_stats_df`

Description

Produces a table of person statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

```
self.person_stats_df(full=False, rsem=False, dp=3, se=True, warm_corr=True, tolerance=0.00001,
                    max_iters=100, ext_score_adjustment=0.5, method='cos', constant=0.1)
```

Arguments

<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>rsem</code>	Boolean: If <code>True</code> , realistic standard error of measurement (RSEM), which takes into account item fit (Wright, 1996), is reported alongside the conditional standard error of measurement (CSEM). Default is <code>rsem=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>se</code>	Boolean: passed through to the internal <code>fit_statistics</code> call (only used if not already computed). If <code>False</code> , skips the bootstrap entirely – this table's own columns (CSEM, RSEM, Infit/Outfit) do not depend on it, so <code>se=False</code> is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is <code>se=True</code> .
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's (1989) bias correction is applied during generation of person estimates (see Section 5.3.4). Default <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 5.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Arguments continue on the next page.

Arguments (continued)

<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 5.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 5.3.1). Default value is <code>constant=0.1</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 5.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 5.3.2). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .

Returns

Attribute `self.person_stats`, a pandas dataframe with one row for each person and the following columns:

<code>Estimate</code>	Item location estimate.
<code>CSEM</code>	Conditional standard error of measurement for person location estimate.
<code>RSEM</code>	Realistic standard error of measurement for person location estimate. Only produced if <code>rsem=True</code>
<code>Score</code>	Number of correct responses.
<code>Max score</code>	Maximum available score (number of items attempted).
<code>p</code>	Proportion of correct responses.
<code>Infit MS</code>	Infit mean square.
<code>Infit Z</code>	Standardised infit z-score.
<code>Outfit MS</code>	Outfit mean square.
<code>Outfit Z</code>	Standardised outfit z-score.

Examples

To produce a summary `person_stats` table with the most commonly reported statistics:

```
my_rsm.person_stats_df()
```

To produce a full `person_stats` table with all statistics:

```
my_rsm.person_stats_df(full=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

5.4.5 test_stats_df

Description

Produces a table of test-level statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or $\text{\LaTeX}$.

Usage

```
self.test_stats_df(dp=3, warm_corr=True, tolerance=0.00001, max_iters=100,  
                  ext_score_adjustment=0.5, method='cos', constant=0.1, alpha=False,  
                  seed=None)
```

Arguments

dp	Integer: Number of decimal places reported in output.
warm_corr	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 5.3.4). Default is <code>warm_corr=True</code> .
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is <code>tolerance=0.00001</code> .
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is <code>max_iters=100</code> .
ext_score_adjustment	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 5.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 5.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 5.3.1). Default value is <code>constant=0.1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>alpha</code>	Boolean: if <code>True</code> , adds a <code>Cronbach alpha</code> row to the <code>Persons</code> column of the returned table (the <code>Items</code> column is left blank, since Cronbach's alpha (Cronbach, 1951) is a person-side reliability statistic). Computed on complete cases; if the data contain missing responses, a warning is raised and alpha is computed after listwise deletion, which may underestimate the true value. Default is <code>alpha=False</code> .
<code>seed</code>	Integer or <code>None</code> : seed passed through to the internal <code>fit_statistics</code> call (only used if not already computed). <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.test_stats`, a pandas dataframe with two columns, `Items` and `Persons` and rows for a range of descriptive statistics describing the distributions of the estimates and different statistics related to reliability – these statistics describe the suitability of the data for estimating and differentiating the parameters, rather than properties of the parameters themselves. The statistics are:

Mean	The mean of the estimates.
SD	The standard deviation of the estimates.
Separation ratio	The separation ratio (Wright, 1996; Wright & Masters, 1982), which is the standard deviation of person locations reported as a ratio of standard error units. For persons, $G_p = \sigma_p / \sqrt{\sum_n SE_n^2}$, where σ_p is the variance of the person estimates and SE_n is the RSEM (see Section 5.4.4) for person n . The formula is symmetrical for items, substituting the standard error of estimation for RSEM.
Strata	The number of statistically distinct levels of either person location or item location as strata with centers three measurement errors apart (Wright & Masters, 1982:106). For persons, $H_p = (4G_p + 1)/3$, with symmetrical results for items.
Reliability	A Rasch-specific reliability statistic (Wright, 1996), derived from PSI and which is a Rasch-specific reliability statistic similar to Cronbach's Alpha (Cronbach, 1951), and which may be interpreted the same way – as the proportion of variance of the estimates which stems from variation in location or location rather than estimation error. For persons, $R_p = G_p^2 / (1 + G_p^2)$, with symmetrical results for items.
Cronbach alpha	Optional (<code>alpha=True</code>), <code>Persons</code> column only – classical Cronbach's alpha (Cronbach, 1951), computed directly from the raw response data rather than derived from Rasch estimates. Complete cases only; a warning is raised if missing data are present, since alpha may then be an underestimate. Not computed by default (<code>alpha=False</code>).

Example

To produce a `self.test_stats` table:

```
my_rsm.test_stats_df()
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

5.4.6 model_selection

Description

Compares RSM against PCM – RSM is the constrained (nested) model, sharing one threshold structure across all items; PCM is the unconstrained model, giving each item its own threshold structure – via a likelihood ratio test, AIC, or BIC, to help decide which model is the more appropriate, parsimonious fit for a given dataset. The comparison is symmetric: both `RSM.model_selection` and `PCM.model_selection` set the same `self.model_comparison_rsm_pcm.*` attribute names regardless of which class the method is called from. Internally constructs and calibrates a PCM object from the same response data for comparison; auto-triggers `calibrate` (Section 5.3.1) and `person` (Section 5.3.4) on `self` directly if not already run.

Usage

```
self.model_selection(test='AIC', aic_sig_test=True, alpha=0.05, sampling='dynamic',
                    warm_corr=True, tolerance=0.00001, max_iters=100,
                    ext_score_adjustment=0.5, constant=0.1, method='cos',
                    log_lik_tol=0.000001, seed=None)
```

Arguments

test	String: 'LR', 'AIC', or 'BIC'. Default is test='AIC'.
aic_sig_test	Boolean: When test='AIC', if True, applies a significance test with RSM as the null hypothesis. The p-value is the relative likelihood of RSM vs PCM, $p = e^{-\Delta/2}$ where $\Delta = \text{AIC}_{\text{RSM}} - \text{AIC}_{\text{PCM}}$. PCM is preferred only if $p < \alpha$; otherwise RSM is retained as the more parsimonious default. Default is aic_sig_test=True.
alpha	Float: significance level for test='LR' or test='AIC' with aic_sig_test=True. Default is alpha=0.05.
sampling	None, 'dynamic', or Integer: Subsamples T non-extreme persons before computing log-likelihoods: at very large N , the LR/AIC/BIC comparison becomes biased toward the more complex (PCM) model, since log-likelihood differences scale with N ; capping the effective N mitigates this. An integer defines T . 'dynamic' uses $T = \min(20(I - 1)(m - 1), 1500)$, where I is the number of items and m the shared maximum score; subsampling is skipped whenever the actual non-extreme sample size is $\leq T$. None applies no subsampling. Default is sampling='dynamic'.
warm_corr, tolerance, max_iters, ext_score_adjustment	Person location estimation keyword arguments, used only if person (Section 5.3.4) has not already been run on self. Always used for the internally-constructed PCM comparison object. See Section 5.3.4 for details of each.

Arguments continue on the next page.

Arguments (continued)

<code>constant, method, log_lik_tol</code>	Calibration keyword arguments, used only if <code>calibrate</code> (Section 5.3.1) has not already been run on <code>self</code> . Always used for the internally-constructed PCM comparison object. See Section 5.3.1 for details of each.
<code>seed</code>	Integer or <code>None</code> : seed for the person-subsampling random number generator, only used when <code>sampling</code> triggers. Pass an integer for a reproducible comparison; <code>None</code> (default) draws fresh entropy. See Section 1.

Returns

Sets whichever of the following attributes correspond to `test` (all named from the RSM-vs-PCM comparison regardless of which class this method is called from, since the comparison itself is symmetric):

<code>self.model_comparison_rsm_pcm_lr, df, _p, _lr_preferred, _lr_summary</code>	Likelihood ratio statistic, degrees of freedom, p-value, the preferred model as a string (RSM is the null; PCM is preferred if $p < \alpha$), and a one-line pandas Series summary (including a Preferred row). Only set if <code>test='LR'</code> .
<code>self.model_comparison_rsm_pcm_aic, _aic_preferred, _aic_summary</code>	AIC values for both models, the preferred model as a string, and a summary Series. Only set if <code>test='AIC'</code> .
<code>self.model_comparison_rsm_pcm_aic_p</code>	The AIC relative-likelihood p-value described above. Only set if <code>test='AIC'</code> and <code>aic_sig_test=True</code> .
<code>self.model_comparison_rsm_pcm_bic, _bic_preferred, _bic_summary</code>	BIC values for both models, the preferred model as a string, and a summary Series. Only set if <code>test='BIC'</code> .

Example

To compare `my_rsm` against the equivalent PCM using the default AIC significance test:

```
my_rsm.model_selection()
my_rsm.model_comparison_rsm_pcm_aic_summary
```

To instead run a likelihood ratio test, with a fixed subsampling size of 800 persons to guard against large- N bias:

```
my_rsm.model_selection(test='LR', sampling=800)
```

5.4.7 andersen_lr_test

Description

Andersen's (1973) likelihood ratio test of parameter invariance (Andersen, 1973a): splits persons into two groups, fits the model separately in each group, and tests whether item location and threshold estimates are invariant across the groups. Groups can be formed either by a median split on person location or raw score (a general model-fit check, not tied to any particular grouping variable), or by an exogenous person covariate such as gender (Differential Item Functioning, or DIF – see Holland and Wainer, 1993), in which case this serves as an omnibus test of whether *any* item shows DIF with respect to that covariate, complementing the per-item and per-threshold tests in `dif_test` (Section 5.4.8).

Usage

```
self.andersen_lr_test(split_by='person_location', covariate=None, warm_corr=True,
                      tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                      constant=0.1, method='cos', matrix_power=3,
                      log_lik_tol=0.000001)
```

Arguments

<code>split_by</code>	String, one of 'person_location', 'score' or 'exogenous': split criterion. 'person_location' splits on the median of person location estimates (Section 5.3.4). 'score' splits on the median of raw scores (a warning is issued if data are missing, since raw scores are then not comparable across persons). 'exogenous' splits by an external person covariate, requiring <code>covariate</code> and <code>self.exogenous</code> to be set (Section 5.1.1), and requires the covariate to have exactly two distinct non-null values – covariates with more than two levels are not supported here; use <code>dif_test</code> (Section 5.4.8) instead, which supports them directly. Default is <code>split_by='person_location'</code> .
<code>covariate</code>	String or <code>None</code> : column name in <code>self.exogenous</code> to split by. Required (and only used) when <code>split_by='exogenous'</code> . Persons with a missing value for this covariate are excluded from both groups and from the full-sample comparison fit, so the likelihood-ratio decomposition stays valid. Default is <code>covariate=None</code> .
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_</code> <code>adjustment</code>	Person-estimation arguments passed to the group models – see Section 5.3.4.
<code>constant</code> , <code>method</code> , <code>matrix_power</code> , <code>log_lik_</code> <code>tol</code>	Calibration arguments passed to the group models – see Section 5.3.1.

Returns

<code>self.andersen_lr</code>	Float: the likelihood ratio statistic.
<code>self.andersen_df</code>	Integer: degrees of freedom $((\text{self.no_of_items} - 1) + (\text{self.max_score} - 1))$.
<code>self.andersen_p</code>	Float: p-value from the chi-squared distribution. <code>self.andersen_lr</code> is floored at 0 (with a warning) if the raw statistic is negative, which can occur since PAIR/CPAT are approximate, not exact, estimation methods; a floored statistic indicates no evidence of misfit.
<code>self.andersen_groups</code>	Dictionary: <code>{group_name: RSM}</code> , the fitted group models, for inspection. Group names are 'low'/'high' for <code>split_by='person_location'/'score'</code> , or the two observed covariate values for <code>split_by='exogenous'</code> .
<code>self.andersen_summary</code>	pandas series: the LR statistic, degrees of freedom and p-value together.

Examples

To test parameter invariance by a median split on person location:

```
self.andersen_lr_test()
```

To test parameter invariance by a median split on raw score:

```
self.andersen_lr_test(split_by='score')
```

To test for DIF by an exogenous covariate 'Gender' (which must have exactly two levels), having supplied exogenous to the constructor:

```
self.andersen_lr_test(split_by='exogenous', covariate='Gender')
```

5.4.8 dif_test

Description

Differential Item Functioning (DIF) test by an exogenous person covariate (e.g. gender, first language) (Holland & Wainer, 1993). Splits non-extreme persons into groups by the named column in `self.exogenous`, designates one group as the reference (by default the largest), and compares each other (“focal”) group against it individually – a reference-group design, not all-pairwise, so covariates with more than two levels are supported directly (unlike `andersen_lr_test`, Section 5.4.7). Two independent components are tested:

1. **Item-location DIF.** Both groups are calibrated independently (each self-centred), then purified onto a common item scale via either a per-item Wald significance test (`selection_method='wald'`, default) or the robust modified z-score outlier-trimming procedure used by `calibrate_anchor` (Section 5.3.3; Iglewicz & Hoaglin, 1993) – the reference group’s item locations supply the “anchor” values, the focal group’s supply the “observed” values – so that genuine DIF items cannot contaminate the common scale used to test for DIF in the first place. Item-level DIF is then tested via a per-item Wald test (`test='wald'`, default), a per-item likelihood-ratio test (`test='lr'`), or both (`test='both'`), using bootstrap standard errors from `std.errors` (Section 5.3.2) for the Wald test. An omnibus LR test (Andersen-style: reference vs. focal, every common item jointly) runs by default (`omnibus=True`). Results in `self.dif_table` and `self.dif_omnibus_table`.
2. **Threshold-structure DIF** (differential category functioning, or DCF). RSM’s shared threshold vector (`self.thresholds`) describes relative category structure common to all items, not item location, so it is not translated by the item-scale purification above – like `calibrate_anchor`’s own threshold handling. Tested via category widths (consecutive threshold differences, `thresholds[k+1] - thresholds[k]`) rather than raw threshold locations, since threshold locations are partial sums of widths and a single genuine width change would otherwise cascade into every downstream threshold location, smearing the signal; widths isolate it to the one category that actually changed. Each of the `max_score - 1` category widths is Wald-tested directly, reference versus focal, using a bootstrap standard error of the width itself (differenced *within* each bootstrap resample before taking the standard deviation, so it already reflects the covariance between adjacent thresholds), with its own multiple-comparison correction pool across widths, independent of the item-location correction pool above. Not affected by `test/omnibus` – those only govern component (1). Results in `self.threshold_dif_table`. Because the threshold vector is shared across the whole test rather than per item, this component only detects a shift in the *test-wide* category structure between groups, not an item-specific structural difference.

An item or category-width cell is flagged only if it clears both a (corrected) p-value threshold and an absolute logit-difference threshold, the latter an ETS-style convention, so that a statistically significant but practically negligible difference is not flagged. `dif_test` also supports ETS-style A/B/C severity-category classification (Zwick et al., 1997) (`category`, `category_thresholds`, `category_alpha`) for the item-location component only – the 0.43/0.64 logit defaults are calibrated for item-location DIF specifically, so this does not extend to `self.threshold_dif_table` – as described in Table 20:

Category	Criteria	Interpretation
A	$ \text{Difference} < 0.43$, or fails the significance test below	Negligible DIF
B+ / B-	Both $ \text{Difference} \geq 0.43$ and $p(\text{DIF} = 0) < \text{category_alpha}$ (two-sided Wald/Welch test), but not C	Slight to moderate DIF
C+ / C-	Both $ \text{Difference} \geq 0.64$ and $p(\text{Difference} \leq 0.43) < \text{category_alpha}$ (one-sided test that the difference significantly exceeds the B boundary)	Moderate to large DIF

Table 20: ETS-style DIF category classification (item-location component only). Sign: + indicates that an item is harder for the focal group; - indicates that an item is easier for the focal group. The 0.43/0.64 logit thresholds are the Rasch-logit equivalents of the classic ETS Mantel-Haenszel delta-scale B/C boundaries (Tristan, 2006); the significance level (default `category_alpha=0.05`) is independent of `alpha`, which governs the separate `Flagged/Flagged_LR` columns.

Usage

```
self.dif_test(covariate, reference=None, selection_method='wald', corr_tol=0.95,
              sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05, test='wald',
              omnibus=True, welch=False, size_adjust=False, reference_n=100,
              correction='bh', alpha=0.05, logit_threshold=0.43, category=False,
              category_thresholds=(0.43, 0.64), category_alpha=0.05, no_of_samples=500,
              plot=False, plot_kwargs=None, warm_corr=True, tolerance=0.00001,
              max_iters=100, ext_score_adjustment=0.5, constant=0.1, method='cos',
              matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>covariate</code>	String: column name in <code>self.exogenous</code> to group persons by.
<code>reference</code>	String or <code>None</code> : value of <code>covariate</code> to use as the reference group. If <code>None</code> , the largest group is used. Default is <code>reference=None</code> .
<code>selection_method</code>	String, one of 'wald', 'robust_z' or 'none': how the common item-location scale is purified before testing (never the threshold-structure component) – see <code>calibrate_anchor</code> (Section 5.3.3) for what each option does. Default is <code>selection_method='wald'</code> .
<code>corr_tol</code> , <code>sd_ratio_tol</code> , <code>min_anchors</code> , <code>wald_alpha</code>	As for <code>calibrate_anchor</code> (Section 5.3.3), used only by the corresponding <code>selection_method</code> . Defaults are <code>corr_tol=0.95</code> , <code>sd_ratio_tol=1.1</code> , <code>min_anchors=6</code> , <code>wald_alpha=0.05</code> .

Arguments continue on the next page.

Arguments (continued)

<code>test</code>	String, one of 'wald', 'lr' or 'both': per-item item-location DIF test(s) to compute – affects only the item-location component, never <code>self.threshold_dif_table</code> . 'wald' (default) matches the original behaviour exactly, at no added cost. 'lr'/'both' additionally compute a per-item likelihood-ratio test (extra cost: two person location re-estimations per item per focal group). The existing <code>Flagged</code> column always reflects the Wald test; a separate <code>Flagged_LR</code> column is added when <code>test</code> is 'lr' or 'both'. Default is <code>test='wald'</code> .
<code>omnibus</code>	Boolean: if <code>True</code> (default), also runs an Andersen-style omnibus LR test (reference vs. focal, every common item jointly) per focal group for the item-location component, stored in <code>self.dif_omnibus_table</code> – unlike PCM, there is no separate scope option here, since RSM's single shared threshold vector has no per-item threshold freedom to additionally account for. Cheap relative to the per-item LR test – one extra combined-group model fit per focal group, not per item.
<code>welch</code>	Boolean: if <code>True</code> , every Wald test here (item-location <i>and</i> threshold-structure) becomes a Welch's t-test: statistic unchanged (<code>Difference / SE</code>), but p-value computed against a t-distribution with Welch-Satterthwaite degrees of freedom instead of a normal distribution. Adds a <code>df</code> column to both <code>self.dif_table</code> and <code>self.threshold_dif_table</code> . Does not affect the LR test or omnibus test (already exact chi-squared tests). Default is <code>welch=False</code> .
<code>size_adjust</code>	Boolean: if <code>True</code> , rescales each group's own SE (item location <i>and</i> threshold-width – same scope as <code>welch</code> , unlike <code>category</code> below) to what it would be at a standard reference sample size (Tristan, 2006) – the opposite problem from <code>welch</code> 's: at very large N , SE shrinks enough that trivial differences become “significant”. Each group's own SE is rescaled independently by $\sqrt{n_{\text{actual}}/\text{reference_n}}$ using that group's own person count, then combined as usual. Applied to the Wald/Welch test, the category tests, and (if <code>welch=True</code>) the Welch-Satterthwaite df, which then also uses <code>reference_n</code> rather than each group's actual n , for internal consistency. Default is <code>size_adjust=False</code> .
<code>reference_n</code>	Integer: reference sample size for <code>size_adjust=True</code> . 100 (the default) is Tristán's own recommendation; around 60 is suggested there for closer alignment with the ETS Category B boundary specifically. Unused unless <code>size_adjust=True</code> .

Arguments continue on the next page.

Arguments (continued)

<code>correction</code>	String or <code>None</code> , one of <code>'bh'</code> , <code>'bonferroni'</code> or <code>None</code> : multiple-comparison correction applied across items (within each focal-group comparison) and, separately, across every category-width cell (within each focal-group comparison, one pool for the whole test since the threshold vector is shared rather than per item) – Benjamini-Hochberg false discovery rate control (Benjamini & Hochberg, 1995) or Bonferroni. Not applied to the omnibus test. Default is <code>correction='bh'</code> .
<code>alpha</code>	Float: p-value threshold for flagging in both components (Wald, per-item LR, and omnibus alike), compared against the corrected p-value if <code>correction</code> is not <code>None</code> . Default is <code>alpha=0.05</code> .
<code>logit_threshold</code>	Float: absolute logit-difference threshold for flagging in both components (ETS-style convention). Default is <code>logit_threshold=0.43</code> .
<code>category</code>	Boolean: if <code>True</code> , adds an ETS-style <code>Category</code> column to <code>self.dif_table</code> (item-location component only – see Table 20 above). Default is <code>category=False</code> .
<code>category_thresholds</code>	Tuple of two floats: (B boundary, C boundary) in logits, per Zwick et al. (1997). Default is <code>category_thresholds=(0.43, 0.64)</code> .
<code>category_alpha</code>	Float: significance level for both category tests, independent of <code>alpha</code> . Default is <code>category_alpha=0.05</code> .
<code>no_of_samples</code>	Integer: bootstrap resamples for each group’s <code>std_errors</code> call (Section 5.3.2). Default is <code>no_of_samples=500</code> .
<code>plot</code>	Boolean: if <code>True</code> , calls <code>plot_anchor_selection</code> (Section 3.5.9) for each focal group that has a selection table (<code>selection_method</code> in <code>'wald'/'robust_z'</code> only) and stores the resulting figures in <code>self.dif_plots</code> . Default is <code>plot=False</code> .
<code>plot_kwargs</code>	Dictionary or <code>None</code> : extra keyword arguments forwarded to <code>plot_anchor_selection</code> for every focal group when <code>plot=True</code> . Default is <code>plot_kwargs=None</code> .
<code>warm_corr,</code> <code>tolerance,</code> <code>max_iters,</code> <code>ext_score_</code> <code>adjustment</code>	Person-estimation arguments, passed through to the group models – see Section 5.3.4. Only used if <code>test</code> is <code>'lr'/'both'</code> or <code>omnibus=True</code> .
<code>constant,</code> <code>method,</code> <code>matrix_power,</code> <code>log_lik_</code> <code>tol</code>	Calibration arguments passed to the group models – see Section 5.3.1.
<code>seed</code>	Integer or <code>None</code> : seed passed through to each group’s internal <code>std_errors</code> call. <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

<code>self.dif_reference</code>	The reference group value.
<code>self.dif_covariate</code>	The covariate column name used.
<code>self.dif_reference_model</code>	RSM object: fitted model for the reference group.
<code>self.dif_focal_models</code>	Dictionary: <code>{focal_value: RSM}</code> , fitted models for each focal group.
<code>self.dif_tc</code>	Dictionary: <code>{focal_value: float}</code> , the item-scale translation constant applied to each focal group.
<code>self.dif_anchor_selection</code>	Dictionary: <code>{focal_value: dataframe or None}</code> , per-item selection diagnostics for each focal group (None if <code>selection_method='none'</code>).
<code>self.dif_plots</code>	Dictionary: <code>{focal_value: figure or None}</code> , the <code>plot_anchor_selection</code> figure for each focal group, if <code>plot=True</code> and a selection table exists for that group.
<code>self.dif_group_sizes</code>	Dictionary: <code>{group_value: int}</code> , non-extreme, non-missing-covariate N used for each group.

The two tested components additionally set:

<code>self.dif_table</code>	pandas dataframe: one row per (item, focal group) pair, indexed by item. Columns: Group, Reference/Focal/Focal (purified) (item location estimates), Difference, SE, z, p, p (corrected), Selected and Flagged. If <code>welch=True</code> , also df. If test is 'lr' or 'both', also LR, p_LR, p_LR (corrected), Flagged_LR. If <code>category=True</code> , also Category.
<code>self.dif_omnibus_table</code>	pandas dataframe or None: one row per focal group – LR, df, p, Flagged – the item-location component only. None if <code>omnibus=False</code> .
<code>self.threshold_dif_table</code>	pandas dataframe: differential category functioning, tested via category widths rather than threshold locations. One row per (category, focal group) pair, where category k is the width between threshold k and $k + 1$ ($k = 1, \dots, \text{max_score} - 1$). Columns: Group, Category, Reference/Focal (category widths), Difference, SE (bootstrap SE of the width itself), z, p, p (corrected) and Flagged – and df if <code>welch=True</code> – corrected independently of <code>self.dif_table</code> .

Examples

To test for DIF by an exogenous covariate 'L1' (which may have any number of levels), having supplied `exogenous` to the constructor, using the largest level as the reference group by default:

```
self.dif_test('L1')
```

To inspect the shared threshold-structure (category-width) DIF results after running the test:

```
self.threshold_dif_table
```

To additionally compute the per-item likelihood-ratio test and ETS-style DIF categories:

```
self.dif_test('L1', test='both', category=True)
```

5.4.9 res_corr_analysis

Description

Analysis of correlations of standardised residuals to test for violations of local item interdependence and unidimensionality requirements.

Usage

```
self.res_corr_analysis(warm_corr=True, tolerance=0.00001, max_iters=100,  
                       ext_score_adjustment=0.5, constant=0.1, method='cos',  
                       matrix_power=3, log_lik_tol=0.000001, se=True)
```

Arguments

warm_corr	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 5.3.4). Default is <code>warm_corr=True</code> .
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is <code>tolerance=0.00001</code> .
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is <code>max_iters=100</code> .
ext_score_adjustment	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 5.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates (see Section 5.3.1). Default value is <code>constant=0.1</code> .
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of central item location estimates (see Section 5.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
matrix_power	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of central item location estimates (see Section 5.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).

Arguments continue on the next page.

Arguments (continued)

<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 5.3.1). Ignored for other methods.
<code>se</code>	Boolean: passed through to the internal <code>fit_statistics</code> call (only used if not already computed). If <code>False</code> , skips the bootstrap entirely – this analysis’s own output (residual correlations, PCA) does not depend on it, so <code>se=False</code> is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is <code>se=True</code> .

Returns

For tests of violation of the requirement for local item independence (Andrich & Kreiner, 2010; Marais, 2012):

<code>self.residual_correlations</code>	A pandas dataframe of pairwise correlations between item standardised residuals.
---	--

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

<code>self.eigenvectors</code>	The eigenvectors of the standardised residual correlations matrix.
<code>self.eigenvalues</code>	The eigenvalues corresponding to the eigenvectors.
<code>self.variance_explained</code>	The variance explained by each principal component.
<code>self.loadings</code>	The loading of each item onto each of the principal components, for the the first of which large loadings ('large' typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce a residual correlation analysis:

```
my_rsm.res_corr_analysis()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

5.4.10 save_stats

Description

Exports `item_stats`, `threshold_stats` (both centred and uncentred), `person_stats` and `test_stats` to file in a single call. Auto-triggers `item_stats_df`, `threshold_stats_df`, `person_stats_df` and `test_stats_df` (Sections 5.4.1, 5.4.2, 5.4.4, 5.4.5) if any have not already been run – with their default arguments plus whichever of `dp/warm_corr/etc.` below are passed, so call those methods directly first (e.g. with `full=True`) if non-default table columns are wanted in the saved output. Does not include `category_stats` (Section 5.4.3) – save that table separately with `self.category_stats.to_csv(...)` if needed.

Usage

```
self.save_stats(filename, format='csv', dp=3, warm_corr=True, tolerance=0.00001,
                max_iters=100, ext_score_adjustment=0.5, method='cos', constant=0.1,
                no_of_samples=500, interval=None)
```

Arguments

<code>filename</code>	String: output filename or path. For <code>format='csv'</code> , used as a prefix (any trailing <code>.csv</code> is stripped first) for five files, <code>_item_stats.csv</code> , <code>_threshold_stats_uncentred.csv</code> , <code>_threshold_stats_centred.csv</code> , <code>_person_stats.csv</code> and <code>_test_stats.csv</code> . For <code>format='xlsx'</code> , used as a single workbook filename (<code>.xlsx</code> appended automatically if not already present), with the five tables on separate sheets.
<code>format</code>	String: <code>'csv'</code> or <code>'xlsx'</code> . Default is <code>format='csv'</code> .
<code>dp</code> , <code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_adjustment</code> , <code>method</code> , <code>constant</code> , <code>no_of_samples</code> , <code>interval</code>	Passed through to <code>item_stats_df/threshold_stats_df/person_stats_df/test_stats_df</code> (Sections 5.4.1, 5.4.2, 5.4.4, 5.4.5), used only for whichever of those tables has not already been computed. See those sections for details of each.

Returns

None. Writes the five tables to file as described above.

Example

To save item, threshold, person and test statistics as five CSV files prefixed `my_rsm_results`:

```
my_rsm.save_stats('my_rsm_results')
```

To save all tables to separate sheets of a single Excel workbook:

```
my_rsm.save_stats('my_rsm_results', format='xlsx')
```

5.4.11 save_residuals

Description

Exports the residual correlation analysis (`self.eigenvectors`, `self.eigenvalues`, `self.variance_explained` and `self.loadings`) to file. Auto-triggers `res_corr_analysis` (Section 5.4.9) if not already run.

Usage

```
self.save_residuals(filename, format='csv', single=True, dp=3, warm_corr=True,
                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                    method='cos', constant=0.1)
```

Arguments

filename	String: output filename or path (extension appended automatically if not already present).
format	String: 'csv' or 'xlsx'. Default is <code>format='csv'</code> .
single	Boolean: if <code>True</code> , writes all four tables (eigenvectors, eigenvalues, variance explained, loadings) into a single file – sequentially, separated by blank lines, for <code>format='csv'</code> ; at successive row offsets on one sheet ('Item residual analysis'), for <code>format='xlsx'</code> . If <code>False</code> , writes each table to its own file (csv) or sheet (xlsx). Default is <code>single=True</code> .
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant	<code>dp</code> controls rounding of the saved values; the rest are passed through to <code>res_corr_analysis</code> (Section 5.4.9), used only if it has not already been run. See that section for details of each.

Returns

None. Writes the residual correlation analysis tables to file as described above.

Example

To save the residual correlation analysis as a single CSV file:

```
my_rsm.save_residuals('my_rsm_residuals')
```

To save each table to its own sheet of a single Excel workbook:

```
my_rsm.save_residuals('my_rsm_residuals', format='xlsx', single=False)
```


5.4.12 `category_counts_df`

Description

Produces a table of counts of scores in each category, plus total and missing responses. All items share the same `max_score` in RSM, so there are no blank cells.

Usage

```
self.category_counts_df(persons=None, items=None, counts_name=None)
```

Arguments

<code>persons</code>	List or String: Person or subset of persons over which to count responses. Default <code>persons=None</code> , which returns counts for all persons.
<code>items</code>	List or String: Item or subset of items over which to count responses. Default <code>items=None</code> , which returns counts for all items.
<code>counts_name</code>	String or Int: Name under which to store counts on a subset of persons or items so multiple sets of counts can be made available. Useful if e.g. user wants to compare counts between different subgroups of persons according to a criterion such as an exogenous variable. If passed, stores counts within a namespace, <code>self.counts</code> . Default <code>counts_name=None</code> , when counts are returned as a simple attribute, <code>self.category_counts_table</code> .

Returns

If `counts_name=None`:

Attribute `self.category_counts_table`, a pandas dataframe of category counts with one row per item and one column per response category (0 to `max_score`), plus total and missing responses per item, and grand total row.

If `counts_name=my_counts_name`:

Attribute `self.counts.my_counts_name` stored in the namespace `self.counts`, a pandas dataframe of category counts with one row per item and one column per response category (0 to `max_score`), plus total and missing responses per item and grand total row.

Example

To produce a dataframe of category counts for all persons and items:

```
my_rsm.category_counts_df()
```

To produce a dataframe of category counts for a subset of persons saved as a list `my_persons`, with a matching attribute name within the namespace `self.counts`:

```
my_rsm.category_counts_df(persons=my_persons, counts_name=my_persons)
```

5.5 Plotting functionality

5.5.1 Shared plotting arguments

All the plotting methods described in this section share a set of arguments which may be used to customise the appearance of the plot or save the plot to file automatically. These arguments are:

<code>title</code>	String: Title for the plot, to appear in the image. Default is <code>title=None</code> .
<code>xmin</code>	Float: Minimum displayed point on x-axis, in logits. Default is <code>xmin=-5</code> .
<code>xmax</code>	Float: Maximum displayed point on x-axis, in logits. Default is <code>xmax=5</code> .
<code>plot_style</code>	String: Plot style to use. Available styles are 'white', producing a plot on a white background, and 'dark', producing a plot on a grey background. The styles correspond to Seaborn (Waskom, 2021) styles <code>whitegrid</code> and <code>darkgrid</code> . Default is <code>plot_style='white'</code> .
<code>palette</code>	String: Controls colours of lines and overplotted class intervals dots. Options are 'dark blue', 'light blue', 'dark red', 'light red', 'dark green', 'light green', 'dark grey', 'light grey', 'dark multi' and 'light multi'. Default is <code>palette='dark blue'</code> . Not used for <code>std_residual_plot</code> .
<code>black</code>	Boolean: If <code>True</code> , the plot will be rendered in black and white. Default is <code>black=False</code> .
<code>font</code>	String: The font to use in the plot. Default is <code>font='Times New Roman'</code> .
<code>title_font_size</code>	Float: The size of the title font in points. Default is <code>title_font_size=15</code> .
<code>axis_font_size</code>	Float: The size of the axis label font in points. Default is <code>axis_font_size=12</code> .
<code>labelsize</code>	Float: The size of the axis tick label font in points. Default is <code>labelsize=12</code> .
<code>filename</code>	String: The filename for the saved plot, with no suffix for format. If <code>None</code> , no file will be saved. Default is <code>filename=None</code> .
<code>file_format</code>	The format of the file: <code>png</code> , <code>jpg</code> or <code>svg</code> . Default is <code>file_format=png</code> .
<code>dpi</code>	The resolution of the plot in dpi (dots per inch) – higher resolution plots are better quality but have larger file sizes. Default is <code>dpi=300</code> .

5.5.2 `icc`

Description

Plots the item characteristic curves (or item response function) for an item: person location on the x-axis against expected score on the y-axis. Options to plot observed responses, item threshold line and lines showing locations corresponding to specified expected scores, and to highlight a specified response category.

Usage

```
self.icc(item, obs=False, no_of_classes=5, thresh_lines=False, central_location=False,  
         score_lines=None, score_labels=False, cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot.
<code>obs</code>	Boolean: If <code>True</code> , mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is <code>obs=False</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is <code>central_location=False</code> .
<code>score_lines</code>	List of floats between 0 and <code>self.max_score</code> : Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score.
<code>score_labels</code>	Boolean: If <code>True</code> , scores and locations corresponding to arguments passed to <code>score_lines</code> will be labelled on the plot. Default is <code>score_labels=False</code> .
<code>cat_highlight</code>	Integer: Passing a score between 0 and <code>self.max_score</code> will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item characteristic curve for `Item_1` and store the output as a variable `my_icc_plot` and save it to file as `my_icc_plot.png`:

```
my_icc_plot = self.icc('Item_1', filename=my_icc_plot)
```

To plot an item characteristic curve for `Item_1` with observed responses for 8 response classes and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_rsm.icc('Item_1', obs=True, no_of_classes=8)
```

To plot an item characteristic curve for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_rsm.icc('Item_1', thresh_lines=True, cat_highlight=0)
```

To plot an item characteristic curve for `Item_1` with lines showing the locations corresponding to expected scores of 0.7 and 1.6, with the expected score and corresponding location labelled, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_rsm.icc('Item_1', score_lines=[0.7, 1.6], score_labels=True)
```

5.5.3 crcs

Description

Plots category response curves for an item: person location on the x-axis against expected the probability of obtaining a score in each category (0 or 1) on the y-axis. Options to plot observed proportions and item threshold line, and to highlight a specified response category.

Usage

```
self.crcs(item=None, obs=None, no_of_classes=5, thresh_lines=False, central_location=False,
          cat_highlight=None)
```

Arguments

<code>item</code>	String: The name of the item to plot. If <code>item=None</code> , the shared set of Rasch-Andrich thresholds with mean zero will be plotted, and any observed proportions overplotted using <code>obs</code> will use data from all items. Default is <code>item=None</code> .
<code>obs</code>	List: List of integers between 0 and <code>self.max_score</code> . For each value, mean observed proportions in each ordered response category scoring in that category are plotted against the mean location of the corresponding response class.
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is <code>central_location=False</code> .
<code>cat_highlight</code>	Integer: Passing a score between 0 and <code>self.max_score</code> will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot basic category response curves and store the output as a variable `my_crcs_plot` and save it to file as `my_crcs_plot.png`:

```
my_crcs_plot = my_rsm.crcs(filename=my_crcs_plot)
```

To plot category response curves for `Item_1` with observed response proportions for category 0 for 8 response classes and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = my_rsm.crcs(item='Item_1', obs=[0], no_of_classes=8)
```

To plot category response curves for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = my_rsm.crcs(item='Item_1', thresh_lines=True, cat_highlight=0)
```

5.5.4 threshold_ccs

Description

Plots the threshold characteristic curves (or threshold response functions) for an item. For threshold τ_k , $k \in \{1, \dots, \text{self.max_score}\}$, the threshold characteristic curve is the probability of obtaining a score of k rather than $k - 1$, conditional on the score being either $k - 1$ rather than k , for a given person location. Each threshold characteristic curve functions as a dichotomous item characteristic curve under the SLM, conditional on scores being in the adjacent categories either side of the threshold (see Sections 3.2.2 and 3.5.2).

For threshold τ_k , `threshold_ccs` plots person location on the x-axis against the probability of obtaining a score of k on the y-axis. Options to plot threshold lines and central item locations, and to highlight a specified response category.

Usage

```
self.threshold_ccs(item=None, obs=None, no_of_classes=5, thresh_lines=False,
                   central_location=False, cat_highlight=None,)
```

Arguments

<code>item</code>	String: The name of the item to plot. If <code>item=None</code> , the shared set of Rasch-Andrich thresholds with mean zero will be plotted, and any observed proportions overplotted using <code>obs</code> will use data from all items. Default is <code>item=None</code> .
<code>obs</code>	List: List of integers corresponding to thresholds τ_1 to τ_m , where $m = \text{self.max_score}$, or 'all' or 'none'. If <code>obs=[k]</code> , mean proportions of persons obtaining a score of k rather than $k - 1$, conditional on the score being either $k - 1$ or k , for each of the ordered response categories, will be plotted against the mean location of the corresponding response class. Multiple thresholds may be passed. Default is <code>obs=None</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is <code>central_location=False</code> .
<code>cat_highlight</code>	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot basic threshold characteristics curves, store the output as variable `my_threshold_ccs_plot`, and save it to file as `my_threshold_ccs_plot.png`:

```
my_threshold_ccs_plot = my_rsm.threshold_ccs(filename='my_iic_plot')
```

To plot threshold characteristics curves for `Item_1` with threshold lines and category 1 highlighted, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = my_rsm.threshold_ccs(item='Item_1', thresh_lines=True,  
                                             cat_highlight=1)
```

To plot threshold characteristics curves for `Item_1` with observed responses plotted for thresholds 2 and 4 and central item location line, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = my_rsm.threshold_ccs(item='Item_1', obs=[2, 4], central_location=True)
```


5.5.5 iic

Description

Plots the item information curve for an item: person location on the x-axis against Fisher information on the y-axis. Options to plot item threshold line and lines showing Fisher information corresponding to specified locations, and to highlight a specified response category.

Usage

```
self.iic(item, ymax=None, thresh_lines=False, central_location=False, point_info_lines=None,
         point_info_labels=False, cat_highlight=None)
```

Arguments

item	String: The name of the item to plot.
ymax	Float: The maximum value to show on the y-axis. If <code>None</code> , will infer, plotting a maximum of 1.1 times the maximum item information. Default is <code>ymax=None</code>
thresh_lines	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
central_location	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is <code>central_location=False</code> .
point_info_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the Fisher information corresponding to the location. Default is <code>point_info_lines=None</code> .
point_info_labels	Boolean: If <code>True</code> , locations and Fisher information corresponding to arguments passed to <code>point_info_lines</code> will be labelled on the plot. Default is <code>point_info_labels=False</code> .
cat_highlight	Integer: Passing a score between 0 and <code>self.max_score</code> will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).
ymax	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item information curve for `Item_1` and store the output as a variable `my_iic_plot` and save it to file as `my_iic_plot.png`:

```
my_iic_plot = my_rsm.iic('Item_1', filename='my_iic_plot')
```

To plot an item information curve for `Item_1` with threshold lines, central item location line and category 1 highlighted, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = my_rsm.iic('Item_1', thresh_lines=True, central_location=True, cat_highlight=1)
```

To plot an item information curve for `Item_1` with lines showing the Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding Fisher information labelled, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = my_rsm.icc('Item_1', point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

5.5.6 tcc

Description

Plots the test characteristic curve (or test response function) for a set of items: person location on the x-axis against expected score on the y-axis. Options to plot observed responses and lines showing locations corresponding to specified expected scores.

Usage

```
self.tcc(items=None, obs=False, no_of_classes=5, score_lines=None, score_labels=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>obs</code>	Boolean: If <code>True</code> , mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is <code>obs=False</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>score_lines</code>	List of floats between 0 and <code>self.max_score</code> : Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is <code>score_lines=None</code> .
<code>score_labels</code>	Boolean: If <code>True</code> , scores and locations corresponding to arguments passed to <code>score_lines</code> will be labelled on the plot. Default is <code>score_labels=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic test characteristic curve for all items and store the output as a variable `my_tcc_plot` and save it to file as `my_tcc_plot.png`:

```
my_tcc_plot = my_rsm.tcc(filename=my_tcc_plot)
```

To plot a test characteristic curve for `Item_1` for a subset of items stored as a list `my_item_list`, with observed responses for 8 response classes and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = my_rsm.tcc(obs=True, no_of_classes=8)
```

To plot a test characteristic curve for `Item_1` for all items with lines showing the locations corresponding to expected scores of 13 and 20, with the expected score and corresponding location labelled, and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = my_rsm.tcc(score_lines=[13, 20], score_labels=True)
```

5.5.7 test_info

Description

Plots the test information curve: person location on the x-axis against total Fisher information on the y-axis. Option to plot lines showing Fisher information corresponding to specified locations.

Usage

```
self.test_info(items=None, ymax=None, point_info_lines=None, point_info_labels=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>point_info_lines</code>	List of floats: Each float represents a location, with a vertical line plotted from the x-axis to where it intersects the item information curve, then from there a horizontal line to the y-axis to show the total Fisher information corresponding to the location. Default is <code>point_info_lines=None</code> .
<code>point_info_labels</code>	Boolean: If <code>True</code> , locations and total Fisher information corresponding to arguments passed to <code>point_info_lines</code> will be labelled on the plot. Default is <code>point_info_labels=False</code> .
<code>ymax</code>	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic test information curve and store the output as a variable `my_test_info_plot` and save it to file as `my_test_info_plot.png`:

```
my_test_info_plot = my_rsm.test_info(filename='my_test_info_plot')
```

To plot a test information curve for a subset of items stored as a list `my_item_list` and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = my_rsm.test_info(items=my_item_list)
```

To plot a test information curve with lines showing the total Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding total Fisher information labelled, and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = my_rsm.test_info(point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

5.5.8 test_csem

Description

Plots the test conditional standard error of measurement (CSEM) curve: person location on the x-axis against CSEM (in logits) on the y-axis. Option to plot lines showing CSEM corresponding to specified locations.

Usage

```
self.test_csem(items=None, point_csem_lines=None, point_csem_labels=False, ymax=5)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>point_csem_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the CSEM curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the CSEM corresponding to the location. Default is <code>point_csem_lines=None</code> .
<code>point_csem_labels</code>	Boolean: If <code>True</code> , locations and CSEM corresponding to arguments passed to <code>point_csem_lines</code> will be labelled on the plot. Default is <code>point_csem_labels=False</code> .
<code>ymax</code>	The maximum point displayed on the y-axis, in logits.

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic CSEM curve and store the output as a variable `my_test_csem_plot` and save it to file as `my_test_csem_plot.png`:

```
my_test_csem_plot = my_rsm.test_csem(filename='my_test_csem_plot')
```

To plot a CSEM curve for a subset of items stored as a list `my_item_list` and store the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = my_rsm.test_csem(items=my_item_list)
```

To plot a CSEM curve with lines showing the CSEM corresponding to locations of -0.3 and 0.7, with the location and corresponding CSEM labelled, and store the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = my_rsm.test_csem(point_csem_lines=[-0.3, 0.7], point_csem_labels=True)
```

5.5.9 std_residuals_plot

Description

Plots histogram of standardised residuals, with optional overplotting of standard Normal distribution.

Usage

```
self.std_residuals_plot(items=None, bin_width=0.5, normal=False)
```

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>bin_width</code>	Float: The width of the histogram bins along the x-axis. Default is <code>bin_width=0.5</code> .
<code>normal</code>	Boolean: If <code>True</code> , plots a standard normal distribution over the standardised residual histogram for comparison. Default is <code>normal=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot and display a basic standardised residuals histogram and save it to file as `my_std_residuals_plot.png`:

```
my_rsm.std_residuals_plot(filename='my_std_residuals_plot')
```

To plot and display a standardised residuals histogram with bin width 1, with standard normal curve:

```
self.std_residuals_plot(bin_width=1, normal=True)
```

To plot and display a standardised residuals histogram on a subset of items stored as a list in a variable `my_item_list`:

```
my_rsm.std_residuals_plot(items=my_item_list)
```

5.5.10 `plot_anchor_selection`

Description

Scatter plot of anchor item locations (x-axis) against observed values for the same items (y-axis) to visualise the outlier-trimming diagnostics produced by `calibrate_anchor` (Section 5.3.3) or `dif_test` (Section 5.4.8). Selected and dropped items are plotted in different colours, with a solid $y = x$ identity line for reference and a dotted OLS best-fit line through the selected points only. `plot_anchor_selection` does not use the shared plotting arguments described in Section 5.5.1; its own, smaller set of arguments is described in full below.

Usage

```
self.plot_anchor_selection(result, xmin=-4, xmax=4, ymin=-3, ymax=3, figsize=(8, 6),
                           font='Times New Roman', title_font_size=15, axis_font_size=12,
                           labels_size=12, title='', dpi=300, filename=None,
                           file_format='png')
```

Arguments

<code>result</code>	dict or pandas dataframe: Either the full dictionary returned internally by the robust anchor-selection routine, or just its 'table', i.e. one of <code>self.anchor_selection</code> (from <code>calibrate_anchor</code>) or <code>self.dif_anchor_selection</code> (from <code>dif_test</code>), each a dataframe with 'Anchor', 'Observed' and boolean 'Selected' columns.
<code>xmin, xmax, ymin, ymax</code>	Floats: axis limits. Defaults are <code>xmin=-4, xmax=4, ymin=-3, ymax=3</code> .
<code>figsize</code>	Tuple of two floats: figure size in inches (width, height). Default is <code>figsize=(8, 6)</code> .
<code>font</code>	String: the font to use in the plot. Default is <code>font='Times New Roman'</code> .
<code>title_font_size,</code> <code>axis_font_size,</code> <code>labels_size</code>	Integers: font sizes in points for the title, axis labels and axis tick labels respectively. Defaults are <code>title_font_size=15, axis_font_size=12, labels_size=12</code> .
<code>title</code>	String: plot title. Default is <code>title=''</code> (no title).
<code>filename</code>	String or None: if provided, saves the plot to this path (with the suffix determined by <code>file_format</code>). Default is <code>filename=None</code> (not saved).
<code>file_format</code>	String: file format for saved plots, e.g. 'png', 'pdf', 'svg'. Default is <code>file_format='png'</code> .
<code>dpi</code>	Integer: output resolution in dpi (dots per inch) when saving to file. Default is <code>dpi=300</code> .

Returns

matplotlib figure object. If a `filename` argument is passed, also saves the image to file. `calibrate_anchor` and `dif_test` both call this method automatically when their own `plot` argument is `True` (see Sections 5.3.3 and 5.4.8), so it need not usually be called directly; it is documented separately here since the resulting figure objects (`self.anchor_plot`, `self.dif_plots`) can also be regenerated with different display arguments by calling it directly on the stored selection table.

Example

To regenerate the anchor-selection plot for an existing calibration's diagnostics, stored in `self.anchor_selection`, with wider axis limits and a custom title:

```
self.plot_anchor_selection(self.anchor_selection, xmin=-6, xmax=6, ymin=-6, ymax=6,  
                           title='Anchor calibration')
```

6 class MFRM

6.1 Preliminaries

6.1.1 MFRM

Description

Creates an object of the class `MFRM` from a pandas multiindex dataframe of polytomously scored data of items, rated by multiple raters which share the same maximum score for analysis. No analysis can be run until an object is created.

Usage

```
MFRM(responses, max_score=0, extreme_persons=True, no_of_classes=5, facet='rater',
      facet_plural=None, validate=True)
```

Arguments

responses	pandas multiindex dataframe with items as columns (item names as column names) and raters and persons as the two levels of the multiindex (rater names as index level 0 names and person names as index level 1 names).
max_score	Integer: The maximum possible score, shared across all items. <code>max_score=0</code> (the default) infers the maximum from the data, although passing an argument explicitly is recommended, to avoid issues when the true maximum is never observed.
extreme_persons	Boolean: if <code>False</code> , all persons with extreme scores (all responses correct or all responses incorrect across all raters) are removed from the response dataframe. Default is <code>extreme_persons=True</code> .
no_of_classes	Integer: the number of classes of persons grouped by location for overplotting observed responses on theoretical curves. Default is <code>no_of_classes=5</code>
facet	String: singular name for the second (non-item, non-person) facet of the data – e.g. 'judge' or 'delivery_mode' if the raters in your design are not literally raters. Since <i>RaschPy</i> 's arguments cannot be renamed per instance, method arguments and the shared, generic <code>self.facet_*</code> attributes always use <code>facet_element/facet_elements</code> regardless of this setting; only the convenience <i>attribute</i> aliases (below) pick up the custom name. Default is <code>facet='rater'</code> .

Arguments continue on the next page.

Arguments (continued)

<code>facet_plural</code>	String or <code>None</code> : plural form of <code>facet</code> , used for the pluralised attribute aliases (e.g. <code>self.no_of_judges</code>). If <code>None</code> (default), <code>facet + 's'</code> is used (so the default <code>'rater'</code> pluralises to <code>'raters'</code> automatically) – only needed for an irregular plural (e.g. <code>facet='judge'</code> with an irregular plural you want to override). Default is <code>facet_plural=None</code> .
<code>validate</code>	Boolean: if <code>True</code> , automatically calls <code>check_data_connectivity</code> (Section 6.1.2) on creation and issues a <code>UserWarning</code> if the item response network is disconnected, or if any item has a structurally unresolvable zero in the pairwise item-location matrix. Default is <code>validate=True</code> .

Returns

Object of class `MFRM`. Analyses are run using methods defined on the `MFRM` object, with results stored as attributes of the `MFRM` object.

Several attributes of object `MFRM` are automatically generated on its creation:

<code>self.responses</code>	pandas multiindex dataframe: Dataframe of valid responses.
<code>self.invalid_responses</code>	pandas multiindex dataframe: Dataframe of invalid responses (persons with no responses to any items, i.e. all missing data).
<code>self.extreme_persons</code>	pandas multiindex dataframe: Dataframe of persons with extreme scores (all-correct or all-incorrect across all raters). When <code>extreme_persons=True</code> (the default), extreme scorers are retained in <code>self.responses</code> and this attribute is empty; when <code>extreme_persons=False</code> , they are removed and stored here.
<code>self.max_score</code>	Integer: The maximum possible score, shared across all items.
<code>self.no_of_items</code>	Integer: Number of items.
<code>self.item_names</code> <code>self.item_ids</code>	List: List of item names. <code>self.item_ids</code> is an alias for users who find <code>*_ids</code> more natural than <code>*_names</code> .
<code>self.no_of_persons</code>	Integer: Number of persons.
<code>self.person_names</code> <code>self.person_ids</code>	List: List of unique person names. <code>self.person_ids</code> is an alias for users who find <code>*_ids</code> more natural than <code>*_names</code> .
<code>self.no_of_facet_elements</code>	Integer: Number of facet elements (raters, by default). Aliased as <code>self.no_of_raters</code> and, if a custom facet was passed to the constructor, as <code>self.no_of_{facet_plural}</code> (e.g. <code>self.no_of_judges</code>).

Returns continue on the next page.

Returns (continued)

<code>self.facet_names</code>	List: List of facet element (rater) names. Aliased as <code>self.rater_names/self.rater_ids</code> and, if a custom facet was passed, as <code>self.{facet}_names/self.{facet}_ids</code> and <code>self.{facet_plural}</code> (e.g. <code>self.judge_names</code> , <code>self.judges</code>).
<code>self.no_of_classes</code>	Integer: Number of response classes, defined by the argument <code>no_of_classes</code> .
<code>self.generating</code>	Namespace object (<code>_SimParams</code>): Only present when the model is instantiated by passing a simulation object (e.g. <code>MFRM_Sim_Global</code>) as the <code>responses</code> argument. Contains all attributes of the simulation object (e.g. <code>self.generating.persons</code> , <code>self.generating.items</code> , <code>self.generating.facet_effects</code>), enabling direct comparison of generating and estimated parameters.
<code>self.connectivity_status</code>	Dictionary: result of <code>check_data_connectivity</code> (Section 6.1.2), present only when <code>validate=True</code> . Always contains at least 'connected' (Boolean) and 'components_count' (Integer).

Example

To create an object from a dataframe `my_mfrm_dataframe`, with a maximum score of 5 and 10 observed classes:

```
my_mfrm = MFRM(my_mfrm_dataframe, max_score=5, no_of_classes=10)
```

To create an object where the second facet represents judges rather than raters, so that `self.judge_names/self.judges/self.no_of_judges` are available alongside the generic `self.facet_*` attributes:

```
my_mfrm = MFRM(my_mfrm_dataframe, max_score=5, facet='judge')
```

6.1.2 `check_data_connectivity`

Description

Validates whether the item response network is fully connected, in the same sense as SLM's `check_data_connectivity` (Section 3.1.2): two items are connected if at least one person's response differentiates between them. For MFRM, the underlying directed matrix is one row per item, with entry (i, j) counting persons who scored exactly one point higher on item i than item j , summed across raters – matching `item_locations`'s own matrix exactly (the item-location estimation step used by `calibrate`, Section 6.3.1).

Because each comparison requires the *same* rater to have scored both items for a given person, two items rated only by disjoint sets of raters will register as disconnected here even if every person was rated by someone on both items – this correctly reflects that `item_locations` itself cannot extract a comparison across raters, only within a rater's own scores for a person. This is a stronger requirement than mere co-occurrence, and worth checking directly if a rating design is heavily spiralled (each rater seeing only a subset of items).

Called automatically on object creation when `validate=True` (the default; see Section 6.1.1), storing its result as `self.connectivity_status`.

Usage

```
self.check_data_connectivity()
```

Returns

Dictionary: 'connected', 'components_count', 'directionally_isolated_items' always present; if disconnected, 'isolated_items'/'all_sub_groups' also present.

Example

To manually re-check connectivity on object `my_mfrm`, e.g. after restricting to a subset of raters, and inspect which items ended up isolated:

```
my_mfrm.connectivity_status = my_mfrm.check_data_connectivity()
my_mfrm.connectivity_status.get('isolated_items', [])
```

6.1.3 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm` from `Item_1` to `my_new_item_name`:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

6.1.4 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm.rename_items_all(my_new_item_names)
```

6.1.5 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the second level of the multiindex of `self.responses` with new name.

Example

To rename a person in object `my_mfrm` from `Person_1` to `my_new_person_name`:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

6.1.6 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the second level of the multiindex of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm.rename_persons_all(my_new_person_names)
```

6.1.7 `rename_facet_element`

Description

Method to rename a single facet element (rater, by default).

Usage

```
self.rename_facet_element(old, new)
```

Arguments

<code>old</code>	String: the old name for the facet element (rater)
<code>new</code>	String: the new name for the facet element (rater)

Returns

Replaces specified facet element (rater) name in the first level of the multiindex of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm.rename_facet_element('Rater_1', 'my_new_rater_name')
```

6.1.8 `rename_facet_elements_all`

Description

Method to rename all facet elements (raters, by default).

Usage

```
self.rename_facet_elements_all(new_names)
```

Arguments

<code>new_names</code>	List of new facet element (rater) names as strings, in the same order as <code>self.facet_names</code>
------------------------	--

Returns

Replaces all facet element (rater) names in the first level of the multiindex of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm.rename_facet_elements_all(my_new_rater_names)
```


6.2 Core functions

6.2.1 `cat_prob`

Description

Category probability function which extends the RSM formulation described in 5.2.1 to include an additional facet, taken here to represent a rater (the default facet alias) for convenience, without loss of generality. Calculates the probability $P(X_{nir} = k)$ of person n scoring k , on item i with $k \in \{0, m\}$, where m is the maximum score, with rater r , from person location, central item location, Rasch-Andrich thresholds and rater representation. The precise formulation depends on the rater representation, which may be:

- `global`: a single global scalar representation which assumes uniform rater behaviour across items and thresholds.
- `items`: an extended vector representation which applies a different rater effect for each item but assumes uniform rater behaviour across thresholds.
- `thresholds`: an extended vector representation which applies a different rater effect for each threshold but assumes uniform rater behaviour across items.
- `matrix`: an extended matrix representation which applies a different rater effect for every possible item/threshold combination.
- `bivector`: an extended decomposed matrix representation in which the rater effect for rater r , item i and threshold k is $\lambda_{rik} = \lambda_{ri} + \lambda_{r,k}$, the sum of a per-(rater, item) effect and a per-(rater, threshold) effect (Elliott & Buttery, 2022a, 2025).
- `pseudo_halo/item_stretch`: a restricted form of `items`, characterised by two parameters – a global shift λ_r and a multiplicative item stretch factor ω_r , by which each item location δ_i is scaled.
- `centrality/threshold_stretch`: a restricted form of `thresholds`, the centrality/extremity model (Jin & Wang, 2018), characterised by two parameters – a global shift λ_r and a multiplicative threshold stretch factor ω_r , by which each threshold location τ_k is scaled.
- `bistretch`: a restricted form of `bivector`, characterised by three parameters – a global shift λ_r , a multiplicative item stretch factor ω_{rI} , by which each item location δ_i is scaled, and a multiplicative threshold stretch factor ω_{rT} , by which each threshold location τ_k is scaled.

See Elliott and Buttery (2022a) and Elliott (2025) for a detailed discussion of extended rater representations.

Formulae

`global`

For a person n with location β_n attempting an item i with central item location δ_i , Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$ and rater r with rater effect λ_r , the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i - \lambda_r) - \sum_{t=0}^k \tau_t}}{\sum_{k=0}^m e^{k(\beta_n - \delta_i - \lambda_r) - \sum_{t=0}^k \tau_t}} \quad (19)$$

items

For a person n with location β_n attempting an item i with central item location δ_i , Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$ and rater r with rater effect $\{\lambda_{ri}\}$ for item i , the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i - \lambda_{ri}) - \sum_{t=0}^k \tau_t}}{\sum_{k=0}^m e^{k(\beta_n - \delta_i - \lambda_{ri}) - \sum_{t=0}^k \tau_t}} \quad (20)$$

thresholds

For a person n with location β_n attempting an item i with central item location δ_i , Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$ and rater r with rater effect vector $\{\lambda_{rj}\}$, $j \in \{0, \dots, m\}$, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i) - \sum_{t=0}^k (\tau_t + \lambda_{rt})}}{\sum_{k=0}^m e^{k(\beta_n - \delta_i) - \sum_{t=0}^k (\tau_t + \lambda_{rt})}} \quad (21)$$

matrix

For a person n with location β_n attempting an item i with central item location δ_i , Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$ and rater r with rater effect vector $\{\lambda_{rij}\}$, $j \in \{0, \dots, m\}$ for item i , the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i) - \sum_{t=0}^k (\tau_t + \lambda_{rit})}}{\sum_{k=0}^m e^{k(\beta_n - \delta_i) - \sum_{t=0}^k (\tau_t + \lambda_{rit})}} \quad (22)$$

bivector

For a person n with location β_n attempting an item i with central item location δ_i , Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$ and rater r with rater effect vector $\{\lambda_{rij}\}$, $j \in \{0, \dots, m\}$ for item i , the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i - \lambda_{ri.}) - \sum_{t=0}^k (\tau_t + \lambda_{r.t})}}{\sum_{k=0}^m e^{k(\beta_n - \delta_i - \lambda_{ri.}) - \sum_{t=0}^k (\tau_t + \lambda_{r.t})}} \quad (23)$$

pseudo_halo/item_stretch

For a person n with location β_n attempting an item i with central item location δ_i , Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$ and rater r with severity shift λ_r and item-difficulty-stretch ω_r , the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \omega_r \delta_i - \lambda_r) - \sum_{t=0}^k \tau_t}}{\sum_{k=0}^m e^{k(\beta_n - \omega_r \delta_i - \lambda_r) - \sum_{t=0}^k \tau_t}} \quad (24)$$

centrality/threshold_stretch

For a person n with location β_n attempting an item i with central item location δ_i , Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$ and rater r with severity shift λ_r and threshold-stretch ω_r , the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i - \lambda_r) - \sum_{t=0}^k \omega_r \tau_t}}{\sum_{k=0}^m e^{k(\beta_n - \delta_i - \lambda_r) - \sum_{t=0}^k \omega_r \tau_t}} \quad (25)$$

bistretch

For a person n with location β_n attempting an item i with central item location δ_i , Rasch-Andrich thresholds $\{\tau_0, \dots, \tau_m\}$ and rater r with severity shift λ_r , item-difficulty-stretch ω_{rI} and threshold-stretch ω_{rT} , the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \omega_{rI}\delta_i - \lambda_r) - \sum_{t=0}^k \omega_{rT}\tau_t}}{\sum_{k=0}^m e^{k(\beta_n - \omega_{rI}\delta_i - \lambda_r) - \sum_{t=0}^k \omega_{rT}\tau_t}} \quad (26)$$

Usage

The global, items, thresholds and matrix rater representations share the same argument format:

```
self.cat_prob_global(person_location, item, item_locations, facet_element, facet_effects,
                    category, thresholds)

self.cat_prob_items(person_location, item, item_locations, facet_element, facet_effects,
                   category, thresholds)

self.cat_prob_thresholds(person_location, item, item_locations, facet_element, facet_effects,
                        category, thresholds)

self.cat_prob_matrix(person_location, item, item_locations, facet_element, facet_effects,
                    category, thresholds)
```

The bivector rater representation takes the decomposed item and threshold facet effects as two separate arguments instead of one combined `facet_effects` argument:

```
self.cat_prob_bivector(person_location, item, item_locations, facet_element,
                      facet_item_effects, facet_threshold_effects, category, thresholds)
```

`centrality`, `pseudo_halo` and `bistretch` instead take the rater's own raw parameters directly – a severity shift λ_r plus one or two stretch parameters – rather than a reconstructed `thresholds/items/matrix-shaped` vector, since each is a closed-form restriction with genuinely fewer free parameters than its parent representation:

```
self.cat_prob_centrality(person_location, item, item_locations, facet_element,
                        lambda_r, omega_r, category, thresholds)
```

```
self.cat_prob_pseudo_halo(person_location, item, item_locations, facet_element,
                        lambda_r, omega_r, category, thresholds)
```

```
self.cat_prob_bistretch(person_location, item, item_locations, facet_element, lambda_r,
                      omega_items_r, omega_thresholds_r, category, thresholds)
```

`facet_item_effects/facet_threshold_effects` may also be passed under the dynamic aliases defined by `self.facet`, e.g. `judge_item_effects` if `self.facet = 'judge'`, default `rater_item_effects/rater_threshold_effects`.

Arguments

<code>person_location</code>	Float: Person location
<code>item</code>	Name of item.
<code>item_locations</code>	pandas series: Item names as keys and central item locations as values.
<code>facet_element</code>	Name of facet_element.
<code>facet_effects</code>	pandas series or dictionary: Rater names as keys and the appropriate rater representations for <code>global</code> , <code>items</code> , <code>thresholds</code> or <code>matrix</code> as values. Not used for <code>bivector</code> (see <code>facet_item_effects/facet_threshold_effects</code> below) or for <code>centrality/pseudo_halo/bistretch</code> (see <code>lambda_r/omega_r</code> below). Details of the formats for facet effects for the different representations are given below.
<code>facet_item_effects</code>	pandas DataFrame or dictionary: <code>bivector</code> only. Rater names as row index (or dictionary keys) and item names as columns (or, for a dictionary, item names as keys of a per-rater pandas series) – e.g. <code>self.facet_effects_bivector_items</code> .
<code>facet_threshold_effects</code>	pandas DataFrame or dictionary: <code>bivector</code> only. Rater names as row index (or dictionary keys) and threshold positions as columns (or array values), zero-summed per rater – e.g. <code>self.facet_effects_bivector_thresholds</code> .
<code>lambda_r</code>	Float: <code>centrality/pseudo_halo/bistretch</code> only. The rater’s own severity shift – e.g. <code>self.lambda_centrality.loc[facet_element]</code> .
<code>omega_r</code>	Float: <code>centrality/pseudo_halo</code> only. The rater’s own stretch parameter – threshold-stretch for <code>centrality</code> (e.g. <code>self.omega_centrality.loc[facet_element]</code>), item-difficulty-stretch for <code>pseudo_halo</code> (e.g. <code>self.omega_pseudo_halo.loc[facet_element]</code>).
<code>omega_items_r</code>	Float: <code>bistretch</code> only. The rater’s own item-difficulty-stretch – e.g. <code>self.omega_items_bistretch.loc[facet_element]</code> .
<code>omega_thresholds_r</code>	Float: <code>bistretch</code> only. The rater’s own threshold-stretch – e.g. <code>self.omega_thresholds_bistretch.loc[facet_element]</code> .
<code>category</code>	Integer: Response category k , with $k \in \{0, 1\}$.
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Rater representations for `facet_effects` argument

Toy examples of each rater representation format, with two raters and two items with three categories for `global`, `items`, `thresholds`, `matrix` and `bivector`:

Rater representation `global`: pandas series with floats (facet effects) for values.

```
facet_effects = pd.Series({'Rater_1': -0.5,
                           'Rater_2': 0.5})
```

Rater representation `items`: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
facet_effects = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),
                 'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
```

Rater representation `thresholds`: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                 'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

Rater representation `matrix`: Nested dictionary. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1': {'Item_1': np.array([0, -1.5, -1, 0.5]),
                             'Item_2': np.array([0, 1, 0, -1])},
                 'Rater_2': {'Item_1': np.array([0, -0.5, 0, 0.5]),
                             'Item_2': np.array([0, 2, 1, 0])}}
```

Rater representation `bivector` uses two separate arguments instead of `facet_effects`: `facet_item_effects` (e.g. `self.facet_effects.bivector_items`) and `facet_threshold_effects` (e.g. `self.facet_effects.bivector_thresholds`), both generated by `calibrate.bivector`.

```
facet_item_effects = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),
                     'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
facet_threshold_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                          'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

`centrality`, `pseudo_halo` and `bistretch` take their own λ_r/ω_r parameters directly instead (see **Usage** above and the worked example below).

Returns

Float: probability of obtaining score k .

Example

To obtain the probability of a person of location 0.5 scoring 1 on 'Item_1' from central item locations set `my_item_diffs` with a set of Rasch-Andrich thresholds `my_thresholds`, rated by rater 'Rater_1' from rater facet effects set `my_facet_effects`, using the matrix rater representation, and store the result as a variable `my_cat_prob`:

```
my_cat_prob = self.cat_prob.matrix(0.5, 'Item_1', my_item_diffs, 'Rater_1', my_facet_effects,
                                   1, my_thresholds)
```

The same format is followed for the `global`, `items` and `thresholds` rater representations, with no changes apart from the method name.

For the bivector rater representation, from item facet effects set `my_facet_item_effects` and from threshold facet effects set `my_facet_threshold_effects`:

```
my_cat_prob = self.cat_prob.bivector(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                     my_facet_item_effects, my_facet_threshold_effects, 1,
                                     my_thresholds)
```

For centrality, `pseudo_halo` and `bistretch`, given a rater's global shift set `my_lambda_r` (or alias `my_global_pseudo_halo_r`, `my_global_centrality_r` or `my_global_bistretch_r`) and stretch parameter(s) `my_omega_r`, or `my_omega_items_r/my_omega_thresholds_r` for `bistretch` (or aliases `my_stretch_pseudo_halo_r`, `my_stretch_centrality_r` or `my_stretch_items_bistretch_r/my_stretch_thresholds_bistretch_r`) – e.g. from `self.lambda_centrality.loc['Rater_1']/self.omega_centrality.loc['Rater_1']` after calling `calibrate(model='centrality')`:

```
my_cat_prob = self.cat_prob.centrality(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                       my_lambda_r, my_omega_r, 1, my_thresholds)
```

`cat_prob_pseudo_halo` follows the identical pattern; `cat_prob_bistretch` takes `my_omega_items_r` and `my_omega_thresholds_r` in place of the single `my_omega_r`.

6.2.2 exp_score

Description

Expected score function which calculates the expected score $E(X_{nir})$, rated by rater r from person location, central item location, Rasch-Andrich thresholds and rater facet effect representation. The expected score is given by:

$$E(X_{nir}) = \sum_{k=0}^m kP(X_{nir} = k) \quad (27)$$

where m is the maximum score and $P(X_{nir} = k)$ is the relevant category probability equation for the rater representation, described in Section 6.2.1.

Usage

The global, items, thresholds and matrix rater representations share the same argument format:

```
self.exp_score_global(person_location, item, item_locations, facet_element, facet_effects,
                      thresholds)

self.exp_score_items(person_location, item, item_locations, facet_element, facet_effects,
                     thresholds)

self.exp_score_thresholds(person_location, item, item_locations, facet_element, facet_effects,
                           thresholds)

self.exp_score_matrix(person_location, item, item_locations, facet_element, facet_effects,
                       thresholds)
```

The bivector rater representation takes the decomposed item and threshold facet effects as two separate arguments instead of one combined `facet_effects` argument:

```
self.exp_score_bivector(person_location, item, item_locations, facet_element,
                        facet_item_effects, facet_threshold_effects, thresholds)

centrality, pseudo_halo and bistretch instead take the rater's own raw parameters directly – see cat_
prob (Section 6.2.1) for the full rationale:

self.exp_score_centrality(person_location, item, item_locations, facet_element,
                           lambda_r, omega_r, thresholds)

self.exp_score_pseudo_halo(person_location, item, item_locations, facet_element,
                            lambda_r, omega_r, thresholds)

self.exp_score_bistretch(person_location, item, item_locations, facet_element, lambda_r,
                          omega_items_r, omega_thresholds_r, thresholds)
```

`facet_item_effects/facet_threshold_effects` may also be passed under the dynamic aliases defined by `self.facet`, e.g. `judge_item_effects` if `self.facet = 'judge'`, default `rater_item_effects/rater_threshold_effects`.

Arguments

<code>person_location</code>	Float: Person location
<code>item</code>	Name of item.
<code>item_locations</code>	pandas series: Item names as keys and central item locations as values.
<code>facet_element</code>	Name of facet_element.
<code>facet_effects</code>	pandas series or dictionary: Rater names as keys and the appropriate rater representations for <code>global</code> , <code>items</code> , <code>thresholds</code> or <code>matrix</code> as values. Not used for <code>bivector</code> (see <code>facet_item_effects/facet_threshold_effects</code> below) or for <code>centrality/pseudo_halo/bistretch</code> (see <code>lambda_r/omega_r</code> below). Details of the formats for facet effects for the different representations are given below.
<code>facet_item_effects</code>	pandas DataFrame or dictionary: <code>bivector</code> only. Rater names as row index (or dictionary keys) and item names as columns (or, for a dictionary, item names as keys of a per-rater pandas series) – e.g. <code>self.facet_effects_bivector_items</code> .
<code>facet_threshold_effects</code>	pandas DataFrame or dictionary: <code>bivector</code> only. Rater names as row index (or dictionary keys) and threshold positions as columns (or array values), zero-summed per rater – e.g. <code>self.facet_effects_bivector_thresholds</code> .
<code>lambda_r</code>	Float: <code>centrality/pseudo_halo/bistretch</code> only. The rater's own severity shift.
<code>omega_r</code>	Float: <code>centrality/pseudo_halo</code> only. The rater's own stretch parameter.
<code>omega_items_r</code>	Float: <code>bistretch</code> only. The rater's own item-difficulty-stretch.
<code>omega_thresholds_r</code>	Float: <code>bistretch</code> only. The rater's own threshold-stretch.
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Rater representations for `facet_effects` argument

Toy examples of each rater representation format, with two raters and two items with three categories for `global`, `items`, `thresholds`, `matrix` and `bivector`:

Rater representation `global`: pandas series with floats (facet effects) for values.

```
facet_effects = pd.Series({'Rater_1': -0.5,
                          'Rater_2': 0.5})
```


Rater representation `items`: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
facet_effects = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),
                 'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
```

Rater representation `thresholds`: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                 'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

Rater representation `matrix`: Nested dictionary. Outer dictionary has item names for values, which are also keys for inner dictionary. Values for inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1': {'Item_1': np.array([0, -1.5, -1, 0.5]),
                             'Item_2': np.array([0, 1, 0, -1])},
                 'Rater_2': {'Item_1': np.array([0, -0.5, 0, 0.5]),
                             'Item_2': np.array([0, 2, 1, 0])}}
```

Rater representation `bivector` uses two separate arguments instead of `facet_effects`: `facet_item_effects` (e.g. `self.facet_effects.bivector_items`) and `facet_threshold_effects` (e.g. `self.facet_effects.bivector_thresholds`), both generated by `calibrate_bivector`.

```
facet_item_effects = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),
                     'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
facet_threshold_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                          'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

`centrality`, `pseudo_halo` and `bistretch` take their own λ_r/ω_r parameters directly instead (see **Usage** above and the worked example below).

Returns

Float: expected score.

Example

To obtain the expected score for a person of location 0.5 on 'Item_1' from central item locations set `my_item_diffs` with a set of Rasch-Andrich thresholds `my_thresholds`, rated by rater 'Rater_1' from rater facet effects set `my_facet_effects`, using the matrix rater representation, and store the result as a variable `my_exp_score`:

```
my_exp_score = self.exp_score_matrix(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                    my_facet_effects, my_thresholds)
```

The same format is followed for the `global`, `items` and `thresholds` rater representations, with no changes apart from the method name.

For the bivector rater representation, from item facet effects set `my_facet_item_effects` and threshold facet effects set `my_facet_threshold_effects`:

```
my_exp_score = self.exp_score_bivector(0.5, 'Item_1', my_item_diffs, 'Rater_1',  
                                       my_facet_item_effects, my_facet_threshold_effects,  
                                       my_thresholds)
```

For centrality, pseudo_halo and bistretch, from that rater's own severity shift set `my_lambda_r` and stretch parameter(s) `my_omega_r` (or `my_omega_items_r/my_omega_thresholds_r` for bistretch):

```
my_exp_score = self.exp_score_centrality(0.5, 'Item_1', my_item_diffs, 'Rater_1',  
                                       my_lambda_r, my_omega_r, my_thresholds)
```

6.2.3 variance

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location, item location and Rasch-Andrich thresholds. The variance is given by:

$$V(X_{nir}) = \sum_{k=0}^m P(X_{nir} = k)(k - E(X_{nir}))^2 \quad (28)$$

where m is the maximum score and $P(X_{nir} = k)$ and $E(X_{nir})$ as described in Sections 6.2.1 and 6.2.2.

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

The global, items, thresholds and matrix rater representations share the same argument format:

```
self.variance_global(person_location, item, item_locations, facet_element, facet_effects,
                    thresholds)

self.variance_items(person_location, item, item_locations, facet_element, facet_effects,
                   thresholds)

self.variance_thresholds(person_location, item, item_locations, facet_element, facet_effects,
                        thresholds)

self.variance_matrix(person_location, item, item_locations, facet_element, facet_effects,
                    thresholds)
```

The bivector rater representation takes the decomposed item and threshold facet effects as two separate arguments instead of one combined `facet_effects` argument:

```
self.variance_bivector(person_location, item, item_locations, facet_element,
                      facet_item_effects, facet_threshold_effects, thresholds)

centrality, pseudo_halo and bistretch instead take the rater's own raw parameters directly – see cat_prob (Section 6.2.1) for the full rationale:

self.variance_centrality(person_location, item, item_locations, facet_element,
                       lambda_r, omega_r, thresholds)

self.variance_pseudo_halo(person_location, item, item_locations, facet_element,
                         lambda_r, omega_r, thresholds)

self.variance_bistretch(person_location, item, item_locations, facet_element, lambda_r,
                      omega_items_r, omega_thresholds_r, thresholds)
```

`facet_item_effects/facet_threshold_effects` may also be passed under the dynamic aliases defined by `self.facet`, e.g. `judge_item_effects` if `self.facet = 'judge'`, default `rater_item_effects/rater_threshold_effects`.

Arguments

<code>person_location</code>	Float: Person location
<code>item</code>	Name of item.
<code>item_locations</code>	pandas series: Item names as keys and central item locations as values.
<code>facet_element</code>	Name of facet_element.
<code>facet_effects</code>	pandas series or dictionary: Rater names as keys and the appropriate rater representations for <code>global</code> , <code>items</code> , <code>thresholds</code> or <code>matrix</code> as values. Not used for <code>bivector</code> (see <code>facet_item_effects</code> / <code>facet_threshold_effects</code> below) or for <code>centrality</code> / <code>pseudo_halo</code> / <code>bistretch</code> (see <code>lambda_r</code> / <code>omega_r</code> below). Details of the formats for facet effects for the different representations are given below.
<code>facet_item_effects</code>	pandas DataFrame or dictionary: <code>bivector</code> only. Rater names as row index (or dictionary keys) and item names as columns (or, for a dictionary, item names as keys of a per-rater pandas series) – e.g. <code>self.facet_effects_bivector_items</code> .
<code>facet_threshold_effects</code>	pandas DataFrame or dictionary: <code>bivector</code> only. Rater names as row index (or dictionary keys) and threshold positions as columns (or array values), zero-summed per rater – e.g. <code>self.facet_effects_bivector_thresholds</code> .
<code>lambda_r</code>	Float: <code>centrality</code> / <code>pseudo_halo</code> / <code>bistretch</code> only. The rater's own severity shift.
<code>omega_r</code>	Float: <code>centrality</code> / <code>pseudo_halo</code> only. The rater's own stretch parameter.
<code>omega_items_r</code>	Float: <code>bistretch</code> only. The rater's own item-difficulty-stretch.
<code>omega_thresholds_r</code>	Float: <code>bistretch</code> only. The rater's own threshold-stretch.
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Rater representations for `facet_effects` argument

Toy examples of each rater representation format, with two raters and two items with three categories for `global`, `items`, `thresholds`, `matrix` and `bivector`:

Rater representation `global`: pandas series with floats (facet effects) for values.

```
facet_effects = pd.Series({'Rater_1': -0.5,
                          'Rater_2': 0.5})
```

Rater representation `items`: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
facet_effects = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),
                 'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
```

Rater representation `thresholds`: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                 'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

Rater representation `matrix`: Nested dictionary. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1': {'Item_1': np.array([0, -1.5, -1, 0.5]),
                             'Item_2': np.array([0, 1, 0, -1])},
                 'Rater_2': {'Item_1': np.array([0, -0.5, 0, 0.5]),
                             'Item_2': np.array([0, 2, 1, 0])}}
```

Rater representation `bivector` uses two separate arguments instead of `facet_effects`: `facet_item_effects` (e.g. `self.facet_effects.bivector_items`) and `facet_threshold_effects` (e.g. `self.facet_effects.bivector_thresholds`), both generated by `calibrate_bivector`.

```
facet_item_effects = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),
                     'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
facet_threshold_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                           'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

`centrality`, `pseudo_halo` and `bistretch` take their own raw λ_r/ω_r parameters directly instead (see **Usage** above and the worked example below).

Returns

Float: variance

Example

To obtain the variance for a person of location 0.5 on 'Item_1' from central item locations set `my_item_diffs` with a set of Rasch-Andrich thresholds `my_thresholds`, rated by rater 'Rater_1' from rater facet effects set `my_facet_effects`, using the matrix rater representation, and store the result as a variable `my_variance`:

```
my_variance = self.variance_matrix(0.5, 'Item_1', my_item_diffs, 'Rater_1', my_facet_effects,
                                  my_thresholds)
```

The same format is followed for the `global`, `items` and `thresholds` rater representations, with no changes apart from the method name.

For the bivector rater representation, from item facet effects set `my_facet_item_effects` and threshold facet effects set `my_facet_threshold_effects`:

```
my_variance = self.variance_bivector(0.5, 'Item_1', my_item_diffs, 'Rater_1',  
                                     my_facet_item_effects, my_facet_threshold_effects,  
                                     my_thresholds)
```

For centrality, pseudo_halo and bistretch, from that rater's own severity shift set `my_lambda_r` and stretch parameter(s) `my_omega_r` (or `my_omega_items_r/my_omega_thresholds_r` for bistretch):

```
my_variance = self.variance_centrality(0.5, 'Item_1', my_item_diffs, 'Rater_1',  
                                       my_lambda_r, my_omega_r, my_thresholds)
```

6.2.4 kurtosis

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location, central item location and Rasch-Andrich thresholds. The variance is given by:

$$\kappa(X_{nir}) = \sum_{k=0}^m P(X_{nir} = k)(k - E(X_{nir}))^4 \quad (29)$$

where m is the maximum score and $P(X_{nir} = k)$ and $E(X_{nir})$ are as described in Sections 6.2.1 and 6.2.2 respectively.

Usage

The global, items, thresholds and matrix rater representations share the same argument format:

```
self.kurtosis_global(person_location, item, item_locations, facet_element, facet_effects,
                    thresholds)

self.kurtosis_items(person_location, item, item_locations, facet_element, facet_effects,
                   thresholds)

self.kurtosis_thresholds(person_location, item, item_locations,
                        facet_element, facet_effects, thresholds)

self.kurtosis_matrix(person_location, item, item_locations, facet_element, facet_effects,
                    thresholds)
```

The bivector rater representation takes the decomposed item and threshold facet effects as two separate arguments instead of one combined `facet_effects` argument:

```
self.kurtosis_bivector(person_location, item, item_locations, facet_element,
                     facet_item_effects, facet_threshold_effects, thresholds)

centrality, pseudo_halo and bistretch instead take the rater's own raw parameters directly – see cat_
prob (Section 6.2.1) for the full rationale:

self.kurtosis_centrality(person_location, item, item_locations, facet_element,
                       lambda_r, omega_r, thresholds)

self.kurtosis_pseudo_halo(person_location, item, item_locations, facet_element,
                        lambda_r, omega_r, thresholds)

self.kurtosis_bistretch(person_location, item, item_locations, facet_element, lambda_r,
                      omega_items_r, omega_thresholds_r, thresholds)

facet_item_effects/facet_threshold_effects may also be passed under the dynamic aliases defined
by self.facet, e.g. judge_item_effects if self.facet = 'judge', default rater_item_effects/rater_
threshold_effects.
```

Arguments

<code>person_location</code>	Float: Person location
<code>item</code>	Name of item.
<code>item_locations</code>	pandas series: Item names as keys and central item locations as values.
<code>facet_element</code>	Name of facet_element.
<code>facet_effects</code>	pandas series or dictionary: Rater names as keys and the appropriate rater representations for <code>global</code> , <code>items</code> , <code>thresholds</code> or <code>matrix</code> as values. Not used for <code>bivector</code> (see <code>facet_item_effects/facet_threshold_effects</code> below) or for <code>centrality/pseudo_halo/bistretch</code> (see <code>lambda_r/omega_r</code> below). Details of the formats for facet effects for the different representations are given below.
<code>facet_item_effects</code>	pandas DataFrame or dictionary: <code>bivector</code> only. Rater names as row index (or dictionary keys) and item names as columns (or, for a dictionary, item names as keys of a per-rater pandas series) – e.g. <code>self.facet_effects_bivector_items</code> .
<code>facet_threshold_effects</code>	pandas DataFrame or dictionary: <code>bivector</code> only. Rater names as row index (or dictionary keys) and threshold positions as columns (or array values), zero-summed per rater – e.g. <code>self.facet_effects_bivector_thresholds</code> .
<code>lambda_r</code>	Float: <code>centrality/pseudo_halo/bistretch</code> only. The rater's own severity shift.
<code>omega_r</code>	Float: <code>centrality/pseudo_halo</code> only. The rater's own stretch parameter.
<code>omega_items_r</code>	Float: <code>bistretch</code> only. The rater's own item-difficulty-stretch.
<code>omega_thresholds_r</code>	Float: <code>bistretch</code> only. The rater's own threshold-stretch.
<code>thresholds</code>	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m , not $m + 1$.

Rater representations for `facet_effects` argument

Toy examples of each rater representation format, with two raters and two items with three categories for `global`, `items`, `thresholds`, `matrix` and `bivector`:

Rater representation `global`: pandas series with floats (facet effects) for values.

```
facet_effects = pd.Series({'Rater_1': -0.5,
                          'Rater_2': 0.5})
```


Rater representation `items`: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
facet_effects = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),
                 'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
```

Rater representation `thresholds`: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                 'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

Rater representation `matrix`: Nested dictionary. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1': {'Item_1': np.array([0, -1.5, -1, 0.5]),
                             'Item_2': np.array([0, 1, 0, -1])},
                 'Rater_2': {'Item_1': np.array([0, -0.5, 0, 0.5]),
                             'Item_2': np.array([0, 2, 1, 0])}}
```

Rater representation `bivector` uses two separate arguments instead of `facet_effects`: `facet_item_effects` (e.g. `self.facet_effects.bivector_items`) and `facet_threshold_effects` (e.g. `self.facet_effects.bivector_thresholds`), both generated by `calibrate_bivector`.

```
facet_item_effects = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),
                     'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
facet_threshold_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                           'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

`centrality`, `pseudo_halo` and `bistretch` take their own λ_r/ω_r parameters directly instead (see **Usage** above and the worked example below).

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 on 'Item_1' from central item locations set `my_item_diffs` with a set of Rasch-Andrich thresholds `my_thresholds`, rated by rater 'Rater_1' from rater facet effects set `my_facet_effects`, using the matrix rater representation, and store the result as a variable `my_kurtosis`:

```
my_kurtosis = self.kurtosis_matrix(0.5, 'Item_1', my_item_diffs, 'Rater_1', my_facet_effects,
                                  my_thresholds)
```

The same format is followed for the `global`, `items` and `thresholds` rater representations, with no changes apart from the method name.

For the bivector rater representation, from item facet effects set `my_facet_item_effects` and threshold facet effects set `my_facet_threshold_effects`:

```
my_kurtosis = self.kurtosis_bivector(0.5, 'Item_1', my_item_diffs, 'Rater_1',  
                                     my_facet_item_effects, my_facet_threshold_effects,  
                                     my_thresholds)
```

For centrality, pseudo_halo and bistretch, from that rater's own severity shift set `my_lambda_r` and stretch parameter(s) `my_omega_r` (or `my_omega_items_r/my_omega_thresholds_r` for bistretch):

```
my_kurtosis = self.kurtosis_centrality(0.5, 'Item_1', my_item_diffs, 'Rater_1',  
                                       my_lambda_r, my_omega_r, my_thresholds)
```

6.3 Parameter estimation

6.3.1 `calibrate`

Description

Produces central item location, Rasch-Andrich threshold and rater facet effect estimates using the conditional pairwise estimation (CPAT) algorithm (Elliott & Buttery, 2022b).

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.calibrate_global(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001)
self.calibrate_items(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001)
self.calibrate_thresholds(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001)
self.calibrate_matrix(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001)
self.calibrate_bivector(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                        matrix_marginals=True)
self.calibrate_centrality(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                          regression='huber')
self.calibrate_pseudo_halo(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                           regression='huber')
self.calibrate_bistretch(constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                         matrix_marginals=True, regression='huber')
```

`centrality`, `pseudo_halo` and `bistretch` are restricted cases of `thresholds`, `items` and `bivector` respectively, recovered directly from the parameters of their corresponding vector representations via robust regression.

`threshold_stretch` and `item_stretch` are descriptive aliases for `centrality` and `pseudo_halo` respectively. They may be used anywhere a model name is accepted throughout this chapter, both as the `model=/models=` value and as the method-name suffix (e.g. `self.calibrate_threshold_stretch()` is identical to `self.calibrate_centrality()`); resulting attributes are still stored under the canonical `_centrality/_pseudo_halo` names regardless of which spelling was used. λ_r is used for the global shift parameter throughout all three representations since it is additive, matching the notation used for facet effects elsewhere in this chapter (see, e.g., the `global` representation's own λ_r in Section 6.2.1, to which each of these three representations collapses when its stretch parameter(s) are neutral). The stretch parameter is given its own distinct letter rather than reusing λ , since it is multiplicative rather than additive: ω_r throughout, following Jin & Wang's (Jin & Wang, 2018) own notation for their rater stretch factor (which they apply to the threshold axis: what they called the centrality/extremity model and what this package calls `centrality`). `pseudo_halo` reuses ω_r for its own item-difficulty-stretch for consistency;; and `bistretch` needs both axes at once, so its two stretch parameters are disambiguated as $\omega_{\text{items},r}$ and $\omega_{\text{thresholds},r}$ – stored as `self.omega_items_bistretch` and `self.omega_thresholds_bistretch` respectively.

λ_r and ω_r (or, for `bistretch`, its two ω variants) also have alternate spellings, matching the pattern `self.global_{model} / self.stretch_{model}` (e.g. `self.global_centrality` and `self.stretch_centrality` are exact synonyms for `self.lambda_centrality` and `self.omega_centrality`, and `self.stretch_items_bistretch/self.stretch_thresholds_bistretch` are synonyms for the two ω variants above) – `global` because $\omega_r = 1$ collapses the representation to `global`, and it matches `global`’s own single-subscript λ_r notation. These alternate spellings work like `threshold_stretch/item_stretch` above: they resolve to the canonical attribute wherever it exists, and any attribute derived from it (bootstrap SEs, CIs) is reachable the same way.

For `centrality`, `pseudo_halo` and `bistretch`, each `facet_element`’s raw parameters are recovered from its free-representation deviation values (`facet_effects_thresholds` for `centrality`, `facet_effects_items` for `pseudo_halo`, both of `bivector`’s marginal vectors for `bistretch`) via a regression of those deviation values against the corresponding reference item/threshold values, one `facet_element` at a time. Two robust methods are offered:

- `regression='huber'` (default): `sklearn.linear_model.HuberRegressor`, which effectively uses a squared error for small differences and MAE for large differences so is more robust to outliers.
- `regression='theil-sen'`: the median of pairwise slopes (slope = median over every pair of points of $(dev_i - dev_j)/(ref_i - ref_j)$; intercept = median of $dev_k - \text{slope} \cdot ref_k$).

‘`lambda_r`’/‘`omega_r`’ are recovered per rater via a robust regression of that rater’s raw free-model deviation values against the reference item/threshold values, controlled by ‘`calibrate(..., regression='huber')`’ (default) or ‘`regression='theil-sen'`’. ‘`huber`’ (an M-estimator that downweights, rather than discards, points with large residuals) was the most accurate of several methods tested empirically, including plain and weighted least-squares — see the manual for the full comparison. ‘`theil-sen`’ (median of pairwise slopes, fully closed-form) is a close second, with nothing to converge. If ‘`huber`’’s underlying M-estimator fit fails to converge for a given rater, automatically falls back to ‘`theil-sen`’ with a ‘`UserWarning`’; other raters are unaffected.

Arguments

<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> .
<code>method</code>	String: method for derivation of central item location and rater facet effect estimates from pairwise reciprocal matrices (Elliott & Buttery, 2022b). Options are ‘ <code>ls</code> ’ for least squares (Choppin, 1968, 1985), ‘ <code>evm</code> ’ for the eigenvector method (Garner & Engelhard, 2002), ‘ <code>cos</code> ’ for cosine similarity (Kou & Lin, 2014) or ‘ <code>log-lik</code> ’ for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>

Arguments continue on the next page.

Arguments (continued)

<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
<code>matrix_marginals</code>	<code>bivector</code> and <code>bistretch</code> representations only. Boolean: if <code>True</code> (default), the item and threshold vectors underlying the bivector decomposition are estimated as marginal means of the full matrix representation's own PAIR estimates (Elliott & Buttery, 2022b). If <code>False</code> , each vector is instead estimated directly from its own pooled PAIR matrix (items pooled across thresholds; thresholds pooled across items, corrected for the mean per-facet-element item effect). Ignored for the other six representations. Default is <code>matrix_marginals=True</code> .
<code>regression</code>	<code>centrality/pseudo_halo/bistretch</code> representations only. String, one of 'huber' or 'theil-sen': the robust regression method used to recover each <code>facet_element</code> 's raw λ_r/ω_r (see the note above for the full rationale and the automatic 'huber' $\rightarrow$ 'theil-sen' fallback behaviour). Ignored for the other five representations. Default is <code>regression='huber'</code> .

Returns

Returns four attributes:

<code>self.items</code>	pandas series: Item location estimates with the item names as keys and estimates as values.
<code>self.thresholds</code>	pandas series: Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, an array of m estimates, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed; the series now has length m , not $m + 1$.

Returns continue on the next page.

Returns (continued)

<code>self.facet_effects</code>	pandas series/DataFrame: Rater names as rows and the appropriate rater representations for <code>global</code>, <code>items</code>, <code>thresholds</code>, <code>matrix</code> or <code>bivector</code> as columns. For <code>bivector</code>, two attributes are stored: <code>self.facet_effects_bivector_items</code> ($R \times I$ DataFrame of per-(rater, item) effects) and <code>self.facet_effects_bivector_thresholds</code> ($R \times K$ DataFrame of per-(rater, threshold) effects). The reconstructed full matrix is available as <code>self.facet_effects_bivector</code>. For <code>centrality</code>, <code>pseudo_halo</code> and <code>bistretch</code>, returns pandas DataFrame with rater names as rows and parameters as columns – <code>lambda</code> plus <code>omega</code> (or, for <code>bistretch</code>, <code>omega_items</code> and <code>omega_thresholds</code>). Full reconstructed vector representatin is stored separately, as <code>self.facet_effects_{model}_full_vector</code>.
<code>self.cat_widths</code>	numpy array: Array of $m - 1$ category width estimates, where m is the maximum score.

Example `self.facet_effects` outputs

Toy examples of each rater representation format, with two raters and two items with three categories:

Rater representation `global`: pandas series, rater names as the index, one facet effect per rater (no column name, since a series has none).

Rater_1	-0.5
Rater_2	0.5

Table 21: `self.facet_effects_global`

Rater representation `items`: pandas DataFrame, rater names as the row index and item names as columns.

	Item_1	Item_2
Rater_1	-1	0
Rater_2	0	1

Table 22: `self.facet_effects_items`

Rater representation `thresholds`: pandas DataFrame, rater names as the row index and threshold positions (1 to `self.max_score`) as columns.

	1	2	3
Rater_1	-1.0	-0.5	0.0
Rater_2	1.5	0.5	-0.5

Table 23: `self.facet_effects_thresholds`

Rater representation `matrix`: pandas MultiIndex DataFrame, rater and item names as a two-level row index, threshold positions (1 to `self.max_score`) as columns.

		1	2	3
Rater_1	Item_1	-1.5	-1.0	0.5
	Item_2	1.0	0.0	-1.0
Rater_2	Item_1	-0.5	0.0	0.5
	Item_2	2.0	1.0	0.0

Table 24: `self.facet_effects_matrix`

Rater representation `bivector`: two DataFrames are produced. `self.facet_effects_bivector_items` has rater names as the row index and item names as columns (per-rater, per-item facet effects):

	Item_1	Item_2
Rater_1	-0.5	-0.3
Rater_2	0.5	0.3

Table 25: `self.facet_effects_bivector_items`

`self.facet_effects_bivector_thresholds` has rater names as the row index and threshold positions as columns (zero-sum per rater):

	1	2	3
Rater_1	-0.2	0.0	0.2
Rater_2	0.3	0.0	-0.3

Table 26: `self.facet_effects_bivector_thresholds`

The reconstructed full matrix `self.facet_effects_bivector` (item effect plus threshold effect per cell) has the same `MultiIndex` format as `self.facet_effects_matrix` above:

		1	2	3
Rater_1	Item_1	-0.7	-0.5	-0.3
	Item_2	-0.5	-0.3	-0.1
Rater_2	Item_1	0.8	0.5	0.2
	Item_2	0.6	0.3	0.0

Table 27: `self.facet_effects_bivector`

Rater representation `centrality` (Jin & Wang, 2018): two parameters per rater, `self.lambda_centrality` (severity shift) and `self.omega_centrality` (threshold-stretch; also reachable as `self.stretch_centrality`), columns of `DataFrame` as `self.facet_effects_centrality`:

	lambda	omega
Rater_1	-0.5	1.5
Rater_2	0.5	0.6

Table 28: `self.lambda_centrality`, `self.omega_centrality`

`self.facet_effects_centrality_full_vector` is the reconstruction of this into the same $R \times K$ format as `self.facet_effects_thresholds`, for compatibility with every downstream method that expects that shape:

	1	2	3
Rater_1	-1.0	-0.5	0.0
Rater_2	0.9	0.5	0.1

Table 29: `self.facet_effects_centrality_full_vector`

Rater representation `pseudo_halo`: two free parameters per rater, `self.lambda_pseudo_halo` (severity shift) and `self.omega_pseudo_halo` (item-difficulty-stretch), columns of `DataFrame` as `self.facet_effects_pseudo_halo`:

	lambda	omega
Rater_1	-0.5	1.5
Rater_2	0.5	0.6

Table 30: `self.lambda_pseudo_halo`, `self.omega_pseudo_halo`

`self.facet_effects_pseudo_halo_full_vector` is the reconstruction of this into the same $R \times I$ format as `self.facet_effects_items`:

	Item_1	Item_2
Rater_1	-1.0	0.0
Rater_2	0.9	0.1

Table 31: `self.facet_effects_pseudo_halo_full_vector`

Rater representation `bistretch`: three free parameters per rater, `self.lambda_bistretch` (severity shift), `self.omega_items_bistretch` (item-stretch; also reachable as `self.stretch_items_bistretch`) and `self.omega_thresholds_bistretch` (threshold-stretch; also reachable as `self.stretch_thresholds_bistretch`), each a pandas series with rater names as the index:

	lambda	omega_items	omega_thresholds
Rater_1	-0.5	1.5	1.5
Rater_2	0.5	0.6	0.6

Table 32: `self.lambda_bistretch`, `self.omega_items_bistretch`, `self.omega_thresholds_bistretch`

For `bistretch` reuses `bivector`, the threshold axis zero-summed per rater (pure shape, no level). So the item component is exactly `pseudo_halo`'s formula, $\lambda_r + (\omega_r - 1)\delta_i$, stored as `self.facet_effects_bistretch_items`:

	Item_1	Item_2
Rater_1	-1.0	0.0
Rater_2	0.9	0.1

Table 33: `self.facet_effects_bistretch_items`

The threshold component drops centrality's λ_r term since this is absorbed in the item axis above – leaving just $(\omega_{\text{thresholds},r} - 1)\tau_k$, stored as `self.facet_effects_bistretch_thresholds`:

	1	2	3
Rater_1	-0.5	0.0	0.5
Rater_2	0.4	0.0	-0.4

Table 34: `self.facet_effects_bistretch_thresholds`

The reconstructed full matrix `self.facet_effects_bistretch_full_vector` (item component plus threshold component per cell) has the same `MultiIndex` format as `self.facet_effects_bivector` above. `self.facet_effects_bistretch` itself, by contrast, is the compact three-column (`lambda`, `omega_items`, `omega_thresholds`) `DataFrame`, one row per rater, matching `self.lambda_bistretch/self.omega_items_bistretch/self.omega_thresholds_bistretch` collected into one table:

		1	2	3
Rater_1	Item_1	-1.5	-1.0	-0.5
	Item_2	-0.5	0.0	0.5
Rater_2	Item_1	1.3	0.9	0.5
	Item_2	0.5	0.1	-0.3

Table 35: `self.facet_effects_bistretch_full_vector`

Examples

To generate a set of estimates for the global rater representation using the cosine similarity method for central item locations and rater facet effects, with additive smoothing constant of 0.1:

```
self.calibrate_global()
```

To generate a set of estimates for the matrix rater representation using the log-likelihood method for central item locations and rater facet effects, with matrix raised to power 7 and a convergence stopping criterion of 0.00000001:

```
self.calibrate_matrix(method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```

To generate a set of bivector estimates using the default cosine similarity method:

```
self.calibrate_bivector()
```

To generate a set of centrality estimates (recovered closed-form from an internally-calibrated thresholds fit):

```
self.calibrate_centrality()
```

To generate a set of pseudo_halo estimates, using 'theil-sen' rather than the default 'huber' to recover λ_r/ω_r :

```
self.calibrate_pseudo_halo(regression='theil-sen')
```

To generate a set of bistretch estimates, with the underlying bivector fit's item and threshold vectors estimated as marginal means of the full matrix representation rather than the default pooled-PAIR approach:

```
self.calibrate_bistretch(matrix_marginals=True)
```

‘Mixed’ rater representation

A ninth representation, `model='mixed'`, is available on `std_errors` (Section 6.3.3), `item_stats_df`, `threshold_stats_df`, `category_stats_df`, `person_stats_df`, `test_stats_df` and `rater_stats_df` (Section 6.4.4), but it has no `calibrate_mixed()` convenience alias and is *not* a valid `model=` value for `calibrate` itself – ‘mixed’ has no calibration of its own. Instead it depends on a per-rater assignment generated by `per_rater_model_selection` (Section 6.3.11) – each rater is represented under whichever of the eight representations above was found adequate for that specific rater, rather than one representation being applied uniformly to every rater. Calling any of the methods above with `model='mixed'` lazily runs `per_rater_model_selection` (with its own default selection criteria) if it has not already been run for the requested anchor set, exactly as `calibrate` is auto-triggered lazily for the other eight representations.

6.3.2 `calibrate_anchor`

Description

Anchors a calibration to a defined set of ‘gold standard’ anchor raters, who provide a frame of reference and for whom mean facet effect is defined as zero; other raters’ facet effects shifted accordingly. For extended rater representations, this process has a knock-on effect on the central item location and/or Rasch-Andrich threshold estimates. Full details of the anchoring procedure can be found in Elliott and Buttery (2022a).

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.calibrate_global_anchor(anchors, calibrate=False, constant=0.1, method='cos',
                             matrix_power=3, log_lik_tol=0.000001, adj=None)

self.calibrate_items_anchor(anchors, calibrate=False, constant=0.1, method='cos',
                             matrix_power=3, log_lik_tol=0.000001, adj=None)

self.calibrate_thresholds_anchor(anchors, calibrate=False, constant=0.1,
                                 method='cos', matrix_power=3, log_lik_tol=0.000001,
                                 adj=None)

self.calibrate_matrix_anchor(anchors, calibrate=False, constant=0.1, method='cos',
                              matrix_power=3, log_lik_tol=0.000001, adj=None)

self.calibrate_bivector_anchor(anchors, calibrate=False, constant=0.1, method='cos',
                               matrix_power=3, log_lik_tol=0.000001, adj=None)

self.calibrate_centrality_anchor(anchors, calibrate=False, constant=0.1,
                                 method='cos', matrix_power=3, log_lik_tol=0.000001,
                                 adj=None, regression='huber')

self.calibrate_pseudo_halo_anchor(anchors, calibrate=False, constant=0.1,
                                  method='cos', matrix_power=3, log_lik_tol=0.000001,
                                  adj=None, regression='huber')

self.calibrate_bistretch_anchor(anchors, calibrate=False, constant=0.1,
                                method='cos', matrix_power=3, log_lik_tol=0.000001,
                                adj=None, regression='huber')
```

Arguments

<code>anchors</code>	List: List of names of ‘gold standard’ anchor raters.
<code>calibrate</code>	Boolean: Only needed when a standard unanchored calibration has not been run, so unanchored parameters have not yet been generated. If <code>True</code> , a standard unanchored calibration will first be run. Default value is <code>calibrate=False</code> .
<code>constant</code>	Float: Only relevant when <code>calibrate=True</code> . Additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> .
<code>method</code>	String: Only relevant when <code>calibrate=True</code> . Method for derivation of central item location and rater facet effect estimates from pairwise reciprocal matrices (Elliott & Buttery, 2022b). Options are ‘ls’ for least squares (Choppin, 1968, 1985), ‘evm’ for the eigenvector method (Garner & Engelhard, 2002), ‘cos’ for cosine similarity (Kou & Lin, 2014) or ‘log-lik’ for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: Only relevant when <code>calibrate=True</code> . Power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: Only relevant when <code>calibrate=True</code> . Convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
<code>adj</code>	Float, dictionary, or <code>None</code> : if provided, applied directly as the anchor translation/adjustment rather than being computed from <code>anchors</code> . Intended for reuse across bootstrap resamples, where recomputing the anchor adjustment from scratch every time would inflate standard errors with anchor-rater sampling variance. Default is <code>adj=None</code> .
<code>regression</code>	<code>centrality/pseudo_halo/bistretch</code> representations only. String, one of ‘huber’ or ‘theil-sen’: same meaning as <code>calibrate</code> ’s own <code>regression</code> argument (Section 6.3.1), applied when re-deriving λ_r/ω_r from the anchored facet effects. Also used for the internal <code>calibrate</code> call when <code>calibrate=True</code> . Ignored for the other five representations. Default is <code>regression='huber'</code> .

Returns

Returns four attributes:

<code>self.anchor_items</code>	pandas series: Item location estimates with the item names as keys and estimates as values. Depending on the rater representation, the attribute will be stored as: <code>self.anchor_items_global</code>, <code>self.anchor_items_items</code>, <code>self.anchor_items_thresholds</code>, <code>self.anchor_items_matrix</code>, <code>self.anchor_items_bivector</code>, <code>self.anchor_items_centrality</code>, <code>self.anchor_items_pseudo_halo</code> or <code>self.anchor_items_bistretch</code>.
<code>self.anchor_thresholds</code>	pandas series: Rasch-Andrich threshold estimates $\tau_1, \dots, \tau_m$, an array of m estimates, where m is the maximum score, which sum to zero. Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed; the series now has length m, not $m + 1$. Depending on the rater representation, the attribute will be stored as: <code>self.anchor_thresholds_global</code>, <code>self.anchor_thresholds_items</code>, <code>self.anchor_thresholds_thresholds</code>, <code>self.anchor_thresholds_matrix</code>, <code>self.anchor_thresholds_bivector</code>, <code>self.anchor_thresholds_centrality</code>, <code>self.anchor_thresholds_pseudo_halo</code> or <code>self.anchor_thresholds_bistretch</code>.

Returns continue on the next page.

Returns (continued)

<code>self.anchor_facet_effects</code>	pandas series, dictionary, or DataFrame: Rater names as keys and the appropriate rater representations for <code>global</code> , <code>items</code> , <code>thresholds</code> , <code>matrix</code> , <code>bivector</code> , <code>centrality</code> , <code>pseudo_halo</code> or <code>bistretch</code> as values. For <code>bivector</code> and <code>bistretch</code> , two attributes are stored: <code>self.anchor_facet_effects_bivector_items</code> and <code>self.anchor_facet_effects_bivector_thresholds</code> (or, for <code>bistretch</code> , the <code>_bistretch_items/_bistretch_thresholds</code> equivalents); the reconstructed full matrix is available as <code>self.anchor_facet_effects_bivector</code> (or <code>_bistretch</code>). Depending on the rater representation, the attribute will be stored as: <code>self.anchor_facet_effects_global</code> , <code>self.anchor_facet_effects_items</code> , <code>self.anchor_facet_effects_thresholds</code> , <code>self.anchor_facet_effects_matrix</code> or <code>self.anchor_facet_effects_bivector</code> .
<code>self.anchor_cat_widths</code>	numpy array Array of $m - 1$ category width estimates, where m is the maximum score, as values. Depending on the rater representation, the attribute will be stored as: <code>self.anchor_cat_widths_global</code> , <code>self.anchor_cat_widths_items</code> , <code>self.anchor_cat_widths_thresholds</code> , <code>self.anchor_cat_widths_matrix</code> , <code>self.anchor_cat_widths_bivector</code> , <code>self.anchor_cat_widths_centrality</code> , <code>self.anchor_cat_widths_pseudo_halo</code> or <code>self.anchor_cat_widths_bistretch</code> .

Example `self.facet_effects` outputs

Toy examples of each rater representation format, with two raters and two items with three categories:

Rater representation `global`: pandas series with floats (facet effects) for values.

```
self.facet_effects_global = pd.Series({'Rater_1': -0.5,  
                                     'Rater_2': 0.5})
```

Rater representation `items`: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
self.facet_effects_items = {'Rater_1': pd.Series({'Item_1': -1, 'Item_2': 0}),  
                           'Rater_2': pd.Series({'Item_1': 0, 'Item_2': 1})}
```

Rater representation `thresholds`: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
self.facet_effects_thresholds = {'Rater_1': np.array([0, -1, -0.5, 0]),
                                'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

Rater representation `matrix`: Nested dictionary. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
self.facet_effects_matrix = {'Rater_1': {'Item_1': np.array([0, -1.5, -1, 0.5]),
                                         'Item_2': np.array([0, 1, 0, -1])},
                             'Rater_2': {'Item_1': np.array([0, -0.5, 0, 0.5]),
                                         'Item_2': np.array([0, 2, 1, 0])}}
```

Rater representation `bivector`: Two DataFrames as for `calibrate_bivector` above: `self.anchor_facet_effects_bivector_items` and `self.anchor_facet_effects_bivector_thresholds`, with the reconstructed full matrix in `self.anchor_facet_effects_bivector`.

Examples

To generate a set of anchored estimates for the global rater representation from an existing unanchored calibration, anchored to raters 'Rater_1' and 'Rater_2':

```
self.calibrate_global_anchor(['Rater_1', 'Rater_2'])
```

To generate a set of estimates for the matrix rater representation anchored to raters 'Rater_1' and 'Rater_2', first running an unanchored calibration, using the log-likelihood method for central item locations and rater facet effects, with matrix raised to power 7 and a convergence stopping criterion of 0.00000001:

```
self.calibrate_matrix_anchor(['Rater_1', 'Rater_2'], calibrate=True,
                             method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```


6.3.3 std_errors

Description

Produces bootstrapped estimates for the standard errors of central item location estimates, threshold estimates and category width estimates for bounded (non-extreme) categories.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.std_errors_global(anchors=None, interval=None, no_of_samples=500, constant=0.1,
                       method='cos', matrix_power=3, log_lik_tol=0.000001,
                       store_bootstrap=False, seed=None)

self.std_errors_items(anchors=None, interval=None, no_of_samples=500, constant=0.1,
                      method='cos', matrix_power=3, log_lik_tol=0.000001,
                      store_bootstrap=False, seed=None)

self.std_errors_thresholds(anchors=None, interval=None, no_of_samples=500,
                           constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                           store_bootstrap=False, seed=None)

self.std_errors_matrix(anchors=None, interval=None, no_of_samples=500, constant=0.1,
                       method='cos', matrix_power=3, log_lik_tol=0.000001,
                       store_bootstrap=False, seed=None)

self.std_errors_bivector(anchors=None, interval=None, no_of_samples=500,
                         constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                         store_bootstrap=False, seed=None)

self.std_errors_centrality(anchors=None, interval=None, no_of_samples=500,
                           constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                           store_bootstrap=False, seed=None)

self.std_errors_pseudo_halo(anchors=None, interval=None, no_of_samples=500,
                            constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                            store_bootstrap=False, seed=None)

self.std_errors_bistretch(anchors=None, interval=None, no_of_samples=500,
                          constant=0.1, method='cos', matrix_power=3, log_lik_tol=0.000001,
                          store_bootstrap=False, seed=None)
```

Arguments

<code>anchors</code>	List: List of names of ‘gold standard’ anchor raters. Only used if anchored standard errors are being produced. Default is <code>anchors=None</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval. Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate. More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default value is <code>constant=0.1</code> .
<code>method</code>	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix for central item location estimates (Elliott & Buttery, 2022b:991) is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
<code>store_bootstrap</code>	Boolean: if <code>True</code> , stores the fitted (unanchored) bootstrap samples as <code>self._bootstrap_samples_{model}</code> rather than discarding them once the SEs above are computed. Costs memory (<code>no_of_samples</code> fitted MFRM objects) but allows <code>anchor_std_errors</code> (Section 6.3.4) to reuse them – recalibrating each stored sample to a chosen anchor set – instead of refitting the whole bootstrap from scratch for every anchor set tried. Default is <code>store_bootstrap=False</code> .
<code>seed</code>	Integer or <code>None</code> : seed for the bootstrap resampling random number generator (and, for <code>model='mixed'</code> , passed through to the lazily-triggered <code>per_rater_model_selection</code> call if it has not already been run). Pass an integer for fully reproducible standard errors; <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Creates a set of attributes, which depend on whether an argument is passed to `anchors`. Where `store_bootstrap=True`, the underlying bootstrap replicas themselves remain retrievable too, with the full complement of raw per-sample values behind every summary below – see the note at the end of this section.

If `anchors=None`:

<code>self.item_se</code>	pandas series: Item names as keys and item standard errors as values. Central item locations are shared across all rater representations, so this attribute carries no <code>_{model}</code> suffix.
<code>self.threshold_se</code>	pandas series: Threshold numbers as keys and item standard errors as values: <code>self.threshold_se_global</code> , <code>self.threshold_se_items</code> , <code>self.threshold_se_thresholds</code> , <code>self.threshold_se_matrix</code> , <code>self.threshold_se_bivector</code> , <code>self.threshold_se centrality</code> , <code>self.threshold_se_pseudo_halo</code> or <code>self.threshold_se_bistretch</code>
<code>self.cat_width_bootstrap</code>	dict: Category numbers as keys and arrays of the per-sample bootstrap replicate category width values as values: <code>self.cat_width_bootstrap_global</code> , <code>self.cat_width_bootstrap_items</code> , <code>self.cat_width_bootstrap_thresholds</code> , <code>self.cat_width_bootstrap_matrix</code> , <code>self.cat_width_bootstrap_bivector</code> , <code>self.cat_width_bootstrap centrality</code> , <code>self.cat_width_bootstrap_pseudo_halo</code> or <code>self.cat_width_bootstrap_bistretch</code>
<code>self.cat_width_se</code>	dict: Category numbers as keys and item standard errors as values: <code>self.cat_width_se_global</code> , <code>self.cat_width_se_items</code> , <code>self.cat_width_se_thresholds</code> , <code>self.cat_width_se_matrix</code> , <code>self.cat_width_se_bivector</code> , <code>self.cat_width_se centrality</code> , <code>self.cat_width_se_pseudo_halo</code> or <code>self.cat_width_se_bistretch</code>
<code>self.rater_se</code>	pandas series or DataFrame: Item names as keys and item standard errors as values. For bivector and bistretch, two additional attributes <code>self.rater_se_marginal_items</code> and <code>self.rater_se_marginal_thresholds</code> are also stored. Depending on the rater representation: <code>self.rater_se_global</code> , <code>self.rater_se_items</code> , <code>self.rater_se_thresholds</code> , <code>self.rater_se_matrix</code> , <code>self.rater_se_bivector</code> , <code>self.rater_se centrality</code> , <code>self.rater_se_pseudo_halo</code> or <code>self.rater_se_bistretch</code>

If an argument is passed to `interval`, also returns:

<code>self.item_low</code>	Lower bound of the specified interval for item estimates. Like <code>self.item_se</code> , unsuffixed – shared across all rater representations.
<code>self.item_high</code>	Upper bound of the specified interval for item estimates. Also unsuffixed, for the same reason.
<code>self.threshold_low</code>	Lower bound of the specified interval for threshold estimates: <code>self.threshold_low_global</code> , <code>self.threshold_low_items</code> , <code>self.threshold_low_thresholds</code> , <code>self.threshold_low_matrix</code> , <code>self.threshold_low_bivector</code> , <code>self.threshold_low centrality</code> , <code>self.threshold_low_pseudo_halo</code> or <code>self.threshold_low_bistretch</code>
<code>self.threshold_high</code>	Upper bound of the specified interval for threshold estimates: <code>self.threshold_high_global</code> , <code>self.threshold_high_items</code> , <code>self.threshold_high_thresholds</code> , <code>self.threshold_high_matrix</code> , <code>self.threshold_high_bivector</code> , <code>self.threshold_high centrality</code> , <code>self.threshold_high_pseudo_halo</code> or <code>self.threshold_high_bistretch</code>
<code>self.cat_width_low</code>	Lower bound of the specified interval for category estimates: <code>self.cat_width_low_global</code> , <code>self.cat_width_low_items</code> , <code>self.cat_width_low_thresholds</code> , <code>self.cat_width_low_matrix</code> , <code>self.cat_width_low_bivector</code> , <code>self.cat_width_low centrality</code> , <code>self.cat_width_low_pseudo_halo</code> or <code>self.cat_width_low_bistretch</code>
<code>self.cat_width_high</code>	Upper bound of the specified interval for category estimates: <code>self.cat_width_high_global</code> , <code>self.cat_width_high_items</code> , <code>self.cat_width_high_thresholds</code> , <code>self.cat_width_high_matrix</code> , <code>self.cat_width_high_bivector</code> , <code>self.cat_width_high centrality</code> , <code>self.cat_width_high_pseudo_halo</code> or <code>self.cat_width_high_bistretch</code>
<code>self.rater_low</code>	Lower bound of the specified interval for rater facet effect estimates: <code>self.rater_low_global</code> , <code>self.rater_low_items</code> , <code>self.rater_low_thresholds</code> , <code>self.rater_low_matrix</code> , <code>self.rater_low_bivector</code> , <code>self.rater_low centrality</code> , <code>self.rater_low_pseudo_halo</code> or <code>self.rater_low_bistretch</code>
<code>self.rater_high</code>	Upper bound of the specified interval for rater facet effect estimates: <code>self.rater_high_global</code> , <code>self.rater_high_items</code> , <code>self.rater_high_thresholds</code> , <code>self.rater_high_matrix</code> , <code>self.rater_high_bivector</code> , <code>self.rater_high centrality</code> , <code>self.rater_high_pseudo_halo</code> or <code>self.rater_high_bistretch</code>

If `anchors` is not `None`:

<code>self.anchor_item_se</code>	pandas series: Item names as keys and item standard errors as values. Like <code>self.item_se</code> , unsuffixed – shared across all rater representations.
<code>self.anchor_threshold_se</code>	pandas series: Threshold numbers as keys and item standard errors as values: <code>self.anchor_threshold_se_global</code> , <code>self.anchor_threshold_se_items</code> , <code>self.anchor_threshold_se_thresholds</code> , <code>self.anchor_threshold_se_matrix</code> , <code>self.anchor_threshold_se_bivector</code> , <code>self.anchor_threshold_se_centrality</code> , <code>self.anchor_threshold_se_pseudo_halo</code> or <code>self.anchor_threshold_se_bistretch</code>
<code>self.anchor_cat_width_bootstrap</code>	dict: Category numbers as keys and arrays of the per-sample bootstrap replicate category width values as values: <code>self.anchor_cat_width_bootstrap_global</code> , <code>self.anchor_cat_width_bootstrap_items</code> , <code>self.anchor_cat_width_bootstrap_thresholds</code> , <code>self.anchor_cat_width_bootstrap_matrix</code> , <code>self.anchor_cat_width_bootstrap_bivector</code> , <code>self.anchor_cat_width_bootstrap_centrality</code> , <code>self.anchor_cat_width_bootstrap_pseudo_halo</code> or <code>self.anchor_cat_width_bootstrap_bistretch</code>
<code>self.cat_width_se</code>	dict: Category numbers as keys and item standard errors as values: <code>self.anchor_cat_width_se_global</code> , <code>self.anchor_cat_width_se_items</code> , <code>self.anchor_cat_width_se_thresholds</code> , <code>self.anchor_cat_width_se_matrix</code> , <code>self.anchor_cat_width_se_bivector</code> , <code>self.anchor_cat_width_se_centrality</code> , <code>self.anchor_cat_width_se_pseudo_halo</code> or <code>self.anchor_cat_width_se_bistretch</code>
<code>self.anchor_rater_se</code>	pandas series: Item names as keys and item standard errors as values: <code>self.anchor_rater_se_global</code> , <code>self.anchor_rater_se_items</code> , <code>self.anchor_rater_se_thresholds</code> , <code>self.anchor_rater_se_matrix</code> , <code>self.anchor_rater_se_bivector</code> , <code>self.anchor_rater_se_centrality</code> , <code>self.anchor_rater_se_pseudo_halo</code> or <code>self.anchor_rater_se_bistretch</code>

If an argument is passed to `interval`, also returns:

<code>self.anchor_item_low</code>	Lower bound of the specified interval for item estimates. Like <code>self.anchor_item_se</code> , unsuffixed – shared across all rater representations.
<code>self.anchor_item_high</code>	Upper bound of the specified interval for item estimates. Also unsuffixed, for the same reason.

Returns continue on the next page.

Returns (continued)

<code>self.anchor_threshold_low</code>	Lower bound of the specified interval for threshold estimates: <code>self.anchor_threshold_low_global, self.anchor_threshold_low_items, self.anchor_threshold_low_thresholds, self.anchor_threshold_low_matrix, self.anchor_threshold_low_bivector, self.anchor_threshold_low_centrality, self.anchor_threshold_low_pseudo_halo</code> or <code>self.anchor_threshold_low_bistretch</code>
<code>self.anchor_threshold_high</code>	Upper bound of the specified interval for threshold estimates: <code>self.anchor_threshold_high_global, self.anchor_threshold_high_items, self.anchor_threshold_high_thresholds, self.anchor_threshold_high_matrix, self.anchor_threshold_high_bivector, self.anchor_threshold_high_centrality, self.anchor_threshold_high_pseudo_halo</code> or <code>self.anchor_threshold_high_bistretch</code>
<code>self.anchor_cat_width_low</code>	Lower bound of the specified interval for category estimates: <code>self.anchor_cat_width_low_global, self.anchor_cat_width_low_items, self.anchor_cat_width_low_thresholds, self.anchor_cat_width_low_matrix, self.anchor_cat_width_low_bivector, self.anchor_cat_width_low_centrality, self.anchor_cat_width_low_pseudo_halo</code> or <code>self.anchor_cat_width_low_bistretch</code>
<code>self.anchor_cat_width_high</code>	Upper bound of the specified interval for category estimates: <code>self.anchor_cat_width_high_global, self.anchor_cat_width_high_items, self.anchor_cat_width_high_thresholds, self.anchor_cat_width_high_matrix, self.anchor_cat_width_high_bivector, self.anchor_cat_width_high_centrality, self.anchor_cat_width_high_pseudo_halo</code> or <code>self.anchor_cat_width_high_bistretch</code>
<code>self.anchor_rater_low</code>	Lower bound of the specified interval for rater facet effect estimates: <code>self.anchor_rater_low_global, self.anchor_rater_low_items, self.anchor_rater_low_thresholds, self.anchor_rater_low_matrix, self.anchor_rater_low_bivector, self.anchor_rater_low_centrality, self.anchor_rater_low_pseudo_halo</code> or <code>self.anchor_rater_low_bistretch</code>
<code>self.anchor_rater_high</code>	Upper bound of the specified interval for rater facet effect estimates: <code>self.anchor_rater_high_global, self.anchor_rater_high_items, self.anchor_rater_high_thresholds, self.anchor_rater_high_matrix, self.anchor_rater_high_bivector, self.anchor_rater_high_centrality, self.anchor_rater_high_pseudo_halo</code> or <code>self.anchor_rater_high_bistretch</code>

Retrieving the raw bootstrap samples

The tables above report summary statistics (standard errors and optional interval bounds) derived from the bootstrap. If `store_bootstrap=True` is passed, the `no_of_samples` bootstrapped samples are stored too, as a list assigned to `self._bootstrap_samples_{model}`. Each replica is a fully-calibrated MFRM object in its own right, carrying every attribute an ordinary `calibrate` (or, if `anchors` was passed, `calibrate_anchor`) call would produce for that model, giving direct access to the per-sample values behind every summary statistic reported above. For example, `[s.items for s in self._bootstrap_samples_global]` recovers the full set of bootstrapped item vectors – with no separate `item_bootstrap`, `threshold_bootstrap` or `rater_bootstrap` attribute needed. If `store_bootstrap=False` (the default, since storing the bootstrap makes the object large), the samples are discarded once the summary statistics above have been computed.

Examples

To generate unanchored item standard errors for the global representation with a 95% interval from 200 samples:

```
self.std_errors_global(interval=0.95, no_of_samples=200)
```

To generate anchored item standard errors for the matrix representation with a 90% interval from 200 samples, anchored to raters 'Rater_1' and 'Rater_2':

```
self.anchor_std_errors_matrix(anchors=['Rater_1', 'Rater_2'], no_of_samples=200)
```

Modifications to the estimation method are discussed in Section 6.3.1.

‘Mixed’ rater representation

`std_errors(model='mixed')` produces bootstrap standard errors under the per-rater representation assignment from `per_rater_model_selection` (Section 6.3.11), lazily run with its own default selection criteria if it has not already been run (or was run with a different anchor set) – see the ‘mixed’ representation note under `calibrate` (Section 6.3.1). There is no `std_errors_mixed()` convenience alias; call `self.std_errors(model='mixed', ...)` instead.

6.3.4 anchor_std_errors

Description

Produces standard errors from pre-existing bootstraps when `std_errors` was previously run with `store_bootstrap=True`, as a means of avoiding the need to re-run bootstrapping when re-anchoring to a different set of anchor raters.

Confidence interval width is inherited from the `std_errors` run for this model if not explicitly provided, so anchored and unanchored confidence intervals stay on a consistent basis; pass `interval=0` to explicitly suppress this inheritance. Stores `anchor_item_se` (and `_low/_high` if an interval is set), `anchor_threshold_se_{model}`, `anchor_rater_se_{model}`, and the corresponding low/high attributes, mirroring `std_errors`'s own naming convention.

When the fast path is taken (stored samples reused), each replica in `self._bootstrap_samples_{model}` gains the anchor-calibrated attributes too, so it remains the same list to consult for the raw per-sample values behind these summaries – see “Retrieving the raw bootstrap replicas” under `std_errors` (Section 6.3.3).

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.anchor_std_errors_global(anchors=None, interval=None, no_of_samples=500,
                              constant=0.1, method='cos', matrix_power=3,
                              log_lik_tol=0.000001, seed=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`).

Arguments

<code>anchors</code>	List: raters whose mean facet effect is anchored to zero. If <code>None</code> , falls back to <code>anchor_rater_names_{model}</code> set by <code>calibrate_anchor</code> (Section 6.3.2) – so <code>calibrate_anchor</code> must already have been run for this model if <code>anchors</code> is left as <code>None</code> . Default is <code>anchors=None</code> .
<code>interval</code>	Float or <code>None</code> : as for <code>std_errors</code> (Section above), but inherits the interval used by that model's <code>std_errors</code> run if left as <code>None</code> . Pass <code>interval=0</code> to explicitly suppress CIs even if <code>std_errors</code> used one. Default is <code>interval=None</code> .
<code>no_of_samples</code>	Integer: number of bootstrap samples. Only used on the slow (full refit) path, when stored samples from a prior <code>store_bootstrap=True</code> run are not available. Default is <code>no_of_samples=500</code> .

Returns continue on the next page.

Returns (continued)

<code>constant,</code> <code>matrix_power,</code> <code>log_lik_tol</code>	<code>method,</code>	Calibration keyword arguments, as for <code>std_errors</code> (Section above). See Section 6.3.1 for details of each.
<code>seed</code>		Integer or <code>None</code> : seed for the bootstrap resampling random number generator. Only used on the slow path. <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

`None` returned directly; sets `self.anchor_item_se` (plus `_low/_high` if interval is set), `self.anchor_threshold_se_{model}` and `self.anchor_rater_se_{model}` (plus their own `_low/_high`), mirroring `std_errors`'s unanchored attribute names with an `anchor_` prefix.

Example

To run the unanchored bootstrap once, storing samples for reuse, then compute anchored SEs for two different candidate anchor sets under the matrix representation without refitting the bootstrap each time:

```
self.std_errors_matrix(store_bootstrap=True)
self.anchor_std_errors_matrix(anchors=['Rater_1', 'Rater_2'])
self.anchor_std_errors_matrix(anchors=['Rater_1', 'Rater_3'])
```

If `calibrate_anchor` has already been run for a model, `anchors` can be omitted to reuse its anchor set:

```
self.calibrate_anchor('matrix', ['Rater_1', 'Rater_2'])
self.anchor_std_errors_matrix()
```

6.3.5 person

Description

Generates a location estimate for a person using the Newton-Raphson method to produce a maximum likelihood estimate, with optional Warm bias correction (Warm, 1989).

Usage

```
self.person(persons, model='global', anchor=False, items=None, facet_elements=None,
            warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
            missing_as_incorrect=False)
```

Arguments

persons	String or list: person name(s) for the location(s) being estimated. Pass 'all' to estimate for all persons in <code>self.responses</code> .
model	String, one of 'global', 'items', 'thresholds', 'matrix', 'bivector', 'centrality', 'pseudo_halo' or 'bistretch': the rater representation to use. Default is <code>model='global'</code> .
anchor	Boolean: If <code>True</code> , parameters from an anchored calibration will be used. Default is <code>anchor=False</code> .
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
facet_elements	List: List of names of a subset of rater, based on which to generate the location estimate. Default is <code>facet_elements=None</code> , which generates a location based on the full set of raters. Only use when an estimate based on a subset of raters or an individual rater is required.
warm_corr	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
max_iters	Integer: maximum number of Newton-Raphson iterations. Default is <code>max_iters=100</code> .

Arguments continue on the next page.

Arguments (continued)

<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>missing_as_incorrect</code>	Boolean: if <code>True</code> , treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is <code>missing_as_incorrect=False</code> .

Returns

pandas series: person location estimate(s), in logits, indexed by person identifier. Returns `NaN` for any person for whom estimation fails.

Example

To generate an unanchored person location estimate for `Person_1` using a global rater representation using the default settings and store the result as a variable, `my_person_location`:

```
my_person_location = self.person('Person_1')
```

To generate an anchored MLE person location estimate under the matrix rater representation without Warm bias correction for `Person_1` based on the first three items rated by `Rater_1` and store the result as a variable, `my_person_location`:

```
my_person_location = self.person('Person_1', model='matrix',  
                                items=['Item_1', 'Item_2', 'Item_3'],  
                                facet_elements=['Rater_1'], warm_corr=False)
```

6.3.6 person_estimates

Description

Generates location estimates for all persons using the Newton-Raphson method to produce maximum likelihood estimates, with optional Warm bias correction (Warm, 1989).

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.person_estimates_global(anchor=False, items=None, facet_elements=None, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                             missing_as_incorrect=False)

self.person_estimates_items(anchor=False, items=None, facet_elements=None, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                             missing_as_incorrect=False)

self.person_estimates_thresholds(anchor=False, items=None, facet_elements=None, warm_corr=True,
                                  tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                  missing_as_incorrect=False)

self.person_estimates_matrix(anchor=False, items=None, facet_elements=None, warm_corr=True,
                              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                              missing_as_incorrect=False)

self.person_estimates_bivector(anchor=False, items=None, facet_elements=None, warm_corr=True,
                                tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                missing_as_incorrect=False)

self.person_estimates_centrality(anchor=False, items=None, facet_elements=None, warm_corr=True,
                                  tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                  missing_as_incorrect=False)

self.person_estimates_pseudo_halo(anchor=False, items=None, facet_elements=None, warm_corr=True,
                                   tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                   missing_as_incorrect=False)

self.person_estimates_bistretch(anchor=False, items=None, facet_elements=None, warm_corr=True,
                                 tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                 missing_as_incorrect=False)
```

Arguments

<code>anchor</code>	Boolean: If <code>True</code> , parameters from an anchored calibration will be used. Default is <code>anchor=False</code> .
<code>items</code>	List: List of names of a subset of items, based on which to generate the location estimate. Default is <code>items=None</code> , which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
<code>facet_elements</code>	List: List of names of a subset of rater, based on which to generate the location estimate. Default is <code>facet_elements=None</code> , which generates a location based on the full set of raters. Only use when an estimate based on a subset of raters or an individual rater is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations. Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>missing_as_incorrect</code>	Boolean: if <code>True</code> , treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is <code>missing_as_incorrect=False</code> .

Returns

- If `anchor=False`, attribute `self.persons_global`, `self.persons_items`, `self.persons_thresholds`, `self.persons_matrix`, `self.persons_bivector`, `self.persons centrality`, `self.persons_pseudo_halo` or `self.persons_bistretch`: pandas series with person names as keys and location estimates as values.
- If `anchor=True`, attribute `self.anchor_persons_global`, `self.anchor_persons_items`, `self.anchor_persons_thresholds`, `self.anchor_persons_matrix`, `self.anchor_persons_bivector`, `self.anchor_persons centrality`, `self.anchor_persons_pseudo_halo` or `self.anchor_persons_bistretch`: pandas series with person names as keys and location estimates as values.

Examples

To generate a set of unanchored person location estimates for the global rater representation from all items and raters with Warm bias correction:

```
self.person_estimates_global()
```

To generate a set of anchored person location estimates for the matrix rater representation without Warm bias correction, on a subset of the first three items only, rated by Rater_1:

```
self.person_estimates_matrix(anchor=True, items=['Item_1', 'Item_2', 'Item_3'],  
                             facet_elements=['Rater_1'], warm_corr=False)
```

6.3.7 score_lookup

Description

Generates a location estimate for a given raw score on responses to a given set of items using the Newton-Raphson method to produce a maximum likelihood estimate, with optional Warm bias correction (Warm, 1989).

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.score_lookup_global(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                        tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_items(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                        tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_thresholds(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_matrix(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                         tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_bivector(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_centrality(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_pseudo_halo(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_bistretch(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)
```

Arguments

score	Integer: The raw score for which location is being estimated.
anchor	Boolean: If True, parameters from an anchored calibration will be used. Default is anchor=False.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.

Arguments continue on the next page.

Arguments (continued)

<code>facet_elements</code>	List: List of names of a subset of rater, based on which to generate the location estimate. Default is <code>facet_elements=None</code> , which generates a location based on the full set of raters. Only use when an estimate based on a subset of raters or an individual rater is required.
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations. Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Returns

Float: Location estimate corresponding to raw score.

Examples

To generate an unanchored location estimate under the global rater representation for a score of 10 on all items rated by all raters, with Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = self.score_lookup_global(10)
```

To generate an anchored location estimate under the matrix rater representation for a score of 10 on a subset of items saved as a variable `my_items`, rated by `Rater_1` without Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = self.score_lookup_matrix(10, anchor=True, items=my_items,  
                                           facet_elements=['Rater_1'], warm_corr=False)
```


6.3.8 score_lookup_table

Description

Generates a lookup table of location estimates corresponding to all available raw scores on a set of items with no missing responses, using the Newton-Raphson method to produce maximum likelihood estimates and with optional Warm bias correction (Warm, 1989).

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.score_lookup_table_global(anchor=False, attribute=True, items=None,
                                facet_elements=None, ext_scores=True, warm_corr=True,
                                tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_table_items(anchor=False, attribute=True, items=None,
                                facet_elements=None, ext_scores=True, warm_corr=True,
                                tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_table_thresholds(anchor=False, attribute=True, items=None,
                                    facet_elements=None, ext_scores=True, warm_corr=True,
                                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_table_matrix(anchor=False, attribute=True, items=None,
                                facet_elements=None, ext_scores=True, warm_corr=True,
                                tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_table_bivector(anchor=False, attribute=True, items=None,
                                   facet_elements=None, ext_scores=True, warm_corr=True,
                                   tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_table_centrality(anchor=False, attribute=True, items=None,
                                    facet_elements=None, ext_scores=True, warm_corr=True,
                                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_table_pseudo_halo(anchor=False, attribute=True, items=None,
                                      facet_elements=None, ext_scores=True, warm_corr=True,
                                      tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_table_bistretch(anchor=False, attribute=True, items=None,
                                   facet_elements=None, ext_scores=True, warm_corr=True,
                                   tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)
```

Arguments

<code>anchor</code>	Boolean: If <code>True</code> , parameters from an anchored calibration will be used. Default is <code>anchor=False</code> .
<code>attribute</code>	Boolean: if <code>True</code> , stores the result as <code>self.score_table_{model}</code> . If <code>False</code> , the table is only returned, not stored. Default is <code>attribute=True</code> .
<code>items</code>	List: List of subset of items on which to generate the location estimate. Default <code>items=None</code> , which generates a location based on all items. Only use when an estimate based on a subset of items is required.
<code>facet_elements</code>	List: List of names of a subset of rater, based on which to generate the location estimate. Default is <code>facet_elements=None</code> , which generates a location based on the full set of raters. Only use when an estimate based on a subset of raters or an individual rater is required.
<code>ext_scores</code>	Boolean: If <code>True</code> , location estimates for extreme scores (all correct/all incorrect) will be generated using the <code>ext_score_adjustment</code> argument. Default is <code>ext_scores=True</code> .
<code>warm_corr</code>	Boolean: if <code>True</code> , Warm's bias correction (Warm, 1989) is applied to the estimate. Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations. Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum Newton-Raphson iterations. Default <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0,1) to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Returns

pandas series with raw scores as keys and corresponding location estimates as values. If `attribute=True` (the default), also stored as `self.score_table_{model}` (e.g. `self.score_table_global`).

Examples

To generate an unanchored location lookup table under the global rater representation for all items rated by all raters, including extreme scores, with Warm bias correction:

```
self.score_lookup_table()
```

To generate an anchored location lookup table under the matrix rater representation for a subset of items saved as a variable `my_items`, rated by `Rater_1`, with extreme scores and Warm bias correction:

```
self.score_lookup_table(anchor=True, items=my_items, facet_elements=['Rater_1'])
```

6.3.9 csem

Description

Calculates the conditional standard error of measurement ($CSEM = 1/\sqrt{I}$, where I is the total Fisher information summed across all observed facet-element/item combinations) for one or more persons, from a set of items and facet elements.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.csem_global(anchor=False, persons=None, person_locations=None, items=None,
                 facet_elements=None)

self.csem_items(anchor=False, persons=None, person_locations=None, items=None,
                facet_elements=None)

self.csem_thresholds(anchor=False, persons=None, person_locations=None, items=None,
                     facet_elements=None)

self.csem_matrix(anchor=False, persons=None, person_locations=None, items=None,
                 facet_elements=None)

self.csem_bivector(anchor=False, persons=None, person_locations=None, items=None,
                   facet_elements=None)

self.csem_centrality(anchor=False, persons=None, person_locations=None, items=None,
                     facet_elements=None)

self.csem_pseudo_halo(anchor=False, persons=None, person_locations=None, items=None,
                       facet_elements=None)

self.csem_bistretch(anchor=False, persons=None, person_locations=None, items=None,
                    facet_elements=None)
```

Arguments

<code>anchor</code>	Boolean: If <code>True</code> , parameters from an anchored calibration will be used. Default is <code>anchor=False</code> .
<code>persons</code>	String, list, or <code>None</code> : subset of persons to compute CSEM for. Overrides <code>person_locations</code> if both are supplied (i.e. <code>person_locations</code> is filtered down to just these persons). Default is <code>persons=None</code> , which uses all persons.

Arguments continue on the next page.

Arguments (continued)

<code>person_locations</code>	pandas series, float, list, numpy array, or <code>None</code> : person location estimate(s) to compute CSEM for. If <code>None</code> (default), uses the stored <code>self.persons_{model}</code> (or <code>self.anchor_persons_{model}</code> if <code>anchor=True</code>) location estimates, auto-triggering <code>person_estimates</code> (Section 6.3.6) if not already present. A raw float, list, or numpy array of locations (i.e. not indexed by a real person) is treated as <i>hypothetical</i> : since there is no observed response row to consult for such a location, every item in <code>items</code> is treated as answered by every facet element in <code>facet_elements</code> . Default is <code>person_locations=None</code> .
<code>items</code>	List or <code>None</code> : List of names of a subset of items, based on which to compute CSEM. Default is <code>items=None</code> , which uses the full set of items.
<code>facet_elements</code>	List, string, or <code>None</code> : subset of facet elements (raters), based on which to compute CSEM. Default is <code>facet_elements=None</code> , which uses the full set of facet elements.

Returns

pandas series: CSEM values, in logits, indexed by person (or, for hypothetical locations passed directly via `person_locations`, indexed by a generated label for each value).

Examples

To generate the unanchored CSEM for all persons under the global rater representation, on all items and raters, and save the result as a variable, `my_csem`:

```
my_csem = self.csem_global()
```

To generate the anchored CSEM for `Person_1` under the matrix rater representation on a subset of items `['Item_1', 'Item_2']`, and a subset of raters `['Rater_1', 'Rater_2']`, and save the result as a variable, `my_csem`:

```
my_csem = self.csem_matrix(anchor=True, persons='Person_1',
                           items=['Item_1', 'Item_2'],
                           facet_elements=['Rater_1', 'Rater_2'])
```

To generate the CSEM for a hypothetical person location of 0.5 on a subset of items saved as a variable `my_items`:

```
my_csem = self.csem_global(person_locations=0.5, items=my_items)
```

6.3.10 `model_selection`

Description

Compares the eight MFRM rater representations (`global`, `items`, `thresholds`, `matrix`, `bivector`, `centrality`, `pseudo_halo`, `bistretch` – see Section 6.3.1) via AIC, BIC or a likelihood ratio test, to choose which single representation is the most appropriate, parsimonious fit for every rater at once. Contrast with `per_rater_model_selection` (Section 6.3.11), which instead assigns an individual, potentially different, representation to each rater, which may vary. Calibrates and runs `person_estimates` (Section 6.3.6) for all active representations if not already done – by default all eight, or only those listed in `models` if it is set.

Usage

```
self.model_selection(test='AIC', models=None, aic_sig_test=True, alpha=0.05,
                    sampling='dynamic', warm_corr=True, tolerance=0.00001, max_iters=100,
                    ext_score_adjustment=0.5, constant=0.1, method='cos',
                    matrix_power=3, log_lik_tol=0.000001, seed=None)
```

Arguments

<code>test</code>	String: 'AIC', 'BIC', or 'LR'. For 'LR', nested pairwise tests are computed rather than a single ranking, since the eight representations do not form one single nested chain: <code>global</code> vs. each of <code>items</code> , <code>thresholds</code> , <code>centrality</code> , <code>pseudo_halo</code> and <code>bistretch</code> ; <code>items</code> vs. <code>bivector</code> and <code>pseudo_halo</code> ; <code>thresholds</code> vs. <code>bivector</code> and <code>centrality</code> ; <code>bivector</code> vs. <code>matrix</code> and <code>bistretch</code> ; <code>centrality</code> vs. <code>bistretch</code> ; <code>pseudo_halo</code> vs. <code>bistretch</code> . Degrees of freedom fall directly out of the parameter-count difference between each pair. Default is <code>test='AIC'</code> .
<code>models</code>	List/tuple of strings, or <code>None</code> : restricts the comparison to the models in the list. For <code>test='LR'</code> , pairs of models must be nested for comparison, but for AIC/BIC, this is not necessary. <code>None</code> (default) uses all eight representations.
<code>aic_sig_test</code>	Boolean: when <code>True</code> (requires <code>test='AIC'</code>), applies a significance test against a baseline representation – whichever representation in the active set has the fewest free parameters, ties broken by lowest AIC. This is <code>global</code> whenever it is in the active set, since it always has the fewest parameters of all eight representations, but otherwise the simplest representation actually available (e.g. <code>centrality/pseudo_halo</code> if <code>models</code> excludes <code>global</code>). The p-value is the relative likelihood of the baseline vs. the best-fitting representation, $p = e^{- \Delta /2}$ where $\Delta = \text{AIC}_{\text{baseline}} - \text{AIC}_{\text{best}}$. A more complex representation is preferred only if $p < \text{alpha}$; otherwise the baseline is retained as the more parsimonious default. Default is <code>aic_sig_test=True</code> .

Arguments continue on the next page.

Arguments (continued)

<code>alpha</code>	Float: significance level for the AIC relative-likelihood test (used only when <code>test='AIC'</code> and <code>aic_sig_test=True</code>). Default is <code>alpha=0.05</code> .
<code>sampling</code>	None, 'dynamic', or Integer: subsamples non-extreme persons before computing log-likelihoods – parameters are always estimated on the full data; only the log-likelihood evaluation itself is subsampled. Not primarily about speed: at very large N , the LR/AIC/BIC comparison becomes biased toward the more complex rater representation, since log-likelihood differences scale with N ; capping the effective N mitigates this. 'dynamic' uses $T = \min(20(I - 1)(R - 1), 1500)$, where I is the number of items and R the number of raters; an integer fixes T directly; None disables subsampling. Default is <code>sampling='dynamic'</code> .
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_adjustment</code>	Person location estimation keyword arguments, used only for representations not already estimated. See Section 6.3.5 for details of each.
<code>constant</code> , <code>method</code> , <code>matrix_power</code> , <code>log_lik_tol</code>	Calibration keyword arguments, used only for representations not already calibrated. See Section 6.3.1 for details of each.
<code>seed</code>	Integer or None: seed for the person-subsampling random number generator, only used when <code>sampling</code> triggers. None (default) draws fresh entropy. See Section 1.

Returns

Sets whichever of the following attributes correspond to `test`:

<code>self.model_comparison_mfrm_aic</code> , <code>_aic_baseline</code> , <code>_aic_p</code> , <code>_aic_preferred</code> , <code>_aic_summary</code>	{ <code>representation</code> : AIC value} dictionary, the baseline representation tested against (see <code>aic_sig_test</code> above) as a string, the AIC relative-likelihood p-value, the preferred representation as a string, and a ranked comparison table (its Δ AIC column is relative to the baseline). Only set if <code>test='AIC'</code> .
<code>self.model_comparison_mfrm_bic</code> , <code>_bic_preferred</code> , <code>_bic_summary</code>	{ <code>representation</code> : BIC value} dictionary, the preferred representation, and a ranked comparison table. Only set if <code>test='BIC'</code> .
<code>self.model_comparison_mfrm_lr</code> , <code>_lr_summary</code>	{ <code>pair_label</code> : (LR, df, p)} dictionary of nested pairwise tests, and a summary table. Only set if <code>test='LR'</code> .

Examples

To compare all eight representations using the default AIC significance test:

```
self.model_selection()  
self.model_comparison_mfrm_aic_summary
```

To instead run pairwise likelihood ratio tests, with a fixed subsampling size of 800 persons to guard against large- N bias:

```
self.model_selection(test='LR', sampling=800)
```

To restrict the comparison to a single nested pair – testing whether `bistretch`'s combined threshold-and-item stretch is needed over the unrestricted `bivector` representation, without calibrating the other six:

```
self.model_selection(test='LR', models=['bivector', 'bistretch'])
```

6.3.11 `per_rater_model_selection`

Description

Assigns the simplest adequate rater-facet effect parameterisation to each rater individually, producing a “mixed” model in which different raters may be represented under different rater representations (any of the eight in Section 6.3.1) according to which is actually needed to capture that rater’s own behaviour. This differs from the global `model_selection` comparison (Section 6.3.10), which picks a single representation for every rater at once: here, e.g. one rater exhibiting extreme or complex behaviour does not force every other rater into a potentially overparameterised representation.

The matrix model (the saturated model) is calibrated; every more restricted representation is then derived from it as marginal means of the matrix facet effects (`centrality/pseudo_halo/bistretch` as restricted forms of those marginals, via the same recipe as `calibrate` – see Section 6.3.1). By default (`test='AIC'`), each rater’s representation is then chosen directly as the minimum-criterion representation among the active set, guarded by a significance test against the simplest active baseline (`aic_sig_test`) and, if set, the `min_effect` practical-difference gate — exactly as for the global `model_selection` comparison (Section 6.3.10), just applied per rater; `test='BIC'` works the same way, substituting BIC for AIC. Under `test='LR'`, each rater is instead tested top-down along a fixed ladder, in two stages.

In the first stage, matrix v bivector is tested first; a rater who needs the matrix representation is assigned it directly. A rater for whom bivector suffices is then tested on two further, simultaneous forks – bivector v items and bivector v thresholds – and provisionally assigned `global` if both simplifications hold, `items` or `thresholds` if only one does, or `bivector` if neither does.

The second stage runs one extra, targeted stretch test depending on which of those five representations the first stage produced – a simplified design used in place of a full recursive walk over all eight representations’ nesting relationships, still reaching every one of the eight possible outcomes:

- `matrix` winner: no extra test (`centrality`, `pseudo_halo` and `bistretch` all restrict representations `matrix` has already beaten).
- `bivector` winner: tested v `bistretch`.
- `thresholds` winner: tested v `centrality`.
- `items` winner: tested v `pseudo_halo`.
- `global` winner: tested v both `centrality` and `pseudo_halo` ($df = 1$ each). If exactly one beats `global`, that one is assigned. If both do, resolved via a further test of `bistretch` against `global`; if `bistretch` itself is not confirmed, falls back to whichever single axis had the stronger evidence. If neither beats `global`, the assignment stays `global`.

At every rung, a more complex representation is retained only if it clears both the likelihood-ratio significance test and, if set, the `min_effect` practical-difference gate.

An anchored variant (called using argument `anchors`) first calibrates the matrix model in anchor mode (Section 6.3.2) before running the same ladder, since a different anchoring frame of reference can result in different model selections for raters, as their facet effect profiles change. Downstream methods that support `model='mixed'` (Sections 6.4.1, 6.3.3 and 6.4.4) read the assignment produced here.

Usage

```
self.per_rater_model_selection(anchors=None, test='AIC', models=None,
                               aic_sig_test=True,
                               sampling='dynamic', alpha=0.05, min_effect=0, warm_corr=True,
                               tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                               constant=0.1, method='cos', matrix_power=3,
                               log_lik_tol=0.000001, seed=None)
```

Arguments

anchors	List or <code>None</code> : rater identifiers used to define the anchoring frame of reference (see Section 6.3.2). Different anchor sets can give different, individually valid, per-rater assignments, since the zero-sum constraint means one non-uniform rater can induce apparent non-uniformity in others. All raters, including anchors, are still tested. Default <code>anchors=None</code> (unanchored).
test	String, one of 'AIC', 'BIC' or 'LR': model-selection criterion. Default is <code>test='AIC'</code> .
models	List/tuple of strings, or <code>None</code> : restricts which representations each rater can be assigned (each one of the eight in Section 6.3.1, or their <code>threshold_stretch/item_stretch</code> aliases). For <code>test='AIC'/'BIC'</code> , a direct filter – the minimum is taken only over the given representations (any non-empty subset works; see <code>aic_sig_test</code> below for how the significance test adapts when <code>global</code> is excluded). For <code>test='LR'</code> , <code>{global, items, thresholds, bivector}</code> must all be included; raises <code>ValueError</code> if the LR backbone is not fully included. <code>None</code> (default) uses all eight representations.
aic_sig_test	Boolean: when <code>test='AIC'</code> , imposes a significance test requiring the more complex model to be more likely with $p > 0.95$, based on the relative-likelihood p-value $p = e^{-\Delta\text{AIC}/2} < \alpha$. Adds <code>p_vs_baseline</code> and <code>baseline_model</code> columns to <code>per_rater_model_selection_table</code> . Default is <code>aic_sig_test=True</code> .
sampling	String, integer, or <code>None</code> : per-rater subsampling used to calculate log-likelihood, to avoid bias towards more complex models. 'dynamic' uses $T = \min(20(I-1)(m-1), 1500)$; an integer fixes T directly; <code>None</code> uses every non-extreme person the rater has scored. Default is <code>sampling='dynamic'</code> .
seed	Integer or <code>None</code> : seed for the per-rater subsampling random number generator (only used when <code>sampling</code> triggers). <code>None</code> (default) draws fresh entropy each call, matching every other stochastic method in <i>RaschPy</i> – see Section 1.

Arguments continue on the next page.

Arguments (continued)

<code>alpha</code>	Float: significance level – for <code>test='LR'</code> , the threshold for accepting a restriction at a ladder rung; for <code>test='AIC'</code> with <code>aic_sig_test=True</code> , the threshold for the relative-likelihood p-value. Default is <code>alpha=0.05</code> .
<code>min_effect</code>	Float: minimum effect size, in logits, required to retain a more complex representation at each ladder rung, measured as the maximum absolute difference between the more complex and simpler representation’s facet effect surfaces at that rung. Applied as a post-hoc walk-back from the selected representation toward <code>global</code> for AIC/BIC, or as an additional condition alongside the p-value for LR. Default <code>min_effect=0</code> disables the gate.
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_adjustment</code>	Person-estimation arguments for the matrix model – see Section 6.3.6.
<code>constant</code> , <code>method</code> , <code>matrix_power</code> , <code>log_lik_tol</code>	Calibration arguments for the matrix model – see Section 6.3.1.

Returns

If `anchors=None`:

<code>self.rater_models</code>	pandas series: the selected representation (any of the eight in Section 6.3.1, or whichever subset <code>models</code> restricted the assignment to) for each rater.
<code>self.facet_effects_mixed</code>	pandas dataframe: each rater’s facet effects under their own assigned representation, in the common (item, threshold) matrix layout.
<code>self.per_rater_model_selection_table</code>	pandas dataframe: full per-rater, per-rung test statistics.
<code>self.per_rater_model_selection_counts</code>	pandas series: count of raters assigned to each representation.

If `anchors` is provided, the equivalent `self.anchor_rater_models`, `self.anchor_facet_effects_mixed`, `self.anchor_per_rater_model_selection_table` and `self.anchor_per_rater_model_selection_counts` are set instead. In both cases:

<code>self.per_rater_model_selection_runs</code>	Dictionary: every call’s results above, keyed by <code>tuple(anchors)</code> (or <code>None</code> for an unanchored call), so results from an earlier call survive a later call with a different anchor set. e.g. <code>self.per_rater_model_selection_runs[tuple(my_anchors)].per_rater_model_selection_table</code> .
--	--

Examples

To assign each rater the simplest adequate representation using the default AIC criterion:

```
self.per_rater_model_selection()
```

To use a likelihood-ratio ladder with a stricter significance level and a minimum practical effect size of 0.3 logits, anchored to a set of raters stored in `my_anchor_raters`:

```
self.per_rater_model_selection(anchors=my_anchor_raters, test='LR', alpha=0.01,  
                               min_effect=0.3)
```

To restrict every rater's assignment to the original five representations – excluding `matrix` and all three stretch representations entirely, so the ladder never considers them:

```
self.per_rater_model_selection(test='LR',  
                               models=['global', 'items', 'thresholds', 'bivector'])
```

6.4 Statistical output

6.4.1 `item_stats_df`

Description

Produces a table of item statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.item_stats_df_global(anchors=None, full=False, ext_scores=True, zstd=False,
                           point_measure_corr=False, dp=3, se=True, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                           method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                           no_of_samples=500, interval=None, seed=None)

self.item_stats_df_items(anchors=None, full=False, ext_scores=True, zstd=False,
                           point_measure_corr=False, dp=3, se=True, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                           method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                           no_of_samples=500, interval=None, seed=None)

self.item_stats_df_thresholds(anchors=None, full=False, ext_scores=True, zstd=False,
                               point_measure_corr=False, dp=3, se=True, warm_corr=True,
                               tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                               method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                               no_of_samples=500, interval=None, seed=None)

self.item_stats_df_matrix(anchors=None, full=False, ext_scores=True, zstd=False,
                           point_measure_corr=False, dp=3, se=True, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                           method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                           no_of_samples=500, interval=None, seed=None)

self.item_stats_df_bivector(anchors=None, full=False, ext_scores=True, zstd=False,
                             point_measure_corr=False, dp=3, se=True, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                             method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                             no_of_samples=500, interval=None, seed=None)

self.item_stats_df_centrality(anchors=None, full=False, ext_scores=True, zstd=False,
                               point_measure_corr=False, dp=3, se=True, warm_corr=True,
                               tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                               method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                               no_of_samples=500, interval=None, seed=None)
```

```

self.item_stats.df_pseudo_halo(anchors=None, full=False, ext_scores=True, zstd=False,
                                point_measure_corr=False, dp=3, se=True, warm_corr=True,
                                tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                                no_of_samples=500, interval=None, seed=None)

self.item_stats.df_bistretch(anchors=None, full=False, ext_scores=True, zstd=False,
                              point_measure_corr=False, dp=3, se=True, warm_corr=True,
                              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                              method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                              no_of_samples=500, interval=None, seed=None)

```

Arguments

<code>anchors</code>	List: If <code>None</code> , unanchored central item location estimates and standard errors will be used. If a list of rater names is passed, anchored estimates and standard errors, anchored to the raters passed, will be used. Other statistics are unaffected by <code>anchors</code> . Default is <code>anchors=None</code> .
<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>zstd</code>	Boolean: If <code>True</code> , infit and outfit standardised z-scores are reported alongside mean square statistics. Default is <code>zstd=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>point_measure_corr</code>	Boolean: If <code>True</code> , point-polyserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure polyserial correlation for an ideal item. Default is <code>point_measure_corr=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>se</code>	Boolean: If <code>True</code> , computes and reports the SE column (and CI bound columns, if <code>interval</code> is set). If <code>False</code> , skips the bootstrap entirely – useful when only Infit MS/Outfit MS are needed (e.g. repeated simulation runs), since those do not depend on the bootstrap and are unaffected by this flag. Forces <code>interval=None</code> when <code>False</code> . Default is <code>se=True</code> .
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's (1989) bias correction applied during generation of person estimates (see Section 6.3.5). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>tolerance=0.00001</code> .

Arguments continue on the next page.

Arguments (continued)

<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 6.3.1). Default value is <code>constant=0.1</code> .
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates (see Section 6.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 6.3.1). Ignored for other methods.
<code>no_of_samples</code>	Integer: Number of bootstrap samples for standard error estimates (see Section 6.3.3). More samples lead to more accurate standard error estimates, but take longer to compute. Default <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 6.3.3). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .
<code>seed</code>	Integer or None: seed passed through to the internal <code>std_errors/fit_statistics</code> calls (and to the lazily-triggered <code>per_rater_model_selection</code> call for <code>model='mixed'</code>) if not already computed. None (default) draws fresh entropy each call. See Section 1.

Returns

Attribute: one of `self.item_stats_global`, `self.item_stats_items`, `self.item_stats_thresholds`, `self.item_stats_matrix`, `self.item_stats_bivector`, `self.item_stats_centrality`, `self.item_stats_pseudo_halo` or `self.item_stats_bistretch`, a pandas dataframe with one row for each item and the following columns:

Estimate	Central item location estimate.
SE	Bootstrapped SE of central item estimate. Only produced if <code>se=True</code> .
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 2.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 5%. Only produced if <code>se=True</code> and <code>interval</code> is set.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value as for lower bound above.
Count	Count of responses.
Facility	Item facility: mean score as a proportion of the maximum possible score.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if <code>zstd=True</code> .
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if <code>zstd=True</code> .
PM corr	Point-polyserial correlation. Only produced if <code>point_measure_corr=True</code> .
Exp PM corr	Expected value of point-measure polyserial correlation. Only produced if <code>point_measure_corr=True</code> .
Disordered	If <code>True</code> , threshold estimates indicate the presence of disordered thresholds within the item (Andrich, 2010).

Examples

To produce summary unanchored `self.item_stats_global` table with most commonly reported statistics:

```
self.item_stats_df_global()
```

To produce a full `self.item_stats_items` table with all statistics, anchored to raters `Rater_1` and `Rater_2`:

```
self.item_stats_df_items(anchors=['Rater_1', 'Rater_2'], full=True)
```

To produce an unanchored `self.item_stats_matrix` table with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
self.item_stats_df_matrix(zstd=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

6.4.2 threshold_stats_df

Description

Produces a table of threshold statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or $\text{\LaTeX}$.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.threshold_stats_df_global(anchors=None, full=False, zstd=False, disc=False,
                                point_measure_corr=False, dp=3, se=True,
                                warm_corr=True, tolerance=0.00001, max_iters=100,
                                ext_score_adjustment=0.5, method='cos', constant=0.1,
                                matrix_power=3, log_lik_tol=0.000001, no_of_samples=500,
                                interval=None, seed=None)

self.threshold_stats_df_items(anchors=None, full=False, zstd=False, disc=False,
                                point_measure_corr=False, dp=3, se=True,
                                warm_corr=True, tolerance=0.00001, max_iters=100,
                                ext_score_adjustment=0.5, method='cos', constant=0.1,
                                matrix_power=3, log_lik_tol=0.000001, no_of_samples=500,
                                interval=None, seed=None)

self.threshold_stats_df_thresholds(anchors=None, full=False, zstd=False, disc=False,
                                    point_measure_corr=False, dp=3, se=True,
                                    warm_corr=True, tolerance=0.00001, max_iters=100,
                                    ext_score_adjustment=0.5, method='cos', constant=0.1,
                                    matrix_power=3, log_lik_tol=0.000001,
                                    no_of_samples=500, interval=None, seed=None)

self.threshold_stats_df_matrix(anchors=None, full=False, zstd=False, disc=False,
                                point_measure_corr=False, dp=3, se=True,
                                warm_corr=True, tolerance=0.00001, max_iters=100,
                                ext_score_adjustment=0.5, method='cos', constant=0.1,
                                matrix_power=3, log_lik_tol=0.000001, no_of_samples=500,
                                interval=None, seed=None)

self.threshold_stats_df_bivector(anchors=None, full=False, zstd=False, disc=False,
                                  point_measure_corr=False, dp=3, se=True,
                                  warm_corr=True, tolerance=0.00001, max_iters=100,
                                  ext_score_adjustment=0.5, method='cos', constant=0.1,
                                  matrix_power=3, log_lik_tol=0.000001, no_of_samples=500,
                                  interval=None, seed=None)
```



```

self.threshold_stats_df_centrality(anchors=None, full=False, zstd=False, disc=False,
                                   point_measure_corr=False, dp=3, se=True,
                                   warm_corr=True, tolerance=0.00001, max_iters=100,
                                   ext_score_adjustment=0.5, method='cos', constant=0.1,
                                   matrix_power=3, log_lik_tol=0.000001, no_of_samples=500,
                                   interval=None, seed=None)

self.threshold_stats_df_pseudo_halo(anchors=None, full=False, zstd=False, disc=False,
                                    point_measure_corr=False, dp=3, se=True,
                                    warm_corr=True, tolerance=0.00001, max_iters=100,
                                    ext_score_adjustment=0.5, method='cos', constant=0.1,
                                    matrix_power=3, log_lik_tol=0.000001, no_of_samples=500,
                                    interval=None, seed=None)

self.threshold_stats_df_bistretch(anchors=None, full=False, zstd=False, disc=False,
                                  point_measure_corr=False, dp=3, se=True,
                                  warm_corr=True, tolerance=0.00001, max_iters=100,
                                  ext_score_adjustment=0.5, method='cos', constant=0.1,
                                  matrix_power=3, log_lik_tol=0.000001, no_of_samples=500,
                                  interval=None, seed=None)

```

Arguments

anchors	List: If <code>None</code> , unanchored central item location estimates and standard errors will be used. If a list of rater names is passed, anchored estimates and standard errors, anchored to the raters passed, will be used. Other statistics are unaffected by <code>anchors</code> . Default is <code>anchors=None</code> .
full	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
zstd	Boolean: If <code>True</code> , infit and outfit standardised z-scores are reported alongside mean square statistics. Default is <code>zstd=False</code> . Automatically <code>True</code> if <code>full=True</code> .
disc	Boolean: If <code>True</code> , item discrimination is reported. The discrimination of the empirical item slope relative to the ideal logistic ogive, with 1 perfect, greater than 1 showing overfit and less than 1 showing underfit; discrimination is similar to the 2PL IRT discrimination parameter (Linacre, 2023), but is a descriptive statistic in the SLM rather than an item parameter.

Arguments continue on the next page.

Arguments (continued)

<code>point_measure_corr</code>	Boolean: If <code>True</code> , point-polyserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure polyserial correlation for an ideal item. Default is <code>point_measure_corr=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>se</code>	Boolean: If <code>True</code> , computes and reports the SE column (and CI bound columns, if <code>interval</code> is set). If <code>False</code> , skips the bootstrap entirely – useful when only Infit MS/Outfit MS are needed (e.g. repeated simulation runs), since those do not depend on the bootstrap and are unaffected by this flag. Forces <code>interval=None</code> when <code>False</code> . Default is <code>se=True</code> .
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & BATTERY, 2022b) during central item location estimation (see section 6.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & BATTERY, 2022b) for central item location estimation (see section 6.3.1). Default value is <code>constant=0.1</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 6.3.3). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Unused if <code>se=False</code> . Default is <code>no_of_samples=500</code> .

Arguments continue on the next page.

Arguments (continued)

<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 6.3.3). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Ignored if <code>se=False</code> . Default is <code>interval=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed passed through to the internal <code>std_errors/fit_statistics</code> calls (and to the lazily-triggered <code>per_rater_model_selection</code> call for <code>model='mixed'</code>) if not already computed. <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.threshold_stats_global`, `self.threshold_stats_items`, `self.threshold_stats_thresholds`, `self.threshold_stats_matrix` or `self.threshold_stats_bivector`, a pandas dataframe with one row for each threshold and the following columns:

Estimate	Rasch-Andrich threshold location estimate.
SE	Bootstrapped standard error of threshold estimate. Only produced if <code>se=True</code> .
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 2.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 5%. Only produced if <code>se=True</code> and <code>interval</code> is set.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: 97.5% represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled 95%. Only produced if <code>se=True</code> and <code>interval</code> is set.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if <code>zstd=True</code> .
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if <code>zstd=True</code> .
Discrim	Item discrimination. Only produced if <code>disc=True</code> .
PM corr	Point-polyserial correlation. Only produced if <code>point_measure_corr=True</code> .
Exp PM corr	Expected value of point-measure polyserial correlation. Only produced if <code>point_measure_corr=True</code> .

Examples

To produce an unanchored summary `self.threshold_stats` table under the global representation with the most commonly reported statistics:

```
self.threshold_stats_df_global()
```

To produce a full `self.threshold_stats` table under the item representation with all statistics, anchored to raters `Rater_1` and `Rater_2`:

```
self.threshold_stats_df(anchors=['Rater_1', 'Rater_2'], full=True)
```

To produce an unanchored `self.threshold_stats` table under the matrix representation with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
self.threshold_stats_df_matrix(zstd=True)
```

Other arguments may be used to alter parameters of central item location, threshold and person location estimation.

6.4.3 category_stats_df

Description

Produces a table of category *width* statistics – `threshold_stats_df` (Section 6.4.2) reports raw threshold locations; this reports the difference between consecutive thresholds for each item, $\tau_{k+1} - \tau_k$. The shared, zero-summed threshold vector has only `max_score-1` true degrees of freedom, so the per-threshold standard errors in `threshold_stats_df` are correlated rather than independent, and understate the true amount of error. No infit/outfit or other fit statistics are reported, since these apply to thresholds rather than categories. Auto-triggers `calibrate` (Section 6.3.1) and `std_errors` (Section 6.3.3) directly if not already run.

A category width can be negative – a negative width at category k means thresholds τ_k and τ_{k+1} are disordered for that item (category k is never the most likely response at any person location). Prop `disordered` makes this a continuous diagnostic rather than a straightforward yes/no: the proportion of bootstrap resamples in which that item’s category width was negative, reasonably read as the probability that the true category is disordered.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.category_stats_df_global(anchors=None, dp=3, warm_corr=True, tolerance=0.00001,
                              max_iters=100, ext_score_adjustment=0.5, method='cos',
                              constant=0.1, matrix_power=3, log_lik_tol=0.000001,
                              no_of_samples=500, interval=None, seed=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.category_stats_df(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>anchors</code>	List: If <code>None</code> , unanchored thresholds and standard errors are used. If a list of rater names is passed, anchored thresholds and standard errors, anchored to the raters passed, are used. Default is <code>anchors=None</code> .
<code>dp</code>	Integer: Number of decimal places reported in output. Default is <code>dp=3</code> .
<code>warm_corr</code> , <code>tolerance</code> , <code>max_iters</code> , <code>ext_score_</code> <code>adjustment</code>	Person location estimation keyword arguments, used only if not already run. See Section 6.3.5 for details of each.
<code>method</code> , <code>constant</code> , <code>matrix_power</code> , <code>log_lik_</code> <code>tol</code>	Calibration keyword arguments, used only if <code>calibrate</code> (Section 6.3.1) has not already been run. See Section 6.3.1 for details of each.
<code>no_of_samples</code> , <code>interval</code> , <code>seed</code>	Bootstrap standard error keyword arguments, used only if <code>std_errors</code> (Section 6.3.3) has not already been run. See Section 6.3.3 for details of each.

Returns

Attribute `self.category_stats_global`, `self.category_stats_items`, `self.category_stats_thresholds`, `self.category_stats_matrix`, `self.category_stats_bivector`, `self.category_stats_centrality`, `self.category_stats_pseudo_halo` or `self.category_stats.bistretch` (as appropriate), a pandas dataframe with one row per category, indexed `Category 1` to `Category max_score-1` (one fewer row than `threshold_stats`, since a width is defined between each pair of consecutive thresholds), with the following columns:

Estimate	Category width (can be negative).
SE	Bootstrapped standard error of the width, differenced <i>within</i> each bootstrap resample before taking the standard deviation, so it correctly reflects the covariance between adjacent thresholds rather than naively combining their individual standard errors.
2.5% / 97.5%	Bounds of the empirical bootstrap estimate interval, if <code>interval</code> is available (see <code>threshold_stats_df</code> , Section 6.4.2, for the column-naming convention).
Disordered	Boolean: True if Estimate is negative.
Prop disordered	Proportion of bootstrap resamples in which this category's width was negative – reasonably read as the probability that the true category is disordered. Useful for interpreting <code>Disordered</code> in both directions: a True that is only weakly supported (<code>Prop disordered</code> close to 0.5) versus robust, or a False that is nonetheless uncertain versus clear-cut.

Example

To produce the `self.category_stats_global` table, alongside the threshold-level table, under the global representation:

```
self.threshold_stats_df_global()
self.category_stats_df_global()
```

To check which categories, if any, are disordered under the matrix representation:

```
self.category_stats_df_matrix()
self.category_stats_matrix[self.category_stats_matrix['Disordered']]
```

6.4.4 `rater_stats_df`

Description

Produces a table of rater statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.rater_stats_df_global(anchors=None, full=False, zstd=False, dp=3, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                           method='cos', constant=0.1, matrix_power=3,
                           log_lik_tol=0.000001, no_of_samples=500, interval=None,
                           seed=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.rater_stats_df(model='global', ...)` with `model` set to any of the eight representation names.

A ninth representation, `model='mixed'`, is also available on the base `self.rater_stats_df(model='mixed', ...)` call (no `rater_stats_df_mixed()` alias), reporting each rater's statistics under whichever representation `per_rater_model_selection` (Section 6.3.11) assigned it, lazily run with its own default selection criteria if it has not already been run (or was run with a different anchor set).

For `centrality`, `pseudo_halo` and `bistretch`, a single call stores two tables. The primary `self.rater_stats_{model}` reports the model's own raw shift/stretch parameters (`lambda`, plus `omega` – or, for `bistretch`, `omega_items` and `omega_thresholds`) with bootstrap SE and, if `interval` is set, CI. The reconstructed vectors these representations imply (per item for `pseudo_halo`, per threshold for `centrality`, both for `bistretch`) is stored separately as `self.rater_stats_{model}_full_vector`.

Passing `alias=True` relabels the primary table's parameter columns with a human-readable name instead, purely cosmetically – the values themselves are identical either way:

Model	Raw parameter	alias=True label
centrality	lambda	Global
centrality	omega	Threshold stretch
pseudo_halo	lambda	Global
pseudo_halo	omega	Item stretch
bistretch	lambda	Global
bistretch	omega_items	Item stretch
bistretch	omega_thresholds	Threshold stretch

Table 36: `rater_stats_df(alias=True)` column labels

Arguments

<code>anchors</code>	List: If <code>None</code> , unanchored central item location estimates and standard errors will be used. If a list of rater names is passed, anchored estimates and standard errors, anchored to the raters passed, will be used. Other statistics are unaffected by <code>anchors</code> . Default is <code>anchors=None</code> .
<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>zstd</code>	Boolean: If <code>True</code> , infit and outfit standardised z-scores are reported alongside mean square statistics. Default is <code>zstd=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>marginal</code>	<code>matrix</code> representation only. Boolean: If <code>True</code> , marginal mean facet effect vectors by item and threshold are reported, rather than the full matrix representation. Default is <code>marginal=True</code> .
<code>alias</code>	<code>centrality/pseudo_halo/bistretch</code> only. Boolean: If <code>True</code> , labels the primary table's parameter columns with a human-readable alias instead of the raw parameter name – 'Global' for <code>lambda</code> , 'Item stretch'/'Threshold stretch' for <code>omega/omega_items/omega_thresholds</code> depending on which axis that model's stretch parameter acts on (see Table 36). Purely cosmetic – the values themselves are unchanged, and <code>self.rater_stats_{model}_full_vector</code> is unaffected (it has no <code>lambda/omega</code> columns to alias). Ignored (no effect) for every other model. Default is <code>alias=False</code> .
<code>divergence_test</code>	<code>centrality/pseudo_halo/bistretch</code> only. <code>None</code> , 'wald' or 't': if set, runs an item-by-item (or threshold-by-threshold, or both for <code>bistretch</code>) test comparing the free vector model's own per-element estimates against the stretch model's reconstruction of them, flagging elements where the simpler representation diverges significantly. SE is estimated via a paired bootstrap of the free and stretch estimates. 'wald' references the test statistic against the standard normal distribution; 't' uses a Student's <i>t</i> -distribution with <code>no_of_samples-1</code> degrees of freedom. <code>None</code> (default) skips this entirely; requires <code>store_bootstrap=True</code> on the stretch model's own bootstrap SE computation. Adds 'Flagged items' and/or 'Flagged categories' column(s) to the primary table (present regardless of <code>full</code>). If <code>full=True</code> , also adds one plain boolean column per item and/or threshold directly after that summary, and a per-element boolean 'Flagged' row to <code>rater_stats_{model}_full_vector</code> . Ignored (no effect) for every other model. Default is <code>divergence_test=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>correction</code>	Only relevant when <code>divergence_test</code> is set. <code>None</code> or <code>'bh'</code> : if <code>'bh'</code> , Benjamini-Hochberg FDR correction is applied within each rater's own family of tests (that rater's items, or that rater's thresholds – not pooled across raters). Default <code>correction=None</code> compares raw p -values against α directly. No-op when <code>divergence_test=None</code> .
<code>alpha</code>	Only relevant when <code>divergence_test</code> is set. Float: significance threshold for flagging, applied after correction (if any). Default is <code>alpha=0.05</code> .
<code>plot</code>	Only relevant when <code>divergence_test</code> is set. Boolean: if <code>True</code> , also builds and stores <code>self.fit_plot_{model}</code> (e.g. <code>self.fit_plot_pseudo_halo</code>) – a scatter of the free vector model's estimates against the stretch model's reconstruction, flagged elements in a different colour, in the same style as <code>self.anchor_plot</code> elsewhere in this package's DIF/anchor-selection machinery. Like every other statistic in this family, it also picks up the usual model-alias spelling (e.g. <code>self.fit_plot_item_stretch</code>) automatically – it is a stored attribute, not a separate callable method. Default is <code>plot=False</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 6.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>

Arguments continue on the next page.

Arguments (continued)

<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 6.3.1). Default value is <code>constant=0.1</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 6.3.3). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 6.3.3). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed passed through to the internal <code>std_errors/fit_statistics</code> calls (and to the lazily-triggered <code>per_rater_model_selection</code> call for <code>model='mixed'</code>), only used if not already computed. <code>None</code> (default) draws fresh entropy each call. See Section 1.

Returns

Attribute: one of `self.rater_stats_global`, `self.rater_stats_items`, `self.rater_stats_thresholds`, `self.rater_stats_matrix`, `self.rater_stats_bivector`, `self.rater_stats_centrality`, `self.rater_stats_pseudo_halo`, `self.rater_stats_bistretch` or `self.rater_stats_mixed`, a pandas dataframe with one row for each rater and the following columns. For `centrality/pseudo_halo/bistretch`, the same call also stores `self.rater_stats_{model}_full_vector` (see above), with the same column structure applied per item/threshold instead of per parameter:

<code>Estimate</code>	Rater facet effect estimate (structure depends on <code>model</code> : a scalar for <code>global</code> ; a vector by item, threshold, or <code>item×threshold</code> cell for <code>items/thresholds/matrix</code> with <code>marginal=False</code> ; a vector by parameter – <code>lambda</code> , plus <code>omega</code> (or, for <code>bistretch</code> , <code>omega_items</code> and <code>omega_thresholds</code>) – for <code>centrality/pseudo_halo/bistretch</code>).
<code>SE</code>	Bootstrapped standard error of the facet effect estimate.
<code>2.5%</code>	Lower bound of empirical bootstrap estimate interval. Column name will change to fit <code>interval</code> value: <code>2.5%</code> represents the lower bound for <code>interval=0.95</code> but, for example, <code>interval=0.9</code> will produce a column labelled <code>5%</code> .

Returns continue on the next page.

Returns (continued)

97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 97.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 95%.
Count	Count of responses made by the rater.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if zstd=True.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if zstd=True.

Examples

To produce a summary unanchored `self.rater_stats_global` table with the most commonly reported statistics:

```
self.rater_stats_df_global()
```

To produce a full `self.rater_stats_items` table with all statistics, anchored to raters `Rater_1` and `Rater_2`:

```
self.rater_stats_df_items(anchors=['Rater_1', 'Rater_2'], full=True)
```

To produce an unanchored `self.item_stats_matrix` table with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
self.item_stats_df_matrix(zstd=True)
```

To produce a full `self.rater_stats_pseudo_halo` table (λ/ω with SEs and 95% CIs) and, from the same call, the reconstructed full ($\text{rater} \times \text{item}$) severity vector as `self.rater_stats_pseudo_halo_full_vector`:

```
self.rater_stats_df_pseudo_halo(full=True)
```

To produce that same table with human-readable column labels ('Global'/'Item stretch' instead of λ/ω – see Table 36) instead:

```
self.rater_stats_df_pseudo_halo(full=True, alias=True)
```

To run the item-by-item divergence test (Wald, with Benjamini-Hochberg correction), then inspect the per-rater flagged-item count and, for one specific item, which raters were flagged:

```
self.rater_stats_df_pseudo_halo(full=True, divergence_test='wald',  
                                correction='bh')  
print(self.rater_stats_pseudo_halo['Flagged items'])  
print(self.rater_stats_pseudo_halo['Item_1'])  
print(self.rater_stats_pseudo_halo_full_vector['Item_1'])
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

The ‘mixed’ rater representation

`rater_stats_df(model='mixed')` reports rater statistics under the per-rater representation assignment from `per_rater_model_selection` (Section 6.3.11), lazily run with its own default selection criteria if it has not already been run (or was run with a different anchor set) – see the ‘mixed’ representation note under `calibrate` (Section 6.3.1). Since different raters may sit under different representations, the resulting table adds a `Model` column recording each rater’s assigned representation, and defaults to `marginal=True`: item and threshold marginal statistics only, since the underlying facet effect structures are not directly comparable cell-by-cell across raters assigned different representations. Pass `marginal=False` for the full (item, threshold) cell-level table regardless. There is no `rater_stats_df_mixed()` convenience alias; call the dispatcher form directly.

6.4.5 `check_anchor_homogeneity`

Description

Tests whether the raters (or other additional facet) selected as an anchor set (Section 6.3.2) are sufficiently homogenous in their behaviours, before `calibrate_anchor` recentres their group mean to zero. On its own, `calibrate_anchor` simply subtracts the mean of facet effect element across the anchor set, thereby setting the mean of the anchors for each element to zero; it does not test whether that mean is a stable, shared reference point, or an artefact of averaging over raters who do not actually behave alike.

If one or more anchor raters have a genuinely different facet location from the rest, the anchored zero becomes unstable and the whole anchored scale inherits that instability. Separate from `calibrate_anchor`, since the default 'cochran' test requires calculation of additional bootstrap standard errors. Unrelated to `per_rater_model_selection` (Section 6.3.11) – this test asks whether the selected anchor raters agree with each other. Free (unanchored) facet effects are used, so the test is not circularly biased toward finding homogeneity by construction.

Homogeneity is tested by one of two methods:

1. Cochran's (1954) Q heterogeneity test (the standard meta-analysis approach), $Q = \sum_r w_r (\lambda_r - \bar{\lambda})^2$ with weights $w_r = 1/SE_r^2$ from bootstrap standard errors, chi-squared with $df = n_{\text{anchors}} - 1$, evaluated on the anchor set as supplied. Which specific rater or raters are responsible are then identified by sequential exclusion (a standard meta-analysis outlier diagnostic): repeatedly drop the rater with the largest $|z|$ against the current remaining group's own weighted mean and retest, until Q is no longer significant – re-centring on each remaining subgroup this way isolates the true offender(s), rather than a single-pass test against the full group's mean, under which a single aberrant rater can impute seemingly significant in the rest of the cohort. 'robust'
2. Iglewicz & Hoaglin modified (robust) z-score (Iglewicz & Hoaglin, 1993), testing each rater's component against the anchor group's own median and MAD.

Usage

```
self.check_anchor_homogeneity(model, anchors, rater_homogeneity_test='cochran',
                              per_component='auto', alpha=0.05, correction='bh',
                              robust_z_threshold=3.5, no_of_samples=500, constant=0.1,
                              method='cos', matrix_power=3, log_lik_tol=0.000001,
                              seed=None)
```

Arguments

<code>model</code>	String: one of 'global', 'items', 'thresholds', 'matrix', 'bivector', 'centrality', 'pseudo_halo' or 'bistretch'.
<code>anchors</code>	List: anchor rater to test. At least two anchor raters required to test homogeneity.

Arguments continue on the next page.

Arguments (continued)

<code>rater_homogeneity_test</code>	String, 'cochran' for Cochran's (1954) Q heterogeneity test, or 'robust' for robust z. Default is <code>rater_homogeneity_test='cochran'</code> .
<code>per_component</code>	'auto', True or False: for the <code>items/thresholds/matrix/bivector</code> representations, where facet effect is a vector or surface rather than a scalar value, also tests homogeneity component by component (per item/threshold/etc.), in addition to the scalar whole-profile-mean test. Default is <code>per_component='auto'</code> .
<code>alpha</code>	Float: significance level for every Cochran's Q test run here – the initial omnibus test, each step of the sequential-exclusion procedure, and the per-component tests (<code>rater_homogeneity_test='cochran'</code> only). Default is <code>alpha=0.05</code> .
<code>correction</code>	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction across the per-component tests only (Benjamini-Hochberg false discovery rate control (Benjamini & Hochberg, 1995) or Bonferroni) – these are genuinely independent tests, one per component, so correction is appropriate. Not applied to per-rater flagging, since sequential exclusion already decides that via repeated omnibus tests rather than a family of independent per-rater p-values. <code>rater_homogeneity_test='cochran'</code> only. Default is <code>correction='bh'</code> .
<code>robust_z_threshold</code>	Float: flagging threshold for <code>rater_homogeneity_test='robust'</code> (Iglewicz & Hoaglin's own recommended default). Default is <code>robust_z_threshold=3.5</code> .
<code>no_of_samples</code>	Integer: bootstrap samples, only used if unanchored standard errors for this representation are not already stored (<code>rater_homogeneity_test='cochran'</code> only). Default is <code>no_of_samples=500</code> .
<code>constant,</code> <code>matrix_power,</code> <code>log_lik_tol</code>	<code>method,</code> <code>matrix_power,</code> <code>log_lik_tol</code> Calibration/bootstrap arguments, used only if calibration or standard errors for this representation are not already computed – see Sections 6.3.1 and 6.3.3.
<code>seed</code>	Integer or None: seed passed through to the internal <code>std.errors</code> call (only used if unanchored standard errors are not already computed, <code>rater_homogeneity_test='cochran'</code> only). None (default) draws fresh entropy each call. See Section 1.

Returns

<code>self.anchor_homogeneity_test</code>	pandas series or None: Q , df , p , Flagged and <code>N_dropped_for_homogeneity</code> for the scalar (whole-profile) omnibus test, evaluated on the anchor set as supplied – <code>N_dropped_for_homogeneity</code> is how many raters the sequential-exclusion procedure needed to remove before Q stopped being significant. None if <code>rater_homogeneity_test='robust'</code> , since no omnibus statistic exists for robust z .
<code>self.anchor_homogeneity_per_rater</code>	pandas dataframe: one row per anchor rater. For <code>rater_homogeneity_test='cochran'</code> : Facet effect, SE, z , p , Flagged (whether sequential exclusion removed this rater – z/p are relative to whichever group they were tested against: their own removal step if dropped, or the final retained group if not). For <code>rater_homogeneity_test='robust'</code> : Facet effect, Robust z , Flagged.
<code>self.anchor_homogeneity_per_component</code>	pandas dataframe or None: one row per component (item name, threshold number, or item-threshold pair) with that component's own omnibus result: Q , df , p , $p_{corrected}$, Flagged ('cochran'), or Flagged only ('robust', meaning at least one rater exceeded the threshold on that component). None if <code>per_component</code> resolves to False.

Examples

To test whether a proposed anchor set stored in `my_anchor_raters` agrees with itself under the matrix representation, using the default Cochran's Q approach:

```
self.check_anchor_homogeneity('matrix', my_anchor_raters)
```

To use the purely descriptive robust z -score approach instead, without per-component breakdown:

```
self.check_anchor_homogeneity('matrix', my_anchor_raters,  
                              rater_homogeneity_test='robust', per_component=False)
```

6.4.6 `person_stats_df`

Description

Produces a table of person statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.person_stats_df_global(anchors=None, full=False, rsem=False, zstd=False, dp=3,
                             warm_corr=True, tolerance=0.00001, max_iters=100,
                             ext_score_adjustment=0.5, method='cos', constant=0.1,
                             matrix_power=3, log_lik_tol=0.000001, interval=None,
                             no_of_samples=500)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.person_stats_df(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>anchors</code>	List: If <code>None</code> , unanchored central item location estimates will be used to generate person location estimates. If a list of rater names is passed, anchored estimates and standard errors, anchored to the raters passed, will be used. Other statistics are unaffected by <code>anchors</code> . Default is <code>anchors=None</code> .
<code>full</code>	Boolean: If <code>True</code> , a full table with all available statistics is produced. Default is <code>full=False</code> for a summary table with the most commonly reported statistics.
<code>rsem</code>	Boolean: If <code>True</code> , realistic standard error of measurement (RSEM), which takes into account item fit (Wright, 1996), is reported alongside the conditional standard error of measurement (CSEM). Default is <code>rsem=False</code> . Automatically <code>True</code> if <code>full=True</code> .
<code>zstd</code>	Boolean: If <code>True</code> , infit and outfit standardised z-scores are reported alongside mean square statistics. Default is <code>zstd=False</code> .
<code>dp</code>	Integer: Number of decimal places reported in output.
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is <code>warm_corr=True</code> .

Arguments continue on the next page.

Arguments (continued)

<code>tolerance</code>	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 6.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 6.3.1). Default value is <code>constant=0.1</code> .
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for central item location estimation (see Section 6.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for central item location estimation. Ignored for other methods.
<code>interval</code>	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 6.3.3). Defines a central interval of proportion p , e.g. <code>interval=0.95</code> defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is <code>interval=None</code> .
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 6.3.3). More samples mean more accurate standard error estimates, but take longer to compute. Default is <code>no_of_samples=500</code> .

Returns

Attribute `self.person_stats_global`, `self.person_stats_items`, `self.person_stats_thresholds`, `self.person_stats_matrix`, `self.person_stats_bivector`, `self.person_stats centrality`, `self.person_stats_pseudo_halo` or `self.person_stats_bistretch`, a pandas dataframe with one row for each person and the following columns:

Estimate	Person location estimate.
CSEM	Conditional standard error of measurement for person location estimate.
RSEM	Realistic standard error of measurement for person estimate if <code>rsem=True</code>
Score	Total score summed across all items and raters.
Max score	Maximum available score (items attempted $\times$ raters $\times$ <code>max_score</code>).
p	Mean score as a proportion of the maximum possible score.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score.

Examples

To produce an unanchored summary `self.person_stats` table under the global rater representation with the most commonly reported statistics:

```
self.person_stats_df_global()
```

To produce a full `self.person_stats` table under the matrix rater representation with all statistics, anchored to raters `Rater_1` and `Rater_2`:

```
self.person_stats_df(anchors=['Rater_1', 'Rater_2'], full=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

6.4.7 test_stats_df

Description

Produces a table of test-level statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or L^AT_EX.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.test_stats_df_global(dp=3, warm_corr=True, tolerance=0.00001, max_iters=100,
                           ext_score_adjustment=0.5, method='cos', constant=0.1,
                           matrix_power=3, log_lik_tol=0.000001, no_of_samples=500,
                           seed=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.test_stats_df(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>dp</code>	Integer: Number of decimal places reported in output.
<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's (1989) bias correction is applied during generation of person estimates (see Section 6.3.5). Default <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.

Arguments continue on the next page.

Arguments (continued)

<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 6.3.1). Options are <code>'ls'</code> for least squares (Choppin, 1968, 1985), <code>'evm'</code> for the eigenvector method (Garner & Engelhard, 2002), <code>'cos'</code> for cosine similarity (Kou & Lin, 2014) or <code>'log-lik'</code> for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 6.3.1). Default value is <code>constant=0.1</code> .
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for central item location estimation (see Section 6.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for central item location estimation. Ignored for other methods.
<code>no_of_samples</code>	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 6.3.3). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is <code>no_of_samples=500</code> .
<code>seed</code>	Integer or None: seed passed through to the internal <code>std_errors/fit_statistics</code> calls (and to the lazily-triggered <code>per_rater_model_selection</code> call for <code>model='mixed'</code>) if not already computed. None (default) draws fresh entropy each call. See Section 1.

Returns

Attribute `self.test_stats`, a pandas dataframe with two columns, `Items` and `Persons` and rows for a range of descriptive statistics describing the distributions of the estimates and different statistics related to reliability – these statistics describe the suitability of the data for estimating and differentiating the parameters, rather than properties of the parameters themselves. The statistics are:

Mean	The mean of the estimates.
SD	The standard deviation of the estimates.
Separation ratio	The separation ratio (Wright, 1996; Wright & Masters, 1982), which is the SD of person locations reported as a ratio of SE units. For persons, $G_p = \sigma_p / \sqrt{\sum_n SE_n^2}$, where σ_p is the variance of the person estimates and SE_n is the RSEM (see Section 6.4.6) for person n . The formula is symmetrical for items, substituting the SE of estimation for RSEM.

Returns continue on the next page.

Returns (continued)

Strata	The number of statistically distinct levels of either person location or item location as strata with centers three measurement errors apart (Wright & Masters, 1982:106). For persons, $H_p = (4G_p + 1)/3$, with symmetrical results for items.
Reliability	A Rasch-specific reliability statistic (Wright, 1996), derived from PSI and which is a Rasch-specific reliability statistic similar to Cronbach's Alpha (Cronbach, 1951), and which may be interpreted the same way – as the proportion of variance of the estimates which stems from variation in location or location rather than estimation error. For persons, $R_p = G_p^2/(1 + G_p^2)$, with symmetrical results for items.

Example

To produce a `self.test_stats` table under the global rater representation:

```
self.test_stats_df_global()
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

6.4.8 item_res_corr_analysis

Description

Analysis of correlations of standardised residuals by item to test for violations of local item interdependence and unidimensionality requirements.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.item_res_corr_analysis_global(warm_corr=True, tolerance=0.00001, max_iters=100,  
                                   ext_score_adjustment=0.5, constant=0.1, method='cos',  
                                   matrix_power=3, log_lik_tol=0.000001)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.item_res_corr_analysis(model='global', ...)` with `model` set to any of the eight representation names. These forward any extra keyword arguments (e.g. `se=False` – see below) to the internal `fit_statistics` call, only used if the residuals have not already been computed for this model.

Arguments

<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates (see Section 6.3.1). Default value is <code>constant=0.1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of central item location estimates (see Section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>se</code>	Boolean: not part of the fixed signature above, but forwarded via extra keyword arguments to the internal <code>fit_statistics</code> call. Standardised residuals (and hence this analysis) do not depend on the bootstrap, so <code>se=False</code> is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is <code>se=True</code> .
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of central item location estimates (see Section 6.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 6.3.1). Ignored for other methods.

Returns

For tests of violation of the requirement for local item independence (Andrich & Kreiner, 2010; Marais, 2012):

<code>self.item_residual_correlations_global,</code> <code>self.item_residual_correlations_items,</code> <code>self.item_residual_correlations_</code> <code>thresholds,</code> <code>self.item_residual_correlations_matrix,</code> <code>self.item_residual_correlations_bivector,</code> <code>self.item_residual_correlations_</code> <code>centrality,</code> <code>self.item_residual_correlations_pseudo_</code> <code>halo</code> or <code>self.item_residual_correlations_bistretch</code>	A pandas dataframe of pairwise correlations between item standardised residuals.
---	--

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

<code>self.item.eigenvectors_global,</code> <code>self.item.eigenvectors_items,</code> <code>self.item.eigenvectors_thresholds,</code> <code>self.item.eigenvectors_matrix,</code> <code>self.item.eigenvectors_bivector,</code> <code>self.item.eigenvectors_centrality,</code> <code>self.item.eigenvectors_pseudo_halo</code> or <code>self.item.eigenvectors_bistretch</code>	The eigenvectors of the standardised residual correlations matrix.
<code>self.item.eigenvalues_global,</code> <code>self.item.eigenvalues_items,</code> <code>self.item.eigenvalues_thresholds,</code> <code>self.item.eigenvalues_matrix,</code> <code>self.item.eigenvalues_bivector,</code> <code>self.item.eigenvalues_centrality,</code> <code>self.item.eigenvalues_pseudo_halo</code> or <code>self.item.eigenvalues_bistretch</code>	The eigenvalues corresponding to the eigenvectors.
<code>self.item.variance_explained_global</code> <code>self.item.variance_explained_items,</code> <code>self.item.variance_explained_thresholds,</code> <code>self.item.variance_explained_matrix,</code> <code>self.item.variance_explained_bivector,</code> <code>self.item.variance_explained_centrality,</code> <code>self.item.variance_explained_pseudo_halo</code> or <code>self.item.variance_explained_bistretch</code>	The variance explained by each principal component.
<code>self.item.loadings_global,</code> <code>self.item.loadings_items,</code> <code>self.item.loadings_thresholds,</code> <code>self.item.loadings_matrix,</code> <code>self.item.loadings_bivector,</code> <code>self.item.loadings_centrality,</code> <code>self.item.loadings_pseudo_halo</code> or <code>self.item.loadings_bistretch</code>	The loading of each item onto each of the principal components, for the first of which large loadings ('large' typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce an item residual correlation analysis under the global rater representation:

```
self.item_res_corr_analysis_global()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

6.4.9 rater_res_corr_analysis

Description

Analysis of correlations of standardised residuals by item to test for violations of local item interdependence and unidimensionality requirements.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.rater_res_corr_analysis_global(warm_corr=True, tolerance=0.00001, max_iters=100,  
                                   ext_score_adjustment=0.5, constant=0.1, method='cos',  
                                   matrix_power=3, log_lik_tol=0.000001)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.rater_res_corr_analysis(model='global', ...)` with `model` set to any of the eight representation names. These forward any extra keyword arguments (e.g. `se=False` – see below) to the internal `fit_statistics` call, only used if the residuals have not already been computed for this model.

Arguments

<code>warm_corr</code>	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is <code>warm_corr=True</code> .
<code>tolerance</code>	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>tolerance=0.00001</code> .
<code>max_iters</code>	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is <code>max_iters=100</code> .
<code>ext_score_adjustment</code>	Float: Value in range (0, 1) to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to <code>ext_score_adjustment</code> (if zero) or the maximum possible score minus <code>ext_score_adjustment</code> (if maximum score) before estimating location.
<code>constant</code>	Float: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates (see Section 6.3.1). Default value is <code>constant=0.1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>method</code>	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of central item location estimates (see Section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method (Garner & Engelhard, 2002), 'cos' for cosine similarity (Kou & Lin, 2014) or 'log-lik' for (iterative) log-likelihood (Bradley & Terry, 1952). Default is <code>method='cos'</code>
<code>matrix_power</code>	Integer: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of central item location estimates (see Section 6.3.1). Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b).
<code>log_lik_tol</code>	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 6.3.1). Ignored for other methods.
<code>se</code>	Boolean: not part of the fixed signature above, but forwarded via extra keyword arguments to the internal <code>fit_statistics</code> call. Standardised residuals (and hence this analysis) do not depend on the bootstrap, so <code>se=False</code> is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is <code>se=True</code> .

Returns

For tests of violation of the requirement for local rater independence (Andrich & Kreiner, 2010; Marais, 2012):

<code>self.rater_residual_correlations_global,</code> <code>self.rater_residual_correlations_items,</code> <code>self.rater_residual_correlations_</code> <code>thresholds,</code> <code>self.rater_residual_correlations_matrix,</code> <code>self.rater_residual_correlations_bivector,</code> <code>self.rater_residual_correlations_</code> <code>centrality,</code> <code>self.rater_residual_correlations_pseudo_</code> <code>halo</code> or <code>self.rater_residual_correlations_bistretch</code>	A pandas dataframe of pairwise correlations between item standardised residuals.
---	--

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

<code>self.rater_eigenvectors_global,</code> <code>self.rater_eigenvectors_items,</code> <code>self.rater_eigenvectors_thresholds,</code> <code>self.rater_eigenvectors_matrix,</code> <code>self.rater_eigenvectors_bivector,</code> <code>self.rater_eigenvectors_centrality,</code> <code>self.rater_eigenvectors_pseudo_halo</code> or <code>self.rater_eigenvectors_bistretch</code>	The eigenvectors of the standardised residual correlations matrix.
<code>self.rater_eigenvalues_global,</code> <code>self.rater_eigenvalues_items,</code> <code>self.rater_eigenvalues_thresholds,</code> <code>self.rater_eigenvalues_matrix,</code> <code>self.rater_eigenvalues_bivector,</code> <code>self.rater_eigenvalues_centrality,</code> <code>self.rater_eigenvalues_pseudo_halo</code> or <code>self.rater_eigenvalues_bistretch</code>	The eigenvalues corresponding to the eigenvectors.
<code>self.rater_variance_explained_global</code> <code>self.rater_variance_explained_items,</code> <code>self.rater_variance_explained_thresholds,</code> <code>self.rater_variance_explained_matrix,</code> <code>self.rater_variance_explained_bivector,</code> <code>self.rater_variance_explained_centrality,</code> <code>self.rater_variance_explained_pseudo_halo</code> or <code>self.rater_variance_explained_bistretch</code>	The variance explained by each principal component.
<code>self.rater_loadings_global,</code> <code>self.rater_loadings_items,</code> <code>self.rater_loadings_thresholds,</code> <code>self.rater_loadings_matrix,</code> <code>self.rater_loadings_bivector,</code> <code>self.rater_loadings_centrality,</code> <code>self.rater_loadings_pseudo_halo</code> or <code>self.rater_loadings_bistretch</code>	The loading of each rater onto each of the principal components, for the first of which large loadings ('large' typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce a rater residual correlation analysis under the global rater representation:

```
self.rater_res_corr_analysis_global()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

6.4.10 save_stats

Description

Exports `item_stats`, `threshold_stats`, `rater_stats`, `person_stats` and `test_stats` for a given rater representation to file in a single call. Auto-triggers `item_stats_df`, `threshold_stats_df`, `rater_stats_df`, `person_stats_df` and `test_stats_df` (Sections 6.4.1, 6.4.2, 6.4.4, 6.4.6 and the section above) for that representation if any have not already been run – with their default arguments plus whichever of `dp/warm_corr/etc.` below are passed, so call those methods directly first if non-default table columns are wanted in the saved output. Does not include `category_stats` (Section 6.4.3) – save that table separately with `self.category_stats_{model}.to_csv(...)` if needed.

Usage

```
self.save_stats(model='global', filename='', format='csv', alias=False, dp=3,
               warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
               method='cos', constant=0.1, matrix_power=3, log_lik_tol=0.000001,
               no_of_samples=500, interval=None)
```

or equivalently `self.save_stats_global(...)` (and `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), which fix `model` and take the remaining arguments unchanged.

Arguments

<code>model</code>	String, one of 'global', 'items', 'thresholds', 'matrix', 'bivector', 'centrality', 'pseudo_halo' or 'bistretch': the rater representation whose statistics to save. Default is <code>model='global'</code> .
<code>filename</code>	String: output filename or path. For <code>format='csv'</code> , used as a prefix (any trailing <code>.csv</code> is stripped first) for five files, <code>_item_stats.csv</code> , <code>_threshold_stats.csv</code> , <code>_rater_stats.csv</code> , <code>_person_stats.csv</code> and <code>_test_stats.csv</code> . For <code>format='xlsx'</code> , used as a single workbook filename (<code>.xlsx</code> appended automatically if not already present), with the five tables on separate sheets. For <code>model</code> one of 'centrality'/'pseudo_halo'/'bistretch', a sixth file/sheet is also saved – <code>_rater_stats_full_vector.csv</code> or the Rater statistics (full vector) sheet – holding <code>self.rater_stats_{model}_full_vector</code> (see Section 6.4.4).
<code>format</code>	String: 'csv' or 'xlsx'. Default is <code>format='csv'</code> .

Arguments continue on the next page.

Arguments (continued)

<code>alias</code>	centrality/pseudo.halo/bistretch only. Boolean: passed straight through to the internal <code>rater_stats_df</code> call (Section 6.4.4, Table 36), labelling the saved rater statistics table's parameter columns with a human-readable alias instead of the raw parameter name. Ignored (no effect) for every other model. As with the other arguments below, only takes effect if <code>rater_stats_{model}</code> has not already been computed – see the row below. Default is <code>alias=False</code> .
<code>divergence_test,</code> <code>correction,</code> <code>alpha,</code> <code>plot</code>	centrality/pseudo.halo/bistretch only. Passed straight through to the internal <code>rater_stats_df</code> call (Section 6.4.4). Note the per-element 'Flagged' breakdown in <code>rater_stats_{model}_full_vector</code> requires <code>rater_stats_df</code> 's own <code>full=True</code> , which <code>save_stats</code> does not currently expose – only the 'Flagged items'/'Flagged categories' summary column(s) in the primary table will appear here. Defaults <code>divergence_test=None</code> , <code>correction=None</code> , <code>alpha=0.05</code> , <code>plot=False</code> .
<code>dp,</code> <code>warm_corr,</code> <code>tolerance,</code> <code>max_iters,</code> <code>ext_score_adjustment,</code> <code>method,</code> <code>constant,</code> <code>matrix_power,</code> <code>log_lik_</code> <code>tol,</code> <code>no_of_samples,</code> <code>interval</code>	Passed through to whichever of <code>item_stats_df</code> / <code>threshold_stats_df</code> / <code>rater_stats_df</code> / <code>person_stats_df</code> / <code>test_stats_df</code> has not already been computed for <code>model</code> . See those sections for details of each.

Returns

None. Writes the five tables to file as described above.

Example

To save item, threshold, rater, person and test statistics under the global representation, as five CSV files prefixed `my_mfrm_results`:

```
self.save_stats_global('my_mfrm_results')
```

To save the matrix representation's tables to separate sheets of a single Excel workbook:

```
self.save_stats_matrix('my_mfrm_results', format='xlsx')
```

To save the centrality representation's tables with the rater statistics table's parameter columns aliased ('Global'/'Threshold stretch' rather than λ/ω):

```
self.save_stats_centrality('my_mfrm_centrality_results', alias=True)
```

6.4.11 `save_residuals_items` / `save_residuals_raters`

Description

Export the item residual correlation analysis (`item_res_corr_analysis`, Section 6.4.8) or rater residual correlation analysis (`rater_res_corr_analysis`, Section above) to file, for a given rater representation. Auto-triggers the corresponding analysis if not already run. Two separate method families exist since the two analyses produce independent sets of eigenvectors/eigenvalues/variance explained/loadings tables (Sections 6.4.8 and above), which are not usually wanted in the same output file.

Usage

```
self.save_residuals_items_global(filename, format='csv', single=True, dp=3)
self.save_residuals_raters_global(filename, format='csv', single=True, dp=3)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch` in place of `_global`, for both families). Any extra keyword arguments (e.g. `warm_corr`, `constant`) are forwarded to the underlying residual correlation analysis, used only if it has not already been run for that representation.

Arguments

<code>filename</code>	String: output filename or path (extension appended automatically if not already present).
<code>format</code>	String: 'csv' or 'xlsx'. Default is <code>format='csv'</code> .
<code>single</code>	Boolean: if <code>True</code> , writes all four tables (eigenvectors, eigenvalues, variance explained, loadings) into a single file – sequentially, separated by blank lines, for <code>format='csv'</code> ; at successive row offsets on one sheet ('Residual analysis'), for <code>format='xlsx'</code> . If <code>False</code> , writes each table to its own file (csv) or sheet (xlsx). Default is <code>single=True</code> .
<code>dp</code>	Integer: Number of decimal places for rounding the saved values. Default is <code>dp=3</code> .

Returns

None. Writes the four tables to file as described above.

Example

To save the item residual correlation analysis under the global representation as a single CSV file:

```
self.save_residuals_items_global('my_mfrm_item_residuals')
```

To save the rater residual correlation analysis under the matrix representation, each table to its own sheet of a single Excel workbook:

```
self.save_residuals_raters_matrix('my_mfrm_rater_residuals', format='xlsx', single=False)
```

6.4.12 `category_counts_df`

Description

Produces a table of counts of scores in each category, plus total and missing responses, aggregated across the requested facet elements (e.g. raters) – and, alongside it, the same breakdown by each individual facet element within that same subset. All items share the same `max_score` in `MFRM`, so there are no blank cells.

If the user wants to compare counts between different subgroups of persons according to a criterion such as an exogenous variable, this may be done via the argument `counts_name`. This stores tables within a namespace, `self.counts`, meaning that a succession of tables, e.g. one per exogenous-variable group, can be kept side by side for comparison rather than overwriting each other.

Usage

```
self.category_counts_df(persons=None, items=None, facet_elements=None, counts_name=None)
```

Arguments

<code>persons</code>	List or String: Person or subset of persons over which to count responses. Default <code>persons=None</code> , which returns counts for all persons.
<code>items</code>	List or String: Item or subset of items over which to count responses. Default <code>items=None</code> , which returns counts for all items.
<code>facet_elements</code>	List or String: Facet element (e.g. rater) or subset of facet elements over which to count responses – restricts both the aggregated table and the per-facet-element breakdown to this subset. Default <code>facet_elements=None</code> , which uses all facet elements (e.g. <code>self.rater_names</code>).
<code>counts_name</code>	String or Int: Name under which to store counts on a subset of persons, items or facet elements, so multiple sets of counts can be made available. If <code>counts_name='my_counts_name'</code> , the aggregated table under <code>self.counts.my_counts_name</code> , the breakdown under <code>self.counts.my_counts_name_facet_elements</code> . If <code>counts_name=None</code> , stores the aggregated table as <code>self.category_counts_table</code> and the per-facet-element breakdown as <code>self.category_counts_facet_elements</code> (also aliased as <code>self.category_counts_raters</code> under the default facet naming). Default is <code>counts_name=None</code> .

Returns

If `counts_name=None`:

<code>self.category_counts_table</code>	pandas dataframe: category counts with one row per item and one column per response category, plus <code>Total</code> and <code>Missing</code> columns, with a <code>Total</code> row, aggregated across the facet elements.
<code>self.category_counts_facet_elements</code> (also <code>self.category_counts_raters</code> under the default facet naming)	pandas multiindex dataframe: the same category counts broken down by facet element within the requested subset, with facet elements as the outer index level and items (plus a <code>Total</code> row per facet element) as the inner index level.

If `counts_name=my_counts_name`:

<code>self.counts.my_counts_name</code>	The aggregated table described above, stored in the namespace <code>self.counts</code> .
<code>self.counts['my_counts_name_facet_elements']</code>	The per-facet-element breakdown described above, stored in the same namespace.

Example

To produce the aggregated and per-rater dataframes of category counts for all persons, items and raters:

```
my_mfrm.category_counts_df()
```

To produce category counts for a subset of persons saved as a list `my_persons`, restricted to rater `'Rater_1'`, with a matching attribute name within the namespace `self.counts`:

```
my_mfrm.category_counts_df(persons=my_persons, facet_elements='Rater_1',
                           counts_name=my_persons)
```


6.5 Plotting functionality

6.5.1 Shared plotting arguments

All the plotting methods described in this section share a set of arguments which may be used to customise the appearance of the plot or save the plot to file automatically. These arguments are:

<code>title</code>	String: Title for the plot, to appear in the image. Default is <code>title=None</code> .
<code>xmin</code>	Float: Minimum displayed point on x-axis, in logits. Default is <code>xmin=-5</code> .
<code>xmax</code>	Float: Maximum displayed point on x-axis, in logits. Default is <code>xmax=5</code> .
<code>plot_style</code>	String: Plot style to use. Available styles are 'white', producing a plot on a white background, and 'dark', producing a plot on a grey background. The styles correspond to Seaborn (Waskom, 2021) styles <code>whitegrid</code> and <code>darkgrid</code> . Default is <code>plot_style='white'</code> .
<code>palette</code>	String: Controls colours of lines and overplotted class intervals dots. Options are 'dark blue', 'light blue', 'dark red', 'light red', 'dark green', 'light green', 'dark grey', 'light grey', 'dark multi' and 'light multi'. Default is <code>palette='dark blue'</code> . Not used for <code>std_residual_plot</code> .
<code>black</code>	Boolean: If <code>True</code> , the plot will be rendered in black and white. Default is <code>black=False</code> .
<code>font</code>	String: The font to use in the plot. Default is <code>font='Times New Roman'</code> .
<code>title_font_size</code>	Float: The size of the title font in points. Default is <code>title_font_size=15</code> .
<code>axis_font_size</code>	Float: The size of the axis label font in points. Default is <code>axis_font_size=12</code> .
<code>labelsize</code>	Float: The size of the axis tick label font in points. Default is <code>labelsize=12</code> .
<code>filename</code>	String: The filename for the saved plot, with no suffix for format. If <code>None</code> , no file will be saved. Default is <code>filename=None</code> .
<code>file_format</code>	The format of the file: <code>png</code> , <code>jpg</code> or <code>svg</code> . Default is <code>file_format=png</code> .
<code>dpi</code>	The resolution of the plot in dpi (dots per inch) – higher resolution plots are better quality but have larger file sizes. Default is <code>dpi=300</code> .

6.5.2 `icc`

Description

Plots the item characteristic curve (or item response function) for an item: person location on the x-axis against expected score on the y-axis. Options to plot observed responses, item threshold line and lines showing locations corresponding to specified expected scores, and to highlight a specified response category.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.icc_global(item, anchor=False, facet_element=None, obs=None, warm=True,
                no_of_classes=5, thresh_lines=False, central_location=False,
                score_lines=None, score_labels=False, cat_highlight=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.icc(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>item</code>	String: The name of the item to plot.
<code>anchor</code>	Boolean: If <code>True</code> , anchored parameters are used; if <code>False</code> , unanchored parameters are used. Default is <code>anchor=False</code> .
<code>facet_element</code>	String: If a <code>facet_element</code> argument is passed, plots the item characteristic curve for the item as rated by the given rater. If <code>facet_element=None</code> , plots a neutral item characteristic curve for a rater with zero facet effect. Default is <code>facet_element=None</code> .
<code>obs</code>	Boolean: If <code>True</code> , mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is <code>obs=None</code> .
<code>warm</code>	Boolean: If <code>True</code> , Warm's bias correction (Warm, 1989) is applied when locating any <code>score_lines</code> on the curve. Default is <code>warm=True</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the thresholds between each response category will be plotted: uncentred thresholds when a <code>facet_element</code> argument is passed, centred thresholds when <code>facet_element=None</code> . Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is <code>thresh_lines=False</code> .

Arguments continue on the next page.

Arguments (continued)

<code>central_location</code>	Boolean: If <code>True</code> , a vertical line is plotted showing the threshold corresponding to the central item location, for uncentred thresholds – see Section 6.2.1); if <code>facet_element=None</code> , the line will be at zero facet effect (the mean of the centred thresholds by definition). Default is <code>central_location=False</code> .
<code>score_lines</code>	List of floats between 0 and <code>self.max_score</code> : Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score.
<code>score_labels</code>	Boolean: If <code>True</code> , scores and locations corresponding to arguments passed to <code>score_lines</code> will be labelled on the plot. Default is <code>score_labels=False</code> .
<code>cat_highlight</code>	Integer: Passing a score between 0 and <code>self.max_score</code> will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic unanchored ICC for `Item_1` under the global rater representation and store the output as a variable `my_icc_plot` and save it to file as `my_icc_plot.png`:

```
my_icc_plot = self.icc_global('Item_1', filename=my_icc_plot)
```

To plot an anchored ICC for `Item_1` under the item rater representation, rated by `Rater_1`, with observed responses and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc_items('Item_1', anchor=True, facet_element='Rater_1', obs=True)
```

To plot an ICC for `Item_1` under the threshold rater representation with a threshold line and highlighted zero category, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc_thresholds('Item_1', thresh_lines=True, cat_highlight=0)
```

To plot an ICC for `Item_1` under the matrix rater representation with lines showing the locations corresponding to expected scores of 0.7 and 1.6, with the expected score and corresponding location labelled, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc_matrix('Item_1', score_lines=[0.7, 1.6], score_labels=True)
```

6.5.3 crcs

Description

Plots category response curves for an item: person location on the x-axis against expected the probability of obtaining a score in each category (0 or 1) on the y-axis. Options to plot observed proportions and item threshold line, and to highlight a specified response category.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.crcs_global(item=None, anchor=False, facet_element=None, obs=None,
                 no_of_classes=5, thresh_lines=False, central_location=False,
                 cat_highlight=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.crcs(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>item</code>	String: The name of the item to plot. If <code>item=None</code> , the shared set of Rasch-Andrich thresholds with mean zero will be plotted, and any observed proportions overplotted using <code>obs</code> will use data from all items. Default is <code>item=None</code> .
<code>anchor</code>	Boolean: If <code>True</code> , anchored parameters are used; if <code>False</code> , unanchored parameters are used. Default is <code>anchor=False</code> .
<code>facet_element</code>	String: If a <code>facet_element</code> argument is passed, plots the category response curves as rated by the given rater, adjusted by rater facet effect. If <code>facet_element=None</code> , plots a neutral category response curves for a rater with zero facet effect. Default is <code>facet_element=None</code> .
<code>obs</code>	List: List of integers between 0 and <code>self.max_score</code> . For each value, mean observed proportions in each ordered response category scoring in that category are plotted against the mean location of the corresponding response class.
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .

Arguments continue on the next page.

Arguments (continued)

<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 6.2.1). Default is <code>central_location=False</code> .
<code>cat_highlight</code>	Integer: Passing a score between 0 and <code>self.max_score</code> will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot basic unanchored category response curves under the global rater representation, rated by rater `Rater_1`, and store the output as a variable `my_crcs_plot` and save it to file as `my_crcs_plot.png`:

```
my_crcs_plot = self.crcs_global(facet_element='Rater_1', filename=my_crcs_plot)
```

To plot anchored category response curves for `Item_1` under the threshold rater representation, rated by rater `Rater_1`, with observed response proportions for category 1 for 8 response classes and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = self.crcs_thresholds(item='Item_1', anchor=True, facet_element='Rater_1', obs=[1],  
                                   no_of_classes=8)
```

To plot unanchored category response curves for `Item_1` under the matrix rater representation, rated by a neutral ‘zero facet effect’ rater, with a threshold line and highlighted zero category, and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = self.crcs_matrix(item='Item_1', thresh_lines=True, cat_highlight=0)
```

6.5.4 threshold_ccs

Description

Plots the threshold characteristic curves (or threshold response functions) for an item. For threshold τ_k , $k \in \{1, \dots, \text{self.max_score}\}$, the threshold characteristic curve is the probability of obtaining a score of k rather than $k - 1$, conditional on the score being either $k - 1$ rather than k , for a given person location. Each threshold characteristic curve functions as a dichotomous item characteristic curve under the SLM (see Sections 3.2.2 and 3.5.2).

For threshold τ_k , `threshold_ccs` plots person location against the probability of obtaining a score of k . Options to plot threshold lines and central item locations, and to highlight a specified response category.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.threshold_ccs_global(item=None, anchor=False, facet_element=None, obs=None,
                          no_of_classes=5, thresh_lines=False,
                          central_location=False, cat_highlight=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.threshold_ccs(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>item</code>	String: The name of the item to plot. If <code>item=None</code> , the shared set of Rasch-Andrich thresholds with mean zero plotted, and any observed proportions overplotted using <code>obs</code> will use data from all items. Default <code>item=None</code> .
<code>anchor</code>	Boolean: If <code>True</code> , anchored parameters are used; if <code>False</code> , unanchored parameters are used. Default is <code>anchor=False</code> .
<code>facet_element</code>	String: If a <code>facet_element</code> argument is passed, plots the category response curves for the item as rated by the given rater. If <code>facet_element=None</code> , plots neutral category response curves for a rater with zero facet effect. Default is <code>facet_element=None</code> .
<code>obs</code>	List: List of integers corresponding to thresholds τ_1 to τ_m , where $m = \text{self.max_score}$, or 'all' or 'none'. If <code>obs=[k]</code> , mean proportions of persons obtaining a score of k rather than $k - 1$, conditional on the score being either $k - 1$ or k , for each of the ordered response categories, will be plotted against the mean location of the corresponding response class. Multiple thresholds may be passed. Default is <code>obs=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 6.2.1). Default is <code>central_location=False</code> .
<code>cat_highlight</code>	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is <code>cat_highlight=None</code> (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot basic unanchored threshold characteristics curves under the global rater representation, rated by `Rater_1`, and store the output as a variable `my_threshold_ccs_plot`, and save it to file as `my_threshold_ccs_plot.png`:

```
my_threshold_ccs_plot = self.threshold_ccs_global(facet_element='Rater_1', filename='my_iic_plot')
```

To plot anchored threshold characteristics curves for `Item_1` under the item rater representation, for a neutral ‘zero’ rater with threshold lines and category 1 highlighted, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = self.threshold_ccs_items(item='Item_1', anchor=True,
                                                thresh_lines=True, cat_highlight=1)
```

To plot unanchored threshold characteristics curves for `Item_1` under the matrix rater representation, rated by `Rater_1`, with observed responses plotted for thresholds 2 and 4 and central item location line, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = self.threshold_ccs_matrix(item='Item_1', facet_element='Rater_1',
                                                obs=[2, 4], central_location=True)
```

6.5.5 iic

Description

Plots the item information curve for an item: person location on the x-axis against Fisher information on the y-axis. Options to plot item threshold line and lines showing Fisher information corresponding to specified locations, and to highlight a specified response category.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.iic_global(item, anchor=False, facet_element=None, ymax=None,
                thresh_lines=False, central_location=False, point_info_lines=None,
                point_info_labels=False, cat_highlight=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.iic(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>item</code>	String: The name of the item to plot.
<code>anchor</code>	Boolean: If <code>True</code> , anchored parameters are used; if <code>False</code> , unanchored parameters are used. Default is <code>anchor=False</code> .
<code>facet_element</code>	String: If a <code>facet_element</code> argument is passed, plots the item information curve for the item as rated by the given rater. If <code>facet_element=None</code> , plots a neutral item information curve for a rater with zero facet effect. Default is <code>facet_element=None</code> .
<code>ymax</code>	Float: The maximum value to show on the y-axis. If <code>None</code> , will infer, plotting a maximum of 1.1 times the maximum item information. Default is <code>ymax=None</code>
<code>thresh_lines</code>	Boolean: If <code>True</code> , vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k - 1$ and k are equally probable. Default is <code>thresh_lines=False</code> .
<code>central_location</code>	Boolean: If <code>True</code> , a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 6.2.1). Default is <code>central_location=False</code> .

Arguments continue on the next page.

Arguments (continued)

<code>point_info_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the Fisher information corresponding to the location.
<code>point_info_labels</code>	Boolean: If <code>True</code> , locations and Fisher information corresponding to arguments passed to <code>point_info_lines</code> will be labelled on the plot. Default is <code>point_info_labels=False</code> .
<code>cat_highlight</code>	Integer: Passing a score between 0 and <code>self.max_score</code> will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is <code>cat_highlight=None</code> (no category highlighted).
<code>ymax</code>	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic unanchored item information curve for `Item_1` under the global rater representation, rated by `Rater_1`, and store the output as a variable `my_iic_plot` and save it to file as `my_iic_plot.png`:

```
my_iic_plot = self.iic_global('Item_1', facet_element='Rater_1', filename='my_iic_plot')
```

To plot an anchored item information curve for `Item_1` under the item rater representation, for a neutral ‘zero’ rater with threshold lines, central item location line and category 1 highlighted, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = self.iic_items('Item_1', anchor=True, thresh_lines=True, central_location=True,
                             cat_highlight=1)
```

To plot an unanchored item information curve for `Item_1` under the matrix rater representation, rated by `Rater_1`, with lines showing the Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding Fisher information labelled, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = self.icc_matrix('Item_1', point_info_lines=[-0.3, 0.7],
                              point_info_labels=True)
```

6.5.6 tcc

Description

Plots the test characteristic curve (or test response function) for a set of items: person location on the x-axis against expected score on the y-axis. Options to plot observed responses and lines showing locations corresponding to specified expected scores.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.tcc_global(anchor=False, items=None, facet_elements=None, obs=False,  
               no_of_classes=5, score_lines=None, score_labels=False)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.tcc(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>anchor</code>	Boolean: If <code>True</code> , anchored parameters are used; if <code>False</code> , unanchored parameters are used. Default is <code>anchor=False</code> .
<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>facet_elements</code>	List: The names of the raters to be used in the plot. <code>facet_elements='all'</code> uses all raters. Default is <code>facet_elements=None</code> , which generates a location based on a single neutral ‘zero’ rater. If multiple raters are selected, the maximum score will be the sum of the maximum scores across raters.
<code>obs</code>	Boolean: If <code>True</code> , mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is <code>obs=False</code> .
<code>no_of_classes</code>	Integer: The number of observed response categories. Default is <code>no_of_classes=5</code> .
<code>score_lines</code>	List of floats between 0 and <code>self.max_score</code> : Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is <code>score_lines=None</code> .
<code>score_labels</code>	Boolean: If <code>True</code> , scores and locations corresponding to arguments passed to <code>score_lines</code> will be labelled on the plot. Default is <code>score_labels=False</code> .

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic unanchored test characteristic curve for all items under the global rater representation for a neutral 'zero' rater and store the output as a variable `my_tcc_plot`, and save it to file as `my_tcc_plot.png`:

```
my_tcc_plot = self.tcc_global(filename=my_tcc_plot)
```

To plot a test characteristic curve for `Item_1` for a subset of items stored as a list `my_item_list` under the threshold rater representation for rater `Rater_1`, with observed responses for 8 response classes and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = self.tcc_thresholds(obs=True, no_of_classes=8)
```

To plot a test characteristic curve for `Item_1` for all items under the matrix rater representation for all raters, with lines showing the locations corresponding to expected scores of 13 and 20, with the expected score and corresponding location labelled, and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = self.tcc_matrix(facet_elements='all', score_lines=[13, 20], score_labels=True)
```

6.5.7 test_info

Description

Plots the test information curve: person location on the x-axis against total Fisher information on the y-axis. Option to plot lines showing Fisher information corresponding to specified locations.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.test_info_global(anchor=False, items=None, facet_elements=None, ymax=None,
                     point_info_lines=None, point_info_labels=False)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.test_info(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>anchor</code>	Boolean: If <code>True</code> , anchored parameters are used; if <code>False</code> , unanchored parameters are used. Default is <code>anchor=False</code> .
<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>facet_elements</code>	List: The names of the raters to be used in the plot. <code>facet_elements='all'</code> uses all raters. Default is <code>facet_elements=None</code> , which generates a location based on a single neutral ‘zero’ rater. If multiple raters are selected, the test information will be the sum of the information across raters.
<code>point_info_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the total Fisher information corresponding to the location. Default is <code>point_info_lines=None</code> .
<code>point_info_labels</code>	Boolean: If <code>True</code> , locations and total Fisher information corresponding to arguments passed to <code>point_info_lines</code> will be labelled on the plot. Default is <code>point_info_labels=False</code> .
<code>ymax</code>	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic anchored test information curve under the global rater representation using all items and a neutral ‘zero’ rater, storing the output as a variable `my_test_info_plot` and saving it to file as `my_test_info_plot.png`:

```
my_test_info_plot = self.test_info_global(anchor=True, filename='my_test_info_plot')
```

To plot an unanchored test information curve under the item rater representation using a subset of items stored as a list `my_item_list` and `Rater_1`, storing the output as a variable `my_test_info_plot`:

```
my_test_info_plot = self.test_info_items(items=my_item_list, facet_elements='Rater_1')
```

To plot an anchored test information curve under the matrix rater representation with lines showing the total Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding total Fisher information labelled, and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = self.test_info_matrix(anchor=True, point_info_lines=[-0.3, 0.7],  
                                         point_info_labels=True)
```

6.5.8 test_csem

Description

Plots the test conditional standard error of measurement (CSEM) curve: person location on the x-axis against CSEM (in logits) on the y-axis. Option to plot lines showing CSEM corresponding to specified locations.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.test_csem_global(anchor=False, items=None, facet_elements=None, point_csem_lines=None,
                      point_csem_labels=False, ymax=5)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.test_csem(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>anchor</code>	Boolean: If <code>True</code> , anchored parameters are used; if <code>False</code> , unanchored parameters are used. Default is <code>anchor=False</code> .
<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>facet_elements</code>	List: The names of the raters to be used in the plot. Default is <code>facet_elements=None</code> , which generates a location based on a single neutral ‘zero’ rater. If multiple raters are selected, the CSEM will be based on the sum of the information across raters.
<code>point_csem_lines</code>	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the CSEM curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the CSEM corresponding to the location. Default is <code>point_csem_lines=None</code> .
<code>point_csem_labels</code>	Boolean: If <code>True</code> , locations and CSEM corresponding to arguments passed to <code>point_csem_lines</code> will be labelled on the plot. Default is <code>point_csem_labels=False</code> .
<code>ymax</code>	The maximum point displayed on the y-axis, in logits.

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic anchored CSEM curve for all items under the global rater representation for a neutral ‘zero’ rater, storing the output as a variable `my_test_csem_plot` and saving it to file as `my_test_csem_plot.png`:

```
my_test_csem_plot = self.test_csem_global(anchor=True, filename='my_test_csem_plot')
```

To plot an unanchored CSEM curve under the item rater representation using a subset of items stored as a list `my_item_list` and `Rater_1`, storing the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = self.test_csem_items(items=my_item_list, facet_elements='Rater_1')
```

To plot an anchored CSEM curve under the matrix rater representation with lines showing the CSEM corresponding to locations of -0.3 and 0.7, with the location and corresponding CSEM labelled, and store the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = self.test_csem_matrix(anchor=True, point_info_lines=[-0.3, 0.7],  
                                         point_info_labels=True)
```

6.5.9 std_residuals_plot

Description

Plots histogram of standardised residuals, with optional overplotting of standard Normal distribution.

Usage

Eight methods are defined, one for each rater representation; all methods share the same argument format:

```
self.std_residuals_plot_global(items=None, facet_elements=None, bin_width=0.5, normal=False)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.std_residuals_plot(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

<code>items</code>	List: The names of the items to be used in the plot. If <code>None</code> , the full set of items will be used. Default is <code>items=None</code> .
<code>facet_elements</code>	List: The names of the raters to be used in the plot. If <code>None</code> , the full set of raters will be used. Default is <code>facet_elements=None</code> .
<code>bin_width</code>	Float: Width of the histogram bins along x-axis. Default is <code>bin_width=0.5</code> .
<code>normal</code>	Boolean: If <code>True</code> , plots a standard normal distribution over the standardised residual histogram for comparison. Default is <code>normal=False</code> .

Since the anchoring process does not affect residuals, there is no `anchor` argument for `std_residuals_plot`. Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot and display a basic standardised residuals histogram under the global rater representation for all items and raters, saving it to file as `my_std_residuals_plot.png`:

```
self.std_residuals_plot_global(filename='my_std_residuals_plot')
```

To plot and display a standardised residuals histogram under the threshold rater representation for all items and rater `'Rater_1'`, with bin width 1 and a standard normal curve:

```
self.std_residuals_plot_thresholds(facet_element='Rater_1', bin_width=1, normal=True)
```

To plot and display a standardised residuals histogram under the matrix rater representation on a subset of items stored as a list in a variable `my_item_list`, and all raters:

```
self.std_residuals_plot_matrix(items=my_item_list)
```


7 class SLM_Sim

7.1 Generating an SLM_Sim simulation

7.1.1 SLM_Sim

Description

Creates an object of the class `SLM_Sim`. Simulates response data fitting the simple logistic model (SLM) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
SLM_Sim(no_of_items, no_of_persons, item_range=3, person_sd=1.5, offset=0, missing=0,  
        manual_persons=None, manual_items=None, manual_person_names=None,  
        manual_item_names=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>item_range</code>	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is <code>item_range=3</code> . Ignored if specified item locations are passed using the <code>manual_items</code> argument.
<code>person_sd</code>	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if specified person locations are passed using the <code>manual_persons</code> argument.
<code>offset</code>	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data in the simulation. If <code>missing</code> is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of <code>missing</code> of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly <code>missing</code> . Default is <code>missing=0</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control the parameters of the simulation. <code>manual_persons=None</code> , random person locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified item locations. Allows the user to control the parameters of the simulation. <code>manual_items=None</code> , random item locations are generated. Default is <code>manual_items=None</code> .
<code>manual_person_names</code>	List: Optional list of person names specified by user. If <code>manual_person_names=None</code> , person names are generated automatically. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names specified by user. If <code>manual_item_names=None</code> , item names are generated automatically. Default is <code>manual_item_names=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed for the random number generator. Pass an integer for a fully reproducible simulation; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Returns

Object of class `SLM_Sim`. Several attributes of object `SLM_Sim` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an <code>SLM</code> object for analysis.
<code>self.no_of_items</code>	Integer: The number of items, specified by argument <code>no_of_items</code> .
<code>self.no_of_persons</code>	Integer: The number of persons, specified by argument <code>no_of_persons</code> .
<code>self.item_range</code>	Float: The generating range of item locations, specified by argument <code>item_range</code> .
<code>self.person_sd</code>	The generating standard deviation of person locations, specified by argument <code>person_sd</code> .
<code>self.offset</code>	The generating offset between the means of person location and item location, specified by argument <code>offset</code> .
<code>self.missing</code>	The generating proportion of missing data, specified by argument <code>missing</code>
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: keys are item names and values are item locations.

Returns continue on the next page.

Returns (continued)

<code>self.person_names</code>	List: Person names. If no person names are passed using the <code>manual_person_names</code> argument, default person names are generated in the format <code>Person_1</code> etc.
<code>self.item_names</code>	List: Item names. If no item names are passed using the <code>manual_item_names</code> argument, default item names are generated in the format <code>Item_1</code> etc.

Examples

To create an `SLM_Sim` object called `my_slm_sim` with randomly generated person locations for 10000 persons and randomly generated item locations for 50 items, specifying an item range of 4 logits, a person SD of 2 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_slm_sim = SLM_Sim(50, 10000, item_range=4, person_sd=2, offset=1, missing=0.3)
```

To create an `SLM_Sim` object called `my_slm_sim` with specified person locations saved as a variable named `my_person_abils`, 50 items with specified item locations saved as a variable named `my_item_diffs`, 100 persons with specified person names saved as a variable named `my_person_names` and specified item names saved as a variable named `my_item_names`, with no missing data:

```
my_slm_sim = SLM_Sim(50, 100, manual_persons=my_person_abils, manual_items=my_item_diffs,  
                    manual_person_names=my_person_names, manual_item_names=my_item_names)
```

7.2 Customising an SLM_Sim simulation

7.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_slm` from `Item_1` to `my_new_item_name`:

```
my_slm.rename_item('Item_1', 'my_new_item_name')
```

7.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_slm` with item names in a list stored as a variable `my_new_item_names`:

```
my_slm.rename_items_all(my_new_item_names)
```

7.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the index of `self.responses` with new name.

Example

To rename a person in object `my_slm` from `Person_1` to `my_new_person_name`:

```
my_slm.rename_person('Person_1', 'my_new_person_name')
```

7.2.4 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the index of `self.responses` with new names.

Example

To rename all persons in object `my_slm` with person names in a list stored as a variable `my_new_person_names`:

```
my_slm.rename_persons_all(my_new_person_names)
```

8 class PCM_Sim

8.1 Generating a PCM_Sim simulation

8.1.1 PCM_Sim

Description

Creates an object of the class `PCM_Sim`. Simulates response data fitting the partial credit model (PCM) (Masters, 1982) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
PCM_Sim(no_of_items, no_of_persons, max_score_vector, item_range=3, category_base=1,
        person_sd=1.5, max_disorder=0, offset=0, missing=0, manual_persons=None,
        manual_items=None, manual_thresholds=None, manual_person_names=None,
        manual_item_names=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>max_score_vector</code>	List or 1-dimensional numpy array of integers: Maximum score for each item.
<code>item_range</code>	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is <code>item_range=3</code> . Ignored if specified item locations are passed using the <code>manual_items</code> argument.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between <code>max_disorder</code> (see below) and twice <code>category_base</code> minus <code>max_disorder</code> . Default is <code>category_base=1</code> .
<code>person_sd</code>	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if specified person locations are passed using the <code>manual_persons</code> argument.
<code>max_disorder</code>	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around <code>category_base</code> . Default is <code>max_disorder=0</code> , with no disordered thresholds.

Arguments continue on the next page.

Arguments (continued)

<code>offset</code>	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data in the simulation. If <code>missing</code> is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of <code>missing</code> of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly <code>missing</code> . Default is <code>missing=0</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control the parameters of the simulation. <code>manual_persons=None</code> , random person locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified item locations. Allows the user to control the parameters of the simulation. <code>manual_items=None</code> , random item locations are generated. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List of lists, or list of 1-dimensional numpy arrays: Optional set of specified Rasch-Andrich thresholds, one list per item (in the same order as the items), each of length <code>max_score_vector</code> 's corresponding entry and summing to zero. Allows the user to control the parameters of the simulation. If <code>manual_thresholds=None</code> , random Rasch-Andrich thresholds are generated independently for each item. Default is <code>manual_thresholds=None</code> .
<code>manual_person_names</code>	List: Optional list of person names specified by user. If <code>manual_person_names=None</code> , person names are generated automatically. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names specified by user. If <code>manual_item_names=None</code> , item names are generated automatically. Default is <code>manual_item_names=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed for the random number generator. Pass an integer for a fully reproducible simulation; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Returns

Object of class `PCM_Sim`. Several attributes of object `PCM_Sim` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an <code>PCM</code> object for analysis.
<code>self.no_of_items</code>	Integer: The number of items, specified by argument <code>no_of_items</code> .
<code>self.no_of_persons</code>	Integer: The number of persons, specified by argument <code>no_of_persons</code> .
<code>self.item_range</code>	Float: The generating range of item locations, specified by argument <code>item_range</code> .
<code>self.max_score_vector</code>	List or 1-dimensional numpy array of integers: List with the maximum possible score for each item.
<code>self.person_sd</code>	The generating standard deviation of person locations, specified by argument <code>person_sd</code> .
<code>self.category_base</code>	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument <code>category_base</code> .
<code>self.max_disorder</code>	The generating maximum permitted threshold disorder, specified by argument <code>max_disorder</code> .
<code>self.offset</code>	The generating offset between the means of person location and item location, specified by argument <code>offset</code> .
<code>self.missing</code>	The generating proportion of missing data, specified by argument <code>missing</code> .
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: keys are item names and values are item locations.
<code>self.thresholds</code>	pandas dataframe: one row per item, one column per threshold index. Since items may have different maximum scores, rows are padded with NaN beyond that item's own <code>max_score_vector</code> entry.
<code>self.person_names</code>	List: Person names. If no person names are passed using the <code>manual_person_names</code> argument, default person names are generated in the format <code>Person_1</code> etc.
<code>self.item_names</code>	List: Item names. If no item names are passed using the <code>manual_item_names</code> argument, default item names are generated in the format <code>Item_1</code> etc.

Example

To create a `PCM_Sim` object called `my_pcm_sim` with randomly generated person locations for 1000 persons and randomly generated item locations for 4 items, specifying maximum scores of 3, 3, 5 and 5, an item range of 4 logits, a person SD of 2 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_pcm_sim = PCM_Sim(4, 1000, max_score_vector=[3, 3, 5, 5], item_range=4,
                    person_sd=2, offset=1, missing=0.3)
```


To create an `PCM_Sim` object called `my_pcm_sim` with 1,000 persons with specified person locations saved as a variable named `my_person_abils`, 30 items with specified item locations saved as a variable named `my_item_diffs`, specified maximum scores saved as a variable named `my_max_score_vector`, specified person names saved as a variable named `my_person_names` and specified item names saved as a variable named `my_item_names`, with no missing data:

```
my_pcm_sim = PCM_Sim(30, 1000, max_score_vector=my_max_score_vector,  
                      manual_persons=my_person_abils, manual_items=my_item_diffs,  
                      manual_person_names=my_person_names, manual_item_names=my_item_names)
```

8.2 Customising an PCM_Sim simulation

8.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_slm` from `Item_1` to `my_new_item_name`:

```
my_slm.rename_item('Item_1', 'my_new_item_name')
```

8.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_slm` with item names in a list stored as a variable `my_new_item_names`:

```
my_slm.rename_items_all(my_new_item_names)
```

8.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the index of `self.responses` with new name.

Example

To rename a person in object `my_slm` from `Person_1` to `my_new_person_name`:

```
my_slm.rename_person('Person_1', 'my_new_person_name')
```

8.2.4 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the index of `self.responses` with new names.

Example

To rename all persons in object `my_slm` with person names in a list stored as a variable `my_new_person_names`:

```
my_slm.rename_persons_all(my_new_person_names)
```

9 class RSM_Sim

9.1 Generating an RSM_Sim simulation

9.1.1 RSM_Sim

Description

Creates an object of the class `RSM_Sim`. Simulates response data fitting the rating scale model (RSM) (Andrich, 1978) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
RSM_Sim(no_of_items, no_of_persons, max_score, item_range=3, category_base=1, person_sd=1.5,
        max_disorder=0, offset=0, missing=0, manual_persons=None, manual_items=None,
        manual_thresholds=None, manual_person_names=None, manual_item_names=None,
        seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>item_range</code>	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is <code>item_range=3</code> . Ignored if specified item locations are passed using the <code>manual_items</code> argument.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between <code>max_disorder</code> (see below) and twice <code>category_base</code> minus <code>max_disorder</code> . Default is <code>category_base=1</code> .
<code>person_sd</code>	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if specified person locations are passed using the <code>manual_persons</code> argument.
<code>max_disorder</code>	Float: If a negative value is passed, disordered Rasch-Andrich thresholds (Andrich, 2010; Pallant & Tennant, 2007) may be generated. A positive value may be passed, which controls the minimum category width and limits the variation in category widths around <code>category_base</code> . Default is <code>max_disorder=0</code> , which does not permit disordered thresholds.

Arguments continue on the next page.

Arguments (continued)

<code>offset</code>	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data in the simulation. If <code>missing</code> is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of <code>missing</code> of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly <code>missing</code> . Default is <code>missing=0</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control the parameters of the simulation. <code>manual_persons=None</code> , random person locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control the parameters of the simulation. <code>manual_items=None</code> , random central item locations are generated. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control the parameters of the simulation. <code>manual_items=None</code> , random Rasch-Andrich thresholds are generated. Default is <code>manual_thresholds=None</code> .
<code>manual_person_names</code>	List: Optional list of person names specified by user. If <code>manual_person_names=None</code> , person names are generated automatically. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names specified by user. If <code>manual_item_names=None</code> , item names are generated automatically. Default is <code>manual_item_names=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed for the random number generator. Pass an integer for a fully reproducible simulation; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Returns

Object of class `RSM_Sim`. Several attributes of object `RSM_Sim` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an RSM object for analysis.
<code>self.no_of_items</code>	Integer: The number of items, specified by argument <code>no_of_items</code> .
<code>self.no_of_persons</code>	Integer: The number of persons, specified by argument <code>no_of_persons</code> .
<code>self.item_range</code>	Float: The generating range of item locations, specified by argument <code>item_range</code> .
<code>self.max_score</code>	Integer: The shared maximum possible score on each item, specified by argument <code>max_score</code> .
<code>self.category_base</code>	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument <code>category_base</code> .
<code>self.person_sd</code>	The generating standard deviation of person locations, specified by argument <code>person_sd</code> .
<code>self.offset</code>	The generating offset between the means of person location and item location, specified by argument <code>offset</code> .
<code>self.missing</code>	The generating proportion of missing data, specified by argument <code>missing</code>
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: keys are item names and values are item locations.
<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> . Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed.
<code>self.person_names</code>	List: Person names. If no person names are passed using the <code>manual_person_names</code> argument, default person names are generated in the format <code>Person_1</code> etc.
<code>self.item_names</code>	List: Item names. If no item names are passed using the <code>manual_item_names</code> argument, default item names are generated in the format <code>Item_1</code> etc.

Examples

To create an `RSM_Sim` object called `my_rsm_sim` with a maximum score of 5, randomly generated person locations for 5000 persons and randomly generated item locations for 8 items, specifying an item range of 4 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_rsm_sim = RSM_Sim(8, 5000, item_range=4, category_base=1.5, person_sd=2, max_disorder=0.5,
                     offset=1, missing=0.3)
```

To create an `RSM_Sim` object called `my_rsm_sim` with a maximum score of 5, specified person locations for 100 persons saved as a variable named `my_person_abils`, specified item locations for 12 items saved as a variable named `my_item_diffs`, a specified set of Rasch-Andrich thresholds saved as a variable named `my_thresholds`, specified person names saved as a variable named `my_person_names` and specified item names saved as a variable named `my_item_names`, with no missing data:

```
my_rsm_sim = RSM_Sim(12, 100, manual_persons=my_person_abils, manual_items=my_item_diffs,  
                    manual_thresholds=my_thresholds, manual_person_names=my_person_names,  
                    manual_item_names=my_item_names)
```

9.2 Customising an RSM_Sim simulation

9.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_rsm` from `Item_1` to `my_new_item_name`:

```
my_rsm.rename_item('Item_1', 'my_new_item_name')
```

9.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_rsm` with item names in a list stored as a variable `my_new_item_names`:

```
my_rsm.rename_items_all(my_new_item_names)
```


9.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the index of `self.responses` with new name.

Example

To rename a person in object `my_rsm` from `Person_1` to `my_new_person_name`:

```
my_rsm.rename_person('Person_1', 'my_new_person_name')
```

9.2.4 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the index of `self.responses` with new names.

Example

To rename all persons in object `my_rsm` with person names in a list stored as a variable `my_new_person_names`:

```
my_rsm.rename_persons_all(my_new_person_names)
```

10 class MFRM_Sim_Global

10.1 Generating an MFRM_Sim_Global simulation

10.1.1 MFRM_Sim_Global

Description

Creates an object of the class `MFRM_Sim_Global`. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the global rater representation (Linacre, 1994) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
MFRM_Sim_Global(no_of_items, no_of_persons, max_score,
                no_of_facet_elements, item_range=2,
                facet_range=2, category_base=1, person_sd=1.5, max_disorder=0, offset=0,
                missing=0, shared_missing=True, manual_persons=None, manual_items=None,
                manual_thresholds=None, manual_facet_effects=None, manual_person_names=None,
                manual_item_names=None, manual_facet_names=None, facet='rater',
                facet_plural=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.
<code>no_of_facet_elements</code>	Integer or None: The number of raters (facet elements) in the simulation.
<code>item_range</code>	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is <code>item_range=2</code> . Ignored if specified item locations are passed using the <code>manual_items</code> argument.
<code>facet_range</code>	Float: The range (maximum - minimum) of rater facet effects for randomly generated rater facet effects, which are drawn from a uniform distribution. Default is <code>facet_range=2</code> . Ignored if specified rater facet effects are passed using the <code>manual_facet_effects</code> argument.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between <code>max_disorder</code> (see below) and twice <code>category_base</code> minus <code>max_disorder</code> . Default is <code>category_base=1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>person_sd</code>	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if specified person locations are passed using the <code>manual_persons</code> argument.
<code>max_disorder</code>	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around <code>category_base</code> . Default is <code>max_disorder=0</code> , with no disordered thresholds.
<code>offset</code>	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data in the simulation. If <code>missing</code> is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of <code>missing</code> of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly <code>missing</code> . Default is <code>missing=0</code> .
<code>shared_missing</code>	Boolean: If <code>True</code> , the same persons are missing across all raters (correlated missingness). If <code>False</code> , missingness is independent across raters. Default is <code>shared_missing=True</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. <code>manual_persons=None</code> , random locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random locations are generated. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random thresholds are generated. Default is <code>manual_thresholds=None</code> .
<code>manual_facet_effects</code>	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If <code>manual_facet_effects=None</code> , random facet effects are generated. Default is <code>manual_facet_effects=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_person_names</code>	List: Optional list of person names specified by user. If <code>manual_person_names=None</code> , person names are generated automatically. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names specified by user. If <code>manual_item_names=None</code> , item names are generated automatically. Default is <code>manual_item_names=None</code> .
<code>manual_facet_names</code>	List: Optional list of item names specified by user. If <code>manual_facet_names=None</code> , item names are generated automatically. Default is <code>manual_facet_names=None</code> .
<code>facet</code>	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format <code>{Facet}_1</code> etc., e.g. <code>Judge_1</code>). Also aliases <code>self.no_of_{facet_plural}</code> and <code>self.{facet_plural}_effects</code> – note this does <i>not</i> alias <code>self.rater_names/self.facet_names</code> , which keep their fixed names regardless. Default is <code>facet='rater'</code> .
<code>facet_plural</code>	String or <code>None</code> : plural form of <code>facet</code> , used for the pluralised attribute aliases described above. If <code>None</code> (default), <code>facet + 's'</code> is used. Default is <code>facet_plural=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed for the random number generator. Pass an integer for a fully reproducible simulation; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Format for `manual_facet_effects` input

Toy example of the input for `manual_facet_effects`, with two items and two persons. Pandas series with rater names as keys and facet effects as values.

```
manual_facet_effects = pd.Series({'Rater_1': -0.5,
                                'Rater_2': 0.5})
```

Returns

Object of class `MFRM_Sim_Global`. Several attributes of object `MFRM_Sim_Global` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an <code>MFRM</code> object for analysis.
<code>self.no_of_items</code>	Integer: The number of items, specified by argument <code>no_of_items</code> .

Returns continue on the next page.

Returns (continued)

<code>self.no_of_persons</code>	Integer: The number of persons, specified by argument <code>no_of_persons</code> .
<code>self.no_of_raters</code>	Integer: The number of raters, specified by argument <code>no_of_raters</code> .
<code>self.item_range</code>	Float: The generating range of item locations, specified by argument <code>item_range</code> .
<code>self.facet_range</code>	Float: The generating range of rater facet effects, specified by argument <code>facet_range</code> .
<code>self.max_score</code>	Integer: The shared maximum possible score on each item, specified by argument <code>max_score</code> .
<code>self.category_base</code>	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument <code>category_base</code> .
<code>self.person_sd</code>	The generating standard deviation of person locations, specified by argument <code>person_sd</code> .
<code>self.offset</code>	The generating offset between the means of person location and item location, specified by argument <code>offset</code> .
<code>self.missing</code>	The generating proportion of missing data, specified by argument <code>missing</code>
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: keys are item names and values are item locations.
<code>self.facet_effects</code>	pandas series: keys are rater names and values are rater facet effects.
<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> . Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed.
<code>self.person_names</code>	List: Person names. If no person names are passed using the <code>manual_person_names</code> argument, default person names are generated as <code>Person_1</code> etc.
<code>self.item_names</code>	List: Item names. If no item names are passed using the <code>manual_item_names</code> argument, default item names are generated as <code>Item_1</code> etc.
<code>self.rater_names</code>	List: Rater names. If no rater names are passed using the <code>manual_facet_names</code> argument, default rater names are generated as <code>Rater_1</code> etc.

Example

To create an `MFRM_Sim_Global` object called `my_mfrm_sim` with a maximum score of 5, randomly generated person locations for 5,000 persons, randomly generated item locations for 10 items and randomly generated rater facet effects for 12 raters, specifying an item range of 4 logits, a rater range of 3 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Global(10, 5000, 12, 5, item_range=4, facet_range=3,
                              category_base=1.5, person_sd=2,
                              max_disorder=0.5, offset=1, missing=0.3)
```

To create an `MFRM_Sim_Global` object called `my_mfrm_sim` with a maximum score of 5, specified person locations for 100 persons saved as a variable named `my_person_abils`, specified item locations for 12 items saved as a variable named `my_item_diffs`, specified rater facet effects for 6 raters saved as a variable named `my_facet_effects`, a specified set of Rasch-Andrich thresholds saved as a variable named `my_thresholds`, specified person names saved as a variable named `my_person_names`, specified item names saved as a variable named `my_item_names` and specified rater names saved as a variable named `my_rater_names`, with no missing data:

```
my_mfrm_sim = MFRM_Sim_Global(12, 100, 6, 5, manual_persons=my_person_abils,
                              manual_items=my_item_diffs,
                              manual_facet_effects=my_facet_effects,
                              manual_thresholds=my_thresholds,
                              manual_person_names=my_person_names,
                              manual_item_names=my_item_names,
                              manual_facet_names=my_rater_names)
```

10.2 Customising an MFRM_Sim_Global simulation

10.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm` from `Item_1` to `my_new_item_name`:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

10.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm.rename_items_all(my_new_item_names)
```

10.2.3 `rename_person`

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

<code>old</code>	String: the old name for the person
<code>new</code>	String: the new name for the person

Returns

Replaces specified person name in the second multiindex level of `self.responses` with new name.

Example

To rename a person in object `my_mfrm` from `Person_1` to `my_new_person_name`:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

10.2.4 `rename_persons_all`

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

<code>new_names</code>	List of new person names as strings
------------------------	-------------------------------------

Returns

Replaces all person names in the second multiindex level of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm.rename_persons_all(my_new_person_names)
```


10.2.5 `rename_rater`

Description

Method to rename a single rater.

Usage

```
self.rename_rater(old, new)
```

Arguments

<code>old</code>	String: the old name for the rater
<code>new</code>	String: the new name for the rater

Returns

Replaces specified rater name in the first multiindex level of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm.rename_rater('Rater_1', 'my_new_rater_name')
```

10.2.6 `rename_raters_all`

Description Method to rename all raters.

Usage

```
self.rename_raters_all(new_names)
```

Arguments

<code>new_names</code>	List of new rater names as strings
------------------------	------------------------------------

Returns

Replaces all rater names in the first multiindex level of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm.rename_raters_all(my_new_rater_names)
```

11 class MFRM_Sim_Items

11.1 Generating an MFRM_Sim_Items simulation

11.1.1 MFRM_Sim_Items

Description

Creates an object of the class `MFRM_Sim_Items`. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the vector-by-item rater representation (Elliott & Buttery, 2022a) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
MFRM_Sim_Items(no_of_items, no_of_persons, max_score,
               no_of_facet_elements, item_range=2,
               facet_range=2, category_base=1, person_sd=1.5, max_disorder=0, offset=0,
               missing=0, shared_missing=True, manual_persons=None, manual_items=None,
               manual_thresholds=None, manual_facet_effects=None, manual_person_names=None,
               manual_item_names=None, manual_facet_names=None, facet='rater',
               facet_plural=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.
<code>no_of_facet_elements</code>	Integer or None: The number of raters (facet elements) in the simulation.
<code>item_range</code>	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is <code>item_range=2</code> . Ignored if specified item locations are passed using the <code>manual_items</code> argument.
<code>facet_range</code>	Float: The range (maximum - minimum) of rater facet effects for randomly generated rater facet effects, which are drawn from a uniform distribution. Default is <code>facet_range=2</code> . Ignored if specified rater facet effects are passed using the <code>manual_facet_effects</code> argument.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between <code>max_disorder</code> (see below) and twice <code>category_base</code> minus <code>max_disorder</code> . Default is <code>category_base=1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>person_sd</code>	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if specified person locations are passed using the <code>manual_persons</code> argument.
<code>max_disorder</code>	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around <code>category_base</code> . Default is <code>max_disorder=0</code> , with no disordered thresholds.
<code>offset</code>	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data in the simulation. If <code>missing</code> is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of <code>missing</code> of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly <code>missing</code> . Default is <code>missing=0</code> .
<code>shared_missing</code>	Boolean: If <code>True</code> , the same persons are missing across all raters (correlated missingness). If <code>False</code> , missingness is independent across raters. Default is <code>shared_missing=True</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. <code>manual_persons=None</code> , random locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random locations are generated. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random thresholds are generated. Default is <code>manual_thresholds=None</code> .
<code>manual_facet_effects</code>	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If <code>manual_facet_effects=None</code> , random facet effects are generated. Default is <code>manual_facet_effects=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_person_names</code>	List: Optional list of person names specified by user. If <code>manual_person_names=None</code> , person names are generated automatically. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names specified by user. If <code>manual_item_names=None</code> , item names are generated automatically. Default is <code>manual_item_names=None</code> .
<code>manual_facet_names</code>	List: Optional list of item names specified by user. If <code>manual_facet_names=None</code> , item names are generated automatically. Default is <code>manual_facet_names=None</code> .
<code>facet</code>	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format <code>{Facet}_1</code> etc., e.g. <code>Judge_1</code>). Also aliases <code>self.no_of_{facet_plural}</code> and <code>self.{facet_plural}_effects</code> – note this does <i>not</i> alias <code>self.rater_names/self.facet_names</code> , which keep their fixed names regardless. Default is <code>facet='rater'</code> .
<code>facet_plural</code>	String or <code>None</code> : plural form of <code>facet</code> , used for the pluralised attribute aliases described above. If <code>None</code> (default), <code>facet + 's'</code> is used. Default is <code>facet_plural=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed for the random number generator. Pass an integer for a fully reproducible simulation; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Format for `manual_facet_effects` input

Toy example of the nested dictionary input for `manual_facet_effects`, with two items and two persons. The outer dictionary has item names for values, which are also the keys for the inner dictionary. The inner dictionaries item names for keys and floats (item facet effects) for values.

```
manual_facet_effects = {'Rater_1': {'Item_1': -1, 'Item_2': 0},
                        'Rater_2': {'Item_1': 0, 'Item_2': 1}}
```

Returns

Object of class `MFRM_Sim_Items`. Several attributes of object `MFRM_Sim_Items` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an <code>MFRM</code> object for analysis.
-----------------------------	--

Returns continue on the next page.

Returns (continued)

<code>self.no_of_items</code>	Integer: The number of items, specified by argument <code>no_of_items</code> .
<code>self.no_of_persons</code>	Integer: The number of persons, specified by argument <code>no_of_persons</code> .
<code>self.no_of_raters</code>	Integer: The number of raters, specified by argument <code>no_of_raters</code> .
<code>self.item_range</code>	Float: The generating range of item locations, specified by argument <code>item_range</code> .
<code>self.facet_range</code>	Float: The generating range of rater facet effects, specified by argument <code>facet_range</code> .
<code>self.max_score</code>	Integer: The shared maximum possible score on each item, specified by argument <code>max_score</code> .
<code>self.category_base</code>	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument <code>category_base</code> .
<code>self.person_sd</code>	The generating standard deviation of person locations, specified by argument <code>person_sd</code> .
<code>self.offset</code>	The generating offset between the means of person location and item location, specified by argument <code>offset</code> .
<code>self.missing</code>	The generating proportion of missing data, specified by argument <code>missing</code>
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: keys are item names and values are item locations.
<code>self.facet_effects</code>	Nested dictionary: Outer keys are rater names, inner keys are item names, as in the <code>manual_facet_effects</code> input example given above.
<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> . Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed.
<code>self.person_names</code>	List: Person names. If no person names are passed using the <code>manual_person_names</code> argument, default person names are generated in the format <code>Person_1</code> etc.
<code>self.item_names</code>	List: Item names. If no item names are passed using the <code>manual_item_names</code> argument, default item names are generated in the format <code>Item_1</code> etc.
<code>self.rater_names</code>	List: Rater names. If no rater names are passed using the <code>manual_facet_names</code> argument, default rater names are generated in the format <code>Rater_1</code> etc.

Examples

To create an `MFRM_Sim_Items` object called `my_mfrm_sim` with a maximum score of 5, randomly generated person locations for 5,000 persons, randomly generated item locations for 10 items and randomly generated rater facet effects for 12 raters, specifying an item range of 4 logits, a rater range of 3 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Items(10, 5000, 12, 5, item_range=4, facet_range=3,
                             category_base=1.5, person_sd=2, max_disorder=0.5,
                             offset=1, missing=0.3)
```

To create an `MFRM_Sim_Items` object called `my_mfrm_sim` with a maximum score of 5, specified person locations for 100 persons saved as a variable named `my_person_abils`, specified item locations for 12 items saved as a variable named `my_item_diffs`, specified rater facet effects for 6 raters saved as a variable named `my_facet_effects`, a specified set of Rasch-Andrich thresholds saved as a variable named `my_thresholds`, specified person names saved as a variable named `my_person_names`, specified item names saved as a variable named `my_item_names` and specified rater names saved as a variable named `my_rater_names`, with no missing data:

```
my_mfrm_sim = MFRM_Sim_Items(12, 100, 6, 5, manual_persons=my_person_abils,
                             manual_items=my_item_diffs,
                             manual_facet_effects=my_facet_effects,
                             manual_thresholds=my_thresholds,
                             manual_person_names=my_person_names,
                             manual_item_names=my_item_names,
                             manual_facet_names=my_rater_names)
```

11.2 Customising an `MFRM_Sim_Items` simulation

11.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm` from `Item_1` to `my_new_item_name`:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

11.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm.rename_items_all(my_new_item_names)
```

11.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the second multiindex level of `self.responses` with new name.

Example

To rename a person in object `my_mfrm` from `Person_1` to `my_new_person_name`:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

11.2.4 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the second multiindex level of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm.rename_persons_all(my_new_person_names)
```


11.2.5 `rename_rater`

Description

Method to rename a single rater.

Usage

```
self.rename_rater(old, new)
```

Arguments

<code>old</code>	String: the old name for the rater
<code>new</code>	String: the new name for the rater

Returns

Replaces specified rater name in the first multiindex level of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm.rename_rater('Rater_1', 'my_new_rater_name')
```

11.2.6 `rename_raters_all`

Description Method to rename all raters.

Usage

```
self.rename_raters_all(new_names)
```

Arguments

<code>new_names</code>	List of new rater names as strings
------------------------	------------------------------------

Returns

Replaces all rater names in the first multiindex level of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm.rename_raters_all(my_new_rater_names)
```

12 class MFRM_Sim_Thresholds

12.1 Generating an MFRM_Sim_Thresholds simulation

12.1.1 MFRM_Sim_Thresholds

Description

Creates an object of the class `MFRM_Sim_Thresholds`. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the vector-by-threshold rater representation (Elliott & Buttery, 2022a) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
MFRM_Sim_Thresholds(no_of_items, no_of_persons, max_score, no_of_facet_elements, item_range=2,
                    facet_range=2, category_base=1, person_sd=1.5, max_disorder=0,
                    offset=0, missing=0, shared_missing=True, manual_persons=None,
                    manual_items=None, manual_thresholds=None, manual_facet_effects=None,
                    manual_person_names=None, manual_item_names=None,
                    manual_facet_names=None, facet='rater', facet_plural=None,
                    seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.
<code>no_of_facet_elements</code>	Integer or None: The number of raters (facet elements) in the simulation.
<code>item_range</code>	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is <code>item_range=2</code> . Ignored if specified item locations are passed using the <code>manual_items</code> argument.
<code>facet_range</code>	Float: The range (maximum - minimum) of rater facet effects for randomly generated rater facet effects, which are drawn from a uniform distribution. Default is <code>facet_range=2</code> . Ignored if specified rater facet effects are passed using the <code>manual_facet_effects</code> argument.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between <code>max_disorder</code> (see below) and twice <code>category_base</code> minus <code>max_disorder</code> . Default is <code>category_base=1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>person_sd</code>	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if specified person locations are passed using the <code>manual_persons</code> argument.
<code>max_disorder</code>	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around <code>category_base</code> . Default is <code>max_disorder=0</code> , with no disordered thresholds.
<code>offset</code>	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data in the simulation. If <code>missing</code> is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of <code>missing</code> of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly <code>missing</code> . Default is <code>missing=0</code> .
<code>shared_missing</code>	Boolean: If <code>True</code> , the same persons are missing across all raters (correlated missingness). If <code>False</code> , missingness is independent across raters. Default is <code>shared_missing=True</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. <code>manual_persons=None</code> , random locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random locations are generated. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random thresholds are generated. Default is <code>manual_thresholds=None</code> .
<code>manual_facet_effects</code>	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If <code>manual_facet_effects=None</code> , random facet effects are generated. Default is <code>manual_facet_effects=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_person_names</code>	List: Optional list of person names specified by user. If <code>manual_person_names=None</code> , person names are generated automatically. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names specified by user. If <code>manual_item_names=None</code> , item names are generated automatically. Default is <code>manual_item_names=None</code> .
<code>manual_facet_names</code>	List: Optional list of item names specified by user. If <code>manual_facet_names=None</code> , item names are generated automatically. Default is <code>manual_facet_names=None</code> .
<code>facet</code>	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format {Facet}_1 etc., e.g. Judge_1). Also aliases <code>self.no_of_{facet_plural}</code> and <code>self.{facet_plural}_effects</code> – note this does <i>not</i> alias <code>self.rater_names/self.facet_names</code> , which keep their fixed names regardless. Default is <code>facet='rater'</code> .
<code>facet_plural</code>	String or None: plural form of <code>facet</code> , used for the pluralised attribute aliases described above. If None (default), <code>facet + 's'</code> is used. Default is <code>facet_plural=None</code> .
<code>seed</code>	Integer or None: seed for the random number generator. Pass an integer for a fully reproducible simulation; None (default) draws fresh entropy each run. See Section 1.

Format for `manual_facet_effects` input

Toy example of the dictionary input for `manual_facet_effects`, with two items, two persons and four categories. Dictionary with item names as keys and numpy arrays as values, of length `self.max_score + 1` of item facet effects for values with the indices of each array corresponding to the threshold, and threshold zero set to zero by convention.

```
manual_facet_effects = {'Rater_1': np.array([0, -1, -0.5, 0]),
                        'Rater_2': np.array([0, 1.5, 0.5, -0.5])}
```

Returns

Object of class `MFRM_Sim_Thresholds`. Several attributes of object `MFRM_Sim_Thresholds` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an MFRM object for analysis.
-----------------------------	---

Returns continue on the next page.

Returns (continued)

<code>self.no_of_items</code>	Integer: The number of items, specified by argument <code>no_of_items</code> .
<code>self.no_of_persons</code>	Integer: The number of persons, specified by argument <code>no_of_persons</code> .
<code>self.no_of_raters</code>	Integer: The number of raters, specified by argument <code>no_of_raters</code> .
<code>self.item_range</code>	Float: The generating range of item locations, specified by argument <code>item_range</code> .
<code>self.facet_range</code>	Float: The generating range of rater facet effects, specified by argument <code>facet_range</code> .
<code>self.max_score</code>	Integer: The shared maximum possible score on each item, specified by argument <code>max_score</code> .
<code>self.category_base</code>	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument <code>category_base</code> .
<code>self.person_sd</code>	The generating standard deviation of person locations, specified by argument <code>person_sd</code> .
<code>self.offset</code>	The generating offset between the means of person location and item location, specified by argument <code>offset</code> .
<code>self.missing</code>	The generating proportion of missing data, specified by argument <code>missing</code>
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: keys are item names and values are item locations.
<code>self.facet_effects</code>	Dictionary: Keys are rater names, values are numpy arrays, as in the <code>manual_facet_effects</code> input example given above.
<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> . Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed.
<code>self.person_names</code>	List: Person names. If no person names are passed using the <code>manual_person_names</code> argument, default person names are generated in the format <code>Person_1</code> etc.
<code>self.item_names</code>	List: Item names. If no item names are passed using the <code>manual_item_names</code> argument, default item names are generated in the format <code>Item_1</code> etc.
<code>self.rater_names</code>	List: Rater names. If no rater names are passed using the <code>manual_facet_names</code> argument, default rater names are generated in the format <code>Rater_1</code> etc.

Example

To create an `MFRM_Sim_Thresholds` object called `my_mfrm_sim` with a maximum score of 5, randomly generated person locations for 5,000 persons, randomly generated item locations for 10 items and randomly generated rater facet effects for 12 raters, specifying an item range of 4 logits, a rater range of 3 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Thresholds(10, 5000, 12, 5, item_range=4,
                                   category_base=1.5, person_sd=2,
                                   max_disorder=0.5, offset=1, missing=0.3)
```

To create an `MFRM_Sim_Thresholds` object called `my_mfrm_sim` with a maximum score of 5, specified person locations for 100 persons saved as a variable named `my_person_abils`, specified item locations for 12 items saved as a variable named `my_item_diffs`, specified rater facet effects for 6 raters saved as a variable named `my_facet_effects`, a specified set of Rasch-Andrich thresholds saved as a variable named `my_thresholds`, specified person names saved as a variable named `my_person_names`, specified item names saved as a variable named `my_item_names` and specified rater names saved as a variable named `my_rater_names`, with no missing data:

```
my_mfrm_sim = MFRM_Sim_Thresholds(12, 100, 6, 5, manual_persons=my_person_abils,
                                   manual_items=my_item_diffs,
                                   manual_facet_effects=my_facet_effects,
                                   manual_thresholds=my_thresholds,
                                   manual_person_names=my_person_names,
                                   manual_item_names=my_item_names,
                                   manual_facet_names=my_rater_names)
```

12.2 Customising an MFRM_Sim_Thresholds simulation

12.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm` from `Item_1` to `my_new_item_name`:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

12.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm.rename_items_all(my_new_item_names)
```

12.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the second multiindex level of `self.responses` with new name.

Example

To rename a person in object `my_mfrm` from `Person_1` to `my_new_person_name`:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

12.2.4 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the second multiindex level of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm.rename_persons_all(my_new_person_names)
```


12.2.5 `rename_rater`

Description

Method to rename a single rater.

Usage

```
self.rename_rater(old, new)
```

Arguments

<code>old</code>	String: the old name for the rater
<code>new</code>	String: the new name for the rater

Returns

Replaces specified rater name in the first multiindex level of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm.rename_rater('Rater_1', 'my_new_rater_name')
```

12.2.6 `rename_raters_all`

Description Method to rename all raters.

Usage

```
self.rename_raters_all(new_names)
```

Arguments

<code>new_names</code>	List of new rater names as strings
------------------------	------------------------------------

Returns

Replaces all rater names in the first multiindex level of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_rsm.rename_raters_all(my_new_rater_names)
```

13 class MFRM_Sim_Matrix

13.1 Generating an MFRM_Sim_Matrix simulation

13.1.1 MFRM_Sim_Matrix

Description

Creates an object of the class `MFRM_Sim_Matrix`. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the matrix representation (Elliott & Buttery, 2022a) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
MFRM_Sim_Matrix(no_of_items, no_of_persons, max_score,
                no_of_facet_elements, item_range=2,
                facet_range=2, category_base=1, person_sd=1.5, max_disorder=0,
                offset=0, missing=0, shared_missing=True, manual_persons=None,
                manual_items=None, manual_thresholds=None, manual_facet_effects=None,
                manual_person_names=None, manual_item_names=None, manual_facet_names=None,
                facet='rater', facet_plural=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.
<code>no_of_facet_elements</code>	Integer or None: The number of raters (facet elements) in the simulation.
<code>item_range</code>	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is <code>item_range=2</code> . Ignored if specified item locations are passed using the <code>manual_items</code> argument.
<code>facet_range</code>	Float: The range (maximum - minimum) of rater facet effects for randomly generated rater facet effects, which are drawn from a uniform distribution. Default is <code>facet_range=2</code> . Ignored if specified rater facet effects are passed using the <code>manual_facet_effects</code> argument.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between <code>max_disorder</code> (see below) and twice <code>category_base</code> minus <code>max_disorder</code> . Default is <code>category_base=1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>person_sd</code>	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if specified person locations are passed using the <code>manual_persons</code> argument.
<code>max_disorder</code>	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around <code>category_base</code> . Default is <code>max_disorder=0</code> , with no disordered thresholds.
<code>offset</code>	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data in the simulation. If <code>missing</code> is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of <code>missing</code> of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly <code>missing</code> . Default is <code>missing=0</code> .
<code>shared_missing</code>	Boolean: If <code>True</code> , the same persons are missing across all raters (correlated missingness). If <code>False</code> , missingness is independent across raters. Default is <code>shared_missing=True</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. <code>manual_persons=None</code> , random locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random locations are generated. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random thresholds are generated. Default is <code>manual_thresholds=None</code> .
<code>manual_facet_effects</code>	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If <code>manual_facet_effects=None</code> , random facet effects are generated. Default is <code>manual_facet_effects=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_person_names</code>	List: Optional list of person names specified by user. If <code>manual_person_names=None</code> , person names are generated automatically. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names specified by user. If <code>manual_item_names=None</code> , item names are generated automatically. Default is <code>manual_item_names=None</code> .
<code>manual_facet_names</code>	List: Optional list of item names specified by user. If <code>manual_facet_names=None</code> , item names are generated automatically. Default is <code>manual_facet_names=None</code> .
<code>facet</code>	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format <code>{Facet}_1</code> etc., e.g. <code>Judge_1</code>). Also aliases <code>self.no_of_{facet_plural}</code> and <code>self.{facet_plural}_effects</code> – note this does <i>not</i> alias <code>self.rater_names/self.facet_names</code> , which keep their fixed names regardless. Default is <code>facet='rater'</code> .
<code>facet_plural</code>	String or <code>None</code> : plural form of <code>facet</code> , used for the pluralised attribute aliases described above. If <code>None</code> (default), <code>facet + 's'</code> is used. Default is <code>facet_plural=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed for the random number generator. Pass an integer for a fully reproducible simulation; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Format for `manual_facet_effects` input

Toy example of the nested dictionary input for `manual_facet_effects`, with two items, two persons and four categories. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the index of the array index corresponding to each threshold (0 to `self.max_score`).

```
manual_facet_effects = {'Rater_1': {'Item_1': np.array([0, -1.5, -1, 0.5]),
                                   'Item_2': np.array([0, 1, 0, -1])},
                       'Rater_2': {'Item_1': np.array([0, -0.5, 0, 0.5]),
                                   'Item_2': np.array([0, 2, 1, 0])}}
```

Returns

Object of class `MFRM_Sim_Matrix`. Several attributes of object `MFRM_Sim_Matrix` are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an <code>MFRM</code> object for analysis.
<code>self.no_of_items</code>	Integer: The number of items, specified by argument <code>no_of_items</code> .
<code>self.no_of_persons</code>	Integer: The number of persons, specified by argument <code>no_of_persons</code> .
<code>self.no_of_raters</code>	Integer: The number of raters, specified by argument <code>no_of_raters</code> .
<code>self.item_range</code>	Float: The generating range of item locations, specified by argument <code>item_range</code> .
<code>self.facet_range</code>	Float: The generating range of rater facet effects, specified by argument <code>facet_range</code> .
<code>self.max_score</code>	Integer: The shared maximum possible score on each item, specified by argument <code>max_score</code> .
<code>self.category_base</code>	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument <code>category_base</code> .
<code>self.person_sd</code>	The generating standard deviation of person locations, specified by argument <code>person_sd</code> .
<code>self.offset</code>	The generating offset between the means of person location and item location, specified by argument <code>offset</code> .
<code>self.missing</code>	The generating proportion of missing data, specified by argument <code>missing</code>
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: keys are item names and values are item locations.
<code>self.facet_effects</code>	Nested dictionary: Outer keys are rater names, inner keys are item names, as in the <code>manual_facet_effects</code> input example given above.
<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> . Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed.
<code>self.person_names</code>	List: Person names. If no person names are passed using the <code>manual_person_names</code> argument, default person names are generated in the format <code>Person_1</code> etc.
<code>self.item_names</code>	List: Item names. If no item names are passed using the <code>manual_item_names</code> argument, default item names are generated in the format <code>Item_1</code> etc.

Returns continue on the next page.

Returns (continued)

<code>self.rater_names</code>	List: Rater names. If no rater names are passed using the <code>manual_facet_names</code> argument, default rater names are generated in the format <code>Rater_1</code> etc.
-------------------------------	---

Example

To create an `MFRM_Sim_Matrix` object called `my_mfrm_sim` with a maximum score of 5, randomly generated person locations for 5,000 persons, randomly generated item locations for 10 items and randomly generated rater facet effects for 12 raters, specifying an item range of 4 logits, a rater range of 3 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Matrix(10, 5000, 12, 5, item_range=4, facet_range=3,
                               category_base=1.5, person_sd=2,
                               max_disorder=0.5, offset=1, missing=0.3)
```

To create an `MFRM_Sim_Matrix` object called `my_mfrm_sim` with a maximum score of 5, specified person locations for 100 persons saved as a variable named `my_person_abils`, specified item locations for 12 items saved as a variable named `my_item_diffs`, specified rater facet effects for 6 raters saved as a variable named `my_facet_effects`, a specified set of Rasch-Andrich thresholds saved as a variable named `my_thresholds`, specified person names saved as a variable named `my_person_names`, specified item names saved as a variable named `my_item_names` and specified rater names saved as a variable named `my_rater_names`, with no missing data:

```
my_mfrm_sim = MFRM_Sim_Matrix(12, 100, 6, 5, manual_persons=my_person_abils,
                               manual_items=my_item_diffs,
                               manual_facet_effects=my_facet_effects,
                               manual_thresholds=my_thresholds,
                               manual_person_names=my_person_names,
                               manual_item_names=my_item_names,
                               manual_facet_names=my_rater_names)
```

13.2 Customising an `MFRM_Sim_Matrix` simulation

13.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm` from `Item_1` to `my_new_item_name`:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

13.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm.rename_items_all(my_new_item_names)
```

13.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the second multiindex level of `self.responses` with new name.

Example

To rename a person in object `my_mfrm` from `Person_1` to `my_new_person_name`:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

13.2.4 rename_persons_all

Description Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the second multiindex level of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm.rename_persons_all(my_new_person_names)
```


13.2.5 `rename_rater`

Description

Method to rename a single rater.

Usage

```
self.rename_rater(old, new)
```

Arguments

<code>old</code>	String: the old name for the rater
<code>new</code>	String: the new name for the rater

Returns

Replaces specified rater name in the first multiindex level of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm.rename_rater('Rater_1', 'my_new_rater_name')
```

13.2.6 `rename_raters_all`

Description Method to rename all raters.

Usage

```
self.rename_raters_all(new_names)
```

Arguments

<code>new_names</code>	List of new rater names as strings
------------------------	------------------------------------

Returns

Replaces all rater names in the first multiindex level of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm.rename_raters_all(my_new_rater_names)
```

14 class MFRM_Sim_Bivector

14.1 Generating an MFRM_Sim_Bivector simulation

14.1.1 MFRM_Sim_Bivector

Description

Creates an object of the class `MFRM_Sim_Bivector`. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the bivector rater representation (Elliott & Buttery, 2022b) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

In the bivector model, rater facet effect is decomposed into two additive components: a per-(rater, item) item effect and a per-(rater, threshold) threshold effect. The true facet effect for rater r , item i and threshold k is:

$$\lambda_{rik} = \lambda_{ri.} + \lambda_{r.k} \quad (30)$$

The item effect carries the overall location of the rater across items (free mean per rater); the threshold effect is zero-sum per rater across thresholds and carries only shape information. `MFRM_Sim_Bivector` is a thin convenience subclass of `MFRM_Sim` (`model='bivector'`); the two additive components are combined internally into the same rater $\times$ item $\times$ threshold representation used by the matrix model, so all attributes of `MFRM_Sim_Matrix` are available on the bivector object, plus `self.item_effects` and `self.threshold_effects`.

Usage

```
MFRM_Sim_Bivector(no_of_items, no_of_persons, max_score, no_of_facet_elements, item_range=2,
                  item_facet_range=2, threshold_facet_range=1, category_base=1,
                  person_sd=1.5, max_disorder=0, offset=0, missing=0,
                  shared_missing=True, manual_persons=None, manual_items=None,
                  manual_thresholds=None, manual_item_effects=None,
                  manual_threshold_effects=None, manual_person_names=None,
                  manual_item_names=None, manual_facet_names=None, facet='rater',
                  facet_plural=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.
<code>no_of_facet_elements</code>	Integer or None: The number of raters (facet elements) in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.

Arguments continue on the next page.

Arguments (continued)

<code>item_range</code>	Float: The range (maximum - minimum) of item locations for randomly generated item locations, drawn from a uniform distribution. Default is <code>item_range=2</code> . Ignored if <code>manual_items</code> is passed.
<code>item_facet_range</code>	Float: The total spread (maximum - minimum) of per-(rater, item) facet effects across the full rater $\times$ item matrix, in logits. Default is <code>item_facet_range=2</code> . Ignored if <code>manual_item_effects</code> is passed.
<code>threshold_facet_range</code>	Float: The total spread (maximum - minimum) of per-(rater, threshold) facet effects across the full rater $\times$ threshold matrix, in logits. Default is <code>threshold_facet_range=1</code> . Ignored if <code>manual_threshold_effects</code> is passed.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds. Default is <code>category_base=1</code> .
<code>person_sd</code>	Float: The standard deviation of person locations for randomly generated locations, drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if <code>manual_persons</code> is passed.
<code>max_disorder</code>	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around <code>category_base</code> . Default is <code>max_disorder=0</code> , with no disordered thresholds.
<code>offset</code>	Float: The difference between the means of person locations and item locations. If positive, mean person location is higher than mean item location. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data in the simulation, introduced according to a missing completely at random (MCAR) pattern. Default is <code>missing=0</code> .
<code>shared_missing</code>	Boolean: If <code>True</code> , the same persons are missing across all raters (correlated missingness). If <code>False</code> , missingness is independent across raters. Default is <code>shared_missing=True</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. <code>manual_persons=None</code> , random locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random locations are generated. Default is <code>manual_items=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If <code>manual_items=None</code> , random thresholds are generated. Default is <code>manual_thresholds=None</code> .
<code>manual_facet_effects</code>	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If <code>manual_facet_effects=None</code> , random facet effects are generated. Default is <code>manual_facet_effects=None</code> .
<code>manual_item_effects</code>	pandas DataFrame or <code>None</code> : Optional specified per-(rater, item) facet effects. Rows are raters, columns are items. If <code>None</code> , random item effects are generated. Default is <code>manual_item_effects=None</code> . Details of the format are given below.
<code>manual_threshold_effects</code>	pandas DataFrame or <code>None</code> : Optional specified per-(rater, threshold) facet effects. Rows are raters, columns are threshold positions (zero-sum per rater). If <code>None</code> , random threshold effects are generated. Default is <code>manual_threshold_effects=None</code> . Details of the format are given below.
<code>manual_person_names</code>	List: Optional list of person names. If <code>None</code> , names are generated automatically in the format <code>Person_1</code> etc. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names. If <code>None</code> , names are generated automatically in the format <code>Item_1</code> etc. Default is <code>manual_item_names=None</code> .
<code>manual_facet_names</code>	List: Optional list of rater names. If <code>None</code> , names are generated automatically in the format <code>Rater_1</code> etc. Default is <code>manual_facet_names=None</code> .
<code>facet</code>	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format <code>{Facet}_1</code> etc., e.g. <code>Judge_1</code>). Also aliases <code>self.no_of_{facet_plural}</code> and <code>self.{facet_plural}_effects</code> – note this does <i>not</i> alias <code>self.rater_names/self.facet_names</code> , which keep their fixed names regardless. Default is <code>facet='rater'</code> .
<code>facet_plural</code>	String or <code>None</code> : plural form of <code>facet</code> , used for the pluralised attribute aliases described above. If <code>None</code> (default), <code>facet + 's'</code> is used. Default is <code>facet_plural=None</code> .
<code>seed</code>	Integer or <code>None</code> : seed for the random number generator, shared by the item-effect and threshold-effect draws and every other random draw in the simulation, so the whole simulation is reproducible from one seed. Pass an integer for full reproducibility; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Format for `manual_item_effects` input

A pandas DataFrame with rater names as the row index and item names as column names. Each cell contains the facet effect for that rater-item combination. Toy example with two raters and two items:

```
manual_item_effects = pd.DataFrame({'Item_1': { 'Rater_1': -0.5, 'Rater_2': 0.5},
                                   'Item_2': { 'Rater_1': 0.3, 'Rater_2': -0.3}})
```

Format for `manual_threshold_effects` input

A pandas DataFrame with rater names as the row index and threshold positions (0 to `max_score - 1`) as columns. Each row must sum to zero. Toy example with two raters and a maximum score of 3:

```
manual_threshold_effects = pd.DataFrame({'Rater_1': [-0.3, 0.0, 0.3],
                                         'Rater_2': [0.2, -0.4, 0.2]}).T
```

Returns

Object of class `MFRM_Sim_Bivector`. Several attributes are automatically generated on its creation:

<code>self.responses</code>	pandas DataFrame: The multiindex dataframe of simulated responses, which may be saved to file or used to create an <code>MFRM</code> object for analysis.
<code>self.no_of_items</code>	Integer: The number of items, specified by argument <code>no_of_items</code> .
<code>self.no_of_persons</code>	Integer: The number of persons, specified by argument <code>no_of_persons</code> .
<code>self.no_of_raters</code>	Integer: The number of raters, specified by argument <code>no_of_raters</code> .
<code>self.max_score</code>	Integer: The shared maximum possible score on each item.
<code>self.item_range</code>	Float: The generating range of item locations.
<code>self.category_base</code>	Float: The generating base category width.
<code>self.person_sd</code>	Float: The generating standard deviation of person locations.
<code>self.offset</code>	Float: The generating offset between person location and item location means.
<code>self.missing</code>	Float: The generating proportion of missing data.
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: Item names as keys and item locations as values.
<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> . Note (breaking change from earlier versions): the leading sentinel $\tau_0 = 0$ has been removed.
<code>self.facet_effects</code>	pandas DataFrame with a MultiIndex (rater, item) as the row index and threshold positions as columns: the reconstructed full facet effect matrix used internally by the MFRM probability machinery.

Returns continue on the next page.

Returns (continued)

<code>self.item_effects</code>	pandas DataFrame: Per-(rater, item) facet effects, with rater names as the row index and item names as column names.
<code>self.threshold_effects</code>	pandas DataFrame: Per-(rater, threshold) facet effects, with rater names as the row index and threshold positions as column names. Each row sums to zero.
<code>self.model</code>	String: Always 'bivector'.
<code>self.person_names</code>	List: Person names.
<code>self.rater_names</code>	List: Rater names.

Examples

To create an `MFRM_Sim_Bivector` object called `my_mfrm_sim` with a maximum score of 5, randomly generated parameters for 5,000 persons, 10 items and 12 raters, specifying an item range of 4 logits, an item facet range of 3 logits, a threshold facet range of 1 logit, a base category width of 1.5 logits, a person SD of 2 logits and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Bivector(10, 5000, 12, 5, item_range=4,
                                item_facet_range=3, threshold_facet_range=1,
                                category_base=1.5, person_sd=2, missing=0.3)
```

To create an `MFRM_Sim_Bivector` object called `my_mfrm_sim` with specified item effects and threshold effects, together with specified person locations, item locations and thresholds:

```
my_mfrm_sim = MFRM_Sim_Bivector(12, 100, 6, 5,
                                manual_persons=my_person_abils,
                                manual_items=my_item_diffs,
                                manual_thresholds=my_thresholds,
                                manual_item_effects=my_item_effects,
                                manual_threshold_effects=my_threshold_effects,
                                manual_person_names=my_person_names,
                                manual_item_names=my_item_names,
                                manual_facet_names=my_rater_names)
```

14.2 Customising an `MFRM_Sim_Bivector` simulation

14.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm_sim` from `Item_1` to `my_new_item_name`:

```
my_mfrm_sim.rename_item('Item_1', 'my_new_item_name')
```

14.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm_sim` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm_sim.rename_items_all(my_new_item_names)
```

14.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the second multiindex level of `self.responses` with new name.

Example

To rename a person in object `my_mfrm_sim` from `Person_1` to `my_new_person_name`:

```
my_mfrm_sim.rename_person('Person_1', 'my_new_person_name')
```

14.2.4 rename_persons_all

Description

Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the second multiindex level of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm_sim` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm_sim.rename_persons_all(my_new_person_names)
```


14.2.5 `rename_rater`

Description

Method to rename a single rater.

Usage

```
self.rename_rater(old, new)
```

Arguments

<code>old</code>	String: the old name for the rater
<code>new</code>	String: the new name for the rater

Returns

Replaces specified rater name in the first multiindex level of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm_sim` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm_sim.rename_rater('Rater_1', 'my_new_rater_name')
```

14.2.6 `rename_raters_all`

Description

Method to rename all raters.

Usage

```
self.rename_raters_all(new_names)
```

Arguments

<code>new_names</code>	List of new rater names as strings
------------------------	------------------------------------

Returns

Replaces all rater names in the first multiindex level of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm_sim` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm_sim.rename_raters_all(my_new_rater_names)
```

15 class MFRM_Sim_Centrality

15.1 Generating an MFRM_Sim_Centrality simulation

15.1.1 MFRM_Sim_Centrality

Description

Creates an object of the class `MFRM_Sim_Centrality`. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the centrality/extremity model (Jin & Wang, 2018) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Unlike `MFRM_Sim_Thresholds`, each rater's threshold-severity profile is generated from just two true parameters: a severity shift λ_r and a threshold-stretch ω_r :

$$\lambda_{rk} = \lambda_r + (\omega_r - 1)\tau_k \quad (31)$$

$\omega_r > 1$ spreads the reference thresholds τ_k apart for that rater ('central' – a larger person-location gap is required to traverse to a more extreme category, so extreme categories are under-used); $0 < \omega_r < 1$ compresses them together ('extreme' – the opposite effect); $\omega_r = 1$ is neutral and collapses to the global representation.

Usage

```
MFRM_Sim_Centrality(no_of_items, no_of_persons, max_score, no_of_facet_elements,
                    global_range=2, stretch_range=1, manual_lambda=None, manual_omega=None,
                    item_range=2, category_base=1, person_sd=1.5, max_disorder=0,
                    offset=0, missing=0, shared_missing=True, manual_persons=None,
                    manual_items=None, manual_thresholds=None, manual_person_names=None,
                    manual_item_names=None, manual_facet_names=None, facet='rater',
                    facet_plural=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>no_of_facet_elements</code>	Integer or None: The number of raters (facet elements) in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.
<code>global_range</code>	Float: The total spread (maximum – minimum) of true λ_r (severity shift) values across raters, analogous to <code>facet_range</code> in <code>MFRM_Sim_Thresholds</code> . Default is <code>global_range=2</code> . Ignored if <code>manual_lambda</code> is passed.

Arguments continue on the next page.

Arguments (continued)

<code>stretch_range</code>	Float: The total spread of true ω_r (threshold-stretch) values around 1 – e.g. <code>stretch_range=1</code> typically gives ω_r roughly in $[0.5, 1.5]$. Default is <code>stretch_range=1</code> . Ignored if <code>manual_omega</code> is passed.
<code>manual_lambda</code>	List or 1-dimensional numpy array: Optional list of specified true λ_r values, length <code>no_of_raters</code> . If <code>None</code> (default), drawn the same way MFRM_Sim's own <code>facet_range</code> scales severities.
<code>manual_omega</code>	List or 1-dimensional numpy array: Optional list of specified true ω_r values, length <code>no_of_raters</code> . If <code>None</code> (default), drawn as 1+ a <code>stretch_range</code> -scaled, zero-mean deviation.
<code>item_range</code>	Float: The range of item locations for the internally-generated preliminary MFRM_Sim_Thresholds simulation, drawn from a uniform distribution. Default is <code>item_range=2</code> . Ignored if <code>manual_items</code> is passed.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds. Default is <code>category_base=1</code> .
<code>person_sd</code>	Float: The standard deviation of person locations for the preliminary simulation, drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if <code>manual_persons</code> is passed.
<code>max_disorder</code>	Float: If negative, the preliminary simulation's Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width. Default is <code>max_disorder=0</code> .
<code>offset</code>	Float: The difference between the means of person locations and item locations in the preliminary simulation. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data, introduced according to a missing completely at random (MCAR) pattern. Default is <code>missing=0</code> .
<code>shared_missing</code>	Boolean: If <code>True</code> , the same persons are missing across all raters (correlated missingness). If <code>False</code> , missingness is independent across raters. Default is <code>shared_missing=True</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. If <code>None</code> , random locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. If <code>None</code> , random locations are generated. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. If <code>None</code> , random thresholds are generated. Default is <code>manual_thresholds=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_person_names</code>	List: Optional list of person names. If <code>None</code> , names are generated automatically in the format <code>Person_1</code> etc. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names. If <code>None</code> , names are generated automatically in the format <code>Item_1</code> etc. Default is <code>manual_item_names=None</code> .
<code>manual_facet_names</code>	List: Optional list of rater names. If <code>None</code> , names are generated automatically in the format <code>Rater_1</code> etc. Default is <code>manual_facet_names=None</code> .
<code>facet</code>	String: singular name for the second (non-item, non-person) facet of the simulated data. Default is <code>facet='rater'</code> .
<code>facet_plural</code>	String or <code>None</code> : plural form of <code>facet</code> . If <code>None</code> (default), <code>facet + 's'</code> is used.
<code>seed</code>	Integer or <code>None</code> : seed for the true λ_r/ω_r draws and every other random draw in the simulation (via both the preliminary and final internal <code>MFRM_Sim</code> calls), so the whole simulation is reproducible from one seed. Pass an integer for full reproducibility; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Format for `manual_lambda/manual_omega` input

Simple lists or 1-dimensional numpy arrays, one true value per rater, in rater order. Toy example with two raters:

```
manual_lambda = [-0.5, 0.5]
manual_omega = [ 1.5, 0.6]
```

Returns

Object of class `MFRM_Sim_Centrality`. Several attributes are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an <code>MFRM</code> object for analysis.
<code>self.no_of_items</code>	Integer: The number of items.
<code>self.no_of_persons</code>	Integer: The number of persons.
<code>self.no_of_raters</code>	Integer: The number of raters.
<code>self.max_score</code>	Integer: The shared maximum possible score on each item.
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: Item names as keys and item locations as values.

Returns continue on the next page.

Returns (continued)

<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> .
<code>self.lambda_</code>	pandas series: True generating λ_r (severity shift) per rater.
<code>self.omega</code>	pandas series: True generating ω_r (threshold-stretch) per rater.
<code>self.facet_effects</code>	pandas DataFrame: the implied rater $\times$ threshold severity table (Equation above), rater names as the row index, threshold positions as columns – same format as <code>self.facet_effects</code> on <code>MFRM_Sim_Thresholds</code> .
<code>self.model</code>	String: Always 'centrality'.
<code>self.person_names</code>	List: Person names.
<code>self.item_names</code>	List: Item names.
<code>self.rater_names</code>	List: Rater names.

Examples

To create an `MFRM_Sim_Centrality` object called `my_mfrm_sim` with a maximum score of 5, randomly generated parameters for 200 persons, 12 items and 8 raters, a true λ_r spread of 2 logits and a true ω_r spread of 1 around neutral:

```
my_mfrm_sim = MFRM_Sim_Centrality(no_of_items=12, no_of_persons=200,  
                                   no_of_raters=8, max_score=5, global_range=2,  
                                   stretch_range=1, seed=42)
```

To create an `MFRM_Sim_Centrality` object with specified true λ_r/ω_r values for 5 raters, saved as variables named `my_lambda` and `my_omega`, together with specified item locations and thresholds:

```
my_mfrm_sim = MFRM_Sim_Centrality(no_of_items=6, no_of_persons=100,  
                                   no_of_raters=5, max_score=5,  
                                   manual_lambda=my_lambda, manual_omega=my_omega,  
                                   manual_items=my_item_diffs,  
                                   manual_thresholds=my_thresholds, seed=42)
```

15.2 Customising an MFRM_Sim_Centrality simulation

15.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm_sim` from `Item_1` to `my_new_item_name`:

```
my_mfrm_sim.rename_item('Item_1', 'my_new_item_name')
```

15.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm_sim` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm_sim.rename_items_all(my_new_item_names)
```

15.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the second multiindex level of `self.responses` with new name.

Example

To rename a person in object `my_mfrm_sim` from `Person_1` to `my_new_person_name`:

```
my_mfrm_sim.rename_person('Person_1', 'my_new_person_name')
```

15.2.4 rename_persons_all

Description

Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the second multiindex level of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm_sim` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm_sim.rename_persons_all(my_new_person_names)
```

15.2.5 `rename_rater`

Description

Method to rename a single rater.

Usage

```
self.rename_rater(old, new)
```

Arguments

<code>old</code>	String: the old name for the rater
<code>new</code>	String: the new name for the rater

Returns

Replaces specified rater name in the first multiindex level of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm_sim` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm_sim.rename_rater('Rater_1', 'my_new_rater_name')
```

15.2.6 `rename_raters_all`

Description

Method to rename all raters.

Usage

```
self.rename_raters_all(new_names)
```

Arguments

<code>new_names</code>	List of new rater names as strings
------------------------	------------------------------------

Returns

Replaces all rater names in the first multiindex level of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm_sim` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm_sim.rename_raters_all(my_new_rater_names)
```


16 class MFRM_Sim_PseudoHalo

16.1 Generating an MFRM_Sim_PseudoHalo simulation

16.1.1 MFRM_Sim_PseudoHalo

Description

Creates an object of the class `MFRM_Sim_PseudoHalo`. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the `pseudo_halo` rater representation – the item-facet analogue of `MFRM_Sim_Centrality` – from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Unlike `MFRM_Sim_Items`, each rater’s item-severity profile is generated from just two true parameters: a severity shift λ_r and an item-difficulty-stretch ω_r :

$$\mu_{ri} = \lambda_r + (\omega_r - 1)\delta_i \quad (32)$$

$\omega_r < 1$ compresses the spread of reference item locations δ_i toward zero for that rater (the pseudo-halo signature – items look more similar in difficulty to that rater than they really are); $\omega_r > 1$ exaggerates real item-difficulty differences instead; $\omega_r = 1$ is neutral and collapses to the `global` representation.

Usage

```
MFRM_Sim_PseudoHalo(no_of_items, no_of_persons, max_score, no_of_facet_elements,
                    global_range=2, stretch_range=1, manual_lambda=None, manual_omega=None,
                    item_range=2, category_base=1, person_sd=1.5, max_disorder=0,
                    offset=0, missing=0, shared_missing=True, manual_persons=None,
                    manual_items=None, manual_thresholds=None, manual_person_names=None,
                    manual_item_names=None, manual_facet_names=None, facet='rater',
                    facet_plural=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>no_of_facet_elements</code>	Integer or <code>None</code> : The number of raters (facet elements) in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.
<code>global_range</code>	Float: The total spread (maximum – minimum) of true λ_r (severity shift) values across raters. Default is <code>global_range=2</code> . Ignored if <code>manual_lambda</code> is passed.

Arguments continue on the next page.

Arguments (continued)

<code>stretch_range</code>	Float: The total spread of true ω_r (item-difficulty-stretch) values around 1. Default is <code>stretch_range=1</code> . Ignored if <code>manual_omega</code> is passed.
<code>manual_lambda</code>	List or 1-dimensional numpy array: Optional list of specified true λ_r values, length <code>no_of_raters</code> . If <code>None</code> (default), drawn the same way MFRM_Sim's own <code>facet_range</code> scales severities.
<code>manual_omega</code>	List or 1-dimensional numpy array: Optional list of specified true ω_r values, length <code>no_of_raters</code> . If <code>None</code> (default), drawn as 1+ a <code>stretch_range</code> -scaled, zero-mean deviation.
<code>item_range</code>	Float: The range of item locations for the internally-generated preliminary MFRM_Sim_Items simulation, drawn from a uniform distribution. Default is <code>item_range=2</code> . Ignored if <code>manual_items</code> is passed.
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds in the preliminary simulation. Default is <code>category_base=1</code> .
<code>person_sd</code>	Float: The standard deviation of person locations for the preliminary simulation, drawn from a normal distribution. Default is <code>person_sd=1.5</code> . Ignored if <code>manual_persons</code> is passed.
<code>max_disorder</code>	Float: If negative, the preliminary simulation's Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width. Default is <code>max_disorder=0</code> .
<code>offset</code>	Float: The difference between the means of person locations and item locations in the preliminary simulation. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data, introduced according to a missing completely at random (MCAR) pattern. Default is <code>missing=0</code> .
<code>shared_missing</code>	Boolean: If <code>True</code> , the same persons are missing across all raters (correlated missingness). If <code>False</code> , missingness is independent across raters. Default is <code>shared_missing=True</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. If <code>None</code> , random locations are generated. Default is <code>manual_persons=None</code> .
<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. If <code>None</code> , random locations are generated. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. If <code>None</code> , random thresholds are generated. Default is <code>manual_thresholds=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_person_names</code>	List: Optional list of person names. If <code>None</code> , names are generated automatically in the format <code>Person_1</code> etc. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names. If <code>None</code> , names are generated automatically in the format <code>Item_1</code> etc. Default is <code>manual_item_names=None</code> .
<code>manual_facet_names</code>	List: Optional list of rater names. If <code>None</code> , names are generated automatically in the format <code>Rater_1</code> etc. Default is <code>manual_facet_names=None</code> .
<code>facet</code>	String: singular name for the second (non-item, non-person) facet of the simulated data. Default is <code>facet='rater'</code> .
<code>facet_plural</code>	String or <code>None</code> : plural form of <code>facet</code> . If <code>None</code> (default), <code>facet + 's'</code> is used.
<code>seed</code>	Integer or <code>None</code> : seed for the true λ_r/ω_r draws and every other random draw in the simulation. Pass an integer for full reproducibility; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Format for `manual_lambda/manual_omega` input

Simple lists or 1-dimensional numpy arrays, one true value per rater, in rater order. Toy example with two raters:

```
manual_lambda = [-0.5, 0.5]
manual_omega = [ 1.5, 0.6]
```

Returns

Object of class `MFRM_Sim_PseudoHalo`. Several attributes are automatically generated on its creation:

<code>self.responses</code>	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an <code>MFRM</code> object for analysis.
<code>self.no_of_items</code>	Integer: The number of items.
<code>self.no_of_persons</code>	Integer: The number of persons.
<code>self.no_of_raters</code>	Integer: The number of raters.
<code>self.max_score</code>	Integer: The shared maximum possible score on each item.
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: Item names as keys and item locations as values.
<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> .
<code>self.lambda_</code>	pandas series: True generating λ_r (severity shift) per rater.

Returns continue on the next page.

Returns (continued)

<code>self.omega</code>	pandas series: True generating ω_r (item-difficulty-stretch) per rater.
<code>self.facet_effects</code>	pandas DataFrame: the implied rater $\times$ item severity table (Equation above), rater names as the row index, item names as columns – same format as <code>self.facet_effects</code> on <code>MFRM_Sim_Items</code> .
<code>self.model</code>	String: Always 'pseudo_halo'.
<code>self.person_names</code>	List: Person names.
<code>self.item_names</code>	List: Item names.
<code>self.rater_names</code>	List: Rater names.

Examples

To create an `MFRM_Sim_PseudoHalo` object called `my_mfrm_sim` with a maximum score of 5, randomly generated parameters for 200 persons, 12 items and 8 raters, a true λ_r spread of 2 logits and a true ω_r spread of 1 around neutral:

```
my_mfrm_sim = MFRM_Sim_PseudoHalo(no_of_items=12, no_of_persons=200,  
                                   no_of_raters=8, max_score=5, global_range=2,  
                                   stretch_range=1, seed=42)
```

To create an `MFRM_Sim_PseudoHalo` object with specified true λ_r/ω_r values for 5 raters, saved as variables named `my_lambda` and `my_omega`, together with specified item locations and thresholds:

```
my_mfrm_sim = MFRM_Sim_PseudoHalo(no_of_items=6, no_of_persons=100,  
                                   no_of_raters=5, max_score=5,  
                                   manual_lambda=my_lambda, manual_omega=my_omega,  
                                   manual_items=my_item_diffs,  
                                   manual_thresholds=my_thresholds, seed=42)
```

16.2 Customising an `MFRM_Sim_PseudoHalo` simulation

16.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm_sim` from `Item_1` to `my_new_item_name`:

```
my_mfrm_sim.rename_item('Item_1', 'my_new_item_name')
```

16.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm_sim` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm_sim.rename_items_all(my_new_item_names)
```

16.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the second multiindex level of `self.responses` with new name.

Example

To rename a person in object `my_mfrm_sim` from `Person_1` to `my_new_person_name`:

```
my_mfrm_sim.rename_person('Person_1', 'my_new_person_name')
```

16.2.4 rename_persons_all

Description

Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the second multiindex level of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm_sim` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm_sim.rename_persons_all(my_new_person_names)
```

16.2.5 `rename_rater`

Description

Method to rename a single rater.

Usage

```
self.rename_rater(old, new)
```

Arguments

<code>old</code>	String: the old name for the rater
<code>new</code>	String: the new name for the rater

Returns

Replaces specified rater name in the first multiindex level of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm_sim` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm_sim.rename_rater('Rater_1', 'my_new_rater_name')
```

16.2.6 `rename_raters_all`

Description

Method to rename all raters.

Usage

```
self.rename_raters_all(new_names)
```

Arguments

<code>new_names</code>	List of new rater names as strings
------------------------	------------------------------------

Returns

Replaces all rater names in the first multiindex level of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm_sim` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm_sim.rename_raters_all(my_new_rater_names)
```

17 class MFRM_Sim_Bistretch

17.1 Generating an MFRM_Sim_Bistretch simulation

17.1.1 MFRM_Sim_Bistretch

Description

Creates an object of the class `MFRM_Sim_Bistretch`. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the bistretch rater representation from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters. Unlike `MFRM_Sim_Bivector`, each rater's (item $\times$ threshold) severity profile is generated from just three true parameters: a severity shift λ_r , an item-difficulty-stretch ω_{rI} and a threshold-stretch ω_{rT} – combining both stretch axes at once.

Usage

```
MFRM_Sim_Bistretch(no_of_items, no_of_persons, max_score, no_of_facet_elements,
                    global_range=2, stretch_range=1, manual_lambda=None, manual_omega_items=None,
                    manual_omega_thresholds=None, item_range=2, category_base=1,
                    person_sd=1.5, max_disorder=0, offset=0, missing=0, shared_missing=True,
                    manual_persons=None, manual_items=None, manual_thresholds=None,
                    manual_person_names=None, manual_item_names=None, manual_facet_names=None,
                    facet='rater', facet_plural=None, seed=None)
```

Arguments

<code>no_of_items</code>	Integer: The number of items in the simulation.
<code>no_of_persons</code>	Integer: The number of persons in the simulation.
<code>no_of_facet_elements</code>	Integer or <code>None</code> : The number of raters (facet elements) in the simulation.
<code>max_score</code>	Integer: The maximum score on each item.
<code>global_range</code>	Float: The total spread (maximum – minimum) of true λ_r (severity shift) values across raters. Default is <code>global_range=2</code> . Ignored if <code>manual_lambda</code> is passed.
<code>stretch_range</code>	Float: The total spread of true $\omega_{\text{items},r}$ and $\omega_{\text{thresholds},r}$ values around 1 (use <code>manual_omega_items/manual_omega_thresholds</code> if you want them to differ). Default is <code>stretch_range=1</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_lambda</code>	List or 1-dimensional numpy array: Optional list of specified true λ_r values, length <code>no_of_raters</code> . If <code>None</code> (default), drawn the same way MFRM_Sim's own <code>facet_range</code> scales severities.
<code>manual_omega_items</code>	List or 1-dimensional numpy array: Optional list of specified true $\omega_{\text{items},r}$ (item-stretch) values, length <code>no_of_raters</code> . If <code>None</code> (default), drawn as 1+ a <code>stretch_range</code> -scaled, zero-mean deviation.
<code>manual_omega_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified true $\omega_{\text{thresholds},r}$ (threshold-stretch) values, length <code>no_of_raters</code> . If <code>None</code> (default), drawn as 1+ a <code>stretch_range</code> -scaled, zero-mean deviation, independently of $\omega_{\text{items},r}$.
<code>item_range</code>	Float: The range of item locations for the internally-generated preliminary MFRM_Sim_Bivector simulation. Default is <code>item_range=2</code> . Ignored if <code>manual_items</code> is passed.
<code>item_facet_range</code>	Float: Spread of the preliminary MFRM_Sim_Bivector simulation's own free per-(rater, item) effects, which are then overwritten by the true stretch structure. Default is <code>item_facet_range=2</code> .
<code>threshold_facet_range</code>	Float: Spread of the preliminary simulation's own free per-(rater, threshold) effects, likewise overwritten. Default is <code>threshold_facet_range=1</code> .
<code>category_base</code>	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds in the preliminary simulation. Default is <code>category_base=1</code> .
<code>person_sd</code>	Float: The standard deviation of person locations for the preliminary simulation. Default is <code>person_sd=1.5</code> . Ignored if <code>manual_persons</code> is passed.
<code>max_disorder</code>	Float: If negative, the preliminary simulation's Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). Default is <code>max_disorder=0</code> .
<code>offset</code>	Float: The difference between the means of person locations and item locations in the preliminary simulation. Default is <code>offset=0</code> .
<code>missing</code>	Float between 0 and 1: The proportion of missing data (MCAR). Default is <code>missing=0</code> .
<code>shared_missing</code>	Boolean: If <code>True</code> , the same persons are missing across all raters. If <code>False</code> , missingness is independent across raters. Default is <code>shared_missing=True</code> .
<code>manual_persons</code>	List or 1-dimensional numpy array: Optional list of specified person locations. Default is <code>manual_persons=None</code> .

Arguments continue on the next page.

Arguments (continued)

<code>manual_items</code>	List or 1-dimensional numpy array: Optional list of specified central item locations. Default is <code>manual_items=None</code> .
<code>manual_thresholds</code>	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Default is <code>manual_thresholds=None</code> .
<code>manual_person_names</code>	List: Optional list of person names. Default is <code>manual_person_names=None</code> .
<code>manual_item_names</code>	List: Optional list of item names. Default is <code>manual_item_names=None</code> .
<code>manual_facet_names</code>	List: Optional list of rater names. Default is <code>manual_facet_names=None</code> .
<code>facet</code>	String: singular name for the second (non-item, non-person) facet of the simulated data. Default is <code>facet='rater'</code> .
<code>facet_plural</code>	String or <code>None</code> : plural form of <code>facet</code> . If <code>None</code> (default), <code>facet + 's'</code> is used.
<code>seed</code>	Integer or <code>None</code> : seed for the true $\lambda_r/\omega_{\text{items},r}/\omega_{\text{thresholds},r}$ draws and every other random draw in the simulation. Pass an integer for full reproducibility; <code>None</code> (default) draws fresh entropy each run. See Section 1.

Format for `manual_lambda/manual_omega_items/manual_omega_thresholds` input

Simple lists or 1-dimensional numpy arrays, one true value per rater, in rater order. Toy example with two raters:

```
manual_lambda = [-0.5, 0.5]
manual_omega_items = [ 1.5, 0.6]
manual_omega_thresholds = [ 1.5, 0.6]
```

Returns

Object of class `MFRM_Sim_Bistretch`. Several attributes are automatically generated on its creation:

<code>self.responses</code>	pandas DataFrame: The multiindex dataframe of simulated responses, which may be saved to file or used to create an <code>MFRM</code> object for analysis.
<code>self.no_of_items</code>	Integer: The number of items.
<code>self.no_of_persons</code>	Integer: The number of persons.
<code>self.no_of_raters</code>	Integer: The number of raters.
<code>self.max_score</code>	Integer: The shared maximum possible score on each item.
<code>self.persons</code>	pandas series: Person names as keys and person locations as values.
<code>self.items</code>	pandas series: Item names as keys and item locations as values.

Returns continue on the next page.

Returns (continued)

<code>self.thresholds</code>	pandas series: set of <code>self.max_score</code> Rasch-Andrich thresholds $\tau_1, \dots, \tau_m$, indexed 1 to <code>self.max_score</code> .
<code>self.lambda_</code>	pandas series: True generating λ_r (severity shift) per rater.
<code>self.omega_items</code>	pandas series: True generating $\omega_{\text{items},r}$ (item-stretch) per rater.
<code>self.omega_thresholds</code>	pandas series: True generating $\omega_{\text{thresholds},r}$ (threshold-stretch) per rater.
<code>self.item_effects</code>	pandas DataFrame: the implied per-(rater, item) effect table, rater names as the row index, item names as columns – same format as <code>self.item_effects</code> on <code>MFRM_Sim_Bivector</code> .
<code>self.threshold_effects</code>	pandas DataFrame: the implied per-(rater, threshold) effect table, rater names as the row index, threshold positions as columns, zero-summed per rater – same format as <code>self.threshold_effects</code> on <code>MFRM_Sim_Bivector</code> .
<code>self.facet_effects</code>	pandas DataFrame with a MultiIndex (rater, item) as the row index and threshold positions as columns: the reconstructed full facet effect matrix (item effect plus threshold effect per cell), same format as <code>self.facet_effects</code> on <code>MFRM_Sim_Bivector</code> .
<code>self.model</code>	String: Always 'bistretch'.
<code>self.person_names</code>	List: Person names.
<code>self.item_names</code>	List: Item names.
<code>self.rater_names</code>	List: Rater names.

Examples

To create an `MFRM_Sim_Bistretch` object called `my_mfrm_sim` with a maximum score of 5, randomly generated parameters for 200 persons, 12 items and 8 raters, a true λ_r spread of 2 logits and a true $\omega_{\text{items},r}/\omega_{\text{thresholds},r}$ spread of 1.2 around neutral:

```
my_mfrm_sim = MFRM_Sim_Bistretch(no_of_items=12, no_of_persons=200,
                                  no_of_raters=8, max_score=5, global_range=2,
                                  stretch_range=1.2, seed=42)
```

To create an `MFRM_Sim_Bistretch` object with specified true $\lambda_r/\omega_{\text{items},r}/\omega_{\text{thresholds},r}$ values for 5 raters, saved as variables named `my_lambda`, `my_omega_items` and `my_omega_thresholds`, together with specified item locations and thresholds:

```
my_mfrm_sim = MFRM_Sim_Bistretch(no_of_items=4, no_of_persons=100,  
                                  no_of_raters=5, max_score=5,  
                                  manual_lambda=my_lambda,  
                                  manual_omega_items=my_omega_items,  
                                  manual_omega_thresholds=my_omega_thresholds,  
                                  manual_items=my_item_diffs,  
                                  manual_thresholds=my_thresholds, seed=42)
```

17.2 Customising an MFRM_Sim_Bistretch simulation

17.2.1 `rename_item`

Description

Method to rename a single item.

Usage

```
self.rename_item(old, new)
```

Arguments

<code>old</code>	String: the old name for the item
<code>new</code>	String: the new name for the item

Returns

Replaces specified item name in the relevant column of `self.responses` with new name.

Example

To rename an item in object `my_mfrm_sim` from `Item_1` to `my_new_item_name`:

```
my_mfrm_sim.rename_item('Item_1', 'my_new_item_name')
```

17.2.2 `rename_items_all`

Description

Method to rename all items.

Usage

```
self.rename_items_all(new_names)
```

Arguments

<code>new_names</code>	List of new item names as strings
------------------------	-----------------------------------

Returns

Replaces all item names in the columns of `self.responses` with new names.

Example

To rename all items in object `my_mfrm_sim` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm_sim.rename_items_all(my_new_item_names)
```

17.2.3 rename_person

Description

Method to rename a single person.

Usage

```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns

Replaces specified person name in the second multiindex level of `self.responses` with new name.

Example

To rename a person in object `my_mfrm_sim` from `Person_1` to `my_new_person_name`:

```
my_mfrm_sim.rename_person('Person_1', 'my_new_person_name')
```

17.2.4 rename_persons_all

Description

Method to rename all persons.

Usage

```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings
-----------	-------------------------------------

Returns

Replaces all person names in the second multiindex level of `self.responses` with new names.

Example

To rename all persons in object `my_mfrm_sim` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm_sim.rename_persons_all(my_new_person_names)
```

17.2.5 `rename_rater`

Description

Method to rename a single rater.

Usage

```
self.rename_rater(old, new)
```

Arguments

<code>old</code>	String: the old name for the rater
<code>new</code>	String: the new name for the rater

Returns

Replaces specified rater name in the first multiindex level of `self.responses` with new name.

Example

To rename a rater in object `my_mfrm_sim` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm_sim.rename_rater('Rater_1', 'my_new_rater_name')
```

17.2.6 `rename_raters_all`

Description

Method to rename all raters.

Usage

```
self.rename_raters_all(new_names)
```

Arguments

<code>new_names</code>	List of new rater names as strings
------------------------	------------------------------------

Returns

Replaces all rater names in the first multiindex level of `self.responses` with new names.

Example

To rename all raters in object `my_mfrm_sim` with rater names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm_sim.rename_raters_all(my_new_rater_names)
```

References

- Andersen, E. B. (1973a). A Goodness of Fit Test for the Rasch Model. *Psychometrika*, 38(1), 123–140.
- Andersen, E. B. (1973b). Conditional inference for multiple-choice questionnaires. *British Journal of Mathematical and Statistical Psychology*, 26(1), 31–44.
- Andrich, D. (1978). A rating formulation for ordered response categories. *Psychometrika*, 43(4), 561–573.
- Andrich, D. (2010). Understanding the Response Structure and Process in the Polytomous Rasch Model. In *Handbook of polytomous item response theory models* (123–152).
- Andrich, D., & Kreiner, S. (2010). Quantifying response dependence between two dichotomous items using the Rasch model. *Applied Psychological Measurement*, 34(3), 181–192.
- Benjamini, Y., & Hochberg, Y. (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society. Series B (Methodological)*, 57(1), 289–300.
- Bradley, R. A., & Terry, M. E. (1952). Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika*, 39(3/4), 324.
- Choppin, B. (1968). Item bank using sample-free calibration. *Nature*, 219(5156), 870–872.
- Choppin, B. (1985). A fully conditional estimation procedure for Rasch model parameters. *Evaluation in Education*, 9(1), 29–42.
- Cochran, W. G. (1954). The Combination of Estimates from Different Experiments. *Biometrics*, 10(1), 101–129.
- Cronbach, L. J. (1951). Coefficient alpha and the internal structure of tests. *Psychometrika*, 16(3), 297–334.
- Elliott, M. (2025). *Extended many-facet Rasch models: Accounting for rater effects in automated essay scoring systems* [Doctoral dissertation, University of Cambridge].
- Elliott, M. (2026). RaschPy: Python Rasch Analysis Package.
- Elliott, M., & Buttery, P. J. (2022a). Extended rater representations in the many-facet Rasch model. *Journal of Applied Measurement*, 22(1), 133–160.
- Elliott, M., & Buttery, P. J. (2022b). Non-iterative conditional pairwise estimation for the rating scale model. *Educational and Psychological Measurement*, 82(5), 989–1019.
- Elliott, M., & Buttery, P. J. (2025). Addressing non-uniform rater effects with extended many-facet Rasch models: A case study. *Paper presented at the Nordic Educational Research Association Conference 2025, Helsinki, Finland, March 5–7*.
- Garner, M., & Engelhard, G. (2002). An eigenvector method for estimating item parameters of the dichotomous and polytomous Rasch models. *Journal of Applied Measurement*, 3(2), 107–128.
- Garner, M., & Engelhard, G. (2009). Using paired comparison matrices to estimate parameters of the partial credit Rasch measurement model for rater-mediated assessments. *Journal of Applied Measurement*, 10(1), 30–41.
- Holland, P. W., & Wainer, H. (Eds.). (1993). *Differential Item Functioning*. Lawrence Erlbaum.
- Iglewicz, B., & Hoaglin, D. C. (1993). *The ASQC Basic References in Quality Control: Statistical Techniques: How to Detect and Handle Outliers* (E. F. Mykytka, Ed.; Vol. 16). ASQC Quality Press.
- Jim, K.-Y., & Wang, W.-C. (2018). A New Facets Model for Rater’s Centrality/Extremity Response Style. *Journal of Educational Measurement*, 55(4), 543–563.
- Kornbrot, D. (2014). Point Biserial Correlation. In N. Balakrishnan, T. Colton, B. Everitt, W. Piegorsch, F. Ruggeri, & J. Teugels (Eds.), *Wiley statsref: Statistics reference online*. Wiley.

- Kou, G., & Lin, C. (2014). A cosine maximization method for the priority vector derivation in AHP. *European Journal of Operational Research*, 235(1), 225–232.
- Linacre, J. M. (1994). *Many-Facet Rasch Measurement*. MESA Press.
- Linacre, J. M. (2023). *Winsteps® Rasch measurement computer program User's Guide. Version 5.6.0*. Winsteps.com.
- Marais, I. (2012). Local independence. In K. B. Christensen, S. Kreiner, & M. Mesbah (Eds.), *Rasch models in health* (111–130). John Wiley & Sons.
- Masters, G. N. (1982). A Rasch model for partial credit scoring. *Psychometrika*, 47(2), 149–174.
- Pallant, J. F., & Tennant, A. (2007). An introduction to the Rasch measurement model: An example using the Hospital Anxiety and Depression Scale (HADS). *British Journal of Clinical Psychology*, 46(1), 1–18.
- Rasch, G. (1960). *Probabilistic models for some intelligence and attainment tests*. Danmarks Pædagogiske Institut.
- Rasch, G. (1968). A Mathematical Theory of Objectivity and its Consequences for Model Construction. *Paper presented at the European Meeting on Statistics, Econometrics and Management Science, Amsterdam, September 2–7, 1968*.
- Smith, E. V. (2002). Detecting and Evaluating the Impact of Multidimensionality Using Item Fit Statistics and Principal Component Analysis of Residuals. *Journal of Applied Measurement*, 3(2), 205–231.
- Tristan, A. (2006). An adjustment for sample size in DIF analysis. *Rasch Measurement Transactions*, 20(3), 1070–1071.
- Warm, T. A. (1989). Weighted likelihood estimation of ability in item response theory. *Psychometrika*, 54(3), 427–450.
- Waskom, M. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021.
- Wright, B. D. (1996). Comparing Rasch measurement and factor analysis. *Structural Equation Modeling*, 3(1), 3–24.
- Wright, B. D., & Masters, G. N. (1982). *Rating Scale Analysis*. MESA Press.
- Zwick, R., Thayer, D. T., & Mazzeo, J. (1997, June). *DESCRIBING AND CATEGORIZING DIF IN POLYTOMOUS ITEMS* (tech. rep. No. 1). Educational Testing Service. John Wiley; Sons Inc.